LAYER: 1_Core

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix.

<file_summary>
This section contains a summary of this file.

<purpose>
This file contains a packed representation of a subset of the repository's contents that is considered the most important context.
It is designed to be easily consumable by AI systems for analysis, code review,
or other automated processes.
</purpose>

<file_format>
The content is organized as follows:
1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
  - File path as an attribute
  - Full contents of the file
</file_format>

<usage_guidelines>
- This file should be treated as read-only. Any changes should be made to the
  original repository files, not this packed version.
- When processing this file, use the file path to distinguish
  between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with
  the same level of security as you would the original repository.
</usage_guidelines>

<notes>
- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: **/obj/**, **/bin/**, **/*.png, **/*.jpg, **/node_modules/**, **/*.Designer.cs
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)
</notes>

</file_summary>

<directory_structure>
Configuration.Web/Components/_Imports.razor
Configuration.Web/Components/Layout/MainLayout.razor
Configuration.Web/Components/Layout/ReconnectModal.razor
Configuration.Web/Components/NavMenu.razor
Configuration.Web/Components/NotFound.razor
Configuration.Web/Components/Routes.razor
Configuration.Web/Configurator/IWebAppConfigurator.cs
Configuration.Web/Configurator/WebAppConfigurator.cs
Configuration.Web/Configurator/WebScanningExtensions.cs
Configuration.Web/Promatis.Net.Configuration.Web.csproj
Configuration.Web/Properties/launchSettings.json
Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs
Configuration.Web/Services/TabNavigationService/TabItem.cs
Configuration.Web/Services/TabNavigationService/TabNavigationService.cs
Configuration.Web/Services/UiModuleService.cs
Configuration.Web/UiCommandBus/IUiCommandBus.cs
Configuration.Web/UiCommandBus/UiCommandBus.cs
Configuration.Web/WebAppBootstrappe.cs
Configuration/AppBootstrapper.cs
Configuration/AppConfigurator.cs
Configuration/AppInfrastructure.cs
Configuration/IAppConfigurator.cs
Configuration/Promatis.Net.Configuration.csproj
Configuration/ServiceScanningExtensions.cs
Data/Configurations/AuditLogConfiguration.cs
Data/Configurations/ModelBuilderConventions.cs
Data/DBContext/ApplicationDbContext.cs

Data/DBContext/ModelBuilderExtensions.cs
Data/Interceptors/ChangeEntryModel.cs
Data/Interceptors/DatabaseTriggerInterceptor.cs
Data/Interceptors/IUserProvider.cs
Data/Promatis.Net.Data.csproj
Data/Triggers/Castom/AuditTrigger.cs
Data/Triggers/Global/FluentValidationTrigger.cs
Data/Triggers/Global/ReferenceTreeParentTrigger.cs
Data/TriggerService/DatabaseTriggerService.cs
Data/TriggerService/EntityStateChangeEnum.cs
Data/TriggerService/IDatabaseTriggerService.cs
Data/TriggerService/PropertyChangeInfo.cs
Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs
Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs
Data/TriggerService/TriggerEvents/EntityChangedArgsBase.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs
Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs
Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs
Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs
Data/Validation/GlobalPolymorphicValidator.cs
Domain.Interface/Enums/EnumExtensions.cs
Domain.Interface/Interfaces/IAudit.cs
Domain.Interface/Interfaces/IDomainObject.cs
Domain.Interface/Interfaces/IDomainObjectHasKey.cs
Domain.Interface/Interfaces/IEntityCloner.cs
Domain.Interface/Interfaces/ISoftDeletable.cs
Domain.Interface/Interfaces/ITreeNode.cs
Domain.Interface/Promatis.Net.Domain.Interface.csproj
Domain/AuditLog/AuditLog.cs
Domain/DomainObject/DomainObject.cs
Domain/DomainObject/DomainObjectBase.cs
Domain/DomainObject/DomainObjectValidator.cs
Domain/EntityCloner/JsonEntityCloner.cs
Domain/Promatis.Net.Domain.csproj
Domain/ReferenceBase/ReferenceBase.cs
Domain/ReferenceBase/ReferenceBaseValidator.cs
Domain/ReferenceTreeBase/ReferenceTreeBase.cs
Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs
Promatis.Net.UI/_Imports.razor
Promatis.Net.UI/Components/ActionContext/DataContext.cs
Promatis.Net.UI/Components/ActionContext/EntityContext.cs
Promatis.Net.UI/Components/ActionContext/GridContext.cs
Promatis.Net.UI/Components/ActionContext/IWorkspaceContext.cs
Promatis.Net.UI/Components/ActionContext/ReferenceContext.cs
Promatis.Net.UI/Components/ActionContext/ReferenceTreeContext.cs
Promatis.Net.UI/Components/ActionContext/TreeContext.cs
Promatis.Net.UI/Components/ActionContext/WorkspaceContext.cs
Promatis.Net.UI/Components/Dialogs/BaseDialogLayout.razor
Promatis.Net.UI/Components/Dialogs/BaseDialogLayout.razor.cs
Promatis.Net.UI/Components/Dialogs/DialogTabConfig.cs
Promatis.Net.UI/Components/Dialogs/ReferenceDialogLayout.razor
Promatis.Net.UI/Components/Dialogs/ReferenceDialogLayout.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/RenderBase.cs
Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor.cs
Promatis.Net.UI/Components/UIControls/BaseUiControl.cs
Promatis.Net.UI/Components/UIControls/IHasOptions.cs
Promatis.Net.UI/Components/UIControls/IHasValue.cs
Promatis.Net.UI/Components/UIControls/IUiControl.cs
Promatis.Net.UI/Components/UIControls/UiControlAlignment.cs
Promatis.Net.UI/Components/Workspaces/GridPage.razor
Promatis.Net.UI/Components/Workspaces/GridPage.razor.cs
Promatis.Net.UI/Components/Workspaces/ReferencePage.razor
Promatis.Net.UI/Components/Workspaces/ReferencePage.razor.cs
Promatis.Net.UI/Components/Workspaces/ReferenceTreePage.razor
Promatis.Net.UI/Components/Workspaces/ReferenceTreePage.razor.cs
Promatis.Net.UI/Components/Workspaces/TreePage.razor
Promatis.Net.UI/Components/Workspaces/TreePage.razor.cs
Promatis.Net.UI/Components/Workspaces/UiToolbar.razor
Promatis.Net.UI/Components/Workspaces/UiToolbar.razor.cs

Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor
Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor.cs
Promatis.Net.UI/Controls/CreateChildButton.cs
Promatis.Net.UI/Controls/CreateEntityButton.cs
Promatis.Net.UI/Controls/DeleteEntityButton.cs
Promatis.Net.UI/Controls/EditEntityButton.cs
Promatis.Net.UI/Controls/ToolbarDivider.cs
Promatis.Net.UI/Data/Cache/FlatOzuMutationStrategy.cs
Promatis.Net.UI/Data/Cache/HierarchicalOzuMutationStrategy.cs
Promatis.Net.UI/Data/Cache/IOzuMutationStrategy.cs
Promatis.Net.UI/Data/Cache/IUiOzuCache.cs
Promatis.Net.UI/Data/Cache/UiOzuCache.cs
Promatis.Net.UI/Data/IUiDataBroker.cs
Promatis.Net.UI/Data/UiDataBroker.cs
Promatis.Net.UI/Extensions/MudColorExtensions.cs
Promatis.Net.UI/Interfaces/IHasSelectedData.cs
Promatis.Net.UI/IUiModule.cs
Promatis.Net.UI/Pages/AuditLogs/AuditLogContext.cs
Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor
Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditActionSelect.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditEntitySelect.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditExportButton.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditPeriodPicker.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditToolbarDivider.cs
Promatis.Net.UI/Pages/Dashboard.razor
Promatis.Net.UI/Pages/Dashboard.razor.css
Promatis.Net.UI/Promatis.Net.UI.csproj
Promatis.Net.UI/Theme/PromatisTheme.cs
Promatis.Net.UI/UiModule.cs
Promatis.Net.UI/wwwroot/App.css
Service/Contracts/AuditLog/AuditLogSearchRequest.cs
Service/Contracts/AuditLog/PagedResult.cs
Service/Promatis.Net.Service.csproj
Service/Services/AuditLogService/AuditLogService.cs
Service/Services/AuditLogService/IAuditLogService.cs
Service/Services/BaseService/BaseService.cs
Service/Services/BaseService/IBaseService.cs
Service/Services/ReferenceService/IReferenceService.cs
Service/Services/ReferenceService/ReferenceService.cs
Service/Services/ReferenceTreeService/IReferenceTreeService.cs
Service/Services/ReferenceTreeService/ReferenceTreeService.cs
Service/Services/TreeService/ITreeService.cs
Service/Services/TreeService/TreeService.cs
Tests/AssemblyInfo.cs
Tests/Domain/DomainLogicTests.cs
Tests/Domain/DomainObjectBaseLogicTests.cs
Tests/Domain/DomainObjectBaseTests.cs
Tests/Domain/DomainObjectExtendedTests.cs
Tests/Domain/ReferenceBaseTests.cs
Tests/Domain/ReferenceTreeTests.cs
Tests/Domain/SoftDeletableTests.cs
Tests/Interceptor/DatabaseTriggerInterceptorTests.cs
Tests/Promatis.Net.ApplicationModel.Tests.csproj
Tests/Service/BaseServiceTests_Crud.cs
Tests/Service/BaseServiceTests.cs
Tests/Service/ReferenceServiceTests_Search.cs
Tests/Service/ReferenceTreeServiceTests.cs
Tests/Service/ServiceIntegrationTests.cs
Tests/Trigger/AuditTriggerTests.cs
Tests/Trigger/DatabaseTriggerServiceTests.cs
Tests/Trigger/FluentValidationTriggerTests.cs
Tests/Trigger/FullTriggerChainTests.cs
Tests/Trigger/ReferenceTreeParentTriggerTests.cs
Tests/Trigger/TreeTriggerChainTests.cs
Tests/Validator/DomainObjectValidatorTests.cs
Tests/Validator/ReferenceBaseValidatorTests.cs
Tests/Validator/ReferenceTreeBaseValidatorTests.cs
</directory_structure>

<files>
This section contains the contents of the repository's files.

<file path="Configuration.Web/Components/_Imports.razor">
@using System.Net.HttpĐ
@using System.Net.Http.JsonĐ

```
@using Microsoft.AspNetCore.Components.FormsÐ
@using Microsoft.AspNetCore.Components.RoutingÐ
@using Microsoft.AspNetCore.Components.WebÐ
@using Microsoft.JSInteropÐ
@using MudBlazorÐ
@using Promatis.Net.UiÐ
@using Promatis.Net.Configuration.WebÐ
@using Promatis.Net.Configuration.Web.ComponentsÐ
@using Promatis.Net.Configuration.Web.Components.LayoutÐ
@using static Microsoft.AspNetCore.Components.Web.RenderMode
</file>

<file path="Configuration.Web/Components/Layout/MainLayout.razor">
@inherits LayoutComponentBaseÐ
@using Promatis.Net.Ui.ThemeÐ
Ð
@inject UiModuleService UiServiceÐ
@inject TabNavigationService TabServiceÐ
@implements IDisposableÐ
Ð
@* A„!Aô A  A´ AÔ : B 2Dô7D´2C 5CÂ ?D >C$0C”4CT@ D$5CÄK D  DC´0C4>CÂ BE <CÔ>C4> D 5Cd8CÄ0 *@Ð
<MudThemeProvider Theme="PromatisTheme.DefaultTheme" @bind-IsDarkMode="_isDarkMode" />Ð
<MudPopoverProvider />Ð
Ð
<MudDialogProvider FullWidth="false"Ð
                   MaxWidth="MaxWidth.False"Ð
                   CloseButton="false"Ð
                   BackdropClick="false"Ð
                   NoHeader="false"Ð
                   Position="DialogPosition.Center"Ð
                   CloseOnEscapeKey="true" />Ð
Ð
<MudSnackbarProvider />Ð
Ð
<CascadingValue Value="this">Ð
    <MudLayout>Ð
        @* A$5D ECÔODò ?C =CT;DÂ ?D 8C´>Cd5CÔ8Dò ¤
        <MudAppBar Color="Color.Default" Elevation="1">Ð
            @* A„!Aô A  A´ AÔ : At0CÄ5CÔ5CÔ> Color.Primary CÔ0 Color.Default *@Ð
            <MudIconButton Icon="@Icons.Material.Filled.Menu"Ð
                           Color="Color.Inherit"Ð
                           Edge="Edge.Start"Ð
                           Size="Size.Large"Ð
                           OnClick="ToggleDrawer" />Ð
            <MudStack Spacing="-1">Ð
                @* AD5DD>C´BCÔ>CR =C 7C$0CÔ8CR ¤
                <MudText Typo="Typo.h6" Class="ml-2">Ð
                    B$5D BCä2D´9 Cô@Cä5C¤B Promatis NetÐ
                </MudText>Ð
            </MudStack>Ð
Ð
            <MudSpacer />Ð
Ð
            <MudIconButton Icon="@(_isDarkMode ? Icons.Material.Filled.LightMode :
Icons.Material.Filled.DarkMode)"Ð
                           Color="Color.Inherit"Ð
                           OnClick="ToggleDarkMode" />Ð
        </MudAppBar>Ð
        Ð
        @* Aô0CÔ5C´L CÄ5CÔN *@Ð
        <MudDrawer @bind-Open="_drawerOpen" ClipMode="DrawerClipMode.Always"
Variant="DrawerVariant.Temporary" Overlay="true" OverlayAutoClose="true" Width="220px"
Elevation="1">Ð
            <NavMenu />Ð
        </MudDrawer>Ð
Ð
        <MudMainContent Class="pt-16 px-4 pb-4 d-flex flex-column"Ð
                        Style="height: calc(100vh - 8px); min-height: calc(100vh - 8px); margin-
top: 10px; display: flex; flex-direction: column; flex-grow: 1; overflow: hidden;">Ð
Ð
            @* A„!Aô A  A´ AÔ : A4;Cä1C ;DÄ=D´9 ErrorBoundary D41D 0CÒ A C$5D ECÔ5C4> D4@Cä2CÔO. B BC„;C, fÆW
            @if (TabService.OpenTabs.Count == 0)Ð
            {Ð
                <div class="flex-grow-1 overflow-auto">Ð
                    <ErrorBoundary>Ð
                        <ChildContent>Ð
```

```
                            @BodyĐ
                        </ChildContent>Đ
                        <ErrorContent Context="exception">Đ
                            <MudAlert Severity="Severity.Error" Class="my-4">Đ
                                Aô@Cä8Ct>D„;C  :D 8D$8Dt5D :C O CäHC„1C¤0 C„=D$5D DCT9D 0: @exception.Message
                            </MudAlert>Đ
                        </ErrorContent>Đ
                    </ErrorBoundary>Đ
                </div>Đ
            }Đ
            elseĐ
            {Đ
                <MudTabs @bind-ActivePanelIndex="TabService.ActiveTabIndex"Đ
                         Position="Position.Top"Đ
                         Color="Color.Default"Đ
                         SliderColor="Color.Primary"Đ
                         Class="mdi-tabs-container w-100 flex-grow-1 d-flex flex-column"Đ
                         Style="height: 100%; max-height: 100%; min-height: 0; display: flex; flex-
direction: column; flex-grow: 1;"Đ
                         TabPanelsClass="pa-0 flex-grow-1 d-flex flex-column overflow-hidden"Đ
                         ApplyEffectsToContainer="true"Đ
                         TabButtonsClass="rounded-t-lg shadow-sm"Đ
                         Outlined="true">Đ
Đ
                    @for (int i = 0; i < TabService.OpenTabs.Count; i++)Đ
                    {Đ
                        int localIndex = i;Đ
                        TabItem tab = TabService.OpenTabs[i];Đ
Đ
                        @* A„!Aô A  A´ AÔ : AD>C 0C$;CT=C  fÆW,ÔAD$@D4:D$CD 0 CD;Dò AC <Cä3Câ BC 1-Cô0CÔ5C´O,
                        <MudTabPanel @key="tab" ID="@localIndex" Class="h-100 d-flex flex-column">Đ
                            <TabContent>Đ
                                <div class="d-flex align-center gap-2" style="cursor: pointer;">Đ
                                    @if (!string.IsNullOrEmpty(tab.Icon))Đ
                                    {Đ
                                        <MudIcon Icon="@tab.Icon" Size="Size.Small" />Đ
                                    }Đ
                                    <MudText Typo="Typo.body2" Class="font-weight-
medium">@tab.Title</MudText>Đ
Đ
                                    <MudIconButton Icon="@Icons.Material.Filled.Close"Đ
                                                   Size="Size.Small"Đ
                                                   Color="Color.Default"Đ
                                                   Class="pa-0 ms-1"Đ
                                                   Style="width: 16px; height: 16px; min-width:
16px;"Đ
                                                   OnClick="@((e) =>
TabService.CloseTabByIndex(localIndex))" />Đ
                                </div>Đ
                            </TabContent>Đ
                            <ChildContent>Đ
                                @* BÔBCäB C¤>CÔBCT9CÔ5D  6CTAD$:Câ 7C =C„<C 5D"    R 2D´ACäBD² 2C¤;C 4C¤8 C,
                                <div class="pa-2 rounded-b-lg w-100 d-flex flex-column flex-grow-1"Đ
                                     Style="height: 100%; max-height: 100%; min-height: 0;
overflow: hidden;">Đ
Đ
                                    <ErrorBoundary>Đ
                                        <ChildContent>Đ
                                            @* A„!Aô A  A´ AÔ : AÄK Cä1Cä@C GC„2C 5CÂ G–æ Ö–46ö× öæVçB 2 DD8C
                                            @* C¤>D$>D KC′ AC„;Cä9 C$KD$OC48C$0CTB D BD 0CÔ8DdC CÔ0D„5C4> CD5
                                            <div class="d-flex flex-column flex-grow-1 w-100"
style="height: 100%; min-height: 0;">Đ
                                                <DynamicComponent Type="@tab.ComponentType" />Đ
                                            </div>Đ
                                        </ChildContent>Đ
                                        <ErrorContent Context="exception">Đ
                                            <MudAlert Severity="Severity.Error" Class="my-4">Đ
                                                Aô@Cä8Ct>D„;C  :D 8D$8Dt5D :C O CäHC„1C¤0 C„=D$5D DCT9D 0 C$:
                                            </MudAlert>Đ
                                        </ErrorContent>Đ
                                    </ErrorBoundary>Đ
Đ
                                </div>Đ
                            </ChildContent>Đ
                        </MudTabPanel>Đ
                    }Đ
```

```
                </MudTabs>Đ
            }Đ
        </MudMainContent>Đ
    </MudLayout>Đ
</CascadingValue>Đ
Đ
@code {Đ
    private bool _drawerOpen = true;Đ
    private bool _isDarkMode; // Aô> D4<Cä;Dt0CÔ8Dâ ?D 8C´>Cd5CÔ8CR 7C ?D4AC¤0CTBD O C" AC$5D$;Cä9 D$5CÄ5Đ
Đ
    protected override void OnInitialized()Đ
    {Đ
        base.OnInitialized();Đ
        TabService.OnTabsChanged += HandleTabsChanged;Đ
        _drawerOpen = false;Đ
    }Đ
Đ
    private void ToggleDrawer() => _drawerOpen = !_drawerOpen;Đ
Đ
    /// <summary>Đ
    /// Aô5D 5C¤;DäGC 5D" DC´0C2 BE <CÔ>C4> D 5Cd8CÄ0 C, 7C AD$0C$;Dô5D" ×VEF†VÖU &÷f–FW" <C4=Cä2CT=CÔ> Cô5D
    /// </summary>Đ
    private void ToggleDarkMode() => _isDarkMode = !_isDarkMode;Đ
Đ
    private void HandleTabsChanged() => InvokeAsync(StateHasChanged);Đ
Đ
    public void CloseDrawer()Đ
    {Đ
        if (_drawerOpen)Đ
        {Đ
            _drawerOpen = false;Đ
            StateHasChanged();Đ
        }Đ
    }Đ
Đ
    public void Dispose() => TabService.OnTabsChanged -= HandleTabsChanged;Đ
}
</file>

<file path="Configuration.Web/Components/Layout/ReconnectModal.razor">
<script type="module" src="@Assets["Components/Layout/ReconnectModal.razor.js"]"></script>Đ
Đ
<dialog id="components-reconnect-modal" data-nosnippet>Đ
    <div class="components-reconnect-container">Đ
        <div class="components-rejoining-animation" aria-hidden="true">Đ
            <div></div>Đ
            <div></div>Đ
        </div>Đ
        <p class="components-reconnect-first-attempt-visible">Đ
            Rejoining the server...Đ
        </p>Đ
        <p class="components-reconnect-repeated-attempt-visible">Đ
            Rejoin failed... trying again in <span id="components-seconds-to-next-attempt"></span>
seconds.Đ
        </p>Đ
        <p class="components-reconnect-failed-visible">Đ
            Failed to rejoin.<br />Please retry or reload the page.Đ
        </p>Đ
        <button id="components-reconnect-button" class="components-reconnect-failed-visible">Đ
            RetryĐ
        </button>Đ
        <p class="components-pause-visible">Đ
            The session has been paused by the server.Đ
        </p>Đ
        <p class="components-resume-failed-visible">Đ
            Failed to resume the session.<br />Please retry or reload the page.Đ
        </p>Đ
        <button id="components-resume-button" class="components-pause-visible components-resume-
failed-visible">Đ
            ResumeĐ
        </button>Đ
    </div>Đ
</dialog>
</file>

<file path="Configuration.Web/Components/NavMenu.razor">
```

```
@using System.ReflectionĐ
@inject UiModuleService UiServiceĐ
@inject TabNavigationService TabServiceĐ
@inject ComponentRegistry RegistryĐ
Đ
<MudNavMenu>Đ
    @{Đ
        List<(string Title, string Href, string Icon, string? Group)> allItems = UiService.ModulesĐ
            .SelectMany(m => m.GetMenuItems())Đ
            .ToList();Đ
Đ
        // AäBD 8D >C$KC$0CT< DÔ;CT<CT=D$K C$5D ECÔ5C4> D4@Cä2CÔOĐ
        IEnumerable<(string Title, string Href, string Icon, string? Group)> topLevelItems =
allItems.Where(i => string.IsNullOrWhiteSpace(i.Group));Đ
        foreach ((string Title, string Href, string Icon, string? Group) item in topLevelItems)Đ
        {Đ
            <!-- A„!Aô A  A´ AÔ : A$<CTAD$> Href C„ACô>C´LCtCCT< OnClick CD;Dò >D$:D KD$8Dò 2C¤;C 4Cä: C 5Cr
            <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title, item.Href,
item.Icon))">Đ
                @item.TitleĐ
            </MudNavLink>Đ
        }Đ
Đ
        // A4@D4?Cô8D CCT< DÔ;CT<CT=D$K Cô> C¤0D$5C4>D 8Dô<Đ
        IOrderedEnumerable<IGrouping<string, (string Title, string Href, string Icon, string?
Group)>> pooledGroups = allItemsĐ
            .Where(i => !string.IsNullOrWhiteSpace(i.Group))Đ
            .GroupBy(i => i.Group!)Đ
            .OrderBy(g => g.Key);Đ
Đ
        foreach (IGrouping<string, (string Title, string Href, string Icon, string? Group)> group
in pooledGroups)Đ
        {Đ
            <MudNavGroup Title="@group.Key" Icon="@Icons.Material.Filled.Folder" Expanded="false">Đ
                @foreach ((string Title, string Href, string Icon, string? Group) item in group)Đ
                {Đ
                    <!-- A„!Aô A  A´ AÔ : B41D 0CÒ ‡&VbÂ 4Cä1C 2C´5CÒ öä6Æ–6² ÒÓí
                    <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title,
item.Href, item.Icon))">Đ
                        @item.TitleĐ
                    </MudNavLink>Đ
                }Đ
            </MudNavGroup>Đ
        }Đ
    }Đ
</MudNavMenu>Đ
Đ
@code {Đ
    /// <summary>Đ
    /// Aô5D 5DT2C BD´2C 5CÂ AC¤2Cä7CÔ>C’ MC¤7CT<Cô;Dô@ D >CD8D$5C´LD :Cä3Câ <C :CTBC
    /// </summary>Đ
    [CascadingParameter]Đ
    protected MainLayout? MainLayoutInstance { get; set; }Đ
Đ
    protected override void OnInitialized()Đ
    {Đ
        base.OnInitialized();Đ
        IEnumerable<Assembly> assemblies = UiService.Modules.Select(m =>
m.GetType().Assembly).Distinct();Đ
        Registry.RegisterModules(assemblies);Đ
    }Đ
Đ
    private void OpenMenuAsTab(string title, string href, string icon)Đ
    {Đ
        Type? componentType = Registry.GetComponentTypeByRoute(href);Đ
Đ
        if (componentType != null)Đ
        {Đ
            // AäBC¤@D´2C 5CÂ 2C¤;C 4C¤CĐ
            TabService.OpenTab(title, href, icon, componentType);Đ
Đ
            // A 2D$>CÄ0D$8Dt5D :C, 7C :D KC$0CT< C >C¤>C$>CR <CT=Dâ ?Cä2CT@DR >C¤=C  ?CäAC´5 C¤;C„:C
            MainLayoutInstance?.CloseDrawer();Đ
        }Đ
        elseĐ
        {Đ
```

```
            System.Diagnostics.Debug.WriteLine($"AôCD$L {href} CÔ5 Ct0D 5C48D BD 8D >C$0CÒ 2 C¤>CÄ?Cä=CT=D$0D
        }Ð
    }Ð
}
</file>

<file path="Configuration.Web/Components/NotFound.razor">
@page "/not-found"Ð
@layout MainLayoutÐ
Ð
<PageTitle>404 - B BD 0CÔ8Dd0 CÔ5 CÔ0C”4CT=C Âõ  vUF—FÆSí
Ð
<MudContainer MaxWidth="MaxWidth.Small" Class="d-flex align-center justify-center" Style="height:
70vh;">Ð
    <MudPaper Elevation="3" Class="pa-8 text-center" Style="width: 100%;">Ð
        <MudIcon Icon="@Icons.Material.Filled.SearchOff" Color="Color.Error" Size="Size.Large"
Class="mb-4" />Ð
        <MudText Typo="Typo.h5" Class="mb-2">AäHC„1C¤0 404</MudText>Ð
        <MudText Typo="Typo.body2" Class="mud-text-secondary mb-6">Ð
            A„7C$8CÔ8D$5, Ct0Cô@C HC„2C 5CÄKC’ 0CD@CTA CÔ5 D CD”5D BC$CCTB, C´8C > D >CäBC$5D$AD$2D4ND”8C’ <C
        </MudText>Ð
        <MudButton Variant="Variant.Filled" Color="Color.Primary" href="/">Ð
            A$5D =D4BDÄADò =C  3C´0C$=D4NÐ
        </MudButton>Ð
    </MudPaper>Ð
</MudContainer>
</file>

<file path="Configuration.Web/Components/Routes.razor">
@using System.ReflectionÐ
@inject UiModuleService UiServiceÐ
Ð
<Router AppAssembly="@typeof(Routes).Assembly"Ð
        AdditionalAssemblies="@_moduleAssemblies"Ð
        NotFoundPage="@typeof(NotFound)">Ð
    <Found Context="routeData">Ð
        <RouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)" />Ð
        <FocusOnNavigate RouteData="@routeData" Selector="h1" />Ð
    </Found>Ð
</Router>Ð
Ð
@code {Ð
    private Assembly[] _moduleAssemblies = [];Ð
Ð
    protected override void OnInitialized()Ð
    {Ð
        _moduleAssemblies = UiService.ModulesÐ
            .Select(m => m.GetType().Assembly)Ð
            .Distinct()Ð
            .ToArray();Ð
            Ð
        base.OnInitialized();Ð
    }Ð
}
</file>

<file path="Configuration.Web/Configurator/IWebAppConfigurator.cs">
namespace Promatis.Net.Configuration.Web;Ð
Ð
public interface IWebAppConfigurator : IAppConfiguratorÐ
{Ð
    void ConfigureMiddleware(WebApplication app);Ð
}
</file>

<file path="Configuration.Web/Configurator/WebAppConfigurator.cs">
using Microsoft.AspNetCore.Components;Ð
using MudBlazor.Services;Ð
using Promatis.Net.UI;Ð
using System.Reflection;Ð
Ð
namespace Promatis.Net.Configuration.Web;Ð
Ð
/// <summary>Ð
/// A 0Ct>C$KC’ :Cä=DD8C4CD 0D$>D  4C´O Web-Cô@C„;Cä6CT=C„9 CÔ0 C 0Ct5 Blazor C, ×VD&Æ ¦÷"í
/// </summary>Ð
```

```
public class WebAppConfigurator<TRootComponent> : AppConfigurator, IWebAppConfiguratorÐ
    where TRootComponent : IComponentÐ
{Ð
    public override void ConfigureServices(IServiceCollection services, IConfiguration
configuration)Ð
    {Ð
        // 1. A 0Ct>C$0Dò 8CÔDD 0D BD CC¤BD4@C  „ ABÂ !CT@C$8D K, A$0C´8CD0D$>D K, B$@C„3C45D K, B :C =CT@ DL
        base.ConfigureServices(services, configuration);Ð
Ð
        // 2. UI C„=DD@C AD$@D4:D$CD 0 (Aô>C„AC¢ •V”ÖöGVÆR 2 D 1Cä@C¤0DRÂ =C 2C„3C FC„O)Ð
        services.AddWebInfrastructure(projectPrefix: "Promatis.");Ð
Ð
        // B  B "AT A   A¤ A  Aä  (MDI): B 5C48D BD 8D CCT< C„=DD@C AD$@D4:D$CD C CÄ=Cä3Cä2C¤;C 4CäGCÔ>C4> C„
        services.AddSingleton<ComponentRegistry>();Ð
        services.AddScoped<TabNavigationService>();Ð
Ð
        // 3. B ?CTFC„DC„:C  &Æ ¦÷" „–çFW& 7F—fR 6W'fW"•
        services.AddRazorComponents()Ð
            .AddInteractiveServerComponents();Ð
Ð
        // 4. B 5C48D BD 0Dd8Dò 8 CÔ0D BD >C”:C  ×VD&Æ ¦÷-
        services.AddMudServices(config =>Ð
        {Ð
            config.SnackbarConfiguration.PositionClass =
MudBlazor.Defaults.Classes.Position.BottomRight;Ð
            config.SnackbarConfiguration.PreventDuplicates = false;Ð
            config.SnackbarConfiguration.NewestOnTop = true;Ð
            config.SnackbarConfiguration.ShowCloseIcon = true;Ð
            config.SnackbarConfiguration.VisibleStateDuration = 5000;Ð
        });Ð
Ð
        // --- B  A4 B "B  Bd Bò  AÔ$B  B "B #A¤"B4 AÔ A4  Bô B   Aô A "BD B  B²  $ôÔ D•2 T’ ÒÒÝ
Ð
        // 1. B 5C48D BD 0Dd8Dò  At#-C¤MD„0 C, 4CTDCä;D$=Cä9 Cô;CäAC¤>C’ AD$@C BCT3C„8 CÄCD$0Dd8C•
        services.AddTransient(typeof(IUiOzuCache<>), typeof(UiOzuCache<>));Ð
        services.AddTransient(typeof(IOzuMutationStrategy<>), typeof(FlatOzuMutationStrategy<>));Ð
Ð
        // 2. B 5C48D BD 0Dd8Dò CCÔ8C$5D AC ;DÄ=Cä3Câ  D >C¤5D 0 CD0CÔ=D´E (3 C45CÔ5D 8C¢Ô?C @C <CTBD 0: TEnt
        services.AddTransient(typeof(IUiDataBroker<,,>), typeof(UiDataBroker<,,>));Ð
Ð
        // 3. B„8CÔ0 UI-D >C KD$8C•
        services.AddScoped<IUiCommandBus, UiCommandBus>();Ð
Ð
        // B 5C48D BD 0Dd8Dò AC„AD$5CÄ=Cä3Câ 0C¤ACTAD >D 0 CD;Dò ?Cä4CD5D 6C¤8 C´5CÔ8C$>C4> D 0Ct@CTHCT=C„O S
        services.AddHttpContextAccessor();Ð
    }Ð
Ð
    /// <summary>Ð
    /// AÔ0D BD >C”:C  :Cä=C$5C”5D 0 Cä1D 0C >D$:C, …EE Ô7C ?D >D >C" „Ö–FFÆWv &R’í
    /// </summary>Ð
    public virtual void ConfigureMiddleware(WebApplication app)Ð
    {Ð
        // A 0Ct>C$KCR Ö–FFÆWv &W2 4C´O D 0C >D$K Web-Cô@C„;Cä6CT=C„OÐ
        app.UseStaticFiles();Ð
        app.UseAntiforgery();Ð
Ð
        // 1. Aô>C´CDt0CT< Ct0D 5C48D BD 8D >C$0CÔ=D´5 UI-D 1Cä@C¤8 C„7 DI-C¤>CÔBCT9CÔ5D 0 DT>D BC
        using IServiceScope scope = app.Services.CreateScope();Ð
        var uiService = scope.ServiceProvider.GetRequiredService<UiModuleService>();Ð
Ð
        Assembly[] moduleAssemblies = uiService.ModulesÐ
            .Select(m => m.GetType().Assembly)Ð
            .Distinct()Ð
            .ToArray();Ð
Ð
        // A„ A„&A„ A´ At Bd Bò  A  B$+ A$ A´ AD A£¢  C ?Cä;CÔOCT< D 5CTAD$@ D >D4BCä2 C¤>CÄ?Cä=CT=D$0CÄ8 C„7
        var componentRegistry = scope.ServiceProvider.GetRequiredService<ComponentRegistry>();Ð
        componentRegistry.RegisterModules(moduleAssemblies);Ð
Ð
        // 2. B 5C48D BD 8D CCT< C¤>D =CT2Cä9 C¤>CÄ?Cä=CT=D" 8 D :C =C„@D45CÂ AD$@C =C„FD² <Cä4D4;CT9 CÔ0 D 5
        app.MapRazorComponents<TRootComponent>()Ð
            .AddInteractiveServerRenderMode()Ð
            .AddAdditionalAssemblies(moduleAssemblies);Ð
    }Ð
Ð
    public override void ConfigureApp(IHost app)Ð
```

```
    {Ð
        // A$KCt>C" 1C 7Cä2Cä9 C„=C„FC„0C´8Ct0Dd8C, „0C$BCä@CT3C„AD$@C FC„O D$@C„3C45D >C"  AB GCT@CT7 D :C =
        base.ConfigureApp(app);Ð
Ð
        // At4CTADÂ <Cä6CÔ> CD>C 0C$8D$L Cô@Cä2CT@C¤C Ct4Cä@Cä2DÄO A   C„;C, =C GC ;DÄ=D´9 Seed CD0CÔ=D´E CD;
    }Ð
}
</file>

<file path="Configuration.Web/Configurator/WebScanningExtensions.cs">
using Promatis.Net.UI;Ð
using Promatis.Net.UI.Components;Ð
using System.Reflection;Ð
Ð
namespace Promatis.Net.Configuration.Web;Ð
Ð
public static class WebInfrastructureExtensionsÐ
{Ð
    public static void AddWebInfrastructure(this IServiceCollection services, string projectPrefix
= "Promatis.")Ð
    {Ð
        Console.WriteLine("[SCANNER] At0CôCD : Cô@Cä3D 5C$0 D 1Cä@Cä: CÄ>CÔ>C´8D$0 (B$>C´LC¤> UI-CÄ>CDCC´8)..
Ð
        string binPath = AppContext.BaseDirectory;Ð
Ð
        // A„!Aô A  A´ AÔ : AÔ0 D4@Cä2CÔ5 DD0C”;Cä2Cä9 D 8D BCT<D² 8D”5CÂ BCä;DÄ:Câ BCR FÆÂÂ :CäBCä@D´5 CÔ0Dt
        string[] assemblyFiles = Directory.GetFiles(binPath, $"{projectPrefix}*.UI.dll",
SearchOption.TopDirectoryOnly);Ð
Ð
        foreach (string file in assemblyFiles)Ð
        {Ð
            tryÐ
            {Ð
                // Aô@C„=D44C„BCT;DÄ=Câ 7C 3D CCd0CT< C"  76VÖ&Ç”Æö D6öçFW‡BäFVf VÇM
                Assembly.LoadFrom(file);Ð
            }Ð
            catch (Exception ex)Ð
            {Ð
                Console.WriteLine($"[SCANNER] AÔ5 D44C ;CäADÂ 7C 3D CCt8D$L DD0C”; D 1Cä@C¤8 {Path.GetFileNam
            }Ð
        }Ð
Ð
        // A„!Aô A  A´ AÔ : BD8C´LD$@D45CÂ    FöÖ –âÂ 1CT@Dò AC >D :C, AD$@Cä3Câ A Cô@CTDC„:D >CÂ 8 Cä:Cä=Dt0
        Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()Ð
            .Where(a => a.FullName != nullÐ
                        && a.FullName.StartsWith(projectPrefix)Ð
                        && a.GetName().Name!.EndsWith(".UI"))Ð
            .Distinct()Ð
            .ToArray();Ð
Ð
        Console.WriteLine($"[SCANNER] B :C =C„@Cä2C =C„5 Ct0C$5D HCT=Câ  C 9CD5CÔ> {assemblies.Length} Dd5C´
Ð
        Console.WriteLine();Ð
        /*Ð
        // A  B$ AÄ B$ Bt B  A / B  A4 B "B  Bd Bò T’Ô Aä B$ A¤!B$ A"  A´ B$$Aä AÄ+Ð
        Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò 22Ô:Cä=D$5C¤AD$>C" AD$@C =C„F Cô> CÄ0D :CT@D2 •v÷&·7
Ð
        // A„!Aô A  A´ AÔ : AÄ0D :CT@ C„7CÄ5CÔ5CÒ =C  0C¤BD40C´LCÔKC’ •v÷&·7  6T6öçFW‡B „1CT7 D ;Cä2C   7F–öâ
        var contextTypes = assembliesÐ
            .SelectMany(s => s.GetTypes())Ð
            .Where(t => typeof(IWorkspaceContext).IsAssignableFrom(t)Ð
                        && !t.IsAbstractÐ
                        && !t.IsInterfaceÐ
                        && !t.IsGenericTypeDefinition);Ð
Ð
        int registeredContextsCount = 0;Ð
        foreach (Type type in contextTypes)Ð
        {Ð
            // 1. B 5C48D BD 8D CCT< D 0CÂ :Cä=C¤@CTBCÔKC’ ?D 8C¤;C 4CÔ>C’ :Cä=D$5C¤AD" „=C ?D 8CÄ5D Â Væ—Död
            services.AddTransient(type);Ð
            registeredContextsCount++;Ð
Ð
            Console.WriteLine($"          % %  [A¤ AÔ"AT B "] {type.Name}");Ð
Ð
            // 2. A 5D 5CÂ AD$@Cä3Câ 1C´8Cd0C”HC„9 C 0Ct>C$KC’ BC„? (CÔ0Cô@C„<CT@, ReferenceContext<UnitOfMea
            Type? baseType = type.BaseType;Ð
```

```
            if (baseType != null && baseType.IsGenericType)Ð
            {Ð
                services.AddTransient(baseType, type);Ð
Ð
                string genericArgs = string.Join(", ", baseType.GetGenericArguments().Select(a =>
a.Name));Ð
                string baseTypeName = baseType.Name.Split('`')[0];Ð
                Console.WriteLine($"          %   % %  AÄ0D :C ¢ ¶& 6UG— Tæ ÖWÓÇ¶vVæW&–4 &w7Óâ"“½
            }Ð
        }Ð
        Console.WriteLine($"[SCANNER] B4ACô5D„=Câ @C 7C$5D =D4BCâ ·&Vv—7FW&VD6öçFW‡G46÷VçGÒ :Cä=D$5C¤AD$>C" C
Ð
        Console.WriteLine();*/Ð
Ð
        Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò T’Ô:Cä<Cô>CÔ5CÔBCä2 C, <Cä4D4;CT9 CÔ0C$8C40Dd8C, GCT@C
Ð
        // A 2D$>CÄ0D$8Dt5D :Cä5 D :C =C„@Cä2C =C„5 C, @CT3C„AD$@C FC„O C„=D$5D DCT9D >C" •V”ÖöGVÆ]
        services.Scan(scan => scanÐ
            .FromAssemblies(assemblies)Ð
            .AddClasses(c => c.AssignableTo<IUiModule>().Where(t => !t.IsAbstract))Ð
            .AsImplementedInterfaces()Ð
            .WithScopedLifetime()Ð
        );Ð
Ð
        // B 5C48D BD 0Dd8Dò 2C HCT3Câ 0C4@CT3C BCä@C  <Cä4D4;CT9Ð
        services.AddScoped<UiModuleService>();Ð
Ð
        // A´>C48D >C$0CÔ8CR >C =C @D46CT=CÔKDR ?D4=C¤BCä2 CÄ5CÔNÐ
        IEnumerable<Type> uiModuleTypes = assembliesÐ
            .SelectMany(a => a.GetTypes())Ð
            .Where(t => typeof(IUiModule).IsAssignableFrom(t) && !t.IsInterface && !t.IsAbstract);Ð
Ð
        foreach (Type type in uiModuleTypes)Ð
        {Ð
            tryÐ
            {Ð
                if (Activator.CreateInstance(type) is IUiModule module)Ð
                {Ð
                    Console.WriteLine($"[SCANNER] UI AÄ>CDCC´L: {module.Name}");Ð
                    List<(string Title, string Href, string Icon, string? Group)> menuItems =
module.GetMenuItems()?.ToList() ?? new();Ð
Ð
                    if (!menuItems.Any())Ð
                    {Ð
                        Console.WriteLine("          % %  [AôCD BCä5 CÄ5CÔN]");Ð
                        continue;Ð
                    }Ð
Ð
                    IEnumerable<(string Title, string Href, string Icon, string? Group)> rootItems
= menuItems.Where(i => string.IsNullOrEmpty(i.Group));Ð
                    IEnumerable<IGrouping<string?, (string Title, string Href, string Icon, string?
Group)>> groupedItems = menuItems.Where(i => !string.IsNullOrEmpty(i.Group)).GroupBy(i => i.Group);Ð
Ð
                    foreach ((string Title, string Href, string Icon, string? Group) item in
rootItems)Ð
                    {Ð
                        Console.WriteLine($"          % %  {item.Title} -> {item.Href}");Ð
                    }Ð
                    foreach (IGrouping<string?, (string Title, string Href, string Icon, string?
Group)> group in groupedItems)Ð
                    {Ð
                        Console.WriteLine($"          % %  [{group.Key}]");Ð
                        List<(string Title, string Href, string Icon, string? Group)> itemsInGroup
= group.ToList();Ð
                        for (int i = 0; i < itemsInGroup.Count; i++)Ð
                        {Ð
                            string prefix = (i == itemsInGroup.Count - 1) ? "          %   % % " :
"          %   % % ";Ð
                            Console.WriteLine($"{prefix} {itemsInGroup[i].Title} ->
{itemsInGroup[i].Href}");Ð
                        }Ð
                    }Ð
                }Ð
            }Ð
            catch (Exception ex)Ð
            {Ð
```

```
                Console.WriteLine($"[SCANNER] AäHC„1C¤0 DtBCT=C„O CÄ5CÔN CD;Dò BC„?C  ·G— Räæ ÖWÓ¢ ¶W,äÖW76 v
            }Ð
        }Ð
Ð
        Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò T’Ô:Cä<Cô>CÔ5CÔBCä2 D4ACô5D„=Câ 7C 2CT@D„5CÔ0");Ð
        Console.WriteLine();Ð
    }Ð
}
</file>

<file path="Configuration.Web/Promatis.Net.Configuration.Web.csproj">
<Project Sdk="Microsoft.NET.Sdk.Web">Ð
Ð
  <PropertyGroup>Ð
    <TargetFramework>net10.0</TargetFramework>Ð
    <ImplicitUsings>enable</ImplicitUsings>Ð
    <Nullable>enable</Nullable>Ð
“Ä÷WG WEG— SäÆ–'& '“Âô÷WG WEG— Sí
  </PropertyGroup>Ð
Ð
  <ItemGroup>Ð
    <PackageReference Include="MudBlazor" Version="9.4.0" />Ð
  </ItemGroup>Ð
Ð
  <ItemGroup>Ð
    <ProjectReference Include="..\Configuration\Promatis.Net.Configuration.csproj" />Ð
    <ProjectReference Include="..\Promatis.Net.UI\Promatis.Net.UI.csproj" />Ð
  </ItemGroup>Ð
Ð
</Project>
</file>

<file path="Configuration.Web/Properties/launchSettings.json">
{Ð
  "profiles": {Ð
    "Promatis.Net.Configuration.Web": {Ð
      "commandName": "Project",Ð
      "launchBrowser": true,Ð
      "environmentVariables": {Ð
        "ASPNETCORE_ENVIRONMENT": "Development"Ð
      },Ð
      "applicationUrl": "https://localhost:56819;http://localhost:56820"Ð
    }Ð
  }Ð
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs">
using System.Reflection;Ð
using Microsoft.AspNetCore.Components;Ð
Ð
namespace Promatis.Net.Configuration.Web;Ð
Ð
public class ComponentRegistryÐ
{Ð
    private readonly Dictionary<string, Type> _routeMap = new(StringComparer.OrdinalIgnoreCase);Ð
Ð
    /// <summary>Ð
    /// B :C =C„@D45D" AC >D :C, <Cä4D4;CT9 C, AD$@Cä8D" :C @D$C: URL -> B Æ 72 "C„? C¤>CÄ?Cä=CT=D$0Ð
    /// </summary>Ð
    public void RegisterModules(IEnumerable<Assembly> assemblies)Ð
    {Ð
        foreach (Assembly assembly in assemblies)Ð
        {Ð
            IEnumerable<Type> componentTypes = assembly.GetTypes()Ð
                .Where(t => typeof(IComponent).IsAssignableFrom(t) && !t.IsAbstract);Ð
Ð
            foreach (Type type in componentTypes)Ð
            {Ð
                // A„ICT< C BD 8C CD" µ&÷WFT GG&–'WFUÒÂ 2 C¤>D$>D KC’ :Cä<Cô8C´8D CCTBD O CD8D 5C¤BC„2C     v
                IEnumerable<RouteAttribute> routeAttributes =
type.GetCustomAttributes<RouteAttribute>();Ð
                foreach (RouteAttribute attr in routeAttributes)Ð
                {Ð
                    _routeMap[attr.Template] = type;Ð
                }Ð
```

```
            }Đ
        }Đ
    }Đ
Đ
    public Type? GetComponentTypeByRoute(string route)Đ
    {Đ
        return _routeMap.TryGetValue(route, out Type? type) ? type : null;Đ
    }Đ
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/TabItem.cs">
namespace Promatis.Net.Configuration.Web;Đ
Đ
public class TabItemĐ
{Đ
    public string Id { get; init; } = string.Empty; // AÔ0D, ‡&Vm
    public string Title { get; init; } = string.Empty;Đ
    public string Icon { get; init; } = string.Empty;Đ
    public Type ComponentType { get; init; } = null!;Đ
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/TabNavigationService.cs">
namespace Promatis.Net.Configuration.Web;Đ
Đ
public class TabNavigationServiceĐ
{Đ
    public List<TabItem> OpenTabs { get; } = new();Đ
    public int ActiveTabIndex { get; set; }Đ
    public event Action? OnTabsChanged;Đ
Đ
    public void OpenTab(string title, string href, string icon, Type componentType)Đ
    {Đ
        TabItem? existingTab = OpenTabs.FirstOrDefault(t => t.Id.Equals(href,
StringComparison.OrdinalIgnoreCase));Đ
Đ
        if (existingTab != null)Đ
        {Đ
            ActiveTabIndex = OpenTabs.IndexOf(existingTab);Đ
        }Đ
        elseĐ
        {Đ
            var newTab = new TabItemĐ
            {Đ
                Id = href,Đ
                Title = title,Đ
                Icon = icon,Đ
                ComponentType = componentTypeĐ
            };Đ
Đ
            OpenTabs.Add(newTab);Đ
            ActiveTabIndex = OpenTabs.Count - 1;Đ
        }Đ
Đ
        NotifyStateChanged();Đ
    }Đ
Đ
    /// <summary>Đ
    /// A 5Ct>Cô0D =Cä5 Ct0C¤@D´BC„5 C$:C´0CD:C, ?Câ 5E  GC„AC´>C$>CÄC C„=CD5C¤AD=
    /// </summary>Đ
    public void CloseTabByIndex(int index)Đ
    {Đ
        if (index < 0 || index >= OpenTabs.Count) return;Đ
Đ
        OpenTabs.RemoveAt(index);Đ
Đ
        // A„=D$5C´;CT:D$CC ;DÄ=D´9 D 0D GCTB DD>C¤CD 0 CÔ0 CäAD$0C$HC„5D O C$:C´0CD:C•
        if (ActiveTabIndex >= OpenTabs.Count)Đ
        {Đ
            ActiveTabIndex = OpenTabs.Count - 1;Đ
        }Đ
Đ
        if (ActiveTabIndex < 0 && OpenTabs.Count > 0)Đ
        {Đ
            ActiveTabIndex = 0;Đ
```

```
        }Ð
Ð
        NotifyStateChanged();Ð
    }Ð
Ð
    private void NotifyStateChanged() => OnTabsChanged?.Invoke();Ð
}
</file>

<file path="Configuration.Web/Services/UiModuleService.cs">
using Promatis.Net.UI;Ð
Ð
namespace Promatis.Net.Configuration.Web;Ð
Ð
// B 5D 2C„A, C¤>D$>D KC’ ACä1C„@C 5D" 2D Q C$>CT4C„=Cí
public class UiModuleService(IEnumerable<IUiModule> modules)Ð
{Ð
    public IEnumerable<IUiModule> Modules => modules;Ð
}
</file>

<file path="Configuration.Web/UiCommandBus/IUiCommandBus.cs">
using MudBlazor;Ð
Ð
namespace Promatis.Net.Configuration.Web;Ð
Ð
/// <summary>Ð
/// A„=D$5D DCT9D  HC„=D² T’ÔACä1D´BC„9Ð
/// </summary>Ð
public interface IUiCommandBusÐ
{Ð
    /// <summary>Ð
    /// B 5C48D BD 0Dd8Dò >C @C 1CäBDt8C¤0 C$KCt>C$0 DD>D <D² „2D´?Cä;CÔOCTBD O Cô@C„=C„<C ND”8CÂ <Cä4D4;CT<,
    /// </summary>Ð
    void RegisterHandler(string commandName, Func<Dictionary<string, object>, Task<DialogResult?>>
handler);Ð
Ð
    /// <summary>Ð
    /// A$7C 8CÄ>CD5C”AD$2C„5 CÄ5Cd4D2 <Cä4D4;Dô<C, 2D ;CT?D4N (C$KCtKC$0CTBD O C„7 MDM)Ð
    /// </summary>Ð
    Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object> parameters);Ð
}
</file>

<file path="Configuration.Web/UiCommandBus/UiCommandBus.cs">
using MudBlazor;Ð
Ð
namespace Promatis.Net.Configuration.Web;Ð
Ð
/// <summary>Ð
/// B„8CÔ0 UI-D >C KD$8C•
/// </summary>Ð
public class UiCommandBus : IUiCommandBusÐ
{Ð
    // Aô>D$>C¤>C 5Ct>CÔ0D =D´9 D ;Cä2C @DÂ 4C´O D 5C48D BD 0Dd8C, :Cä<C =CB 2 D 0CÔBC 9CÄ5 SignalRÐ
    private readonly System.Collections.Concurrent.ConcurrentDictionary<string,
Func<Dictionary<string, object>, Task<DialogResult?>>> _handlers = new();Ð
Ð
    public void RegisterHandler(string commandName, Func<Dictionary<string, object>,
Task<DialogResult?>> handler)Ð
    {Ð
        _handlers[commandName] = handler ?? throw new ArgumentNullException(nameof(handler));Ð
    }Ð
Ð
    public async Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object>
parameters)Ð
    {Ð
        if (_handlers.TryGetValue(commandName, out Func<Dictionary<string, object>,
Task<DialogResult?>>? handler))Ð
        {Ð
            return await handler(parameters);Ð
        }Ð
Ð
        // ATAC´8 CÄ>CDCC´L (CÔ0Cô@C„<CT@, DCA) CÔ5 Cô>CD:C´NDt5CÒ 2 C´0D4=Dt5D 5, D 8D BCT<C  =CR CCô0CD5D"Â
        return null;Ð
    }Ð
```

```
}
</file>

<file path="Configuration.Web/WebAppBootstrappe.cs">
using Microsoft.AspNetCore.Components;Ð
Ð
namespace Promatis.Net.Configuration.Web;Ð
Ð
// AÔ0D ;CT4CÔ8C¢ &ö÷G7G&   W"Â :CäBCä@D´9 D4<CT5D" 7C ?D4AC¤0D$L Web-D 5D 2CT@Ð
public abstract class WebAppBootstrapper<TRootComponent> : AppBootstrapperÐ
    where TRootComponent : IComponentÐ
{Ð
    public override void Run(string[] args)Ð
    {Ð
        // B =Cä2C  7C 3D CCd0CT< DLL (CÔ0 D ;D4GC 9 Cô@Dô<Cä3Câ 7C ?D4AC¤0)Ð
        // LoadProjectAssemblies ("Promatis.") D46CR 2D´7C$0CÔ0 C" & 6Rå'VâÂ
        // CÔ> Ct4CTADÂ <D² 8D ?Cä;DÄ7D45CÂ vV$   Æ–6 F–öä'V–ÆFW"í
Ð
        // Aä Bô A "AT BÄ Aã¢  CT7 DÔBCä3Câ    FöÖ –â =CR CC$8CD8D" <Cä4D4;C, ?D 8 Type.GetTypeÐ
        LoadProjectAssemblies("Promatis.");Ð
Ð
        WebApplicationBuilder builder = WebApplication.CreateBuilder(args);Ð
        List<IAppConfigurator> configs = GetConfigurators(builder.Configuration).ToList();Ð
Ð
        foreach (IAppConfigurator cfg in configs)Ð
            cfg.ConfigureServices(builder.Services, builder.Configuration);Ð
Ð
        WebApplication app = builder.Build();Ð
Ð
        // AÔ0D BD >C”:C  Ö–FFÆWv &R <Cä4D4;CT9Ð
        foreach (IWebAppConfigurator cfg in configs.OfType<IWebAppConfigurator>())Ð
        {Ð
            cfg.ConfigureMiddleware(app);Ð
        }Ð
Ð
        foreach (IAppConfigurator cfg in configs)Ð
            cfg.ConfigureApp(app);Ð
Ð
        app.Run();Ð
    }Ð
}
</file>

<file path="Configuration/AppBootstrapper.cs">
using Microsoft.Extensions.Configuration;Ð
using Microsoft.Extensions.Hosting;Ð
using System.Reflection;Ð
using System.Runtime.Loader;Ð
Ð
namespace Promatis.Net.Configuration;Ð
Ð
public abstract class AppBootstrapperÐ
{Ð
    public virtual void Run(string[] args)Ð
    {Ð
        // 1. Aô A„ B4 A„"AT BÄ A / At A4 B4 A¤  DLL (DtBCä1D² G— RävWEG— R 8DR 2C„4CT;)Ð
        LoadProjectAssemblies("Promatis.");Ð
Ð
        HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);Ð
Ð
        // 2. B4 AÔ+A′  Aä B   A¤ AÔ$A„ B4 A "Aä Aä  A„  JSONÐ
        List<IAppConfigurator> configurators = GetConfigurators(builder.Configuration).ToList();Ð
Ð
        foreach (IAppConfigurator config in configurators)Ð
        {Ð
            config.ConfigureServices(builder.Services, builder.Configuration);Ð
        }Ð
Ð
        IHost host = builder.Build();Ð
Ð
        foreach (IAppConfigurator config in configurators)Ð
        {Ð
            config.ConfigureApp(host);Ð
        }Ð
Ð
        host.Run();Ð
```

```
    }Ð
Ð
    protected virtual IEnumerable<IAppConfigurator> GetConfigurators(IConfiguration configuration)Ð
    {Ð
        string[] typeNames = configuration.GetSection("LauncherSettings:Modules").Get<string[]>()Ð
                             ?? [];Ð
Ð
        foreach (string name in typeNames)Ð
        {Ð
            // A„ICT< D$8Cò 2Câ 2D 5DR 7C 3D CCd5CÔ=D´E D 1Cä@C¤0D]
            Type? type = AppDomain.CurrentDomain.GetAssemblies()Ð
                .Select(a => a.GetType(name))Ð
                .FirstOrDefault(t => t != null);Ð
Ð
            if (type != null && typeof(IAppConfigurator).IsAssignableFrom(type))Ð
            {Ð
                if (Activator.CreateInstance(type) is IAppConfigurator instance)Ð
                {Ð
                    yield return instance;Ð
                }Ð
            }Ð
            elseÐ
            {Ð
                Console.WriteLine($"[BOOTSTRAPPER][WARN] B$8Cò =CR =C 9CD5CÒ 8C´8 CÔ5 C$0C´8CD5CÓ¢ ¶æ ÖWÒ"“½
            }Ð
        }Ð
    }Ð
Ð
    protected void LoadProjectAssemblies(string prefix)Ð
    {Ð
        string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);Ð
        if (path == null) return;Ð
Ð
        foreach (string file in Directory.GetFiles(path, $"{prefix}*.dll"))Ð
        {Ð
            tryÐ
            {Ð
                AssemblyLoadContext.Default.LoadFromAssemblyPath(file);Ð
            }Ð
            catch { /* A„3CÔ>D 8D CCT< CäHC„1C¤8 Ct0C4@D47C¤8 */ }Ð
        }Ð
    }Ð
}
</file>

<file path="Configuration/AppConfigurator.cs">
using Microsoft.Extensions.Configuration;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Microsoft.Extensions.Hosting;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Text.Json;Ð
using System.Text.Json.Serialization;Ð
Ð
namespace Promatis.Net.Configuration;Ð
Ð
public class AppConfigurator : IAppConfiguratorÐ
{Ð
    public virtual void ConfigureServices(IServiceCollection services, IConfiguration configuration)Ð
    {Ð
        // B :C =C„@D45CÂ 2D Q (C$:C´NDt0Dò  VF—EG&–vvW" 2 Cô@Cä5C¤BCR F F •
        services.AddDomainInfrastructure(projectPrefix: "Promatis.");Ð
Ð
        // B 5C48D BD 8D CCT< C 0Ct>C$KCR ACT@C$8D KÐ
        services.AddScoped<DatabaseTriggerService>();Ð
        services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());Ð
        services.AddScoped<DatabaseTriggerInterceptor>();Ð
Ð
        // Aä Bd Aƒ¢ ¥4ôâ =C AD$@Cä9C¤8Ð
        services.AddSingleton(new JsonSerializerOptionsÐ
        {Ð
            PropertyNamingPolicy = JsonNamingPolicy.CamelCase,Ð
            DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,Ð
            WriteIndented = falseÐ
```

```
        });Ð
Ð
        // B  A4 B "B  Bd Bò  A´ B$$Aä AÄ AÔ Aä Aâ  A´ AÔ B   B #B” Aä!B$ A•
        services.AddSingleton<IEntityCloner, JsonEntityCloner>();Ð
    }Ð
Ð
    public virtual void ConfigureApp(IHost app)Ð
    {Ð
        if (app == null) throw new ArgumentNullException(nameof(app));Ð
Ð
        // A„=C„FC„0C´8Ct8D CCT< C4;Cä1C ;DÄ=D´9 Service Locator Cô@C, AD$0D BCR ?D 8C´>Cd5CÔ8Dý
        AppInfrastructure.Initialize(app.Services);Ð
Ð
        using IServiceScope scope = app.Services.CreateScope();Ð
        scope.ServiceProvider.AutoRegisterTriggers();Ð
    }Ð
}
</file>

<file path="Configuration/AppInfrastructure.cs">
namespace Promatis.Net.Configuration;Ð
Ð
/// <summary>Ð
/// A4;Cä1C ;DÄ=C O D 8D BCT<CÔ0Dò BCäGC¤0 CD>D BD4?C  : C„=DD@C AD$@D4:D$CD =D´< D 5D 2C„AC < Cô;C BDD>D <D²
/// A„=C„FC„0C´8Ct8D CCTBD O Cä4CÔ>C¤@C BCÔ> Cô@C, AD$0D BCR ?D 8C´>Cd5CÔ8Dò 8 C„AC¤;DäGC 5D" @C 7CDCC$0CÔ8CR
/// </summary>Ð
public static class AppInfrastructureÐ
{Ð
    private static IServiceProvider? _provider;Ð
    private static readonly object _lock = new();Ð
Ð
    /// <summary>Ð
    /// Read-only CD>D BD4? C¢ 3C´>C 0C´LCÔ>CÄC C¤>CÔBCT9CÔ5D C Ct0C$8D 8CÄ>D BCT9.Ð
    /// </summary>Ð
    public static IServiceProvider ProviderÐ
    {Ð
        getÐ
        {Ð
            if (_provider == null)Ð
            {Ð
                throw new InvalidOperationException(Ð
                    "A¤@C„BC„GCTAC¤0Dò >D„8C :C ¢    –æg& 7G'V7GW&R =CR 8CÔ8Dd8C ;C„7C„@Cä2C =. " +Ð
                    "B41CT4C„BCTADÂÂ GD$> CÄ5D$>CB    –æg& 7G'V7GW&Rä–æ—F– Æ—¦R,’ 2D´7C$0CÒ 2 Program.cs C„;C
            }Ð
            return _provider;Ð
        }Ð
    }Ð
Ð
    /// <summary>Ð
    /// Aô>D$>C¤>C 5Ct>Cô0D =C O C„=C„FC„0C´8Ct0Dd8Dò ;Cä:C BCä@C â  D´7D´2C 5D$ADò AD$@Cä3Câ >CD8CÒ @C 7 Cô@
    /// </summary>Ð
    public static void Initialize(IServiceProvider provider)Ð
    {Ð
        if (provider == null) throw new ArgumentNullException(nameof(provider));Ð
        if (_provider != null) return;Ð
Ð
        lock (_lock)Ð
        {Ð
            _provider ??= provider;Ð
        }Ð
    }Ð
}
</file>

<file path="Configuration/IAppConfigurator.cs">
using Microsoft.Extensions.Configuration;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Microsoft.Extensions.Hosting;Ð
Ð
namespace Promatis.Net.Configuration;Ð
Ð
public interface IAppConfiguratorÐ
{Ð
    void ConfigureServices(IServiceCollection services, IConfiguration configuration);Ð
    void ConfigureApp(IHost app);Ð
}
```

```
</file>

<file path="Configuration/Promatis.Net.Configuration.csproj">
<Project Sdk="Microsoft.NET.Sdk">Ð
Ð
  <PropertyGroup>Ð
    <TargetFramework>net10.0</TargetFramework>Ð
    <ImplicitUsings>enable</ImplicitUsings>Ð
    <Nullable>enable</Nullable>Ð
•
  </PropertyGroup>Ð
Ð
Ð
Ð
“Ä–FVÔw&÷W í
’  Å  6¶ vU&VfW&Væ6R –æ6ÇVFSÒ$Ö–7&÷6ögBäW‡FVç6–öç2äFW VæFVæ7”–æ¦V7F–öâä '7G& 7F–öç2"
Version="10.0.8" />Ð
’  Å  6¶ vU&VfW&Væ6R –æ6ÇVFSÒ$Ö–7&÷6ögBäW‡FVç6–öç2ä†÷7F–ær" fW'6–öãÒ#  ã ã," óí
’  Å  6¶ vU&VfW&Væ6R –æ6ÇVFSÒ$Ö–7&÷6ögBäW‡FVç6–öç2ä†÷7F–ærä '7G& 7F–öç2" fW'6–öãÒ#  ã ã," óí
’  Å  6¶ vU&VfW&Væ6R –æ6ÇVFSÒ$Ö–7&÷6ögBäW‡FVç6–öç2äÆövv–ærä '7G& 7F–öç2" fW'6–öãÒ#  ã ã," óí
’  Å  6¶ vU&VfW&Væ6R –æ6ÇVFSÒ%67'WF÷"" fW'6–öãÒ#rã ã " óí
“Âô–FVÔw&÷W í
Ð
Ð
Ð
“Ä–FVÔw&÷W í
’  Å &ö¦V7E&VfW&Væ6R –æ6ÇVFSÒ"âåÅ6W'f–6UÅ &öÖ F–2äæWBå6W'f–6Ræ77 &ö¢" óí
“Âô–FVÔw&÷W í
Ð
Ð
</Project>
</file>

<file path="Configuration/ServiceScanningExtensions.cs">
using FluentValidation;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Service;Ð
using System.Reflection;Ð
using System.Runtime.Loader;Ð
Ð
namespace Promatis.Net.Configuration;Ð
Ð
public static class ServiceScanningExtensionsÐ
{Ð
    public static void AddDomainInfrastructure(this IServiceCollection services, string
projectPrefix = "Promatis.")Ð
    {Ð
        Console.WriteLine();Ð
        Console.WriteLine("[SCANNER] At0CôCD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, 2 DI...");Ð
Ð
        // 1. Aô A„ B4 A„"AT BÄ A / At A4 B4 A¤  B  Aä Aä Ð
        string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);Ð
        if (path != null)Ð
        {Ð
            foreach (string file in Directory.GetFiles(path, $"{projectPrefix}*.dll"))Ð
            {Ð
                tryÐ
                {Ð
                    AssemblyLoadContext.Default.LoadFromAssemblyPath(file);Ð
                }Ð
                catch (Exception ex)Ð
                {Ð
                    string fileName = Path.GetFileName(file);Ð
                    Console.ForegroundColor = ConsoleColor.Red;Ð
                    Console.WriteLine($"[SCANNER][ERROR] AÔ5 D44C ;CäADÂ 7C 3D CCt8D$L D 1Cä@C¤C {fileName}:
                    Console.ResetColor();Ð
                }Ð
            }Ð
        }Ð
Ð
        // A 5D 5CÂ 2D 5 Ct0C4@D46CT=CÔKCR AC >D :C, A C$0D„8CÂ AC„AD$5CÄ=D´< Cô@CTDC„:D >CÍ
        Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()Ð
            .Where(a => a.FullName != null && a.FullName.StartsWith(projectPrefix))Ð
            .Distinct()Ð
```

```
            .ToArray();Đ
Đ
        int countBefore = services.Count;Đ
Đ
        // 2. A  B$ AÄ B$ Bt B  Aä  B  A  A„ Aä A  A„  A,  AT A„!B$ A &A„/ Bt B  Ar 45%UDõ-
        services.Scan(scan =>Đ
        {Đ
            // A´>C¤0C´LCÔ0Dò DD4=C¤FC„O CD;Dò @CT:D4@D 8C$=Cä9 Cô@Cä2CT@C¤8 C$ACT9 Dd5Cô>Dt:C, =C AC´5CD>C$0
            bool IsSubclassOfRawGeneric(Type generic, Type? toCheck)Đ
            {Đ
                while (toCheck != null && toCheck != typeof(object))Đ
                {Đ
                    Type cur = toCheck.IsGenericType ? toCheck.GetGenericTypeDefinition() : toCheck;Đ
                    if (generic == cur) return true;Đ
                    toCheck = toCheck.BaseType;Đ
                }Đ
                return false;Đ
            }Đ
Đ
            scan.FromAssemblies(assemblies)Đ
                // A„!Aô A  A´ AÔ : B 5C48D BD 0Dd8Dò !AT A$ B  A" A Cô>CD4CT@Cd:Cä9 C4;D41Cä:Cä3Câ =C AC´5CD
                .AddClasses(classes => classes.Where(type =>Đ
                    !type.IsAbstract &&Đ
                    !type.IsGenericTypeDefinition &&Đ
                    IsSubclassOfRawGeneric(typeof(BaseService<,,>), type)))Đ
                .AsSelf()Đ
                .AsImplementedInterfaces()Đ
                .WithScopedLifetime()Đ
Đ
                // B 5C48D BD 0Dd8Dò  A  A„ A "Aä Aä  (FluentValidation)Đ
                .AddClasses(c => c.AssignableTo(typeof(IValidator<>))Đ
                    .Where(t => !t.IsAbstract && !t.IsGenericTypeDefinition))Đ
                .AsImplementedInterfaces()Đ
                .WithTransientLifetime()Đ
Đ
                // B 5C48D BD 0Dd8Dò "B  A4 AT Aä  (Before/After Save)Đ
                .AddClasses(c => c.AssignableToAny(typeof(IBeforeSaveTrigger<>),
typeof(IAfterSaveTrigger<>)).Where(t => !t.IsAbstract))Đ
                .AsSelfWithInterfaces()Đ
                .WithScopedLifetime();Đ
        });Đ
Đ
        // --- B  A4 B "B  Bd Bò  A´ A  A´,AÔ A4  Aô A´ AÄ B $AÔ A4  A$ A´ AD B$ B   Bô B   ---Đ
        // B 5C48D BD 8D CCT< C¤0C¢ >D$:D KD$KC’ 4Cd5CÔ5D 8C¢â "CT?CT@DÂ ?D 8 Ct0Cô@CäACR •f Æ–F F÷#Ä Dä1Cä9A
        // DI-C¤>CÔBCT9CÔ5D  ääUB    3C @C =D$8D >C$0CÔ=Câ >D$4C AD" =C H Cô>C´8CÄ>D DCÔKC’ 4C„ACô5D$GCT@Đ
        services.AddScoped(typeof(IValidator<>), typeof(GlobalPolymorphicValidator<>));Đ
Đ
        // 3. A„!Aô A  A´ AÔ Aä  A,  A  AT AÔ AR  Aä A„ Aä A  A„  B  At#A´,B$ B$ A-
        List<ServiceDescriptor> newRegistrations = services.Skip(countBefore).ToList();Đ
        var loggedTypes = new HashSet<string>();Đ
Đ
        foreach (ServiceDescriptor reg in newRegistrations)Đ
        {Đ
            // A 5D 5CÂ @CT0C´LCÔKC’ BC„? D 5C ;C„7C FC„8 (C" ?D 8Cä@C„BCTBCR 8Cr –× ÆVÖVçF F–öåG— RÂ 8CÔ0Dt5
            Type? implType = reg.ImplementationType ?? reg.ImplementationInstance?.GetType();Đ
Đ
            if (implType == null) continue;Đ
Đ
            // --- A  A  A„  A$ A´ AD B$ B  A" ÒÒÝ
            bool isValidator = reg.ServiceType.IsGenericType &&Đ
                               reg.ServiceType.GetGenericTypeDefinition() == typeof(IValidator<>);Đ
Đ
            if (isValidator && loggedTypes.Add($"Val:{implType.FullName}"))Đ
            {Đ
                var inheritanceChain = new List<string>();Đ
                Type? currentType = implType;Đ
Đ
                while (currentType != null && currentType != typeof(object))Đ
                {Đ
                    string typeName = currentType.Name.Contains('`') ? currentType.Name.Split('`')
[0] : currentType.Name;Đ
                    inheritanceChain.Add(typeName);Đ
                    currentType = currentType.BaseType;Đ
                    if (typeName == "AbstractValidator") break;Đ
                }Đ
Đ
```

```
                string chainDisplay = string.Join(" -> ", inheritanceChain);Đ
                Console.WriteLine($"[SCANNER] A$0C´8CD0D$>D ¢ ¶6† –äF—7 Æ –Ò ¢ •f Æ–F F÷""“½
                continue; // A´>C2 4C´O DÔBCä9 Ct0Cô8D 8 C$KC$5CD5CÒÂ 8CD5CÂ : D ;CT4D4ND”5C•
            }Đ
Đ
            // --- A  A  A„  B  B  A„!Aä  (AÔ0D ;CT4CÔ8C¤>C" & 6U6W'f–6R’ ÒÒÝ
            // A„!Aô A  A´ AÔ : B41D 0CÔ> Cd5D BC¤>CR CD ;Cä2C„5 reg.ServiceType == reg.ImplementationType, Đ
            // D$0C¢ :C : D 5D 2C„AD² BCT?CT@DÂ @CT3C„AD$@C„@D4ND$ADò ?C @C <C,  C´0D A + A„=D$5D DCT9D
            bool isService = false;Đ
            var serviceChain = new List<string>();Đ
            Type? currentServiceType = implType;Đ
Đ
            while (currentServiceType != null && currentServiceType != typeof(object))Đ
            {Đ
                string typeName = currentServiceType.Name.Contains('`') ?
currentServiceType.Name.Split('`')[0] : currentServiceType.Name;Đ
                serviceChain.Add(typeName);Đ
Đ
                if (currentServiceType.IsGenericType &&
currentServiceType.GetGenericTypeDefinition() == typeof(BaseService<,,>))Đ
                {Đ
                    isService = true;Đ
                    break;Đ
                }Đ
                currentServiceType = currentServiceType.BaseType;Đ
            }Đ
Đ
            if (isService && loggedTypes.Add($"Srv:{implType.FullName}"))Đ
            {Đ
                string chainDisplay = string.Join(" -> ", serviceChain);Đ
                Console.WriteLine($"[SCANNER] B 5D 2C„A:    {chainDisplay}");Đ
                continue;Đ
            }Đ
Đ
            // --- A  A  A„  B$ A„ A4 B  A" ÒÒÝ
            bool isTrigger = implType.GetInterfaces().Any(i => i.IsGenericType &&Đ
                (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||Đ
                 i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));Đ
Đ
            if (isTrigger && loggedTypes.Add($"Tri:{implType.FullName}"))Đ
            {Đ
                Console.WriteLine($"[SCANNER] B$@C„3C45D ¢   ¶–× ÅG— Räæ ÖWÒ"“½
            }Đ
        }Đ
Đ
        Console.WriteLine("[SCANNER] A 2D$>CÄ0D$8Dt5D :C O D 5C48D BD 0Dd8Dò CD ?CTHCÔ> Ct0C$5D HCT=C â"“½
        Console.WriteLine();Đ
    }Đ
Đ
    public static void AutoRegisterTriggers(this IServiceProvider serviceProvider)Đ
    {Đ
        Console.WriteLine("[AUTOREG] At0CôCD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, BD 8C43CT@Cä2 C" F F &
Đ
        var triggerService = serviceProvider.GetRequiredService<IDatabaseTriggerService>();Đ
Đ
        // 1. Aô>C„AC¢ 2D 5DR :C´0D ACä2 D$@C„3C45D >C" 2 D 1Cä@C¤0DR  &öÖ F—2â-
        IEnumerable<Type> triggerTypes = AppDomain.CurrentDomain.GetAssemblies()Đ
            .Where(a => a.FullName?.StartsWith("Promatis.") == true)Đ
            .SelectMany(s => s.GetTypes())Đ
            .Where(t => t is { IsClass: true, IsAbstract: false } &&Đ
                        t.GetInterfaces().Any(i => i.IsGenericType &&Đ
                                                   (i.GetGenericTypeDefinition() ==
typeof(IBeforeSaveTrigger<>) ||Đ
                                                    i.GetGenericTypeDefinition() ==
typeof(IAfterSaveTrigger<>))))Đ
            .ToList();Đ
Đ
        MethodInfo? registerMethod =
typeof(IDatabaseTriggerService).GetMethod(nameof(IDatabaseTriggerService.Register));Đ
Đ
        int totalBindings = 0;Đ
Đ
        foreach (Type triggerType in triggerTypes)Đ
        {Đ
            // AÔ0DT>CD8CÂ 2D 5 C„=D$5D DCT9D K D$@C„3C45D >C"Â :CäBCä@D´5 D 5C ;C„7D45D" 4C =CÔKC’ :C´0D AĐ
            IEnumerable<Type> interfaces = triggerType.GetInterfaces()Đ
```

```
                .Where(i => i.IsGenericType &&Ð
                            (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||Ð
                             i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));Ð
Ð
            foreach (Type @interface in interfaces)Ð
            {Ð
                // Aô>C´CDt0CT< D$8Cò AD4ICÔ>D BC, …B 8Cr ”&Vf÷&U6 fUG&–vvW#ÅCâ•
                Type entityType = @interface.GetGenericArguments()[0];Ð
Ð
                // Aä?D 5CD5C´OCT< D$8Cò BD 8C43CT@C  4C´O C¤@C AC„2Cä3Câ ;Cä3C
                string triggerKind = @interface.GetGenericTypeDefinition() ==
typeof(IBeforeSaveTrigger<>)Ð
                    ? "Before"Ð
                    : "After ";Ð
Ð
                tryÐ
                {Ð
                    // A$KCtKC$0CT< Register<TEntity, TTrigger>()Ð
                    MethodInfo genericRegister = registerMethod!.MakeGenericMethod(entityType,
triggerType);Ð
                    genericRegister.Invoke(triggerService, null);Ð
Ð
                    // A´ A4 B  A$ AÔ AR #B  AT(AÔ A’  B  A$/At A•
                    Console.WriteLine($"[AUTOREG] [{triggerKind}] {entityType.Name} <---
{triggerType.Name}");Ð
                    totalBindings++;Ð
                }Ð
                catch (Exception ex)Ð
                {Ð
                    Console.ForegroundColor = ConsoleColor.Red;Ð
                    Console.WriteLine($"[AUTOREG][ERROR] AäHC„1C¤0 Cô@C„2Dô7C¤8 {triggerType.Name} C¢ ¶VçF—G•
{ex.InnerException?.Message ?? ex.Message}");Ð
                    Console.ResetColor();Ð
                }Ð
            }Ð
        }Ð
Ð
        Console.WriteLine($"[AUTOREG] At0C$5D HCT=Cââ !Cä7CD0CÔ> Cô@C„2Dô7Cä:: {totalBindings}");Ð
        Console.WriteLine();Ð
    }Ð
}
</file>

<file path="Data/Configurations/AuditLogConfiguration.cs">
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.EntityFrameworkCore.Metadata.Builders;Ð
using Promatis.Net.Domain;Ð
Ð
namespace Promatis.Net.Data;Ð
Ð
public class AuditLogConfiguration : IEntityTypeConfiguration<AuditLog>Ð
{Ð
    public void Configure(EntityTypeBuilder<AuditLog> builder)Ð
    {Ð
        builder.Property(e => e.EntityName).HasMaxLength(100);Ð
        builder.Property(e => e.Action).HasMaxLength(50);Ð
        builder.Property(e => e.ChangedBy).HasMaxLength(100);Ð
Ð
        // ChangesJson CäAD$0C$;Dô5CÂ 1CT7 C´8CÄ8D$0 (text/jsonb), D$0C¢ :C : D ?C„ACä: C„7CÄ5CÔ5CÔ8C’ <Cä6CT
        builder.Property(e => e.ChangesJson).HasColumnType("jsonb");Ð
    }Ð
}
</file>

<file path="Data/Configurations/ModelBuilderConventions.cs">
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.EntityFrameworkCore.Metadata;Ð
using System.Linq.Expressions;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Data;Ð
Ð
public static class ModelBuilderConventionsÐ
{Ð
    public static void ApplyGlobalConventions(this ModelBuilder modelBuilder)Ð
    {Ð
```

```
        foreach (IMutableEntityType entityType in modelBuilder.Model.GetEntityTypes())Ð
        {Ð
            Type type = entityType.ClrType;Ð
Ð
            // A´8CÄ8D$K CD;Dò AD$@Cä: (255 Cô> D4<Cä;Dt0CÔ8Dâ•
            foreach (IMutableProperty property in entityType.GetProperties())Ð
            {Ð
                if (property.ClrType == typeof(string))Ð
                {Ð
                    if (property.GetMaxLength() == null)Ð
                        property.SetMaxLength(255);Ð
                }Ð
            }Ð
Ð
            // A 2D$>-C¤;DäGC, 4C´O Guid (ValueGeneratedNever, D$0C¢ :C : Id D >Ct4C 5D$ADò 2 DomainObject)Ð
            if (typeof(IDomainObjectHasKey<Guid>).IsAssignableFrom(type))Ð
            {Ð
                modelBuilder.Entity(type).Property("Id").ValueGeneratedNever();Ð
            }Ð
Ð
            // A  B$ -BD A´,B$  C,  A$"AâÔ AÔ AT B  4C´O SoftDelete Cä1D¤5C¤BCä2Ð
            if (typeof(ISoftDeletable).IsAssignableFrom(type))Ð
            {Ð
                IMutableEntityType? baseType = entityType.BaseType;Ð
Ð
                // B BC 2C„< DD8C´LD$@ D$>C´LC¤> CÔ0 C¤>D 5CÔL C„5D 0D EC„8 (C$:C´NDt0Dò 0C AD$@C :D$=D´9 Uni
                if (baseType == null || !typeof(ISoftDeletable).IsAssignableFrom(baseType.ClrType))Ð
                {Ð
                    // B4AD$0CÔ>C$:C   VW'”f–ÇFW-
                    modelBuilder.SetSoftDeleteFilter(type);Ð
Ð
                    // B4AD$0CÔ>C$:C  8CÔ4CT:D 0 CÔ0 DeletedAt CD;Dò CD :Cä@CT=C„O Ct0Cô@CäACä2Ð
                    modelBuilder.Entity(type).HasIndex("DeletedAt");Ð
                }Ð
            }Ð
        }Ð
    }Ð
Ð
    private static void SetSoftDeleteFilter(this ModelBuilder modelBuilder, Type entityType)Ð
    {Ð
        // AD;Dò E BõE , 2C 6CÔ> D BC 2C„BDÂ DC„;DÄBD  =C  :Cä@CT=DÂÂ 4C 6CR 5D ;C, >CÒ 0C AD$@C :D$=D´9.Ð
        // B4AC´>C$8CR –46Æ 72 4CäAD$0D$>Dt=Câí
        if (entityType.IsClass)Ð
        {Ð
            modelBuilder.Entity(entityType).HasQueryFilter(GenerateFilter(entityType));Ð
        }Ð
    }Ð
Ð
    private static LambdaExpression GenerateFilter(Type type)Ð
    {Ð
        ParameterExpression parameter = Expression.Parameter(type, "e");Ð
        // A„ACô>C´LCtCCT< Property CD;Dò 4CäAD$CCô0 C¢ FVÆWFVD B GCT@CT7 C„=D$5D DCT9D  8C´8 D 2Cä9D BC$>Ð
        MemberExpression property = Expression.Property(parameter,
nameof(ISoftDeletable.DeletedAt));Ð
        ConstantExpression nullConstant = Expression.Constant(null, typeof(DateTime?));Ð
        BinaryExpression body = Expression.Equal(property, nullConstant);Ð
Ð
        return Expression.Lambda(body, parameter);Ð
    }Ð
}
</file>

<file path="Data/DBContext/ApplicationDbContext.cs">
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.Extensions.Configuration;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Promatis.Net.Domain;Ð
Ð
namespace Promatis.Net.Data;Ð
Ð
public class ApplicationDbContext(Ð
    DbContextOptions options,Ð
    IConfiguration configuration,Ð
    IServiceProvider? serviceProvider = null)Ð
    : DbContext(options)Ð
{Ð
```

```
    protected virtual string Schema => "public";Ð
    public DbSet<AuditLog> AuditLogs => Set<AuditLog>();Ð
Ð
    protected override void OnModelCreating(ModelBuilder modelBuilder)Ð
    {Ð
        modelBuilder.HasDefaultSchema(Schema);Ð
        modelBuilder.ApplyModuleConfigurations(this);Ð
        modelBuilder.ApplyGlobalConventions();Ð
        base.OnModelCreating(modelBuilder);Ð
    }Ð
Ð
    // A„!Aô A  A´ AÔ B´  A,  A  AT AÔ+A’  A  A„ AÔ"Ð
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)Ð
    {Ð
        base.OnConfiguring(optionsBuilder);Ð
Ð
        // 1. Aô@Cä2CT@Dô5CÂÂ GD$> CÄK C" @C =D$0C”<CRÂ 0 CÔ5 C" ?D >Dd5D ACR <C„3D 0Dd8C, 4Ä•
        if (!EF.IsDesignTime && serviceProvider != null)Ð
        {Ð
            DatabaseTriggerInterceptor? interceptor = null;Ð
Ð
            tryÐ
            {Ð
                // Aô>CôKD$:C   ¢  D >C CCT< D 0Ct@CTHC„BDÂ =C ?D OCÄCDâ „5D ;C, ?D >C$0C”4CT@ Cä:C 7C ;D O S
                interceptor = serviceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as
DatabaseTriggerInterceptor;Ð
            }Ð
            catch (InvalidOperationException)Ð
            {Ð
                // Aô>CôKD$:C  #¢  D ;C, ?D >C$0C”4CT@ C¤>D =CT2Cä9 (Root), D >Ct4C 5CÂ 66÷ R 2D CDt=D4N!Ð
                // BÔBCâ 3C @C =D$8D CCTB, DtBCâ 66÷ VBÔ8CÔBCT@Dd5CôBCä@ D 0Ct@CTHC„BD O C 5Cr >D„8C >C¢ > Ro
                var scopeFactory = serviceProvider.GetService(typeof(IServiceScopeFactory)) as
IServiceScopeFactory;Ð
Ð
                if (scopeFactory != null)Ð
                {Ð
                    // B >Ct4C 5CÂ 2D 5CÄ5CÔ=D4N Cä1C´0D BDÂ 2C„4C„<CäAD$8Ð
                    using IServiceScope scope = scopeFactory.CreateScope();Ð
                    interceptor =
scope.ServiceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as DatabaseTriggerInterceptor;Ð
                }Ð
            }Ð
Ð
            // ATAC´8 C„=D$5D FCT?D$>D  CD ?CTHCÔ> CÔ0C”4CT= Cä4CÔ8CÂ 8Cr ACô>D >C >C" r ?Cä4C¤;DäGC 5CÂ 5C4>
            if (interceptor != null)Ð
            {Ð
                optionsBuilder.AddInterceptors(interceptor);Ð
            }Ð
        }Ð
    }Ð
}
</file>

<file path="Data/DBContext/ModelBuilderExtensions.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Reflection;Ð
using Microsoft.EntityFrameworkCore.Metadata;Ð
using Microsoft.EntityFrameworkCore.Metadata.Builders;Ð
Ð
namespace Promatis.Net.Data;Ð
Ð
public static class ModelBuilderExtensionsÐ
{Ð
    public static void ApplyModuleConfigurations(this ModelBuilder modelBuilder, DbContext context)Ð
    {Ð
        // 1. B :C =C„@D45CÂ BCä;DÄ:Câ AC >D :C, 4C =CÔKDR ?D >CT:D$0 Promatis.*.DataÐ
        List<Assembly> dataAssemblies = AppDomain.CurrentDomain.GetAssemblies()Ð
            .Where(a => {Ð
                string? name = a.GetName().Name;Ð
                return name != null &&Ð
                       name.StartsWith("Promatis.") &&Ð
                       name.EndsWith(".Data");Ð
            }).ToList();Ð
Ð
        // 2. Aô>C´CDt0CT< D$8CôK D CD”=CäAD$5C’Â :CäBCä@D´5 Dô2CÔ> Cä1D¤OC$;CT=D² 2 D$5C¤CD”5CÂ :Cä=D$5C¤AD$
```

```
        HashSet<Type> contextDbSetTypes = context.GetType()Ð
            .GetProperties(BindingFlags.Public | BindingFlags.Instance)Ð
            .Where(p => p.PropertyType.IsGenericType && p.PropertyType.GetGenericTypeDefinition()
== typeof(DbSet<>))Ð
            .Select(p => p.PropertyType.GetGenericArguments()[0])Ð
            .ToHashSet();Ð
Ð
        foreach (Assembly assembly in dataAssemblies)Ð
        {Ð
            // Aô@C„<CT=Dô5CÂ :Cä=DD8C4CD 0Dd8C, BCä;DÄ:Câ 4C´O D$5DR BC„?Cä2, C¤>D$>D KCR <C BCT@C„0C´LCÔK (
            modelBuilder.ApplyConfigurationsFromAssembly(assembly, type =>Ð
            {Ð
                if (type is not { IsClass: true, IsAbstract: false, IsGenericTypeDefinition:
false })Ð
                    return false;Ð
Ð
                // AÔ0DT>CD8CÂ 8CÔBCT@DD5C”A C¤>CÔDC„3D4@C FC„8 C, BC„? D CD”=CäAD$8, C¤>D$>D CDâ >CÔ0 CÔ0D B
                Type? configInterface = type.GetInterfaces()Ð
                    .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(IEntityTypeConfiguration<>));Ð
Ð
                if (configInterface == null) return false;Ð
Ð
                Type entityType = configInterface.GetGenericArguments()[0];Ð
Ð
                // A¤ A„"A„'AT!A¤ A’ $A„ BÄ"B ¢  D ;C, =C AD$@C 8C$0CT<C O D CD”=CäAD$L C 1D BD 0C¤BCÔ0,Ð
                // Cô@Cä2CT@Dô5CÂÂ 5D BDÂ ;C, C CÔ5E  ECäBDÂ >CD8CÒ 6C„2Cä9 CÔ0D ;CT4CÔ8C¢ 2 D$5C¤CD”5CÂ F%6W
                if (entityType.IsAbstract)Ð
                {Ð
                    bool hasConcreteDerivedInDbSet = contextDbSetTypes.Any(t =>
t.IsSubclassOf(entityType));Ð
                    if (!hasConcreteDerivedInDbSet) return false; // Aô@Cä?D4AC¤0CT< C¤>CÔDC„3D4@C FC„N C 1D
                }Ð
Ð
                return type.GetConstructor(Type.EmptyTypes) != null;Ð
            });Ð
        }Ð
Ð
        // 3. A  B$ -A„ AÔ B  B  A$ AÔ AR ´ B  B  B$ A$(A„%» A  B "B  A¤&A„ Ð
        IEnumerable<Type> allAbstractEntities = dataAssembliesÐ
            .SelectMany(a => a.GetTypes())Ð
            .Where(t => t is { IsClass: true, IsAbstract: true } && !t.IsGenericTypeDefinition);Ð
Ð
        foreach (Type abstractType in allAbstractEntities)Ð
        {Ð
            bool isUsedInCurrentContext = contextDbSetTypes.Any(t => t == abstractType ||
t.IsSubclassOf(abstractType));Ð
Ð
            if (!isUsedInCurrentContext)Ð
            {Ð
                modelBuilder.Ignore(abstractType);Ð
            }Ð
        }Ð
Ð
        // 4. A  B$ AÄ B$ Bt B  A / AÔ B "B  A” A   At A´ B  A$ AÔ B´% AD B  A$,AT  A,  AÔ AT B  A-
        foreach (IMutableEntityType entityType in modelBuilder.Model.GetEntityTypes())Ð
        {Ð
            Type? clrType = entityType.ClrType;Ð
            if (clrType == null) continue;Ð
Ð
            // A„ICT< D 5C ;C„7C FC„N C„=D$5D DCT9D 0 ITreeNode<>Ð
            Type? treeInterface = clrType.GetInterfaces()Ð
                .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(ITreeNode<>));Ð
Ð
            if (treeInterface != null)Ð
            {Ð
                Type targetTreeType = treeInterface.GetGenericArguments()[0];Ð
Ð
                // AÔ0D BD 0C„2C 5CÂ BCä;DÄ:Câ =C  1C 7Cä2Cä< D4@Cä2CÔ5 Cä1D¤OC$;CT=C„O D 2Cä9D BC" „GD$>C K
                if (clrType == targetTreeType)Ð
                {Ð
                    EntityTypeBuilder builder = modelBuilder.Entity(clrType);Ð
Ð
                    // 1. AÔ0D BD >C”:C  AC$OCt8 B >CD8D$5C´L-Aô>D$>CÄ>C-
                    builder.HasOne("Parent")Ð
```

```
                        .WithMany("Children")Ð
                        .HasForeignKey("ParentId")Ð
                        .OnDelete(DeleteBehavior.Restrict);Ð
Ð
                    // 2. A  B$ -A„ AD A¤!: B >Ct4C 5CÂ 8CÔ4CT:D  4C´O C$=CTHCÔ5C4> C¤;DäGC    &VçD–M
                    // BÔBCâ CD :Cä@C„B Cô>C„AC¢ MC´5CÄ5CÔBCä2 Cô> ParentId Cô@C, @C 1CäBCR ”Æöö·W  2 D 5D 2C
                    builder.HasIndex("ParentId")Ð
                        .HasDatabaseName($"IX_{clrType.Name}_ParentId");Ð
                }Ð
            }Ð
        }Ð
    }Ð
}
</file>

<file path="Data/Interceptors/ChangeEntryModel.cs">
namespace Promatis.Net.Data;Ð
Ð
/// <summary>Ð
/// AÄ>CD5C´L DD8C¤AC FC„8 C„7CÄ5CÔ5CÔ8C’ „BD 0CÔACô>D BCÔKC’ >C JCT:D"’Â ?D 5CDAD$0C$;DÔND”0Dò ACÔ8CÄ>C¢ ACä
/// CD>CÄ5CÔ=Cä9 D CD”=CäAD$8 C" <Cä<CT=D" 5E  4Cä1C 2C´5CÔ8DòÂ <Cä4C„DC„:C FC„8 C„;C, CCD0C´5CÔ8Dòí
/// </summary>Ð
internal class ChangeEntryModelÐ
{Ð
    /// <summary>Ð
    /// A„7CÄ5CÔOCT<D´9 DÔ:Ct5CÄ?C´OD  4Cä<CT=CÔ>C4> Cä1D¤5C¤BC  „AD4ICÔ>D BC,’í
    /// AÔ@C„2Cä4C„BD O C¢ :Cä=C¤@CTBCÔ>CÄC D$8CôC C$=D4BD 8 D$@C„3C45D >C"í
    /// </summary>Ð
    public object Entity { get; init; } = null!;Ð
Ð
    /// <summary>Ð
    /// B$5C¤CD”8C’ AD$0D$CD  8Ct<CT=CT=C„O D CD”=CäAD$8 C" AC„AD$5CÄ5 (Added, Modified, Deleted, SoftDeleted
    /// </summary>Ð
    public EntityStateChangeEnum State { get; init; }Ð
Ð
    /// <summary>Ð
    /// A¤>C´;CT:Dd8Dò BCäGCTGCÔKDR 8Ct<CT=CT=C„9 D 2Cä9D BC" „?Cä;CT9) D CD”=CäAD$8.Ð
    /// At0CÔ>C´=DÔ5D$ADò 4CT;DÄBCä9 Ct=C GCT=C„9 "B BC @Cä5 -> AÔ>C$>CR"â  C´O CÔ>C$KDR AD4ICÔ>D BCT9 D >CD5
    /// </summary>Ð
    public List<PropertyChangeInfo> Changes { get; init; } = [];Ð
Ð
    /// <summary>Ð
    /// A„4CT=D$8DD8C¤0D$>D  „8CÄO C„;C, ;Cä3C„=) Cô>C´LCt>C$0D$5C´O, D >C$5D HC„2D„5C4> CD0CÔ=Cä5 CD5C”AD$2C
    /// </summary>Ð
    public string? ChangedBy { get; init; }Ð
Ð
    /// <summary>Ð
    /// AD0D$0 C, 2D 5CÄO DD8C¤AC FC„8 C„7CÄ5CÔ5CÔ8Dò 2 DD>D <C BCR UD2í
    /// </summary>Ð
    public DateTime ChangedAt { get; init; }Ð
}
</file>

<file path="Data/Interceptors/DatabaseTriggerInterceptor.cs">
using System.Collections.Concurrent;Ð
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.EntityFrameworkCore.ChangeTracking;Ð
using Microsoft.EntityFrameworkCore.Diagnostics;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Data;Ð
Ð
/// <summary>Ð
/// AÔ5D 5DT2C BDt8C¢ >CÔ5D 0Dd8C’ F$6öçFW‡BÂ >C 5D ?CTGC„2C ND”8C’ @C 1CäBD2 $<Dô3C¤>C4> D44C ;CT=C„O" (Soft
/// CÔ@CT4C$0D 8D$5C´LCÔCDâ 2C ;C„4C FC„N C„7CÄ5CÔ5CÔ8C’ 8 D :C$>Ct=Cä5 D$@C„3C45D =Cä5 D42CT4Cä<C´5CÔ8CR AC„
/// </summary>Ð
/// <param name="scopeFactory">BD0C @C„:C  4C´O D >Ct4C =C„O C„7Cä;C„@Cä2C =CÔKDR >C ;C AD$5C’ 2C„4C„<CäAD$8
public class DatabaseTriggerInterceptor(IServiceScopeFactory scopeFactory) : SaveChangesInterceptorÐ
{Ð
    /// <summary>Ð
    /// AÔ>D$>C¤>C 5Ct>CÔ0D =Cä5 DT@C =C„;C„ICR 7C DC„:D 8D >C$0CÔ=D´E C„7CÄ5CÔ5CÔ8C’ 2 D 0CÄ:C E D$5C¤CD”5C’
    /// A¤;DäG – D4=C„:C ;DÄ=D´9 C„4CT=D$8DD8C¤0D$>D  MC¤7CT<Cô;Dô@C  F$6öçFW‡Bí
    /// </summary>Ð
    private readonly ConcurrentDictionary<Guid, List<ChangeEntryModel>> _capturedChanges = new();Ð
Ð
```

```
    /// <summary>Đ
    /// A AC„=DT@Cä=CÔKC’ ?CT@CTEC$0D$GC„:, C$KCô>C´=Dô5CÄKC’  Aâ DC :D$8Dt5D :Cä3Câ ACäED 0CÔ5CÔ8Dò 8Ct<CT=C
    /// AäBC$5Dt0CTB Ct0 C´>C48C¤C Soft Delete, DD8C¤AC FC„N D =C„<C¤0 C„7CÄ5CÔ5CÔ8C’ 8 Ct0CôCD : D$@C„3C45D
    /// </summary>Đ
    public override async ValueTask<InterceptionResult<int>> SavingChangesAsync(Đ
        DbContextEventData eventData, InterceptionResult<int> result, CancellationToken ct =
default)Đ
    {Đ
        if (eventData.Context == null) return result;Đ
Đ
        // B >Ct4C 5CÂ ;Cä:C ;DÄ=D´9 C40D 0CÔBC„@Cä2C =CÔ> Cd8C$>C’ 66÷ R 4C´O Cô@CT4CäBC$@C ICT=C„O Cô@Cä1C´
        using IServiceScope scope = scopeFactory.CreateScope();Đ
        var triggerService = scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();Đ
Đ
        string? userName = await GetUserNameInternalAsync(scope.ServiceProvider);Đ
Đ
        // 1. Aô@CT4C$0D 8D$5C´LCÔ0Dò >C @C 1CäBC¤0 Soft Delete (AÄOC4:Cä5 D44C ;CT=C„5)Đ
        // AÔ0DT>CD8CÂ 2D 5 D CD”=CäAD$8 D  8CÔBCT@DD5C”ACä< ISoftDeletable, D2 :CäBCä@D´E D BC BD4A "B44C ;C
        IEnumerable<EntityEntry<ISoftDeletable>> entries =
eventData.Context.ChangeTracker.Entries<ISoftDeletable>()Đ
            .Where(e => e.State == EntityState.Deleted);Đ
Đ
        foreach (EntityEntry<ISoftDeletable> entry in entries)Đ
        {Đ
            // Aô>CD<CT=Dô5CÂ DC„7C„GCTAC¤>CR CCD0C´5CÔ8CR =C  >C =Cä2C´5CÔ8CR 4C =CÔKDR 2 A  Đ
            entry.State = EntityState.Modified;Đ
            entry.Entity.DeletedAt = DateTime.UtcNow;Đ
            entry.Entity.DeletedBy = userName;Đ
        }Đ
Đ
        // Aô@C„=D44C„BCT;DÄ=Câ 7C AD$0C$;Dô5CÂ Tb 6÷&R ?CT@CTADt8D$0D$L CD5D 5C$> C„7CÄ5CÔ5CÔ8C’ ?CäAC´5 Cô>
        eventData.Context.ChangeTracker.DetectChanges();Đ
Đ
        // 2. At0DT2C B C„7CÄ5CÔ5CÔ8C’ „ACäED 0CÔOCTB CÔ0D$8C$=D´5 D$8CôK CD0CÔ=D´E object? CD;Dò BCTAD$>C"•
        List<ChangeEntryModel> captured = CaptureChanges(eventData.Context, userName);Đ
Đ
        // Aô@C„2Dô7D´2C 5CÂ :Cä;C´5C¤FC„N C„7CÄ5CÔ5CÔ8C’ : C¤>CÔ:D 5D$=Cä<D2 MC¤7CT<Cô;Dô@D2 :Cä=D$5C¤AD$0 C
        _capturedChanges[eventData.Context.ContextId.InstanceId] = captured;Đ
Đ
        // 3. A$0C´8CD0Dd8Dò „&Vf÷&U6 fR’ GCT@CT7 Cd8C$>C’ G&–vvW%6W'f–6]
        // At0CÔCD :C 5D" 4Cä<CT=CÔKCR ?D 0C$8C´0 C, 2C ;C„4C BCä@D² ?CT@CT4 D$5CÂÂ :C : CD0CÔ=D´5 Cô>Cô0CDCD
        foreach (ChangeEntryModel item in captured)Đ
            await triggerService.ValidateAsync(item.Entity, item.State, item.Changes,
eventData.Context);Đ
Đ
        return result;Đ
    }Đ
Đ
    /// <summary>Đ
    /// A AC„=DT@Cä=CÔKC’ ?CT@CTEC$0D$GC„:, C$KCô>C´=Dô5CÄKC’  Aä!A´  D4ACô5D„=Cä3Câ ACäED 0CÔ5CÔ8Dò 8Ct<CT=C
    /// AäBC$5Dt0CTB Ct0 D42CT4Cä<C´5CÔ8CR ?Cä4CÔ8D GC„:Cä2 C, BD 8C43CT@Cä2 Cô>D B-Cä1D 0C >D$:C, „DC 7C  6
    /// </summary>Đ
    public override async ValueTask<int> SavedChangesAsync(Đ
        SaveChangesCompletedEventData eventData, int result, CancellationToken ct = default)Đ
    {Đ
        // A„7C$;CT:C 5CÂ 7C EC$0Dt5CÔ=D´5 C„7CÄ5CÔ5CÔ8Dò 8 D @C 7D2 0D$>CÄ0D =Câ CCD0C´OCT< C„E C„7 D ;Cä2C
        if (eventData.Context != null &&
_capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out List<ChangeEntryModel>?
entries))Đ
        {Đ
            // B >Ct4C 5CÂ ;Cä:C ;DÄ=D´9 C40D 0CÔBC„@Cä2C =CÔ> Cd8C$>C’ 66÷ R 4C´O DD0CtK Saved (C$KCô>C´=Dô5
            using IServiceScope scope = scopeFactory.CreateScope();Đ
            var triggerService =
scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();Đ
Đ
            // A AC„=DT@Cä=CÔ> D42CT4Cä<C´OCT< D 8D BCT<D2 >C  8Ct<CT=CT=C„ODR „=C ?D 8CÄ5D Â 4C´O Cä1CÔ>C$;C
            foreach (ChangeEntryModel item in entries)Đ
                await triggerService.NotifyAsync(item.Entity, item.State, item.Changes,
item.ChangedBy, item.ChangedAt);Đ
        }Đ
        return result;Đ
    }Đ
Đ
    /// <summary>Đ
    /// Aô5D 5DT2C BDt8C¢ AC >Dò ?D 8 D >DT@C =CT=C„8 C„7CÄ5CÔ5CÔ8C’â  C @C =D$8D CCTB CäGC„AD$:D2 :DÔHC  7C
    /// Cô@CT4CäBC$@C IC O D4BCTGC¤8 CÔ0CÄOD$8 Cô@C, ?C 4CT=C„8 D$@C =Ct0C¤FC„9.Đ
```

```
    /// </summary>Ð
    public override Task SaveChangesFailedAsync(DbContextErrorEventData eventData,
CancellationToken ct = default)Ð
    {Ð
        if (eventData.Context != null)Ð
            _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out _);Ð
Ð
        return base.SaveChangesFailedAsync(eventData, ct);Ð
    }Ð
Ð
    /// <summary>Ð
    /// A$=D4BD 5CÔ=C„9 CÄ5D$>CB 0CÔ0C´8Ct0 D$@CT:CT@C  8Ct<CT=CT=C„9 EF Core. Ð
    /// A$KDt8D ;Dô5D" 4CT;DÄBD2 „AD$0D KCRô=Cä2D´5 Ct=C GCT=C„O Cô>C´5C’’ 8 DD>D <C„@D45D" <Cä4CT;C, 8Ct<CT=
    /// </summary>Ð
    /// <param name="context">B$5C¤CD”8C’ :Cä=D$5C¤AD" 1C 7D² 4C =CÔKDRãÂ÷  & Óí
    /// <param name="userName">A„<Dò ?Cä;DÄ7Cä2C BCT;DòÂ ACä2CT@D„8C$HCT3Câ >Cô5D 0Dd8DâãÂ÷  & Óí
    /// <returns>B ?C„ACä: D BD CC¤BD4@C„@Cä2C =CÔKDR <Cä4CT;CT9 C„7CÄ5CÔ5CÔ8C’ Ç6VR 7&VcÒ$6† ævTVçG'”ÖöFVÂ"ó
    private List<ChangeEntryModel> CaptureChanges(DbContext context, string? userName)Ð
    {Ð
        // BD8C´LD$@D45CÂ BCä;DÄ:Câ AD4ICÔ>D BC, 2 D >D BCäOCÔ8DôE AD>C 0C$;CT=C„O, A„7CÄ5CÔ5CÔ8Dò 8C´8 B44C
        List<EntityEntry> entries = context.ChangeTracker.Entries()Ð
            .Where(e => e.State is EntityState.Added or EntityState.Modified or EntityState.Deleted)Ð
            .Where(e => !e.Entity.GetType().Name.Contains("AuditLog"))Ð
            .ToList();Ð
Ð
        var result = new List<ChangeEntryModel>();Ð
Ð
        foreach (EntityEntry e in entries)Ð
        {Ð
            EntityStateChangeEnum state = MapState(e.State);Ð
Ð
            // B ?CTFC„DC„GCÔKC’ <C ?Cô8CÔ3 D >D BCäOCÔ8Dò 4C´O "CÄOC4:Câ CCD0C´5CÔ=D´E" Ct0Cô8D 5C•
            if (e.Entity is ISoftDeletable soft)Ð
            {Ð
                PropertyEntry prop = e.Property(nameof(ISoftDeletable.DeletedAt));Ð
                if (prop.IsModified && soft.DeletedAt != null) state =
EntityStateChangeEnum.SoftDeleted;Ð
            }Ð
Ð
            List<PropertyChangeInfo> changes = new();Ð
Ð
            // B FCT=C @C„9 1: B CD”=CäAD$L CD>C 0C$;CT=C â  D 5 D$5C¤CD”8CR 7CÔ0Dt5CÔ8Dò AC$>C”AD$2 D GC„BC
            if (state == EntityStateChangeEnum.Added)Ð
            {Ð
                changes = e.PropertiesÐ
                    .Select(p => new PropertyChangeInfoÐ
                    {Ð
                        PropertyName = p.Metadata.Name,Ð
                        OriginalValue = null,Ð
                        CurrentValue = p.CurrentValueÐ
                    }).ToList();Ð
            }Ð
            // B FCT=C @C„9 2: B CD”=CäAD$L C„7CÄ5CÔ5CÔ0 C„;C, <Dô3C¤> D44C ;CT=C â  D´GC„AC´OCT< D 0Ct=C„FD2
            else if (state is EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted)Ð
            {Ð
                PropertyValues originalValues = e.OriginalValues;Ð
Ð
                foreach (PropertyEntry p in e.Properties.Where(p => p.IsModified))Ð
                {Ð
                    string propertyName = p.Metadata.Name;Ð
                    object? originalValue = originalValues[propertyName];Ð
                    object? currentValue = p.CurrentValue;Ð
Ð
                    // BD8C¤AC„@D45CÂ 8Ct<CT=CT=C„5 D$>C´LC¤> CTAC´8 Ct=C GCT=C„O CD5C”AD$2C„BCT;DÄ=Câ @C 7C´
                    if (!Equals(originalValue, currentValue))Ð
                    {Ð
                        changes.Add(new PropertyChangeInfoÐ
                        {Ð
                            PropertyName = propertyName,Ð
                            OriginalValue = originalValue,Ð
                            CurrentValue = currentValueÐ
                        });Ð
                    }Ð
                }Ð
            }Ð
Ð
```

```
            // AD0Cd5 CTAC´8 C¤>C´;CT:Dd8Dò BCäGCTGCÔKDR 8Ct<CT=CT=C„9 Cô>C´5C’ „6† ævW2’ ?D4AD$0 Đ
            // C„7-Ct0 D 0CÔBC 9CÂÔ1C„=CD8CÔ3C  &Æ ¦÷"Â <D²  B   B  A$ Aâ 4Cä1C 2C´OCT< D CD”=CäAD$L C" ACô8D
            // CTAC´8 DD0C @C„:C  7C DC„:D 8D >C$0C´0 D >D BCäOCÔ8CR ÖöF–f–VB  -D$> C40D 0CÔBC„@D45D"    R 4C
            if (state == EntityStateChangeEnum.Modified && changes.Count == 0)Đ
            {Đ
                // B >Ct4C 5CÂ DC„:D$8C$=D4N Ct0Cô8D L C„7CÄ5CÔ5CÔ8Dò 4C´O CD>CÄ5CÔ=D´E D$@C„3C45D >C"Â
                // DtBCä1D² =CR ;Cä<C BDÂ 2C ;C„4C FC„N, CÔ> D >DT@C =C„BDÂ 8CÄ?D4;DÄA CD;Dò T•
                changes.Add(new PropertyChangeInfoĐ
                {Đ
                    PropertyName = "Id",Đ
                    OriginalValue = e.Property("Id").OriginalValue,Đ
                    CurrentValue = e.Property("Id").CurrentValueĐ
                });Đ
            }Đ
Đ
            result.Add(new ChangeEntryModelĐ
            {Đ
                Entity = e.Entity,Đ
                State = state,Đ
                ChangedBy = userName,Đ
                ChangedAt = DateTime.UtcNow,Đ
                Changes = changesĐ
            });Đ
        }Đ
Đ
        return result;Đ
    }Đ
Đ
    /// <summary>Đ
    /// AÄ0Cô?C„=C2 2CÔCD$@CT=CÔ8DR ACäAD$>Dô=C„9 EntityFramework State C$> C$=D4BD 5CÔ=C„9 CD>CÄ5CÔ=D´9 Cô5D
    /// </summary>Đ
    private EntityStateChangeEnum MapState(EntityState state) => state switchĐ
    {Đ
        EntityState.Added => EntityStateChangeEnum.Added,Đ
        EntityState.Deleted => EntityStateChangeEnum.Deleted,Đ
        _ => EntityStateChangeEnum.ModifiedĐ
    };Đ
Đ
    /// <summary>Đ
    /// A 5Ct>CÔ0D =Cä5 Cô>C´CDt5CÔ8CR 8CÄ5CÔ8 D$5C¤CD”5C4> Cô>C´LCt>C$0D$5C´O C„7 C¤>CÔBCT:D BC  0C$BCä@C„7C
    /// </summary>Đ
    /// <param name="provider">A´>C¤0C´LCÔKC’ ?D >C$0C”4CT@ D ;D46C  „D’ 6öçF –æW"’ãÂ÷  & Óí
    private async Task<string?> GetUserNameInternalAsync(IServiceProvider provider)Đ
    {Đ
        tryĐ
        {Đ
            var userProvider = provider.GetService<IUserProvider>();Đ
            if (userProvider != null)Đ
                return await userProvider.GetCurrentUserNameAsync();Đ
        }Đ
        catchĐ
        {Đ
            // ignoredĐ
        }Đ
Đ
        return "System";Đ
    }Đ
}
</file>

<file path="Data/Interceptors/IUserProvider.cs">
namespace Promatis.Net.Data;Đ
Đ
public interface IUserProviderĐ
{Đ
    // AÄ5D$>CB <Cä6CTB C KD$L C AC„=DT@Cä=CÔKCÂÂ 5D ;C, ?Cä;D4GCT=C„5 C„<CT=C, BD 5C CCTB Ct0CÔ@CäAC
    Task<string?> GetCurrentUserNameAsync();Đ
}
</file>

<file path="Data/Promatis.Net.Data.csproj">
<Project Sdk="Microsoft.NET.Sdk">Đ
Đ
  <PropertyGroup>Đ
    <TargetFramework>net10.0</TargetFramework>Đ
    <ImplicitUsings>enable</ImplicitUsings>Đ
```

```
    <Nullable>enable</Nullable>Ð
  </PropertyGroup>Ð
Ð
  <ItemGroup>Ð
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.8" />Ð
    <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />Ð
    <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.1" />Ð
  </ItemGroup>Ð
Ð
  <ItemGroup>Ð
    <ProjectReference Include="..\Domain\Promatis.Net.Domain.csproj" />Ð
  </ItemGroup>Ð
Ð
“Ä–FVÔw&÷W í
™<AssemblyAttribute Include="System.Runtime.CompilerServices.InternalsVisibleTo">Ð
™“Åõ  & ÖWFW# å &öÖ F–2äæWBä   Æ–6 F–öäÖöFVÂåFW7G3Âõõ  & ÖWFW# â Â ÒÒ  CÄO D$5D BCä2Cä3Câ ?D >CT:D$0 -->Ð
™</AssemblyAttribute>Ð
“Âô–FVÔw&÷W í
Ð
Ð
</Project>
</file>

<file path="Data/Triggers/Castom/AuditTrigger.cs">
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Text.Json;Ð
Ð
namespace Promatis.Net.Data;Ð
Ð
public class AuditTrigger(Ð
    IServiceScopeFactory scopeFactory, Ð
    JsonSerializerOptions jsonOptions)Ð
    : IAfterSaveTrigger<DomainObject>Ð
{Ð
    public async Task HandleAsync(EntityChangedEventArgs<DomainObject> args)Ð
    {Ð
        // AäGC„IC 5CÂ @C =D$0C”<-D$8Cò >D" 2Cä7CÄ>Cd=Cä9 CD8CÔ0CÄ8Dt5D :Cä9 Cô@Cä:D 8-Cä1Cä;CäGC¤8 EF Core (
        Type entityType = args.Entity.GetType();Ð
        if (entityType.Namespace == "Castle.Proxies" && entityType.BaseType != null)Ð
        {Ð
            entityType = entityType.BaseType;Ð
        }Ð
Ð
        // 1. BD8C´LD$@: C´>C48D CCT< D$>C´LC¤> D$5 D CD”=CäAD$8, C¤>D$>D KCR ?Cä<CTGCT=D² ” VF–M
        if (entityType.GetInterfaces().All(i => i.Name != nameof(IAudit)))Ð
            return;Ð
Ð
        // 2. BD>D <C„@D45CÂ 4C =CÔKCR 4C´O JSONÐ
        object changesToSerialize = args.State switchÐ
        {Ð
            EntityStateChangeEnum.Added => args.Changes.Select(c => newÐ
            {Ð
                c.PropertyName,Ð
                NewValue = c.CurrentValueÐ
            }),Ð
Ð
            EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted =>
args.Changes.Select(c => newÐ
            {Ð
                c.PropertyName,Ð
                OldValue = c.OriginalValue,Ð
                NewValue = c.CurrentValueÐ
            }),Ð
Ð
            EntityStateChangeEnum.Deleted => new { Message = "Aä1D¤5C¤B D44C ;CT= Cô>C´=CäAD$LDâ" ÒÍ
Ð
            _ => args.ChangesÐ
        };Ð
Ð
        // 3. B >Ct4C 5CÂ 7C ?C„ADÂ ;Cä3C
        var auditLog = new AuditLogÐ
        {Ð
            Id = Guid.NewGuid(),Ð
```

```
            EntityName = entityType.Name, // A„!Aô A  A´ AÔ : Aô8D„5CÂ GC„AD$>CR 8CÄO C¤;C AD 0 Cä@C4AD$@D4:D
            EntityId = args.Entity.Id,Ð
            Action = args.State.ToString(),Ð
            ChangedAt = args.ChangedAt,Ð
            ChangedBy = args.ChangedBy,Ð
            ChangesJson = JsonSerializer.Serialize(changesToSerialize, jsonOptions)Ð
        };Ð
Ð
        // 4. A„!Aô A  A´ AÔ : A 5Ct>Cô0D =Cä5 D >DT@C =CT=C„5 C"  AB GCT@CT7 C$KCD5C´5CÔ=D´9, C40D 0CÔBC„@Cä
        using IServiceScope scope = scopeFactory.CreateScope();Ð
Ð
        // A„ICT< DD0C @C„:D2 1C 7Cä2Cä3Câ :Cä=D$5C¤AD$0 Dt5D 5Cr ?D >C$0C”4CT@ C´>C¤0C´LCÔ>C4> ScopeÐ
        var factory = scope.ServiceProvider.GetService<IDbContextFactory<ApplicationDbContext>>();Ð
Ð
        if (factory != null)Ð
        {Ð
            await using ApplicationDbContext context = await factory.CreateDbContextAsync();Ð
            context.Set<AuditLog>().Add(auditLog);Ð
            await context.SaveChangesAsync();Ð
        }Ð
        elseÐ
        {Ð
            Console.WriteLine("[AuditTrigger][Error] AÔ5 D44C ;CäADÂ =C 9D$8 IDbContextFactory<ApplicationDbC
        }Ð
    }Ð
}
</file>

<file path="Data/Triggers/Global/FluentValidationTrigger.cs">
using FluentValidation;Ð
using FluentValidation.Results;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Data;Ð
Ð
public class FluentValidationTrigger(IServiceScopeFactory scopeFactory) // A„!Aô A  A´ AÔ : A„=Cd5C¤BC„@D45CÂ
IServiceProviderÐ
    : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>Ð
{Ð
    public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)Ð
    {Ð
        Type entityType = args.Entity.GetType();Ð
        Type validatorType = typeof(IValidator<>).MakeGenericType(entityType);Ð
Ð
        // A„!Aô A  A´ AÔ : B >Ct4C 5CÂ 8Ct>C´8D >C$0CÔ=D4N, C 5Ct>Cô0D =D4N Cä1C´0D BDÂ 2C„4C„<CäAD$8 (Scope
        // BÔBCâ 3C @C =D$8D CCTB, DtBCâ •6W'f–6U &÷f–FW" 1D44CTB "Cd8C$KCÂ" 8 D4ACô5D„=Câ @C 7D 5D„8D" =C H
        using IServiceScope scope = scopeFactory.CreateScope();Ð
Ð
        // At0Cô@C HC„2C 5CÂ 2C ;C„4C BCä@ C„7 D 2CT6CT3CâÂ 3C @C =D$8D >C$0CÔ=Câ 6C„2Cä3Câ 66÷ RÔ?D >C$0C”4C
        if (scope.ServiceProvider.GetService(validatorType) is IValidator validator)Ð
        {Ð
            var context = new ValidationContext<object>(args.Entity);Ð
            ValidationResult result = await validator.ValidateAsync(context);Ð
Ð
            if (!result.IsValid)Ð
            {Ð
                args.Cancel = true;Ð
                // BD>D <C„@D45CÂ :D 0D 8C$>CRÂ GC„AD$>CR ACä>C ICT=C„5 Cä1 CäHC„1C¤0DR 4C´O UIÐ
                args.ErrorMessage = string.Join(" | ", result.Errors.Select(e => e.ErrorMessage));Ð
            }Ð
        }Ð
    }Ð
}
</file>

<file path="Data/Triggers/Global/ReferenceTreeParentTrigger.cs">
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.EntityFrameworkCore.Metadata;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Reflection;Ð
Ð
namespace Promatis.Net.Data;Ð
Ð
/// <summary>Ð
/// B4=C„2CT@D 0C´LCÔKC’ 8CÔDD 0D BD CC¤BD4@CÔKC’ BD 8C43CT@ CD;Dò ?D >C$5D :C, FCT;CäAD$=CäAD$8 C„5D 0D EC„G
```

```
/// C, AC¤2Cä7CÔ>C’ 7C IC„BD² >D" FC„:C´8Dt5D :C„E Ct0C$8D 8CÄ>D BCT9 Cô5D 5CB ACäED 0CÔ5CÔ8CT< C" !B4 ABí
/// </summary>Ð
public class ReferenceTreeParentTrigger : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>Ð
{Ð
    private static readonly MethodInfo DbContextSetMethod = typeof(DbContext)Ð
        .GetMethods(BindingFlags.Public | BindingFlags.Instance)Ð
        .First(m => m.Name == nameof(DbContext.Set) && m.IsGenericMethod &&
m.GetParameters().Length == 0);Ð
Ð
    private static readonly MethodInfo AsNoTrackingMethod =
typeof(EntityFrameworkQueryableExtensions)Ð
        .GetMethods(BindingFlags.Public | BindingFlags.Static)Ð
        .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.AsNoTracking) &&
m.IsGenericMethod);Ð
Ð
    private static readonly MethodInfo IgnoreQueryFiltersMethod =
typeof(EntityFrameworkQueryableExtensions)Ð
        .GetMethods(BindingFlags.Public | BindingFlags.Static)Ð
        .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.IgnoreQueryFilters) &&
m.IsGenericMethod);Ð
Ð
    public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)Ð
    {Ð
        // Aô Aä!B$ AR  AT(AT A„ : Bt5D 5Cr @CTDC´5C¤AC„N Cô@Cä2CT@Dô5CÂÂ 5D BDÂ ;C, C Cä1D¤5C¤BC  AC$>C”AD$2
        // ATAC´8 D 2Cä9D BC$0 CÔ5D" r MD$>D" >C JCT:D" =CR 4CT@CT2CâÂ <D² ?D >D BCâ 2D´ECä4C„<!Ð
        PropertyInfo? parentIdProp = args.Entity.GetType().GetProperty("ParentId");Ð
        if (parentIdProp == null) return;Ð
Ð
        // Bt8D$0CT< Ct=C GCT=C„O D 2Cä9D BC" 4C„=C <C„GCTAC¤8Ð
        Guid entityId = args.Entity.Id;Ð
        Guid? parentId = (Guid?)parentIdProp.GetValue(args.Entity);Ð
Ð
        PropertyInfo? deletedAtProp = args.Entity.GetType().GetProperty("DeletedAt");Ð
Ð
        // 1. At0D”8D$0 CäB Cô@Dô<Cä3Câ AC <CäFC„BC„@Cä2C =C„OÐ
        if (parentId.HasValue && parentId == entityId)Ð
        {Ð
            args.Cancel = true;Ð
            args.ErrorMessage = "Aä1D¤5C¤B Cô5 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;CT< D 0CÄ>CÄC D 5C 5.";Ð
            return;Ð
        }Ð
Ð
        if (parentId.HasValue && parentId.Value != Guid.Empty)Ð
        {Ð
            // A„ICT< D >CD8D$5C´O C" 6† ævUG& 6¶W" AD 5CD8 C´NC KDR >C JCT:D$>C"Â C C¤>D$>D KDR ACä2CÔ0CD0CT
            object? localParent = args.Context.ChangeTracker.Entries()Ð
                .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
parentId.Value)?.Entity;Ð
Ð
            bool exists;Ð
            bool isDeleted;Ð
Ð
            if (localParent != null)Ð
            {Ð
                exists = true;Ð
                var localDeletedAt = (DateTime?)deletedAtProp?.GetValue(localParent);Ð
                isDeleted = localDeletedAt != null;Ð
            }Ð
            elseÐ
            {Ð
                // A$0D„0 Cä@C„3C„=C ;DÄ=C O D 5DD;CT:D 8Dò 4C´O C$KDt8D ;CT=C„O C¤>D =Dò 8 Query C" Tb 6÷&]
                IEntityType? entityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());Ð
                Type rootClrType = entityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();Ð
Ð
                object? rawSet =
DbContextSetMethod.MakeGenericMethod(rootClrType).Invoke(args.Context, null);Ð
                object? noTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(rootClrType).Invoke(null, [rawSet]);Ð
                object? ignoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(rootClrType).Invoke(null, [noTrackingQuery]);Ð
Ð
                IQueryable<object> query = ((IQueryable)ignoreFiltersQuery!).Cast<object>();Ð
Ð
                // A„ICT< Cä1D¤5C¤B Cô> CD8CÔ0CÄ8Dt5D :Cä<D2 AC$>C”AD$2D2 $–B" !B4 AM
```

```
                object? dbParent = await query.FirstOrDefaultAsync(x => EF.Property<Guid>(x, "Id")
== parentId.Value);Ð
Ð
                exists = dbParent != null;Ð
                var dbDeletedAt = dbParent != null ? (DateTime?)deletedAtProp?.GetValue(dbParent) :
null;Ð
                isDeleted = dbDeletedAt != null;Ð
            }Ð
Ð
            // 2. Aô@Cä2CT@C¤0 DD8Ct8Dt5D :Cä3Câ AD4ICTAD$2Cä2C =C„O D >CD8D$5C´LD :Cä3Câ CCt;C
            if (!exists)Ð
            {Ð
                args.Cancel = true;Ð
                args.ErrorMessage = $"B4:C 7C =CÔKC’ @Cä4C„BCT;DÄAC¤8C’ >C JCT:D" „”C¢ ·  &VçD–GÒ’ =CR =C 9CD
                return;Ð
            }Ð
Ð
            // 3. Aô@Cä2CT@C¤0 D BC BD4AC  <Dô3C¤>C4> D44C ;CT=C„OÐ
            if (isDeleted)Ð
            {Ð
                args.Cancel = true;Ð
                args.ErrorMessage = "AÔ5C´LCtO CÔ0Ct=C GC„BDÂ @Cä4C„BCT;CT< D44C ;CT=CÔKC’ >C JCT:D"â#½
                return;Ð
            }Ð
Ð
            // 4. A4 B4 Aä A / At B” B$  Aä" Bd A¤ Aä  (A -> B -> C -> A)Ð
            Guid? currentCheckId = parentId;Ð
            var visitedIds = new HashSet<Guid>();Ð
Ð
            while (currentCheckId.HasValue && currentCheckId.Value != Guid.Empty)Ð
            {Ð
                if (currentCheckId == entityId)Ð
                {Ð
                    args.Cancel = true;Ð
                    args.ErrorMessage = "Bd8C¤;C„GCTAC¤0Dò 7C 2C„AC„<CäAD$L: CÔ5C´LCtO Cô5D 5CÄ5D BC„BDÂ @Cä4
                    return;Ð
                }Ð
Ð
                if (!visitedIds.Add(currentCheckId.Value))Ð
                    break;Ð
Ð
                // A„ICT< DÔ;CT<CT=D" FCT?CäGC¤8 D =C GC ;C  2 ChangeTrackerÐ
                object? nextLocalElement = args.Context.ChangeTracker.Entries()Ð
                    .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
currentCheckId.Value)?.Entity;Ð
Ð
                if (nextLocalElement != null)Ð
                {Ð
                    currentCheckId = (Guid?)parentIdProp.GetValue(nextLocalElement);Ð
                }Ð
                elseÐ
                {Ð
                    Guid? id = currentCheckId;Ð
Ð
                    IEntityType? loopEntityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());Ð
                    Type loopRootClrType = loopEntityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();Ð
Ð
                    object? loopRawSet =
DbContextSetMethod.MakeGenericMethod(loopRootClrType).Invoke(args.Context, null);Ð
                    object? loopNoTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopRawSet]);Ð
                    object? loopIgnoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopNoTrackingQuery]);Ð
Ð
                    IQueryable<object> loopQuery =
((IQueryable)loopIgnoreFiltersQuery!).Cast<object>();Ð
Ð
                    object? parentNodeObj = await loopQuery.FirstOrDefaultAsync(x =>
EF.Property<Guid>(x, "Id") == id.Value);Ð
Ð
                    currentCheckId = parentNodeObj != null ?
(Guid?)parentIdProp.GetValue(parentNodeObj) : null;Ð
                }Ð
            }Ð
```

```
        }Đ
    }Đ
}
</file>

<file path="Data/TriggerService/DatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;Đ
using Microsoft.Extensions.DependencyInjection;Đ
using System.Collections.Concurrent;Đ
Đ
namespace Promatis.Net.Data;Đ
Đ
/// <summary>Đ
/// B 5C ;C„7C FC„O Dd5CÔBD 0C´LCÔ>C4> CD8D ?CTBDt5D 0 D$@C„3C45D >C"â #Cô@C 2C´OCTB D 5C48D BD 0Dd8CT9 CD>CÄ
/// C, :Cä>D 4C„=C„@D45D" DC 7D² 2C ;C„4C FC„8 (CD> D >DT@C =CT=C„O) C, CC$5CD>CÄ;CT=C„O (Cô>D ;CR ACäED 0CÔ5
/// </summary>Đ
/// <param name="serviceProvider">A :D$CC ;DÄ=D´9 C¤>CÔBCT:D B Ct0C$8D 8CÄ>D BCT9 (DI Container) D$5C¤CD”5C4>
public class DatabaseTriggerService(IServiceProvider serviceProvider) : IDatabaseTriggerServiceĐ
{Đ
    /// <summary>Đ
    /// A4;Cä1C ;DÄ=Cä5 D >C KD$8CRÂ CC$5CD>CÄ;DôND”5CR AC„AD$5CÄC Cä1 D4ACô5D„=Cä< C¤>CÄ<C„BCR AD4ICÔ>D BC,
    /// A„ACô>C´LCtCCTBD O CD;Dò @CT0C¤BC„2CÔ>C4> Cä1CÔ>C$;CT=C„O C„=D$5D DCT9D 0 (MudBlazor) C„;C, AC¤2Cä7CÔ
    /// </summary>Đ
    public static event Action<EntityStateChangeEnum, object>? OnEntityCommitted;Đ
Đ
    /// <summary>Đ
    /// B 5CTAD$@ Cô>CD?C„ADt8C¤>C"Â 2D´?Cä;CÔODäIC„ED O AD  D >DT@C =CT=C„O C„7CÄ5CÔ5CÔ8C’ „$C 7C   C ;C„4C
    /// AD5C´5C40D" ?D 8CÔ8CÄ0CTB C @C4CCÄ5CÔBD² ACä1D´BC„O C, 0C¤BD40C´LCÔKC’ 66÷ VB 6W'f–6U &÷f–FW"í
    /// </summary>Đ
    private static readonly Dictionary<Type, List<Func<EntityCancelArgsBase, IServiceProvider,
Task>>> BeforeSubscribers = new();Đ
Đ
    /// <summary>Đ
    /// B 5CTAD$@ Cô>CD?C„ADt8C¤>C"Â 2D´?Cä;CÔODäIC„ED O Aô B  AR CD ?CTHCÔ>C4> D >DT@C =CT=C„O C„7CÄ5CÔ5CÔ8C
    /// </summary>Đ
    private static readonly Dictionary<Type, List<Func<EntityChangedArgsBase, IServiceProvider,
Task>>> AfterSubscribers = new();Đ
Đ
    /// <summary>Đ
    /// Aô>D$>C¤>C 5Ct>CÔ0D =D´9 C¤MD, 8CT@C @DT8C, BC„?Cä2. A„AC¤;DäGC 5D" ?Cä2D$>D =Cä5 C„ACô>C´LCt>C$0CÔ8C
    /// Cô@C, >CÔ@CT4CT;CT=C„8 C 0Ct>C$KDR :C´0D ACä2 C, 8CÔBCT@DD5C”ACä2 Cä1D 0C 0D$KC$0CT<D´E D CD”=CäAD$5C
    /// </summary>Đ
    private static readonly ConcurrentDictionary<Type, List<Type>> HierarchyCache = new();Đ
Đ
    /// <summary>Đ
    /// B 5C48D BD 8D CCTB CD>CÄ5CÔ=D´9 D$@C„3C45D  4C´O C¤>CÔ:D 5D$=Cä3Câ BC„?C  AD4ICÔ>D BC,í
    /// A$KCtKC$0CTBD O C 2D$>CÄ0D$8Dt5D :C, ?D 8 D :C =C„@Cä2C =C„8 D 1Cä@Cä: C$> C$@CT<Dò 8CÔ8Dd8C ;C„7C FC
    /// </summary>Đ
    /// <typeparam name="TEntity">B$8Cò 4Cä<CT=CÔ>C’ AD4ICÔ>D BC,ãÂ÷G– W  & Óí
    /// <typeparam name="TTrigger">B$8Cò :C´0D AC ÔBD 8C43CT@C Â @CT0C´8CtCDäICT3Câ ;Cä3C„:D2 >C @C 1CäBC¤8.<
    public void Register<TEntity, TTrigger>()Đ
        where TEntity : classĐ
        where TTrigger : classĐ
    {Đ
        Type entityType = typeof(TEntity);Đ
        Type triggerType = typeof(TTrigger);Đ
Đ
        // Aô@Cä2CT@DÔ5CÂÂ :C :C„5 C„=D$5D DCT9D K Cd8Ct=CT=CÔ>C4> Dd8C¤;C  @CT0C´8CtCCTB CD0CÔ=D´9 D$@C„3C45
        bool isBefore = typeof(IBeforeSaveTrigger<TEntity>).IsAssignableFrom(triggerType);Đ
        bool isAfter = typeof(IAfterSaveTrigger<TEntity>).IsAssignableFrom(triggerType);Đ
Đ
        if (isBefore)Đ
        {Đ
            // BD>D <C„@D45CÂ >D$;Cä6CT=CÔCDâ ;Dô<C 4D2â  Cä=D$5C”=CT@ 'sp' Cô5D 5CD0CTBD O C" <Cä<CT=D" 2D´7
            // DtBCâ ?D 5CD>D$2D 0D”0CTB D4BCTGC¤C CD>C´3Cä6C„2D4IC„E Cä1D¤5C¤BCä2 (Captive Dependency).Đ
            AddSubscriber(BeforeSubscribers, entityType, async (argsBase, sp) =>Đ
            {Đ
                // B 0Ct@CTHC 5CÂ MC¤7CT<Cô;Dô@ D$@C„3C45D 0 C„7 C :D$CC ;DÄ=Cä3Câ 66÷ R BCT:D4ICT3Câ 7C ?D >
                if (sp.GetRequiredService<TTrigger>() is IBeforeSaveTrigger<TEntity> handler)Đ
                {Đ
                    // Aä1Cä@C GC„2C 5CÂ 1C 7Cä2D´5 C @C4CCÄ5CÔBD² 2 D BD >C4> D$8Cô8Ct8D >C$0CÔ=D´9 C¤>CÔBCT
                    var typedArgs = new EntityCancelEventArgs<TEntity>(Đ
                        (TEntity)argsBase.Entity,Đ
                        argsBase.State,Đ
                        argsBase.Changes,Đ
                        argsBase.Context);Đ
```

```
Ð
                    await handler.HandleAsync(typedArgs);Ð
Ð
                    // A$>Ct2D 0D”0CT< D 5CtCC´LD$0D$K C$KCô>C´=CT=C„O D$@C„3C45D 0 Cä1D 0D$=Câ 2 C 0Ct>C$KC’
                    argsBase.Cancel = typedArgs.Cancel;Ð
                    argsBase.ErrorMessage = typedArgs.ErrorMessage;Ð
                    argsBase.Handled = typedArgs.Handled;Ð
                }Ð
            });Ð
        }Ð
Ð
        if (isAfter)Ð
        {Ð
            AddSubscriber(AfterSubscribers, entityType, async (argsBase, sp) =>Ð
            {Ð
                if (sp.GetRequiredService<TTrigger>() is IAfterSaveTrigger<TEntity> handler)Ð
                {Ð
                    var typedArgs = new EntityChangedEventArgs<TEntity>(Ð
                        (TEntity)argsBase.Entity,Ð
                        argsBase.State,Ð
                        argsBase.Changes,Ð
                        argsBase.ChangedBy,Ð
                        argsBase.ChangedAt);Ð
Ð
                    await handler.HandleAsync(typedArgs);Ð
                    argsBase.Handled = typedArgs.Handled;Ð
                }Ð
            });Ð
        }Ð
Ð
        // ATAC´8 C¤;C AD  =CR @CT0C´8CtCCTB CÔ8 Cä4C„= C¤>CÔBD 0C¤B — DÔBCâ :D 8D$8Dt5D :C O CäHC„1C¤0 C¤>CÔ
        if (!isBefore && !isAfter)Ð
        {Ð
            throw new InvalidOperationException(Ð
                $"A¤;C AD  ·G&–vvW%G— Rää ÖWÒ =CR @CT0C´8CtCCTB IBeforeSaveTrigger C„;C, ” gFW%6 fUG&–vvW" 4C
        }Ð
    }Ð
Ð
    /// <summary>Ð
    /// A$KCô>C´=Dô5D" AC¤2Cä7CÔCDâ 2C ;C„4C FC„N C„7CÄ5CÔOCT<Cä9 D CD”=CäAD$8 Cô5D 5CB 5E  >D$?D 0C$:Cä9 C"
    /// Aä?D 0D„8C$0CTB C$ADâ FCT?CäGC¤C CÔ0D ;CT4Cä2C =C„O D CD”=CäAD$8. Aô@CT@D´2C 5D" >Cô5D 0Dd8Dâ ?D 8 CÔ
    /// </summary>Ð
    public async Task ValidateAsync(object entity, EntityStateChangeEnum state, 
List<PropertyChangeInfo> changes, DbContext context)Ð
    {Ð
        var args = new EntityCancelArgsBase(entity, state, changes, context);Ð
Ð
        // Aä1DT>CD8CÂ 8CT@C @DT8Dâ BC„?Cä2 (D 0CÄ>C’ AD4ICÔ>D BC,Â 1C 7Cä2D´E C¤;C AD >C" 8 C„=D$5D DCT9D >C
        foreach (Type type in GetTypesHierarchy(entity.GetType()))Ð
        {Ð
            if (BeforeSubscribers.TryGetValue(type, out List<Func<EntityCancelArgsBase, 
IServiceProvider, Task>>? actions))Ð
            {Ð
                foreach (Func<EntityCancelArgsBase, IServiceProvider, Task> action in actions)Ð
                {Ð
                    // Aô5D 5CD0CT< C´>C¤0C´LCÔKC’ 6W'f–6U &÷f–FW"Â A C¤>D$>D KCÂ 1D´; D >Ct4C = CD0CÔ=D´9 DÔ
                    await action(args, serviceProvider);Ð
Ð
                    // ATAC´8 D$@C„3C45D  2D´AD$0C$8C² DC´0C2 6 æ6VÂ r =CT<CT4C´5CÔ=Câ ?D 5D KC$0CT< D$@C =Ct
                    if (args.Cancel) throw new OperationCanceledException(args.ErrorMessage);Ð
                }Ð
            }Ð
        }Ð
    }Ð
Ð
    /// <summary>Ð
    /// B 0D AD´;C 5D" CC$5CD>CÄ;CT=C„O Cä1 D4ACô5D„=Cä< D >DT@C =CT=C„8 C„7CÄ5CÔ5CÔ8C’ 2 C 0CtC CD0CÔ=D´E.Ð
    /// Aô>CD4CT@Cd8C$0CTB C¤0D :C 4CÔKC’ ?CT@CTEC$0D" „DC´0C2 † æFÆVB’ 8 CôCC ;C„:D45D" ACä1D´BC„5 C" 3C´>C
    /// </summary>Ð
    public async Task NotifyAsync(object entity, EntityStateChangeEnum state, 
List<PropertyChangeInfo> changes, string? user, DateTime at)Ð
    {Ð
        var args = new EntityChangedArgsBase(entity, state, changes, user, at);Ð
Ð
        foreach (Type type in GetTypesHierarchy(entity.GetType()))Ð
        {Ð
```

```
            if (AfterSubscribers.TryGetValue(type, out List<Func<EntityChangedArgsBase,
IServiceProvider, Task>>? actions))Ð
            {Ð
                foreach (Func<EntityChangedArgsBase, IServiceProvider, Task> action in actions)Ð
                {Ð
                    await action(args, serviceProvider);Ð
Ð
                    // ATAC´8 Cä4C„= C„7 D$@C„3C45D >C" 2D´AD$0C$8C² † æFÆVB Ò G'VRÂ >C @C 1CäBC¤0 Dd5Cô>Dt:C
                    if (args.Handled) return;Ð
                }Ð
            }Ð
        }Ð
Ð
        // AôCC ;C„:C FC„O C" AD$0D$8Dt5D :C„9 DÔ2CT=D" 4C´O D 5C :D$8C$=Cä3Câ >C =Cä2C´5CÔ8Dò :Cä<Cô>CÔ5CÔBC
        OnEntityCommitted?.Invoke(state, entity);Ð
    }Ð
Ð
    /// <summary>Ð
    /// A$ACô>CÄ>C40D$5C´LCÔKC’ <CTBCä4 CD;Dò 1CT7Cä?C ACÔ>C4> CD>C 0C$;CT=C„O Cô>CD?C„ADt8C¤>C" 2 D ;Cä2C @C
    /// </summary>Ð
    private void AddSubscriber<TArgs>(Dictionary<Type, List<Func<TArgs, IServiceProvider, Task>>>
dict, Type type, Func<TArgs, IServiceProvider, Task> action)Ð
    {Ð
        if (!dict.TryGetValue(type, out List<Func<TArgs, IServiceProvider, Task>>? list))Ð
        {Ð
            list = new List<Func<TArgs, IServiceProvider, Task>>();Ð
            dict[type] = list;Ð
        }Ð
        list.Add(action);Ð
    }Ð
Ð
    /// <summary>Ð
    /// A„7C$;CT:C 5D" ?Cä;CÔCDâ 8CT@C @DT8Dâ BC„?Cä2 CD;Dò CC¤0Ct0CÔ=Cä3Câ >C JCT:D$0 (C$:C´NDt0Dò 1C 7Cä2D´
    /// B 5CtCC´LD$0D$K C¤MD„8D CDäBD O CD;Dò >C 5D ?CTGCT=C„O C$KD >C¤>C’ ?D >C„7C$>CD8D$5C´LCÔ>D BC, ?Cä4 C
    /// </summary>Ð
    private IEnumerable<Type> GetTypesHierarchy(Type type) =>Ð
        HierarchyCache.GetOrAdd(type, t => {Ð
            var types = new List<Type>();Ð
            // B 5C¤CD AC„2CÔ> Cô>CD=C„<C 5CÄADò 2C$5D E Cô> CD@CT2D2 =C AC´5CD>C$0CÔ8Dò :C´0D ACä2, C„AC¤;Dä
            for (Type? c = t; c != null && c != typeof(object); c = c.BaseType) types.Add(c);Ð
            // AD>C 0C$;DÔ5CÂ 2D 5 D 5C ;C„7Cä2C =CÔKCR 8CÔBCT@DD5C”AD² 8 D41C„@C 5CÂ 4D41C´8C¤0D$KÐ
            return types.Concat(t.GetInterfaces()).Distinct().ToList();Ð
        });Ð
Ð
    /// <summary>Ð
    /// B @CäGCÔKC’ AC @CäA D BC BC„GCTAC¤8DR @CT3C„AD$@C FC„9. Ð
    /// A„ACô>C´LCtCCTBD O Cô@CT8CÄCD”5D BC$5CÔ=Câ 2 Unit-D$5D BC E CD;Dò 8Ct>C´ODd8C, ?D >C4>CÔ>C" BCTAD$>C"
    /// </summary>Ð
    internal static void ClearInternalRegistrations()Ð
    {Ð
        BeforeSubscribers.Clear();Ð
        AfterSubscribers.Clear();Ð
        HierarchyCache.Clear();Ð
    }Ð
}
</file>

<file path="Data/TriggerService/EntityStateChangeEnum.cs">
namespace Promatis.Net.Data;Ð
Ð
public enum EntityStateChangeEnumÐ
{Ð
    Added,Ð
    Modified,Ð
    Deleted,Ð
    SoftDeletedÐ
}
</file>

<file path="Data/TriggerService/IDatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;Ð
Ð
namespace Promatis.Net.Data;Ð
Ð
public interface IDatabaseTriggerServiceÐ
{Ð
```

```
    // AÄ5D$>CB 4C´O CÔ0D BD >C”:C, AC$OCt5C’ $!D4ICÔ>D BDÂÔ"D 8C43CT@"Đ
    void Register<TEntity, TTrigger>()Đ
        where TEntity : classĐ
        where TTrigger : class;Đ
Đ
    // AÄ5D$>CDK C$KCô>C´=CT=C„O (C$KCtKC$0DäBD O C„=D$5D FCT?D$>D >CÂ•
    Task ValidateAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo>
changes, DbContext context);Đ
    Task NotifyAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo> changes,
string? user, DateTime at);Đ
}
</file>

<file path="Data/TriggerService/PropertyChangeInfo.cs">
namespace Promatis.Net.Data;Đ
Đ
public class PropertyChangeInfoĐ
{Đ
    public string PropertyName { get; set; } = string.Empty;Đ
    public object? OriginalValue { get; set; }Đ
    public object? CurrentValue { get; set; }Đ
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs">
using Microsoft.EntityFrameworkCore;Đ
Đ
namespace Promatis.Net.Data;Đ
Đ
public class EntityCancelArgsBase(Đ
    object entity,Đ
    EntityStateChangeEnum state,Đ
    List<PropertyChangeInfo> changes,Đ
    DbContext context) : EntityEventArgsBase(entity, state, changes)Đ
{Đ
    public DbContext Context { get; } = context;Đ
    public bool Cancel { get; set; }Đ
    public string? ErrorMessage { get; set; }Đ
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs">
using Microsoft.EntityFrameworkCore;Đ
Đ
namespace Promatis.Net.Data;Đ
Đ
public class EntityCancelEventArgs<T>(Đ
    T entity,Đ
    EntityStateChangeEnum state,Đ
    List<PropertyChangeInfo> changes,Đ
    DbContext context)Đ
    : EntityCancelArgsBase(entity!, state, changes, context) where T : classĐ
{Đ
    public new T Entity => (T)base.Entity;Đ
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedArgsBase.cs">
namespace Promatis.Net.Data;Đ
Đ
// A 0Ct>C$KC’ :C´0D A CD;Dò 2CÔCD$@CT=CÔ5C4> C„ACô>C´LCt>C$0CÔ8Dò 2 D 5D 2C„AC]
public class EntityChangedArgsBase(Đ
    object entity,Đ
    EntityStateChangeEnum state,Đ
    List<PropertyChangeInfo> changes,Đ
    string? changedBy,Đ
    DateTime changedAt) : EntityEventArgsBase(entity, state, changes) // <-- AÔ0D ;CT4D45CÂ 7CD5D LĐ
{Đ
    public string? ChangedBy { get; } = changedBy;Đ
    public DateTime ChangedAt { get; } = changedAt;Đ
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs">
namespace Promatis.Net.Data;Đ
Đ
```

```
// A$0D, BC„?C„7C„@Cä2C =CÔKC′ :C´0D A CD;Dò BD 8C43CT@Cä2Ð
public class EntityChangedEventArgs<T>(Ð
    T entity, EntityStateChangeEnum state, Ð
    List<PropertyChangeInfo> changes, Ð
    string? changedBy, Ð
    DateTime changedAt)Ð
    : EntityChangedArgsBase(entity!, state, changes, changedBy, changedAt) where T : classÐ
{Ð
    public new T Entity => (T)base.Entity;Ð
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs">
namespace Promatis.Net.Data;Ð
Ð
public abstract class EntityEventArgsBase(Ð
    object entity,Ð
    EntityStateChangeEnum state,Ð
    List<PropertyChangeInfo> changes) : EventArgsÐ
{Ð
    public object Entity { get; } = entity;Ð
    public EntityStateChangeEnum State { get; } = state;Ð
    public List<PropertyChangeInfo> Changes { get; } = changes;Ð
    public bool Handled { get; set; }Ð
}
</file>

<file path="Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs">
namespace Promatis.Net.Data;Ð
Ð
// AD;Dò ?D 0C$8C² $ Aä!A´ " D >DT@C =CT=C„O (D42CT4Cä<C´5CÔ8DòôHC„=C •
public interface IAfterSaveTrigger<T> where T : classÐ
{Ð
    Task HandleAsync(EntityChangedEventArgs<T> args);Ð
}
</file>

<file path="Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs">
namespace Promatis.Net.Data;Ð
Ð
// AD;Dò ?D 0C$8C² $ Aâ" ACäED 0CÔ5CÔ8Dò „2C ;C„4C FC„O)Ð
public interface IBeforeSaveTrigger<T> where T : classÐ
{Ð
    Task HandleAsync(EntityCancelEventArgs<T> args);Ð
}
</file>

<file path="Data/Validation/GlobalPolymorphicValidator.cs">
using FluentValidation;Ð
using FluentValidation.Results;Ð
using Microsoft.Extensions.DependencyInjection;Ð
Ð
namespace Promatis.Net.Domain;Ð
Ð
public class GlobalPolymorphicValidator<T> : AbstractValidator<T> where T : classÐ
{Ð
    private readonly IServiceProvider _serviceProvider;Ð
Ð
    public GlobalPolymorphicValidator(IServiceProvider serviceProvider)Ð
    {Ð
        _serviceProvider = serviceProvider ?? throw new
ArgumentNullException(nameof(serviceProvider));Ð
    }Ð
Ð
    protected override bool PreValidate(ValidationContext<T> context, ValidationResult result)Ð
    {Ð
        if (context.InstanceToValidate == null) return true;Ð
Ð
        // 1. Aô>C´CDt0CT< D 5C ;DÄ=D´9 D$8Cò >C JCT:D$0 C" @C =D$0C”<CR „=C ?D 8CÄ5D Â FW  'FÖVçEVæ–B 8C´8 A
        Type realType = context.InstanceToValidate.GetType();Ð
Ð
        // ATAC´8 D$8Cò ACä2Cô0CD0CTB D  B „B.CRâ MD$> C¤>CÔ5Dt=D´9 C¤;C AD Â 0 CÔ5 C 0Ct>C$KC′ 0C AD$@C :D$=
        // D$> C„AC¤0D$L CD>Dt5D =C„9 C$0C´8CD0D$>D  =CR =D46CÔ>, DtBCä1D² =CR CC”BC, 2 C 5D :Cä=CTGCÔCDâ @CT
        if (realType == typeof(T)) return true;Ð
Ð
        // 2. B BD >C„< D$8Cò 8CÔBCT@DD5C”AC  4C´O D$>Dt5Dt=Cä3Câ 2C ;C„4C BCä@C  „=C ?D 8CÄ5D Â •f Æ–F F÷#ÄF
```

```
        Type specificValidatorType = typeof(IValidator<>).MakeGenericType(realType);Đ
Đ
        // 3. At0Cô@C HC„2C 5CÂ 8Cr D’  B   C$0C´8CD0D$>D K CD;Dò MD$>C4> C¤>CÔ:D 5D$=Cä3Câ BC„?C
        // BÔBCâ 2C 6CÔ>, D$0C¢ :C : CD;Dò >CD=Cä3Câ BC„?C  <Cä6CTB C KD$L Ct0D 5C48D BD 8D >C$0CÔ> CÔ5D :Cä;
        List<IValidator> specificValidators =
_serviceProvider.GetServices(specificValidatorType).Cast<IValidator>().ToList();Đ
Đ
        if (specificValidators.Any())Đ
        {Đ
            // B >Ct4C 5CÂ =CRÔ4Cd5CÔ5D 8C¢ :Cä=D$5C¤AD" 2C ;C„4C FC„8 FluentValidation CD;Dò ?CT@CT4C GC, 4C
            ValidationContext<object>? newContext =
ValidationContext<object>.CreateWithOptions(context.InstanceToValidate, options =>Đ
            {Đ
                // A„!Aô A  A´ AÔ : A¤0CÔ>CÔ8Dt=D´9 CÄ5D$>CB fÇVVçEf Æ–F F–öâ 4C´O Cô@Dô<Cä9 Cô@Cä1D >D :C, A
                if (context.Selector != null)Đ
                {Đ
                    options.UseCustomSelector(context.Selector);Đ
                }Đ
            });Đ
Đ
            // At0CôCD :C 5CÂ :C 6CDKC’ =C 9CD5CÔ=D´9 D$>Dt5Dt=D´9 C$0C´8CD0D$>D  =C AC´5CD=C„:C
            foreach (IValidator validator in specificValidators)Đ
            {Đ
                // BtBCä1D² 8Ct1CT6C BDÂ 7C FC„:C´8C$0CÔ8DòÂ ?D >C$5D OCT<, DtBCâ =C 9CD5CÔ=D´9 C$0C´8CD0D$>D
                if (validator.GetType().IsGenericType &&
validator.GetType().GetGenericTypeDefinition() == typeof(GlobalPolymorphicValidator<>))Đ
                    continue;Đ
Đ
                ValidationResult specificResult = validator.Validate(newContext);Đ
Đ
                // Aô5D 5CÔ>D 8CÂ 2D 5 Cä1CÔ0D CCd5CÔ=D´5 CäHC„1C¤8 C" >C IC„9 D 5CtCC´LD$0D" 2C ;C„4C FC„8 D
                foreach (ValidationFailure error in specificResult.Errors)Đ
                {Đ
                    result.Errors.Add(error);Đ
                }Đ
            }Đ
        }Đ
Đ
        return true;Đ
    }Đ
}
</file>

<file path="Domain.Interface/Enums/EnumExtensions.cs">
using System.ComponentModel;Đ
using System.Reflection;Đ
Đ
namespace Promatis.Net.Domain.Interface;Đ
Đ
public static class EnumExtensionsĐ
{Đ
    /// <summary>Đ
    /// A$>Ct2D 0D”0CTB Ct=C GCT=C„5 C BD 8C CD$0 [Description] CD;Dò ;Dä1Cä3Câ MC´5CÄ5CÔBC  ?CT@CTGC„AC´5CÔ8
    /// ATAC´8 C BD 8C CD" >D$AD4BD BC$CCTB, C$>Ct2D 0D”0CTB D BC =CD0D BCÔ>CR 8CÄO .ToString().Đ
    /// </summary>Đ
    public static string GetDescription(this Enum value)Đ
    {Đ
        FieldInfo? field = value.GetType().GetField(value.ToString());Đ
        if (field == null) return value.ToString();Đ
Đ
        var attribute = field.GetCustomAttribute<DescriptionAttribute>();Đ
        return attribute != null ? attribute.Description : value.ToString();Đ
    }Đ
}
</file>

<file path="Domain.Interface/Interfaces/IAudit.cs">
namespace Promatis.Net.Domain.Interface;Đ
Đ
/// <summary>Đ
/// B CD”=CäAD$8, D 5C ;C„7D4ND”8CR MD$>D" 8CÔBCT@DD5C”A, C CCDCD" 7C ?C„AD´2C BDÄADò 2 AuditLogĐ
/// </summary>Đ
public interface IAuditĐ
{Đ
    Đ
}
```

```
</file>

<file path="Domain.Interface/Interfaces/IDomainObject.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
public interface IDomainObjectÐ
{Ð
    DateTime CreatedAt { get; init; }Ð
}
</file>

<file path="Domain.Interface/Interfaces/IDomainObjectHasKey.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
public interface IDomainObjectHasKey<TKey> : IDomainObjectÐ
{Ð
    TKey Id { get; set; }Ð
}
</file>

<file path="Domain.Interface/Interfaces/IEntityCloner.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
/// <summary>Ð
/// A„=D$5D DCT9D  ?C´0D$DCä@CÄ5CÔ=Cä3Câ 4C$8Cd:C  8Ct>C´8D >C$0CÔ=Cä3Câ :C´>CÔ8D >C$0CÔ8Dò 4Cä<CT=CÔKDR AD4I
/// </summary>Ð
public interface IEntityClonerÐ
{Ð
    T CloneEntity<T>(T entity) where T : class;Ð
}
</file>

<file path="Domain.Interface/Interfaces/ISoftDeletable.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
/// <summary>Ð
/// A„=D$5D DCT9D  4C´O CÄOC4:Cä3Câ CCD0C´5CÔ8Dò >C JCT:D$>C-
/// </summary>Ð
public interface ISoftDeletableÐ
{Ð
    DateTime? DeletedAt { get; set; }Ð
Ð
    string? DeletedBy { get; set; }Ð
}
</file>

<file path="Domain.Interface/Interfaces/ITreeNode.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
Ð
/// <summary>Ð
/// A4;Cä1C ;DÄ=D´9 Cô;C BDD>D <CT=CÔKC’ :Cä=D$@C :D" 4C´O C$ACTE C„5D 0D EC„GCTAC¤8DR „4D 5C$>C$8CD=D´E) D C
/// </summary>Ð
/// <typeparam name="T">B$8Cò 4Cä<CT=CÔ>C’ <Cä4CT;C,ãÂ÷G— W  & Óí
public interface ITreeNode<T> : IDomainObjectHasKey<Guid> where T : classÐ
{Ð
    /// <summary>Ð
    /// A„4CT=D$8DD8C¤0D$>D  @Cä4C„BCT;DÄAC¤>C4> D47C´0. B 0C$5CÒ çVÆÂ 4C´O C¤>D =CT2D´E DÔ;CT<CT=D$>C"í
    /// A„!Aô A  A´ AÔ : AD>C 0C$;CT= set CD;Dò 2Cä7CÄ>Cd=CäAD$8 C„7CÄ5CÔ5CÔ8Dò AC$OCt5C’ „?CT@CT<CTICT=C„O C
    /// </summary>Ð
    Guid? ParentId { get; set; }Ð
Ð
    /// <summary>Ð
    /// AÔ0C$8C40Dd8Cä=CÔ>CR AC$>C”AD$2Câ AD KC´:C, =C  @Cä4C„BCT;DÄAC¤8C’ >C JCT:D"í
    /// </summary>Ð
    T? Parent { get; set; }Ð
Ð
    /// <summary>Ð
    /// A¤>C´;CT:Dd8Dò 4CäGCT@CÔ8DR MC´5CÄ5CÔBCä2 (Cô>CDGC„=CT=CÔKDR CCt;Cä2).Ð
    /// </summary>Ð
    ICollection<T> Children { get; }Ð
}
</file>

<file path="Domain.Interface/Promatis.Net.Domain.Interface.csproj">
<Project Sdk="Microsoft.NET.Sdk">Ð
```

```
Đ
  <PropertyGroup>Đ
    <TargetFramework>net10.0</TargetFramework>Đ
    <ImplicitUsings>enable</ImplicitUsings>Đ
    <Nullable>enable</Nullable>Đ
  </PropertyGroup>Đ
Đ
</Project>
</file>

<file path="Domain/AuditLog/AuditLog.cs">
namespace Promatis.Net.Domain;Đ
Đ
/// <summary>Đ
/// A CCD8D" AC„AD$5CÄKĐ
/// </summary>Đ
public class AuditLog : DomainObjectĐ
{Đ
    /// <summary>Đ
    /// AÔ0C„<CT=Cä2C =C„5 D >C KD$8Dý
    /// </summary>Đ
    public string EntityName { get; init; } = string.Empty;Đ
Đ
    /// <summary>Đ
    /// A„4CT=D$8DD8C¤0D$>D  ACä1D´BC„OĐ
    /// </summary>Đ
    public Guid EntityId { get; init; }Đ
Đ
    /// <summary>Đ
    /// AD5C”AD$2C„5Đ
    /// </summary>Đ
    public string Action { get; init; } = string.Empty;Đ
Đ
    /// <summary>Đ
    /// AD0D$0 C„7CÄ5CÔ5CÔ8Dý
    /// </summary>Đ
    public DateTime ChangedAt { get; init; }Đ
Đ
    /// <summary>Đ
    /// A„7CÄ5CÔ5CÔ>Đ
    /// </summary>Đ
    public string? ChangedBy { get; init; }Đ
Đ
    /// <summary>Đ
    /// B BD >C¤0 C„7CÄ5CÔ5CÔ8C•
    /// </summary>Đ
    public string ChangesJson { get; init; } = string.Empty;Đ
}
</file>

<file path="Domain/DomainObject/DomainObject.cs">
namespace Promatis.Net.Domain;Đ
Đ
/// <summary>Đ
/// AäACÔ>C$=Cä9 C¤;C AD  „uT”B ?Câ CCÄ>C´GC =C„N)Đ
/// </summary>Đ
public abstract class DomainObject : DomainObjectBase<Guid>Đ
{Đ
    protected DomainObject()Đ
    {Đ
        Id = Guid.NewGuid();Đ
    }Đ
}
</file>

<file path="Domain/DomainObject/DomainObjectBase.cs">
using Promatis.Net.Domain.Interface;Đ
Đ
namespace Promatis.Net.Domain;Đ
Đ
/// <summary>Đ
/// B4=C„2CT@D 0C´LCÔKC’ 1C 7Cä2D´9 C¤;C AD  A Cô>CD4CT@Cd:Cä9 C´NC >C4> D$8Cô0 C¤;DäGC
/// </summary>Đ
/// <typeparam name="TKey">B$8Cò :C´NDt0</typeparam>Đ
public abstract class DomainObjectBase<TKey> : IDomainObjectHasKey<TKey>Đ
{Đ
```

```
    public TKey Id { get; set; } = default!;Ð
Ð
    public DateTime CreatedAt { get; init; } = DateTime.UtcNow;Ð
}
</file>

<file path="Domain/DomainObject/DomainObjectValidator.cs">
using FluentValidation;Ð
Ð
namespace Promatis.Net.Domain;Ð
Ð
/// <summary>Ð
/// B4=C„2CT@D 0C´LCÔKC’ 2C ;C„4C BCä@ CD;Dò 2D 5DR :C´0D ACä2, CÔ0D ;CT4D45CÄKDR >D" FöÖ –äö&¦V7M
/// </summary>Ð
public abstract class DomainObjectValidator<T> : AbstractValidator<T> where T : DomainObjectÐ
{Ð
    protected DomainObjectValidator()Ð
    {Ð
        RuleFor(x => x.Id)Ð
            .NotEmpty().WithMessage("A„4CT=D$8DD8C¤0D$>D  >C JCT:D$0 CÔ5 CÄ>Cd5D" 1D´BDÂ ?D4AD$KCÂâ"“½
Ð
        RuleFor(x => x.CreatedAt)Ð
            .Must(date => date <= DateTime.UtcNow.AddSeconds(1))Ð
            .WithMessage("AD0D$0 D >Ct4C =C„O CÔ5 CÄ>Cd5D" 1D´BDÂ 2 C CCDCD”5CÂâ"“½
    }Ð
}
</file>

<file path="Domain/EntityCloner/JsonEntityCloner.cs">
using System.Text.Json;Ð
using System.Text.Json.Serialization;Ð
using System.Text.Json.Serialization.Metadata;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Domain;Ð
Ð
public class JsonEntityCloner : IEntityClonerÐ
{Ð
    private static readonly JsonSerializerOptions JsonOptions = new()Ð
    {Ð
        ReferenceHandler = ReferenceHandler.IgnoreCycles,Ð
        TypeInfoResolver = new DefaultJsonTypeInfoResolverÐ
        {Ð
            Modifiers = { typeInfo =>Ð
            {Ð
                foreach (var property in typeInfo.Properties)Ð
                {Ð
                    // 1. A$KD 5Ct0CT< Dd8C¤;C„GCTAC¤8CR =C 2C„3C FC„>CÔ=D´5 D 2Cä9D BC$0 C„5D 0D EC„9Ð
                    if (property.Name is "Parent" or "Children")Ð
                    {Ð
                        property.ShouldSerialize = (_, _) => false;Ð
                        continue;Ð
                    }Ð
Ð
                    // 2. A$KD 5Ct0CT< D ;Cä6CÔKCR AC$OCt0CÔ=D´5 CD>CÄ5CÔ=D´5 Cä1D¤5C¤BD² 1DÔ:CT=CD0 C, :Cä;C
                    Type propType = property.PropertyType;Ð
                    bool isDomainClass = typeof(Domain.DomainObject).IsAssignableFrom(propType);Ð
                    bool isDomainCollection = propType.IsGenericType &&Ð

typeof(System.Collections.IEnumerable).IsAssignableFrom(propType) &&Ð

typeof(Domain.DomainObject).IsAssignableFrom(propType.GetGenericArguments().FirstOrDefault());Ð
Ð
                    if (isDomainClass || isDomainCollection)Ð
                    {Ð
                        property.ShouldSerialize = (_, _) => false;Ð
                    }Ð
                }Ð
            }}Ð
        }Ð
    };Ð
Ð
    public T CloneEntity<T>(T entity) where T : classÐ
    {Ð
        if (entity == null) throw new ArgumentNullException(nameof(entity));Ð
Ð
```

```
        // B 5D 8C ;C„7D45CÂ 8 CD5D 5D 8C ;C„7D45CÂÂ AD$@Cä3Câ ACäED 0CÔODò @CT0C´LCÔKC’ @C =D$0C”<-D$8Cò =C
        string json = JsonSerializer.Serialize(entity, entity.GetType(), JsonOptions);Đ
        return (T)JsonSerializer.Deserialize(json, entity.GetType(), JsonOptions)!;Đ
    }Đ
}
</file>

<file path="Domain/Promatis.Net.Domain.csproj">
<Project Sdk="Microsoft.NET.Sdk">Đ
Đ
  <PropertyGroup>Đ
    <TargetFramework>net10.0</TargetFramework>Đ
    <ImplicitUsings>enable</ImplicitUsings>Đ
    <Nullable>enable</Nullable>Đ
  </PropertyGroup>Đ
Đ
  <ItemGroup>Đ
    <PackageReference Include="FluentValidation" Version="12.1.1" />Đ
  </ItemGroup>Đ
Đ
  <ItemGroup>Đ
    <ProjectReference Include="..\Domain.Interface\Promatis.Net.Domain.Interface.csproj" />Đ
  </ItemGroup>Đ
Đ
</Project>
</file>

<file path="Domain/ReferenceBase/ReferenceBase.cs">
using Promatis.Net.Domain.Interface;Đ
Đ
namespace Promatis.Net.Domain;Đ
Đ
/// <summary>Đ
/// A 0Ct>C$KC’ :C´0D A CD;Dò 2D 5DR ACô@C 2CäGCÔ8C¤>C-
/// </summary>Đ
public abstract class ReferenceBase : DomainObject, ISoftDeletableĐ
{Đ
    public string Name { get; set; } = string.Empty;Đ
Đ
    public string? Description { get; set; }Đ
Đ
    public string? Code { get; set; }Đ
Đ
    public DateTime? DeletedAt { get; set; }Đ
Đ
    public string? DeletedBy { get; set; }Đ
}
</file>

<file path="Domain/ReferenceBase/ReferenceBaseValidator.cs">
using FluentValidation;Đ
using FluentValidation.Results;Đ
Đ
namespace Promatis.Net.Domain;Đ
Đ
/// <summary>Đ
/// B4=C„2CT@D 0C´LCÔKC’ 2C ;C„4C BCä@ CD;Dò 2D 5DR :C´0D ACä2, CÔ0D ;CT4D45CÄKDR >D" &VfW&Væ6T& 6]
/// </summary>Đ
public abstract class ReferenceBaseValidator<T> : DomainObjectValidator<T> where T : ReferenceBaseĐ
{Đ
    /// <summary>Đ
    /// A$8D BD40C´LCÔKC’ ?C BD$5D = D 5C4CC´OD =Cä3Câ 2D´@C 6CT=C„O CD;Dò ?D >C$5D :C,  Cä4C í
    /// Aô> D4<Cä;Dt0CÔ8Dã¢ AD$@Cä3Câ ;C BC„=C„FC  2 C$5D ECÔ5CÂ @CT3C„AD$@CRÂ FC„DD K C, òí
    /// </summary>Đ
    protected virtual string CodeRegexPattern => @"^[A-Z0-9_]*$";Đ
Đ
    /// <summary>Đ
    /// A$8D BD40C´LCÔ>CR ACä>C ICT=C„5 Cä1 CäHC„1C¤5 C$0C´8CD0Dd8C, @CT3D4;Dô@CÔ>C4> C$KD 0Cd5CÔ8Dò  Cä4C í
    /// </summary>Đ
    protected virtual string CodeValidationMessage => "A¤>CB <Cä6CTB D >CD5D 6C BDÂ BCä;DÄ:Câ ;C BC„=C„FD2 2
Đ
    protected ReferenceBaseValidator()Đ
    {Đ
        RuleFor(x => x.Name)Đ
            .NotEmpty().WithMessage("AÔ0C„<CT=Cä2C =C„5 Cä1Dô7C BCT;DÄ=Câ 4C´O Ct0Cô>C´=CT=C„O")Đ
            // Aô@Cä2CT@C¤0 CÔ0 Cô@Cä1CT;D² 2 CÔ0Dt0C´5 C, :Cä=Dd5Đ
```

```
            .Must(name => name == null || name.Trim() == name)Ð
            .WithMessage("AÔ0C„<CT=Cä2C =C„5 CÔ5 CÄ>Cd5D" =C GC„=C BDÄADò 8C´8 Ct0C¤0CÔGC„2C BDÄADò ?D >C 5C´
            .MinimumLength(2).WithMessage("AÔ0C„<CT=Cä2C =C„5 CD>C´6CÔ> D >CD5D 6C BDÂ <C„=C„<D4< 2 D 8CÄ2Cä;
            .MaximumLength(250).WithMessage("Aô@CT2D´HCT=C  <C :D 8CÄ0C´LCÔ0Dò 4C´8CÔ0 CÔ0C„<CT=Cä2C =C„O (25
Ð
        RuleFor(x => x.Code)Ð
            .Matches(x => CodeRegexPattern)Ð
            .When(x => !string.IsNullOrEmpty(x.Code))Ð
            .WithMessage(x => CodeValidationMessage);Ð
    }Ð
Ð
    public Func<object, string, Task<IEnumerable<string>>> ValidateValue => async (model,
propertyName) =>Ð
    {Ð
        ValidationResult? result = await
ValidateAsync(ValidationContext<T>.CreateWithOptions((T)model, x =>
x.IncludeProperties(propertyName)));Ð
        return result.IsValid ? [] : result.Errors.Select(e => e.ErrorMessage);Ð
    };Ð
}
</file>

<file path="Domain/ReferenceTreeBase/ReferenceTreeBase.cs">
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Domain;Ð
Ð
/// <summary>Ð
/// A 0Ct>C$KC’ :C´0D A CD;Dò 2D 5DR ACô@C 2CäGCÔ8C¤>C" ACä4CT@Cd0D”8DR 4CT@CT2DÄOÐ
/// </summary>Ð
/// <summary>Ð
/// A 0Ct>C$KC’ :C´0D A CD;Dò 2D 5DR 4D 5C$>C$8CD=D´E D ?D 0C$>Dt=C„:Cä2 (MDM).Ð
/// A„ACô>C´LCtCCTB CÔ0D$BCT@CÒ 5%E  4C´O Cô5D 5CD0Dt8 D BD >C4> D$8Cô8Ct8D >C$0CÔ=D´E CÔ0C$8C40Dd8Cä=CÔKDR A
/// </summary>Ð
/// <typeparam name="T">B$8Cò :Cä=C¤@CTBCÔ>C4> CD>CÄ5CÔ=Cä3Câ >C JCT:D$0 (CÔ0D ;CT4CÔ8C¤0).</typeparam>Ð
public abstract class ReferenceTreeBase<T> : ReferenceBase, ITreeNode<T> where T : classÐ
{Ð
    /// <summary>Ð
    /// A„4CT=D$8DD8C¤0D$>D  @Cä4C„BCT;DÄAC¤>C4> D47C´0 C" !B4 ABí
    /// </summary>Ð
    public Guid? ParentId { get; set; }Ð
Ð
    /// <summary>Ð
    /// AÔ0C$8C40Dd8Cä=CÔ>CR AC$>C”AD$2Câ AD KC´:C, =C  @Cä4C„BCT;DÄAC¤8C’ >C JCT:D" 2 Cä?CT@C BC„2CÔ>C’ ?C <
    /// Aô5D 5Cä?D 5CD5C´OCTBD O C¤0C¢ f—'GV Â 2 C¤>CÔ5Dt=D´E C¤;C AD 0DR 4C´O Cô>CD4CT@Cd:C, Æ §’ Æö F–ærí
    /// </summary>Ð
    public virtual T? Parent { get; set; }Ð
Ð
    /// <summary>Ð
    /// A¤>C´;CT:Dd8Dò 4CäGCT@CÔ8DR MC´5CÄ5CÔBCä2 (Cô>CDGC„=CT=CÔKDR CCt;Cä2).Ð
    /// </summary>Ð
    public virtual ICollection<T> Children { get; set; } = new List<T>();Ð
}
</file>

<file path="Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs">
using FluentValidation;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Domain;Ð
Ð
/// <summary>Ð
/// B4=C„2CT@D 0C´LCÔKC’ 2C ;C„4C BCä@ CD;Dò 2D 5DR :C´0D ACä2, CÔ0D ;CT4D45CÄKDR >D" &VfW&Væ6UG&VT& 6]
/// </summary>Ð
public abstract class ReferenceTreeBaseValidator<T> : ReferenceBaseValidator<T>Ð
    where T : ReferenceBase, ITreeNode<T>Ð
{Ð
    protected ReferenceTreeBaseValidator() : base()Ð
    {Ð
        // Aô@Cä2CT@C¤0 CÔ0 D 0CÄ>Dd8D$8D >C$0CÔ8CR „>C JCT:D" =CR <Cä6CTB D4:C 7D´2C BDÂ =C  ACT1Dò :C : CÔ0
        RuleFor(x => x.ParentId)Ð
            .Must((model, parentId) => parentId != model.Id)Ð
            .WithMessage("Aä1D¤5C¤B CÔ5 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;CT< D 0CÄ>CÄC D 5C 5");Ð
Ð
        // A 0Ct>C$0Dò ?D >C$5D :C  DCä@CÄ0D$0 GUIDÐ
        RuleFor(x => x.ParentId)Ð
```

```
            .NotEqual(Guid.Empty)Đ
            .When(x => x.ParentId.HasValue)Đ
            .WithMessage("B >CD8D$5C´LD :C„9 C„4CT=D$8DD8C¤0D$>D  =CR <Cä6CTB C KD$L CôCD BD´< GUID");Đ
    }Đ
}
</file>

<file path="Promatis.Net.UI/_Imports.razor">
@using Microsoft.AspNetCore.Components.FormsĐ
@using Microsoft.AspNetCore.Components.RoutingĐ
@using Microsoft.AspNetCore.Components.WebĐ
@using MudBlazorĐ
@using Promatis.Net.UI.Components.WorkspacesĐ
@using Promatis.Net.Domain.InterfaceĐ
@using Promatis.Net.UI.Components.Dialogs
</file>

<file path="Promatis.Net.UI/Components/ActionContext/DataContext.cs">
using Microsoft.Extensions.DependencyInjection;Đ
using Promatis.Net.Service;Đ
Đ
namespace Promatis.Net.UI.Components;Đ
Đ
/// <summary>Đ
/// A 1D BD 0C¤BCÔ>CR 8CÔDD 0D BD CC¤BD4@CÔ>CR OCD@Câ @C 1CäBD² A CD0CÔ=D´<C,â
/// A„=C¤0CôAD4;C„@D45D" @C 1CäBD2 1D >C¤5D 0, C¤MD„0 C, ACäAD$>Dô=C„O Ct0C4@D47C¤8, Cä1D”0DôADÂ A D 5D 2C„AC
/// </summary>Đ
public abstract class DataContext<TEntity, TKey, TQueryState, TResultData> : WorkspaceContext,Đ
    IHasSelectedData<TEntity>,Đ
    IDisposableĐ
    where TEntity : class, new()Đ
    where TKey : notnullĐ
{Đ
    private readonly IBaseService<TEntity, TKey> _baseService;Đ
    private TEntity? _selectedData;Đ
    private bool _isLoading;Đ
    private bool _isDisposed;Đ
Đ
    public IUiDataBroker<TEntity, TQueryState, TResultData> Broker { get; }Đ
    public IUiOzuCache<TEntity> OzuCache { get; }Đ
Đ
    public TEntity? SelectedDataĐ
    {Đ
        get => _selectedData;Đ
        setĐ
        {Đ
            if (_selectedData != value)Đ
            {Đ
                _selectedData = value;Đ
                OnContextUpdated?.Invoke();Đ
                NotifyStateChanged();Đ
            }Đ
        }Đ
    }Đ
Đ
    public Action? OnContextUpdated { get; set; }Đ
    public override bool IsLoading => _isLoading;Đ
Đ
    protected DataContext(Đ
        IServiceProvider serviceProvider,Đ
        bool isInMemoryMode,Đ
        Action? onDataChangedNotifier = null)Đ
        : base()Đ
    {Đ
        if (serviceProvider == null) throw new ArgumentNullException(nameof(serviceProvider));Đ
Đ
        _baseService = serviceProvider.GetRequiredService<IBaseService<TEntity, TKey>>();Đ
        OzuCache = serviceProvider.GetRequiredService<IUiOzuCache<TEntity>>();Đ
        Broker = serviceProvider.GetRequiredService<IUiDataBroker<TEntity, TQueryState,
TResultData>>();Đ
Đ
        // A„!Aô A  A´ AÔ : A$>Ct2D 0D”0CT< C¤>CÔDC„3D4@C FC„N CÔ0 CTQ Ct0C¤>CÔ=Cä5 CÄ5D BCââ  D >C¤5D  3CäBC
        if (isInMemoryMode)Đ
        {Đ
            Broker.ConfigureInMemoryMode(OzuCache, LoadInitialDataForBrokerAsync,
EvaluateDataStateInMemory);Đ
```

```
        }Đ
        elseĐ
        {Đ
            Broker.ConfigureServerMode(FetchDataFromServerAsync);Đ
        }Đ
    }Đ
Đ
    /// <summary>Đ
    /// AT4C„=C O C, CCÔ8C$5D AC ;DÄ=C O D$>Dt:C  7C ?D >D 0 CD0CÔ=D´E CD;Dò 1C 7Cä2Cä3Câ ECä;D BC  AD$@C =C„
    /// A$:C´NDt0CTB C$8CtCC ;DÄ=D´9 C„=CD8C¤0D$>D  7C 3D CCt:C, …ö—4Æö F–ær’ :C : CD;Dò 6W'fW$ÖöFRÂ BC : C,
    /// </summary>Đ
    public async Task<TResultData> GetDataAsync(TQueryState state, CancellationToken ct = default)Đ
    {Đ
        if (_isLoading) return default!;Đ
Đ
        tryĐ
        {Đ
            _isLoading = true;Đ
            NotifyStateChanged(); // Aô>C¤0CtKC$0CT< C¤@D4BC„;C¤C Ct0C4@D47C¤8 CÔ0 DÔ:D 0CÔ5Đ
Đ
            return await Broker.FetchDataAsync(state, ct);Đ
        }Đ
        finallyĐ
        {Đ
            _isLoading = false;Đ
            NotifyStateChanged(); // B :D KC$0CT< C¤@D4BC„;C¤C Cô>D ;CR >D$@C„ACä2C¤8 CD0CÔ=D´EĐ
        }Đ
    }Đ
Đ
    /// <summary>Đ
    /// B ;D46CT1CÔKC’ ;CT=C„2D´9 Cô>D BC 2D”8C¢ 4C =CÔKDR 4C´O A @Cä:CT@C  „–äÖVÖ÷'’ ÖöFR’í
    /// </summary>Đ
    private async Task<List<TEntity>> LoadInitialDataForBrokerAsync(TQueryState state,
CancellationToken ct)Đ
    {Đ
        tryĐ
        {Đ
            return await _baseService.GetAllAsync(ct) ?? [];Đ
        }Đ
        catch (OperationCanceledException)Đ
        {Đ
            return [];Đ
        }Đ
    }Đ
Đ
    protected abstract TResultData EvaluateDataStateInMemory(TQueryState state,
IReadOnlyList<TEntity> inMemoryList);Đ
    protected abstract Task<TResultData> FetchDataFromServerAsync(TQueryState state,
CancellationToken ct);Đ
    protected IBaseService<TEntity, TKey> GetBaseService() => _baseService;Đ
Đ
    public virtual void Dispose()Đ
    {Đ
        if (_isDisposed) return;Đ
Đ
        Broker?.Dispose();Đ
        _isDisposed = true;Đ
    }Đ
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/EntityContext.cs">
using Microsoft.Extensions.DependencyInjection;Đ
using MudBlazor;Đ
using Promatis.Net.Domain.Interface;Đ
Đ
namespace Promatis.Net.UI.Components;Đ
Đ
/// <summary>Đ
/// A„=DD@C AD$@D4:D$CD =Cä5 Dô4D > D 0C >D$K D  AD4ICÔ>D BDô<C, 8 Cä?CT@C FC„OCÄ8 CRUD.Đ
/// A 5Ct>Cô0D =Câ 8Ct>C´8D CCTB CÄCD$0Dd8C, 4C =CÔKDRÂ CCô@C 2C´OCTB CÄ>CD0C´LCÔKCÄ8 Cä:CÔ0CÄ8 C, CCD0C´5CÔ8
/// </summary>Đ
public abstract class EntityContext<TEntity, TKey, TQueryState, TResultData>Đ
    : DataContext<TEntity, TKey, TQueryState, TResultData>Đ
    where TEntity : class, new()Đ
    where TKey : notnullĐ
```

```
{Đ
    protected IDialogService DialogService { get; }Đ
    private readonly IEntityCloner _entityCloner;Đ
Đ
    protected EntityContext(Đ
        IServiceProvider serviceProvider,Đ
        bool isInMemoryMode,Đ
        Action? onDataChangedNotifier = null)Đ
        : base(serviceProvider, isInMemoryMode, onDataChangedNotifier)Đ
    {Đ
        // A 5Ct>Cô0D =Cä5 CÔ0D$8C$=Cä5 C„7C$;CTGCT=C„5 D ;D46C K CD8C ;Cä3Cä2 MudBlazor Dt5D 5Cr vWE&W V—&VE
        DialogService = serviceProvider.GetRequiredService<IDialogService>();Đ
Đ
        // A„=C$5D AC„O Ct0C$8D 8CÄ>D BCT9. At0Cô@C HC„2C 5CÂ :C´>CÔ5D  GCT@CT7 C„=D$5D DCT9D  8Cr D•
        _entityCloner = serviceProvider.GetRequiredService<IEntityCloner>();Đ
    }Đ
Đ
    /// <summary>Đ
    /// A¤>CÄ0CÔ4C   D 8CÔED >CÔ=Cä3Câ !Cä7CD0CÔ8Dò  C ?C„AC,í
    /// A„=C„FC„0C´8Ct8D CCTB Dt8D BD´9 DÔ:Ct5CÄ?C´OD  AD4ICÔ>D BC, 8 Cô5D 5CD0CTB D4?D 0C$;CT=C„5 C" 0C AD$@
    /// </summary>Đ
    protected async Task ExecuteCreateRecordAsync()Đ
    {Đ
        await OpenDialogWindowAsync(new TEntity(), isNew: true);Đ
    }Đ
Đ
    /// <summary>Đ
    /// A¤>CÄ0CÔ4C   D 8CÔED >CÔ=Cä3Câ  CT4C :D$8D >C$0CÔ8Dò  C ?C„AC,í
    /// A 5Ct>Cô0D =Câ 8Ct>C´8D CCTB CÄCD$0Dd8Dâ 4C =CÔKDR GCT@CT7 C 1D BD 0C¤BCÔKC’ 6ÆöæW" >D" >D =Cä2CÔ>C’
    /// </summary>Đ
    protected async Task ExecuteEditRecordAsync(TEntity? selectedRow)Đ
    {Đ
        if (selectedRow == null) return;Đ
Đ
        // A4;D41Cä:Cä5 C¤;Cä=C„@Cä2C =C„5 D$5Cô5D L Ct0D”8D”5CÔ> C„=D$5D DCT9D >CÂ „@CTHC 5D" ?D >C ;CT<D2 F
        TEntity clone = _entityCloner.CloneEntity(selectedRow);Đ
Đ
        await OpenDialogWindowAsync(clone, isNew: false);Đ
    }Đ
Đ
    /// <summary>Đ
    /// A¤>CÄ0CÔ4C   D 8CÔED >CÔ=Cä3Câ #CD0C´5CÔ8Dò  C ?C„AC,í
    /// At0Cô@C HC„2C 5D" =C BC„2CÔ>CR ?Cä4D$2CT@Cd4CT=C„5 C, 2D´?Cä;CÔOCTB Cä?CT@C FC„N Dt5D 5Cr 8CÔBCT@DD5C
    /// </summary>Đ
    protected async Task ExecuteDeleteRecordAsync(TEntity? selectedRow)Đ
    {Đ
        if (selectedRow == null) return;Đ
Đ
        bool? confirm = await DialogService.ShowMessageBoxAsync(Đ
            "B44C ;CT=C„5 Ct0Cô8D 8",Đ
            "A$K D42CT@CT=D²Â GD$> DT>D$8D$5 C 5Ct2Cä7C$@C BCÔ> D44C ;C„BDÂ 2D´1D 0CÔ=D4N Ct0Cô8D L?",Đ
            yesText: "B44C ;C„BDÂ"Í
            noText: "AäBCÄ5CÔ0");Đ
Đ
        if (confirm == true)Đ
        {Đ
            // A„7C$;CT:C 5CÂ AD$@Cä3Câ BC„?C„7C„@Cä2C =CÔKC’ :C´NDr GCT@CT7 Cô@Cä2CT@C¤C C„=D$5D DCT9D 0 IDo
            if (selectedRow is IDomainObjectHasKey<TKey> hasKeyEntity)Đ
            {Đ
                TKey idValue = hasKeyEntity.Id;Đ
Đ
                // Aô>C´CDt0CT< CD>D BD4? C¢ ACT@C$8D C CD0CÔ=D´E C„7 C$5D ECÔ5C4> D ;CäO Dô4D 0 C, 2D´7D´2C
                await GetBaseService().DeleteAsync(idValue);Đ
            }Đ
            elseĐ
            {Đ
                // B 5Ct5D 2CÔKC’ AD$@Cä3Câ BC„?C„7C„@Cä2C =CÔKC’ ?D4BDÂÂ 5D ;C, 8CÔBCT@DD5C”A CÔ5 D 5C ;C„7C
                // AÔ> C @DT8D$5C¤BD4@CÔ> DD8C¤AC„@D45CÂÂ GD$> CD>CÄ5CÔ=D´5 Cä1D¤5C¤BD² >C OCt0CÔK C„<CTBDÂ :
                throw new InvalidOperationException(Đ
                    $"B CD”=CäAD$L '{typeof(TEntity).Name}' CÔ5 D 5C ;C„7D45D" >C OCt0D$5C´LCÔKC’ 8CÔBCT@DD5C
            }Đ
        }Đ
    }Đ
Đ
    /// <summary>Đ
    /// A 1D BD 0C¤BCÔ0Dò BCäGC¤0 C$KCt>C$0 Cä:CÔ0 DD>D <D²â
```

```
    /// B ?CTFC„DC„:D2 2D´7Cä2C  „:C :Cä9 C„<CT=CÔ> Razor-C¤>CÄ?Cä=CT=D" 4C„0C´>C40 CäBC¤@D´BDÂ’ 7C 4C AD" :C
    /// </summary>Ð
    protected abstract Task OpenDialogWindowAsync(TEntity model, bool isNew);Ð
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/GridContext.cs">
using Microsoft.Extensions.DependencyInjection;Ð
using MudBlazor;Ð
using Promatis.Net.Service;Ð
Ð
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// Aô@Cä<CT6D4BCäGCÔKC’ :Cä=D$5C¤AD" ?D 5CDAD$0C$;CT=C„O CD;Dò BC 1C´8Dt=D´E D BD 0CÔ8Db „w&-B ÖöFR’í
/// AÄ0CÔ8D" 0C AD$@C :D$=D´5 CD0CÔ=D´5 Dô4D 0 C" AD$@Cä3Câ BC„?C„7C„@Cä2C =CÔKCR AD$@D4:D$CD K Cô0C48CÔ0Dd8C
/// </summary>Ð
public abstract class GridContext<TEntity, TKey>Ð
    : EntityContext<TEntity, TKey, GridState<TEntity>, GridData<TEntity>>Ð
    where TEntity : class, new()Ð
    where TKey : notnullÐ
{Ð
    protected GridContext(Ð
        IServiceProvider serviceProvider,Ð
        bool isInMemoryMode,Ð
        Action? onDataChangedNotifier = null)Ð
        : base(serviceProvider, isInMemoryMode, onDataChangedNotifier)Ð
    {Ð
        // A„!Aô A  A´ AÔ : A„=C$5D AC„O Ct0C$8D 8CÄ>D BCT9. A„7C$;CT:C 5CÂ AD$@C BCT3C„N CÄCD$0Dd8C’  At# Dt
        var strategy = serviceProvider.GetRequiredService<IOzuMutationStrategy<TEntity>>();Ð
        OzuCache.SetMutationStrategy(strategy);Ð
    }Ð
Ð
    /// <summary>Ð
    /// A„!Aô A  A´ AÔ : B 5C ;C„7C FC„O C´>C¤0C´LCÔ>C’ DC„;DÄBD 0Dd8C,Â ACä@D$8D >C$:C, 8 Cô0C48CÔ0Dd8C, 2 C
    /// AÄ5D$>CB ?D 8CÔ8CÄ0CTB C 5Ct>Cô0D =D´9 IReadOnlyList C, 8D ?Cä;DÄ7D45D" =C BC„2CÔKCR <CTBCä4D² ×VD&Æ
    /// </summary>Ð
    protected override GridData<TEntity> EvaluateDataStateInMemory(GridState<TEntity> state,
IReadOnlyList<TEntity> inMemoryList)Ð
    {Ð
        // Aô@CT2D 0D”0CT< C 5Ct>Cô0D =D´9 D ?C„ACä: C" Ä”å Ô2D´@C 6CT=C„5Ð
        IQueryable<TEntity> query = inMemoryList.AsQueryable();Ð
Ð
        // 1. Aô@C„<CT=DÔ5CÂ ?Cä;DÄ7Cä2C BCT;DÄAC¤8CR DC„;DÄBD K C¤>C´>CÔ>C¢ ×VD&Æ ¦÷" =C  ;CTBD=
        if (state.FilterDefinitions != null && state.FilterDefinitions.Any())Ð
        {Ð
            query = query.Where(state.FilterDefinitions);Ð
        }Ð
Ð
        // 2. Aô@C„<CT=DÔ5CÂ 4C„=C <C„GCTAC¤CDâ ACä@D$8D >C$:D2 :Cä;Cä=Cä: MudBlazorÐ
        if (state.SortDefinitions != null && state.SortDefinitions.Any())Ð
        {Ð
            query = query.OrderBy(state.SortDefinitions);Ð
        }Ð
Ð
        // A$KDt8D ;DÔ5CÂ >C ICT5 C¤>C´8Dt5D BC$> Ct0CÔ8D 5C’Â ?D >D„5CDHC„E DD8C´LD$@C FC„N, CD> D 0Ct4CT;CT
        int totalCount = query.Count();Ð
Ð
        // 3. A$KD 5Ct0CT< D$5C¤CD”CDâ AD$@C =C„FD2 4C =CÔKDR „ C 3C„=C FC„O)Ð
        List<TEntity> pagedItems = queryÐ
            .Skip(state.Page * state.PageSize)Ð
            .Take(state.PageSize)Ð
            .ToList();Ð
Ð
        return new GridData<TEntity>Ð
        {Ð
            Items = pagedItems,Ð
            TotalItems = totalCountÐ
        };Ð
    }Ð
Ð
    /// <summary>Ð
    /// B 5C ;C„7C FC„O Dt5D BCÔ>C4> D 5D 2CT@CÔ>C4> Cô>D BD 0CÔ8Dt=Cä3Câ 7C ?D >D 0 C¢ !B4 AB …6W'fW" ÖöFR’í
    /// </summary>Ð
    protected override async Task<GridData<TEntity>> FetchDataFromServerAsync(GridState<TEntity>
state, CancellationToken ct)Ð
```

```
    {Ð
        // A$KCtKC$0CT< C 0Ct>C$KC’ ACT@C$8D  4C =CÔKDRÂ BD 0CÔAC´8D CDò AD$5C”B MudBlazor C" ?C @C <CTBD K C
        PagedResult<TEntity> pagedResult = await GetBaseService().GetPagedAsync(state.Page,
state.PageSize, ct);Ð
Ð
        return new GridData<TEntity>Ð
        {Ð
            Items = pagedResult.Items,Ð
            TotalItems = pagedResult.TotalCountÐ
        };Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/IWorkspaceContext.cs">
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// AT4C„=D´9 C¤>CÔBD 0C¤B C¤>CÔBCT:D BC  @C 1CäGCT9 Cä1C´0D BC,Â CCô@C 2C´ODäIC„9 C45Cä<CTBD 8CT9 CôOD$8 Ct>
/// D >D BC 2Cä< CD>D BD4?CÔKDR T’Ô4CT9D BC$8C’ 8 D$5C¤CD”8CÂ 2D´4CT;CT=C„5CÂ 4C =CÔKDR =C  MC¤@C =CRí
/// </summary>Ð
public interface IWorkspaceContextÐ
{Ð
    // --- B  Aä  B4 B  A$ AT A„/ A¤ AÄ Aä AT B$ AÄ  A,  A  AÔ+AÄ  ---Ð
Ð
    /// <summary>Ð
    /// A¤>C´;CT:Dd8Dò 8CÔBCT@C :D$8C$=D´E DÔ;CT<CT=D$>C" CCô@C 2C´5CÔ8Dò 4C =CÔ>C4> DT>C´AD$0.Ð
    /// </summary>Ð
    IEnumerable<IUiControl> Controls { get; }Ð
Ð
    event Action? OnContextStateChanged;Ð
    void NotifyStateChanged();Ð
Ð
    /// <summary>Ð
    /// AT4C„=C O D$>Dt:C  8CÔ8Dd8C ;C„7C FC„8 Cd8Ct=CT=CÔ>C4> Dd8C¤;C  ;Dä1Cä3Câ :Cä=D$5C¤AD$0.Ð
    /// A$KCtKC$0CTBD O C 0Ct>C$KCÂ ECä;D BCä< (WorkspacePage) Cô@C, AD$0D BCR DCä@CÄK.Ð
    /// </summary>Ð
    void InitializeContext();Ð
Ð
    /// <summary>Ð
    /// A„!Aô A  A´ AÔ : AT4C„=C O D$>Dt:C  0D 8CÔED >CÔ=Cä9 Ct0C4@D47C¤8 CÄ5D$0CD0CÔ=D´E DD8C´LD$@Cä2 C, ACÔ
    /// A$KCtKC$0CTBD O C 2D$>CÄ0D$8Dt5D :C, 1C 7Cä2D´< DT>C´AD$>CÂ !B$ Aä Aâ  Aä!A´  Cô5D 2C„GCÔ>C4> D 5CÔ4C
    /// </summary>Ð
    Task LoadMetadataAsync(CancellationToken ct);Ð
Ð
    /// <summary>Ð
    /// A„!Aô A  A´ AÔ : BD;C 3 C„=CD8C¤0Dd8C, 7C 3D CCt:C, ?Cä4CÔOD" =C  AC <D´9 C$5D E C¤>CÔBD 0C¤BC Í
    /// DtBCä1D² 1C 7Cä2D´9 DT>C´AD" AD$@C =C„FD² <Cä3 D 5C :D$8C$=Câ >D$>C @C 6C BDÂ :D CD$8C´:D2 ×VD÷fW&Æ ’
    /// </summary>Ð
    bool IsLoading { get; }Ð
Ð
    // --- Aô B  AÄ B$ B² !B$ A´ At Bd A,   A4 Aä AT"B  A, R  Aä  ---Ð
    int PaperElevation { get; }Ð
    string PaperClass { get; }Ð
    string WorkspaceHeight { get; }Ð
    string TopZoneHeight { get; }Ð
    string BottomZoneHeight { get; }Ð
    string LeftZoneWidth { get; }Ð
    string RightZoneWidth { get; }Ð
Ð
    // A„!Aô A  A´ AÔ : B$5Cô5D L C„=D$5D DCT9D  4CT:C´0D 8D CCTB Cô>C´=CäFCT=CÔ>CR @CT0C¤BC„2CÔ>CR ?Cä2CT4CT
    bool IsTopZoneCollapsed { get; set; }Ð
    bool IsBottomZoneCollapsed { get; set; }Ð
    bool IsLeftZoneCollapsed { get; set; }Ð
    bool IsRightZoneCollapsed { get; set; }Ð
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/ReferenceContext.cs">
using Promatis.Net.Domain;Ð
using Promatis.Net.UI.Controls;Ð
Ð
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// A 8Ct=CTA-A 3D 5C40D$>D  4C´O C$ACTE Cô;CäAC¤8DR ACô@C 2CäGCÔ8C¤>C"  B   D 8D BCT<D²í
```

```
/// BD8C¤AC„@D45D" BC„?D² 4C =CÔKDRÂ 2C¤;DäGC 5D" –äÖVÖ÷'' ÖöFR 8 C 2D$>CÄ0D$8Dt5D :C, ACä1C„@C 5D" BC„?Cä2Cä
/// </summary>Ð
public abstract class ReferenceContext<TEntity> : GridContext<TEntity, Guid>Ð
    where TEntity : ReferenceBase, new()Ð
{Ð
    // A„!AÔ A  A´ AÔ : AÔ0D 0CÄ5D$@ isInMemoryMode D$5CÔ5D L C48C :Câ CCô@C 2C´OCTBD O Dt5D 5Cr :Cä=D BD CC¤
    // Cô>Ct2Cä;DôO D ?D 0C$>Dt=C„:C < AÔ!A, 8D ?Cä;DÄ7Cä2C BDÂ 2D 5 Cô@CT8CÄCD”5D BC$0 C¤MD„8D >C$0CÔ8Dò 2 A
    protected ReferenceContext(Ð
        IServiceProvider serviceProvider,Ð
        bool isInMemoryMode,Ð
        Action? onDataChangedNotifier = null)Ð
        : base(serviceProvider, isInMemoryMode, onDataChangedNotifier)Ð
    {Ð
    }Ð
Ð
    /// <summary>Ð
    /// A 2D$>CÄ0D$8Dt5D :C, 2D´7D´2C 5D$ADò 1C 7Cä2D´< DT>C´AD$>CÂ …v÷&·7  6U  vR’ 2 D >C KD$8C, öä–æ–F– Æ–¦
    /// C¤>C44C  3D 0DB >C JCT:D$>C" =C AC´5CD=C„:Cä2 D46CR 3C @C =D$8D >C$0CÔ=Câ ?CäAD$@Cä5CÒ 2 Cô0CÄOD$8.Ð
    /// </summary>Ð
    public override void InitializeContext()Ð
    {Ð
        PopulateDefaultCrudToolbar();Ð
    }Ð
Ð
    /// <summary>Ð
    /// A 2D$>CÄ0D$8Dt5D :C O D 1Cä@C¤0 Cô0CÔ5C´8 D4?D 0C$;CT=C„O.Ð
    /// A¤>CÔ5Dt=D´5 D ?D 0C$>Dt=C„:C, ?D 8 Cd5C´0CÔ8C, <Cä3D4B Cô5D 5Cä?D 5CD5C´8D$L CÄ5D$>CBÂ >Dt8D BC„BDÂ
    /// </summary>Ð
    protected virtual void PopulateDefaultCrudToolbar()Ð
    {Ð
        // A¤=Cä?C¤0 B >Ct4C BDÂ r 2D´7D´2C 5D" 3CäBCä2D4N Cô@Cä2Cä4C¤C Dô4D 0Ð
        AddControl(new CreateEntityButton<TEntity>().OnExecute(ExecuteCreateRecordAsync));Ð
Ð
        // A„!AÔ A  A´ AÔ : AÔ>C´=CäAD$LDâ 8Ct1C 2C„;C„ADÂ >D" 0CÔ>CÔ8CÄ=D´E C´OCÄ1CD0-Ct0CÄKC¤0CÔ8C’  7-æ2 ‡
        // B AD´;C¤8 CÔ0 C 0Ct>C$KCR 5%TBÔ:Cä<C =CDK CÔ5D 5CD0DäBD O CÔ0Cô@Dô<D4N, D BD 0DTCDò t2 >D" CD$5Dt5
        AddControl(new EditEntityButton<TEntity>().OnExecute(ExecuteEditRecordAsync));Ð
Ð
        // A¤=Cä?C¤0 B44C ;C„BDÂ r 7C ?D 0D„8C$0CTB MessageBox C, 2D´?Cä;CÔOCTB C¤>CÄ0CÔ4D2 CCD0C´5CÔ8Dý
        AddControl(new DeleteEntityButton<TEntity>().OnExecute(ExecuteDeleteRecordAsync));Ð
Ð
        // B BC =CD0D BCÔKC’ 2C„7D40C´LCÔKC’ @C 7CD5C´8D$5C´LÐ
        AddControl(new ToolbarDivider());Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/ReferenceTreeContext.cs">
using Promatis.Net.Domain;Ð
using Promatis.Net.UI.Controls;Ð
Ð
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// A 8Ct=CTA-A 3D 5C40D$>D  4C´O C$ACTE CD@CT2Cä2C„4CÔKDR „8CT@C @DT8Dt5D :C„E) D ?D 0C$>Dt=C„:Cä2 AÔ!A, AC„
/// BD8C¤AC„@D45D" BC„?D² 4C =CÔKDRÂ =C AD$@C 8C$0CTB D 5C¤CD AC„2CÔKC’  At#-C¤MD, 8 C 2D$>CÄ0D$8Dt5D :C, ACä
/// </summary>Ð
public abstract class ReferenceTreeContext<TEntity> : TreeContext<TEntity, Guid>Ð
    where TEntity : ReferenceTreeBase<TEntity>, new()Ð
{Ð
    // A„!AÔ A  A´ AÔ : A¤>CÔAD$@D4:D$>D  AD$0C² :D 8D BC ;DÄ=Câ GC„AD$KCÂâ  C AD$@Cä9C¤0 D BD 0D$5C48C’ <D4B
    // C, 2D´7Cä2D² 2C„@D$CC ;DÄ=D´E CÄ5D$>CD>C" >D$ADä4C  ?Cä;CÔ>D BDÄN D44C ;CT=D²
    protected ReferenceTreeContext(Ð
        IServiceProvider serviceProvider,Ð
        Action? onDataChangedNotifier = null)Ð
        : base(serviceProvider, isInMemoryMode: true, onDataChangedNotifier)Ð
    {Ð
    }Ð
Ð
    /// <summary>Ð
    /// A„!AÔ A  A´ AÔ : A 5Ct>CÔ0D =D´9 CÄ5D$>CB 6C„7CÔ5CÔ=Cä3Câ FC„:C´0 C¤>CÔBCT:D BC í
    /// A 2D$>CÄ0D$8Dt5D :C, 2D´7D´2C 5D$ADò 1C 7Cä2D´< DT>C´AD$>CÂ …v÷&·7  6U  vR’ 2 D >C KD$8C, öä–æ–F– Æ–¦
    /// C40D 0CÔBC„@D4O Ct0D”8D$C C¤;C AD >C"Ô=C AC´5CD=C„:Cä2 CäB NullReferenceException.Ð
    /// </summary>Ð
    public override void InitializeContext()Ð
    {Ð
        PopulateDefaultTreeToolbar();Ð
```

```
    }Ð
Ð
    /// <summary>Ð
    /// A 2D$>CÄ0D$8Dt5D :C O D 1Cä@C¤0 Cô0CÔ5C´8 D4?D 0C$;CT=C„O CD;Dò 4CT@CT2C í
    /// </summary>Ð
    protected virtual void PopulateDefaultTreeToolbar()Ð
    {Ð
        // 1. A¤=Cä?C¤0 D >Ct4C =C„O C¤>D =CT2Cä3Câ MC´5CÄ5CÔBC  „1CT7 D >CD8D$5C´O)Ð
        AddControl(new CreateEntityButton<TEntity> { Title = "AD>C 0C$8D$L C¤>D 5CÔL" }Ð
            .OnExecute(ExecuteCreateRecordAsync));Ð
Ð
        // 2. A¤=Cä?C¤0 D >Ct4C =C„O CD>Dt5D =CT3Câ ?Cä4D47C´0 (C 2D$>CÄ0D$8Dt5D :C, 7C 2Dô7C =C  =C  =C ;C„G
        AddControl(new CreateChildButton<TEntity>().OnExecute(ExecuteCreateChildRecordAsync));Ð
Ð
        // 3. A„!Aô A  A´ AÔ : Aô@Dô<C O Cô5D 5CD0Dt0 D AD´;Cä: CÔ0 CÄ5D$>CDK C 5Cr 0CÔ>CÔ8CÄ=D´E C´OCÄ1CD0-C
        AddControl(new EditEntityButton<TEntity>().OnExecute(ExecuteEditRecordAsync));Ð
Ð
        // 4. A¤=Cä?C¤0 D44C ;CT=C„O (MessageBox Cô>CDBC$5D 6CD5CÔ8Dò 8 C¤>CÄ0CÔ4C  CCD0C´5CÔ8Dò 2 B #A  )Ð
        AddControl(new DeleteEntityButton<TEntity>().OnExecute(ExecuteDeleteRecordAsync));Ð
Ð
        // B BC =CD0D BCÔKC’ 2C„7D40C´LCÔKC’ @C 7CD5C´8D$5C´LÐ
        AddControl(new ToolbarDivider());Ð
    }Ð
Ð
    /// <summary>Ð
    /// B ?CTFC„DC„GCÔ0Dò 1C„7CÔ5D Ô:Cä<C =CD0 D >Ct4C =C„O Cô>CDGC„=CT=CÔ>C4> D47C´0.Ð
    /// B GC„BD´2C 5D" ”B 2D´4CT;CT=CÔ>C4> D >CD8D$5C´O C, 7C ?C„AD´2C 5D" 5C4> C"   &VçD–B =Cä2Cä9 D CD”=CäA
    /// </summary>Ð
    protected async Task ExecuteCreateChildRecordAsync()Ð
    {Ð
        if (SelectedData == null) return;Ð
Ð
        // B >Ct4C 5CÂ GC„AD$KC’ >C JCT:D"Â 8D ?Cä;DÄ7D4O CÔ>C$KC’ AC„=D$0C¤AC„A C# Dd5C´5C$>C4> D$8Cô0, Ð
        // C, ?D 8C$OCtKC$0CT< CT3Câ : D$5C¤CD”5CÄC C$KCD5C´5CÔ=Cä<D2 CCt;D=
        TEntity childModel = new()Ð
        {Ð
            ParentId = SelectedData.IdÐ
        };Ð
Ð
        await OpenDialogWindowAsync(childModel, isNew: true);Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/TreeContext.cs">
using Microsoft.Extensions.DependencyInjection;Ð
Ð
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// Aô@Cä<CT6D4BCäGCÔKC’ :Cä=D$5C¤AD" ?D 5CDAD$0C$;CT=C„O CD;Dò 4D 5C$>C$8CD=D´E D BD 0CÔ8Db …G&VR ÖöFR’í
/// Aô5D 5C$>CD8D" 0C AD$@C :D$=D´5 CD0CÔ=D´5 Dô4D 0 CÔ0 Dô7D´: C„5D 0D EC„GCTAC¤8DR 2C„7D40C´8Ct0D$>D >C"í
/// </summary>Ð
public abstract class TreeContext<TEntity, TKey>Ð
    : EntityContext<TEntity, TKey, object, IReadOnlyList<TEntity>> Ð
    where TEntity : class, new()Ð
    where TKey : notnullÐ
{Ð
    protected TreeContext(Ð
        IServiceProvider serviceProvider,Ð
        bool isInMemoryMode,Ð
        Action? onDataChangedNotifier = null)Ð
        : base(serviceProvider, isInMemoryMode, onDataChangedNotifier)Ð
    {Ð
        // A„!Aô A  A´ AÔ : A„=C$5D AC„O Ct0C$8D 8CÄ>D BCT9. A„7C$;CT:C 5CÂ 8CT@C @DT8Dt5D :D4N D BD 0D$5C48D
        var treeStrategy = serviceProvider.GetRequiredService<IOzuMutationStrategy<TEntity>>();Ð
        OzuCache.SetMutationStrategy(treeStrategy);Ð
    }Ð
Ð
    /// <summary>Ð
    /// A„!Aô A  A´ AÔ : AÄ5D$>CB ?D 8CÔ8CÄ0CTB C, 2Cä7C$@C IC 5D" 1CT7Cä?C ACÔKC’ •&V DöæÇ”Æ–7Bí
    /// AD;Dò 4CT@CT2C  2 CÔ0CÄOD$8 CÄK Cô@CäAD$> C$>Ct2D 0D”0CT< C$5D L CÔ0C¤>Cô;CT=CÔKC’ 3D 0DB >C JCT:D$>C
    /// </summary>Ð
    protected override IReadOnlyList<TEntity> EvaluateDataStateInMemory(object state,
IReadOnlyList<TEntity> inMemoryList)Ð
    {Ð
```

```
        return inMemoryList;Đ
    }Đ
Đ
    /// <summary>Đ
    /// Bt5D BCÔKC′ ACT@C$5D =D´9 Ct0Cô@CäA CD;Dò 4CT@CT2C â  C 3D CCd0CTB C$5D L C´8CÔ5C″=D´9 D ?C„ACä: Ct0C
    /// CÔ0 CäACÔ>C$5 C¤>D$>D >C4> C$8CtCC ;C„7C BCä@ Cô>D BD >C„B C4@C D D 2Dô7CT9 ParentId -> Children.Đ
    /// </summary>Đ
    protected override async Task<IReadOnlyList<TEntity>> FetchDataFromServerAsync(object state,
CancellationToken ct)Đ
    {Đ
        // A$KCt>C" ?D 8C$5CD5CÒ 2 D >CäBC$5D$AD$2C„5 D  ACT@C$8D =D´< C¤>CÔBD 0C¤BCä< C 5Cr ?C @C <CTBD >C-
        List<TEntity> result = await GetBaseService().GetAllAsync();Đ
        return result ?? [];Đ
    }Đ
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/WorkspaceContext.cs">
namespace Promatis.Net.UI.Components;Đ
Đ
/// <summary>Đ
/// A 0Ct>C$KC′ :Cä=D$5C¤AD" ;Dä1Cä9 D 0C >Dt5C′ >C ;C AD$8 (C$:C´0CD:C,′ 2 D 8D BCT<CRí
/// Aä1D″8C′ 7CÔ0CÄ5CÔ0D$5C´L CD;Dò <CÔ5CÄ>D ECT<, D ?D 0C$>Dt=C„:Cä2, CD0D„1Cä@CD>C" 8 C4@C DC„:Cä2.Đ
/// </summary>Đ
public class WorkspaceContext : IWorkspaceContextĐ
{Đ
    // Aô@C„<CT=Dô5CÂ :Cä;C´5C¤FC„>CÔ=D´5 C$KD 0Cd5CÔ8Dò 2=
    protected readonly List<IUiControl> _controls = [];Đ
Đ
    // A„ACô>C´LCtCCT< CÔ>C$KC′ Æö6² 4C´O Ct0D″8D$K C¤>C´;CT:Dd8C, >D" DCä=Cä2D´E Cô>D$>C¤>C" „BD 8C43CT@Cä2
    private readonly Lock _lockObject = new();Đ
Đ
    public IEnumerable<IUiControl> ControlsĐ
    {Đ
        getĐ
        {Đ
            // AäBCD0CT< D =C ?D„>D" 2Câ 8Ct1CT6C =C„5 InvalidOperationException Cô@C, @CT=CD5D 8CÔ3CR 2Câ 2D
            lock (_lockObject)Đ
            {Đ
                return _controls.ToArray();Đ
            }Đ
        }Đ
    }Đ
Đ
    public event Action? OnContextStateChanged;Đ
    public void NotifyStateChanged() => OnContextStateChanged?.Invoke();Đ
Đ
    /// <summary>Đ
    /// A 0Ct>C$0Dò ?D4AD$0Dò @CT0C´8Ct0Dd8Dòâ  CäBCä<Cä: CÄ>Cd5D" ?CT@CT>Cô@CT4CT;C„BDÂ 5E
    /// CD;Dò 0C$BCä<C BC„GCTAC¤>C′ AC >D :C, BD4;C 0D >C" 8C´8 CÔ0Dt0C´LCÔ>C′ =C AD$@Cä9C¤8.Đ
    /// </summary>Đ
    public virtual void InitializeContext()Đ
    {Đ
        // Aô> D4<Cä;Dt0CÔ8Dâ =C„GCT3Câ =CR 4CT;C 5D"Â ?D 5CD>D BC 2C´ODò ECä;D B CD;Dò @C AD„8D 5CÔ8Dý
    }Đ
Đ
    /// <summary>Đ
    /// A„!Aô A  A´ AÔ : A 0Ct>C$0Dò ?D4AD$0Dò 7C 3C´CD„:C  0D 8CÔED >CÔ=Cä3Câ 6C„7CÔ5CÔ=Cä3Câ FC„:C´0.Đ
    /// B ;Cä6CÔKCR :Cä=D$5C¤AD$K CäBDt5D$>C" 8 CdCD =C ;Cä2 (CÔ0Cô@C„<CT@, AuditLogContext) Cô5D 5Cä?D 5CD5C
    /// CTQ CD;Dò ;CT=C„2Cä3Câ =C ?Cä;CÔ5CÔ8Dò ACô8D :Cä2 C$KC >D 0 DD8C´LD$@Cä2 Cô>D ;CR >D$@C„ACä2C¤8 C¤0D
    /// </summary>Đ
    public virtual Task LoadMetadataAsync(CancellationToken ct)Đ
    {Đ
        return Task.CompletedTask;Đ
    }Đ
Đ
    /// <summary>Đ
    /// A„!Aô A  A´ AÔ : A 0Ct>C$>CR ACäAD$>Dô=C„5 DD;C 3C  4C´O D BC BC„GCTAC¤8DR AD$@C =C„F.Đ
    /// AD8CÔ0CÄ8Dt5D :C„5 C¤>CÔBCT:D BD² „F F 6öçFW‡B′ ?CT@CT>Cô@CT4CT;DôB CT3Câ AC$>CT9 true/false C´>C48C¤
    /// </summary>Đ
    public virtual bool IsLoading => false;Đ
Đ
    // --- AD BD A´"AÔ Bò  A !B$ Aä A¤  A4 Aä AT"B  A,   B "A„ AT  ---Đ
    public virtual int PaperElevation => 1;Đ
    public virtual string PaperClass => "pa-4 d-flex flex-column flex-grow-1 w-100 h-100";Đ
    public virtual string WorkspaceHeight => "100%";Đ
```

```
    public virtual string TopZoneHeight => "auto";Đ
    public virtual string BottomZoneHeight => "auto";Đ
    public virtual string LeftZoneWidth => "250px";Đ
    public virtual string RightZoneWidth => "300px";Đ
Đ
    // A„!Aô A  A´ AÔ : B 2Cä9D BC$0 Ct>CÒ BCT?CT@DÂ ?D 8CÔCCD8D$5C´LCÔ> Cô8CÔ0DäB UI Cô@C, 8Ct<CT=CT=C„8 D >
    public bool IsTopZoneCollapsedĐ
    {Đ
        get;Đ
        setĐ
        {Đ
            if (field == value) return;Đ
            field = value;Đ
            NotifyStateChanged();Đ
        }Đ
    }Đ
Đ
    public bool IsBottomZoneCollapsedĐ
    {Đ
        get;Đ
        setĐ
        {Đ
            if (field == value) return;Đ
            field = value;Đ
            NotifyStateChanged();Đ
        }Đ
    }Đ
Đ
    public bool IsLeftZoneCollapsedĐ
    {Đ
        get;Đ
        setĐ
        {Đ
            if (field == value) return;Đ
            field = value;Đ
            NotifyStateChanged();Đ
        }Đ
    }Đ
Đ
    public bool IsRightZoneCollapsedĐ
    {Đ
        get;Đ
        setĐ
        {Đ
            if (field == value) return;Đ
            field = value;Đ
            NotifyStateChanged();Đ
        }Đ
    }Đ
Đ
    // --- Aô B$ A¤ A  At Aô B  B´  AÄ B$ AD+ B4 B  A$ AT A„/ Aô AÔ A´,Bâ  A´/ Aô B$ AÄ Aä  ---Đ
    protected void AddControl(IUiControl control)Đ
    {Đ
        if (control == null) throw new ArgumentNullException(nameof(control));Đ
Đ
        lock (_lockObject)Đ
        {Đ
            _controls.Add(control);Đ
        }Đ
        NotifyStateChanged();Đ
    }Đ
Đ
    protected void RemoveControl(string controlId)Đ
    {Đ
        lock (_lockObject)Đ
        {Đ
            _controls.RemoveAll(c => c.Id == controlId);Đ
        }Đ
        NotifyStateChanged();Đ
    }Đ
}
</file>

<file path="Promatis.Net.UI/Components/Dialogs/BaseDialogLayout.razor">
@typeparam TModel where TModel : classĐ
Đ
```

```
<MudDialog>Ð
    <TitleContent>Ð
        <MudText Typo="Typo.h6">Ð
            @TitleÐ
        </MudText>Ð
    </TitleContent>Ð
Ð
    <DialogContent>Ð
        @* A„=D$5C4@C„@D45CÂ fÇVVçEf Æ–F F–öâ =C ?D OCÄCDâ 2 CÔ0D$8C$=D4N DD>D <D2 ×VD&Æ ¦÷" ¤
        <MudForm @ref="_form"Ð
                 Model="@Model"Ð
                 Validation="@(async (object model) => await ValidateModelViaFluentAsync())"Ð
                 ValidationDelay="0">Ð
Ð
            @* B BD >C48C′ 3CT>CÄ5D$@C„GCTAC¤8C′ :Cä=D$5C″=CT@ Dô4D 0 — Ct0D″8D″0CTB DD>D <D2 >D" ?D KCd:Cä2
            <div class="d-flex flex-column pt-2" style="min-height: 340px; height: 60vh; max-
height: 60vh; min-width: 0; overflow: hidden;">Ð
                @ChildContentÐ
            </div>Ð
Ð
        </MudForm>Ð
    </DialogContent>Ð
Ð
    <DialogActions>Ð
        @* AÔ0D$8C$=Cä5 C¤0D :C 4CÔ>CR CCô@C 2C´5CÔ8CR >C¤=Cä< MudBlazor *@Ð
        <MudButton Variant="Variant.Text"Ð
                   Color="Color.Default"Ð
                   OnClick="HandleCancelAsync">Ð
            AäBCÄ5CÔ0Ð
        </MudButton>Ð
Ð
        <MudButton Variant="Variant.Filled"Ð
                   Color="Color.Primary"Ð
                   OnClick="HandleSubmitAsync">Ð
            B >DT@C =C„BDÍ
        </MudButton>Ð
    </DialogActions>Ð
</MudDialog>
</file>

<file path="Promatis.Net.UI/Components/Dialogs/BaseDialogLayout.razor.cs">
using FluentValidation;Ð
using Microsoft.AspNetCore.Components;Ð
using MudBlazor;Ð
Ð
namespace Promatis.Net.UI.Components.Dialogs;Ð
Ð
public partial class BaseDialogLayout<TModel> : ComponentBase where TModel : classÐ
{Ð
    protected MudForm _form = null!;Ð
Ð
    [CascadingParameter]Ð
    protected IMudDialogInstance MudDialog { get; set; } = null!;Ð
Ð
    [Parameter]Ð
    public string Title { get; set; } = "B 5CD0C¤BC„@Cä2C =C„5 Ct0Cô8D 8";Ð
Ð
    [Parameter]Ð
    public required TModel Model { get; set; }Ð
Ð
    [Parameter]Ð
    public IValidator<TModel>? Validator { get; set; }Ð
Ð
    /// <summary>Ð
    /// A¤>CÔBCT=D"Â :CäBCä@D´9 C CCD5D" @C 7C$5D =D4B C$=D4BD 8 DD>D <D² „CCÔ8C¤0C´LCÔKCR ?Cä;Dò 2C$>CD0 C„;
    /// </summary>Ð
    [Parameter]Ð
    public required RenderFragment ChildContent { get; set; }Ð
Ð
    protected override void OnInitialized()Ð
    {Ð
        base.OnInitialized();Ð
        if (Model == null)Ð
            throw new ArgumentNullException(nameof(Model), "Aô0D 0CÄ5D$@ Model Dô2C´OCTBD O Cä1Dô7C BCT;DÄ=D´
    }Ð
Ð
```

```
    public async Task HandleSubmitAsync()Đ
    {Đ
        if (_form == null) return;Đ
Đ
        // AÔ0D$8C$=Câ AC¤0CÔ8D CCT< C$ADâ DCä@CÄC D 8C´0CÄ8 MudBlazorĐ
        await _form.ValidateAsync();Đ
Đ
        if (_form.IsValid)Đ
        {Đ
            // At0C¤@D´2C 5CÂ >C¤=Câ 8 C$>Ct2D 0D”0CT< C$0C´8CD=D4N CÄ>CD5C´L CÔ0 D BD 0CÔ8DdCĐ
            MudDialog.Close(DialogResult.Ok(Model));Đ
        }Đ
    }Đ
Đ
    public void HandleCancelAsync()Đ
    {Đ
        MudDialog.Cancel();Đ
    }Đ
Đ
    protected async Task<IEnumerable<string>> ValidateModelViaFluentAsync()Đ
    {Đ
        if (Validator == null) return Array.Empty<string>();Đ
Đ
        var result = await Validator.ValidateAsync(Model);Đ
        if (result.IsValid) return Array.Empty<string>();Đ
Đ
        // AäBCD0CT< MudForm Cô;CäAC¤8C’ ACô8D >C¢ BCT:D BCä2D´E CäHC„1Cä: CD;Dò ?Cä4D 2CTBC¤8 C„=CôCD$>C-
        return result.Errors.Select(e => e.ErrorMessage);Đ
    }Đ
}
</file>

<file path="Promatis.Net.UI/Components/Dialogs/DialogTabConfig.cs">
using Microsoft.AspNetCore.Components;Đ
Đ
namespace Promatis.Net.UI.Components.Dialogs;Đ
Đ
/// <summary>Đ
/// B BD >C4> D$8Cô8Ct8D >C$0CÔ=D´9 C¤;C AD Ô:Cä=DD8C4CD 0D$>D  4C´O CD8CÔ0CÄ8Dt5D :C„E C$:C´0CD>C¢í
/// Aô>Ct2Cä;Dô5D" ?CT@CT4C 2C BDÂ :C AD$>CÄ=D4N D 0Ct<CTBC¤C Cô>C´5C’ 2C$>CD0, Cô@C„2Dô7C =CÔCDâ : Cd8C$>C’
/// </summary>Đ
public class DialogTabConfig<TModel> where TModel : classĐ
{Đ
    /// <summary>Đ
    /// B$5C¤AD$>C$KC’ 7C 3Cä;Cä2Cä: C$:C´0CD:C, =C  ?C =CT;C, ×VEF '2í
    /// </summary>Đ
    public string Title { get; }Đ
Đ
    /// <summary>Đ
    /// B 8D BCT<CÔ0Dò 8C¤>CÔ:C  ×VD&Æ ¦÷" 4C´O Ct0C4>C´>C$:C  2C¤;C 4C¤8 (Cä?Dd8Cä=C ;DÄ=Câ’í
    /// </summary>Đ
    public string? Icon { get; }Đ
Đ
    /// <summary>Đ
    /// B BD >C4> D$8Cô8Ct8D >C$0CÔ=C O D 0Ct<CTBC¤0 Cô>C´5C’ 2C$>CD0, Cô@C„=C„<C ND”0Dò 6C„2D4N CD>CÄ5CÔ=D4N
    /// </summary>Đ
    public RenderFragment<TModel> Content { get; }Đ
Đ
    public DialogTabConfig(string title, RenderFragment<TModel> content, string? icon = null)Đ
    {Đ
        if (string.IsNullOrWhiteSpace(title))Đ
            throw new ArgumentException("At0C4>C´>C$>C¢ 2C¤;C 4C¤8 CÔ5 CÄ>Cd5D" 1D´BDÂ ?D4AD$KCÂâ"Â æ ÖVöb‡F—
Đ
        Title = title;Đ
        Content = content ?? throw new ArgumentNullException(nameof(content));Đ
        Icon = icon;Đ
    }Đ
}
</file>

<file path="Promatis.Net.UI/Components/Dialogs/ReferenceDialogLayout.razor">
@typeparam TModel where TModel : Promatis.Net.Domain.ReferenceBaseĐ
Đ
@* Aä1Cä@C GC„2C 5CÂ 2D Q C" 1C 7Cä2D´9 C¤0D :C A Dô4D 0, Cô@Cä1D 0D KC$0Dò <Cä4CT;DÂÂ 7C 3Cä;Cä2Cä: C, 2C ;C
<BaseDialogLayout TModel="TModel"Đ
                  Model="@Model"Đ
```

```
                 Title="@Title"Ð
                 Validator="@Validator">Ð
Ð
    @* A$=D4BD 5CÔ=C„9 C¤>CÔBCT=D" :C @C¤0D 0: Cä?D 5CD5C´OCT<, CÔCCd=Câ ;C, @C 7C$>D 0Dt8C$0D$L C$:C´0CD:C,
    @if (CustomTabs == null || !CustomTabs.Any())Ð
    {Ð
        @* B &AT A  A„  A ¢  C AD$>CÄ=D´E C$:C´0CD>C¢ =CTB. A$KC$>CD8CÂ BCä;DÄ:Câ 1C 7Cä2D´5 Cô>C´O D ?D 0C$>
        <div class="flex-grow-1" style="height: 100%; overflow-y: auto; padding-right: 4px;">Ð
            @RenderBaseFields()Ð
        </div>Ð
    }Ð
    elseÐ
    {Ð
        @* B &AT A  A„  A ¢  C AC´5CD=C„: Cô5D 5CD0C² AC$>C, 2C¤;C 4C¤8. B 0Ct2Cä@C GC„2C 5CÂ <CÔ>C4>D BD 0CÔ
        <MudTabs Elevation="0"Ð
                Rounded="true"Ð
                Class="d-flex flex-column flex-grow-1 h-100"Ð
                @bind-ActivePanelIndex="_activeTabIndex">Ð
Ð
            @* B 8D BCT<CÔ0Dò  C¤;C 4C¤0 1: Aô>C´O ReferenceBase CäB Dô4D 0 D 8D BCT<D² ¤
            <MudTabPanel Text="AäACÔ>C$=Cä5" Icon="@Icons.Material.Filled.Info" PanelClass="pa-4 flex-grow-1"
                <div style="max-height: 48vh; overflow-y: auto; padding-right: 4px;">Ð
                    @RenderBaseFields()Ð
                </div>Ð
            </MudTabPanel>Ð
Ð
            @* AD8CÔ0CÄ8Dt5D :C„5 A$:C´0CD:C, "³¢  CT@CT1C„@C 5CÂ 8 D 5CÔ4CT@C„< C$ACR :C AD$>CÄ=D´5 D$0C K D
            @foreach (DialogTabConfig<TModel> tab in CustomTabs)Ð
            {Ð
                <MudTabPanel Text="@tab.Title" Icon="@tab.Icon" PanelClass="pa-4 flex-grow-1">Ð
                    <div style="max-height: 48vh; overflow-y: auto; padding-right: 4px;">Ð
                        @tab.Content(Model)Ð
                    </div>Ð
                </MudTabPanel>Ð
            }Ð
Ð
        </MudTabs>Ð
    }Ð
Ð
</BaseDialogLayout>Ð
Ð
@code {Ð
    /// <summary>Ð
    /// B BC =CD0D BC„7C„@Cä2C =CÔ0Dò @C 7CÄ5D$:C  1C 7Cä2D´E Cô>C´5C′  B  , Cä1D”0Dò 4C´O C$ACTE 100+ D ?D 0
    /// </summary>Ð
    private RenderFragment RenderBaseFields() => __builder =>Ð
    {Ð
        <div class="d-flex flex-column gap-4">Ð
            <MudTextField @bind-Value="Model.Name"Ð
                          Label="AÔ0C„<CT=Cä2C =C„5"Ð
                          For="@(() => Model.Name)"Ð
                          Required="true" />Ð
Ð
            <MudTextField @bind-Value="Model.Code"Ð
                          Label="B 8D BCT<CÔKC′ :Cä4 (B4=C„:C ;DÄ=D´9)"Ð
                          For="@(() => Model.Code)" />Ð
Ð
            <MudTextField @bind-Value="Model.Description"Ð
                          Label="Aä?C„AC =C„5 / AD>Cô>C´=C„BCT;DÄ=Cä5 Cô@C„<CTGC =C„5"Ð
                          Lines="4"Ð
                          For="@(() => Model.Description)" />Ð
        </div>Ð
    };Ð
}
</file>

<file path="Promatis.Net.UI/Components/Dialogs/ReferenceDialogLayout.razor.cs">
using FluentValidation;Ð
using Microsoft.AspNetCore.Components;Ð
using Promatis.Net.Domain;Ð
Ð
namespace Promatis.Net.UI.Components.Dialogs;Ð
Ð
public partial class ReferenceDialogLayout<TModel> : ComponentBase where TModel : ReferenceBaseÐ
{Ð
    protected int _activeTabIndex;Ð
```

```
Đ
    /// <summary>Đ
    /// B$5C¤AD$>C$KC’ 7C 3Cä;Cä2Cä: C" HC ?C¤5 CÄ>CD0C´LCÔ>C4> Cä:CÔ0. Aô@Cä1D 0D KC$0CTBD O CÔ0Cô@Dô<D4N C"
    /// </summary>Đ
    [Parameter]Đ
    public string Title { get; set; } = "B 5CD0C¤BC„@Cä2C =C„5 D ?D 0C$>Dt=C„:C #½
Đ
    /// <summary>Đ
    /// Ad8C$0Dò @CT4C :D$8D CCT<C O CD>CÄ5CÔ=C O CÄ>CD5C´L (CÔ0D ;CT4CÔ8C¢ &VfW&Væ6T& 6R’í
    /// </summary>Đ
    [Parameter]Đ
    public required TModel Model { get; set; }Đ
Đ
    /// <summary>Đ
    /// Aô>C´8CÄ>D DCÔKC’ fÇVVçBÔ2C ;C„4C BCä@ CD;Dò :Cä=C¤@CTBCÔ>C4> D ?D 0C$>Dt=C„:C í
    /// </summary>Đ
    [Parameter]Đ
    public IValidator<TModel>? Validator { get; set; }Đ
Đ
    /// <summary>Đ
    /// A¤>C´;CT:Dd8Dò AD$@Cä3Câ BC„?C„7C„@Cä2C =CÔKDR :C AD$>CÄ=D´E C$:C´0CD>C¢ „4CTAC¤@C„?D$>D >C"’Â
    /// C¤>D$>D KCR :Cä=C¤@CTBCÔKC’ ACô@C 2CäGCÔ8C¢ ECäGCTB CD>C 0C$8D$L CÔ0 DD>D <D2 2 CD>Cô>C´=CT=C„5 C¢ 2C
    /// </summary>Đ
    [Parameter]Đ
    public IReadOnlyCollection<DialogTabConfig<TModel>>? CustomTabs { get; set; }Đ
Đ
    protected override void OnInitialized()Đ
    {Đ
        base.OnInitialized();Đ
        if (Model == null)Đ
            throw new ArgumentNullException(nameof(Model), "Aô0D 0CÄ5D$@ Model Dô2C´OCTBD O Cä1Dô7C BCT;DÄ=D´
    }Đ
}
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor">
@inherits RenderBaseĐ
<MudTooltip Text="@(Control.Tooltip ?? Control.Title)" Arrow="true" Placement="Placement.Bottom">Đ
    <MudButton Variant="@Variant"Đ
               StartIcon="@(Control.IsRunning ? null : Control.Icon)"Đ
               Disabled="@(!Control.IsEnabled || Control.IsRunning || !
Control.IsEnabledForData(CurrentSelectedData))"Đ
               Color="@Color" Đ
               OnClick="@HandleClickAsync">Đ
Đ
        @if (Control.IsRunning)Đ
        {Đ
            <MudProgressCircular Class="ms-n1 me-2" Size="Size.Small" Indeterminate="true"
Color="Color.Inherit" />Đ
        }Đ
Đ
        @Control.TitleĐ
    </MudButton>Đ
</MudTooltip>
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor.cs">
using Microsoft.AspNetCore.Components;Đ
using MudBlazor;Đ
Đ
namespace Promatis.Net.UI.Components.ElementRenderBase;Đ
Đ
public partial class ButtonRenderBase : RenderBaseĐ
{Đ
    [Parameter] public Color Color { get; set; } = Color.Inherit;Đ
    [Parameter] public Variant Variant { get; set; } = Variant.Text;Đ
Đ
    protected async Task HandleClickAsync()Đ
    {Đ
        await Control.TriggerAsync(CurrentSelectedData);Đ
    }Đ
}
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor">
@inherits RenderBaseĐ
```

```
Ð
<MudDateRangePicker Label="@Control.Title"Ð
                    DateRange="@GetDateRange()"Ð
                    DateRangeChanged="@HandleDateRangeChangedAsync"Ð
                    Disabled="@(!Control.IsEnabled || Control.IsRunning)"Ð
                    Margin="Margin.Dense"Ð
                    DateFormat="dd.MM.yyyy"Ð
                    Variant="Variant.Outlined"Ð
                    Style="max-width: 260px;" />
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor.cs">
using MudBlazor;Ð
Ð
namespace Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
public partial class DateRangeRenderBase : RenderBaseÐ
{Ð
    /// <summary>Ð
    /// A 5Ct>CÔ0D =Câ 8Ct2C´5C¤0CTB Cä1D¤5C¤B DateRange C„7 CÄ>CD5C´8 C¤>CÔBD >C´0 Dt5D 5Cr 8CÔBCT@DD5C”A-D
    /// </summary>Ð
    protected DateRange? GetDateRange()Ð
    {Ð
        if (Control is IHasValue valueProvider)Ð
        {Ð
            return valueProvider.Value as DateRange;Ð
        }Ð
        return null;Ð
    }Ð
Ð
    /// <summary>Ð
    /// At0Cô8D KC$0CTB CÔ>C$>CR 7CÔ0Dt5CÔ8CR 2D´1D 0CÔ=Cä3Câ 4C„0Cô0Ct>CÔ0 CD0D" 2 CÄ>CD5C´L C, 7C ?D4AC¤0CT
    /// </summary>Ð
    protected async Task HandleDateRangeChangedAsync(DateRange? newRange)Ð
    {Ð
        if (Control is IHasValue valueProvider)Ð
        {Ð
            valueProvider.Value = newRange;Ð
            await Control.TriggerAsync(CurrentSelectedData);Ð
        }Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor">
@inherits RenderBaseÐ
Ð
<MudDivider Vertical="true" FlexItem="true" Class="my-2mx-1" />
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor.cs">
namespace Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
public partial class DividerRenderBase : RenderBaseÐ
{Ð
}
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/RenderBase.cs">
using Microsoft.AspNetCore.Components;Ð
Ð
namespace Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
/// <summary>Ð
/// A 1D BD 0C¤BCÔKC’ 1C 7Cä2D´9 C¤>CÄ?Cä=CT=D" 4C´O C$ACTE Razor-D 5CÔ4CT@CT@Cä2 DÔ;CT<CT=D$>C" CCô@C 2C´5CÔ
/// </summary>Ð
public abstract class RenderBase : ComponentBase, IDisposableÐ
{Ð
    /// <summary>Ð
    /// A 8Ct=CTA-CÄ>CD5C´L DÔ;CT<CT=D$0 D4?D 0C$;CT=C„O, Cô5D 5CD0C$0CT<C O C„7 DynamicComponent.Ð
    /// </summary>Ð
    [Parameter]Ð
    public IUiControl Control { get; set; } = null!;Ð
Ð
    /// <summary>Ð
    /// A¤0D :C 4CÔKC’ :Cä=D$5C¤AD" ECä;D BC  @C 1CäGCT9 Cä1C´0D BC,í
```

```
    /// </summary>Đ
    [CascadingParameter]Đ
    protected IWorkspaceContext? ActionContext { get; set; }Đ
Đ
    /// <summary>Đ
    /// A 5Ct>Cô0D =Câ 8Ct2C´5C¤0CTB D$5C¤CD”8CR 2D´4CT;CT=CÔKCR 4C =CÔKCR 8Cr :Cä=D$5C¤AD$0 DD>D <D²í
    /// </summary>Đ
    protected object? CurrentSelectedData => GetSelectedDataFromContext();Đ
Đ
    protected override void OnInitialized()Đ
    {Đ
        base.OnInitialized();Đ
Đ
        // B 5C48D BD 8D CCT< D 5C :D$8C$=Cä5 Cä1CÔ>C$;CT=C„5 C„=D$5D DCT9D 0 Cô@C, ACÄ5CÔ5 D >D BCäOCÔ8Dò 1C
        Control.OnStateChanged += HandleStateChanged;Đ
    }Đ
Đ
    private void HandleStateChanged() => InvokeAsync(StateHasChanged);Đ
Đ
    private object? GetSelectedDataFromContext()Đ
    {Đ
        if (ActionContext == null) return null;Đ
Đ
        Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");Đ
        return interfaceType?.GetProperty("SelectedData")?.GetValue(ActionContext);Đ
    }Đ
Đ
    public virtual void Dispose() => Control.OnStateChanged -= HandleStateChanged;Đ
}
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor">
@inherits RenderBaseĐ
<MudSelect T="string"Đ
           Value="@GetControlValue()"Đ
           ValueChanged="@HandleValueChangedAsync"Đ
           Disabled="@(!Control.IsEnabled || Control.IsRunning)"Đ
           Label="@Control.Title"Đ
           Margin="Margin.Dense"Đ
           Variant="Variant.Outlined"Đ
           Style="min-width: 160px; max-width: 250px;">Đ
Đ
    @foreach (string option in GetOptions())Đ
    {Đ
        <MudSelectItem T="string" Value="@option">@option</MudSelectItem>Đ
    }Đ
</MudSelect>
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor.cs">
namespace Promatis.Net.UI.Components.ElementRenderBase;Đ
Đ
public partial class SelectRenderBase : RenderBaseĐ
{Đ
    protected string GetControlValue()Đ
    {Đ
        if (Control is IHasValue valueProvider && valueProvider.Value != null)Đ
        {Đ
            return valueProvider.Value.ToString() ?? string.Empty;Đ
        }Đ
        return string.Empty;Đ
    }Đ
Đ
    protected async Task HandleValueChangedAsync(string newValue)Đ
    {Đ
        if (Control is IHasValue valueProvider)Đ
        {Đ
            valueProvider.Value = newValue;Đ
            await Control.TriggerAsync(CurrentSelectedData);Đ
        }Đ
    }Đ
Đ
    protected IEnumerable<string> GetOptions()Đ
    {Đ
        if (Control is IHasOptions optionsProvider)Đ
        {Đ
```

```
            return optionsProvider.Options;Đ
        }Đ
        return Array.Empty<string>();Đ
    }Đ
}
</file>

<file path="Promatis.Net.UI/Components/UIControls/BaseUiControl.cs">
namespace Promatis.Net.UI.Components;Đ
Đ
/// <summary>Đ
/// A 0Ct>C$0Dò 0C AD$@C :Dd8Dò 4C´O C$ACTE DÔ;CT<CT=D$>C" CCô@C 2C´5CÔ8Dò ?C´0D$DCä@CÄK (C¤=Cä?Cä:, Dt5C¤1Cä
/// B 5C ;C„7D45D" 8CÔDD 0D BD CC¤BD4@CÔCDâ @D4BC„=D2 8 Cô@CT4CäAD$0C$;Dô5D" <CäAD" : CD8CÔ0CÄ8Dt5D :Cä<D2 @C
/// </summary>Đ
public abstract class BaseUiControl : IUiControlĐ
{Đ
    private bool _isVisible = true;Đ
    private bool _isEnabled = true;Đ
    private bool _isRunning;Đ
Đ
    public abstract string Id { get; }Đ
    public abstract Type ComponentType { get; }Đ
Đ
    public virtual string? Title { get; init; }Đ
    public virtual string? Icon { get; init; }Đ
    public virtual string? Tooltip { get; init; }Đ
Đ
    public Dictionary<string, object> ComponentParameters { get; } = new();Đ
Đ
    public virtual UiControlAlignment Alignment { get; init; } = UiControlAlignment.Left;Đ
Đ
    public bool IsVisibleĐ
    {Đ
        get => _isVisible;Đ
        set { if (_isVisible != value) { _isVisible = value; NotifyStateChanged(); } }Đ
    }Đ
Đ
    public bool IsEnabledĐ
    {Đ
        get => _isEnabled;Đ
        set { if (_isEnabled != value) { _isEnabled = value; NotifyStateChanged(); } }Đ
    }Đ
Đ
    public bool IsRunning => _isRunning;Đ
Đ
    public event Action? OnStateChanged;Đ
Đ
    protected BaseUiControl()Đ
    {Đ
        ComponentParameters.Add("Control", this);Đ
    }Đ
Đ
    protected void NotifyStateChanged() => OnStateChanged?.Invoke();Đ
Đ
    public virtual bool IsEnabledForData(object? targetData) => true;Đ
Đ
    public async Task TriggerAsync(object? targetData)Đ
    {Đ
        if (!_isEnabled || _isRunning || !IsEnabledForData(targetData)) return;Đ
Đ
        tryĐ
        {Đ
            _isRunning = true;Đ
            NotifyStateChanged();Đ
            await HandleTriggerAsync(targetData);Đ
        }Đ
        finallyĐ
        {Đ
            _isRunning = false;Đ
            NotifyStateChanged();Đ
        }Đ
    }Đ
Đ
    protected abstract Task HandleTriggerAsync(object? targetData);Đ
}
</file>
```

```
<file path="Promatis.Net.UI/Components/UIControls/IHasOptions.cs">
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// A¤>CÔBD 0C¤B CD;Dò MC´5CÄ5CÔBCä2 D4?D 0C$;CT=C„O, Cä1C´0CD0DäIC„E D ?C„AC¤>CÂ 4CäAD$CCô=D´E Cä?Dd8C’ 2D´1
/// </summary>Ð
public interface IHasOptions : IUiControlÐ
{Ð
    IEnumerable<string> Options { get; }Ð
}
</file>

<file path="Promatis.Net.UI/Components/UIControls/IHasValue.cs">
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// A¤>CÔBD 0C¤B CD;Dò MC´5CÄ5CÔBCä2 D4?D 0C$;CT=C„O, C¤>D$>D KCR ED 0CÔOD" 8 C„7CÄ5CÔODäB D 0CÔBC 9CÂÔ7CÔ0Dt
/// </summary>Ð
public interface IHasValue : IUiControlÐ
{Ð
    object? Value { get; set; }Ð
}
</file>

<file path="Promatis.Net.UI/Components/UIControls/IUiControl.cs">
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// B4=C„2CT@D 0C´LCÔKC’ :Cä=D$@C :D" MC´5CÄ5CÔBC  CCô@C 2C´5CÔ8Dò „:CÔ>Cô:C Â GCT:C >C¤A, C$KCô0CD0DäIC„9 D
/// </summary>Ð
public interface IUiControlÐ
{Ð
    string Id { get; }Ð
    Type ComponentType { get; }Ð
    Dictionary<string, object> ComponentParameters { get; }Ð
Ð
    string? Title { get; }Ð
    string? Icon { get; }Ð
    string? Tooltip { get; }Ð
Ð
    bool IsVisible { get; set; }Ð
    bool IsEnabled { get; set; }Ð
    bool IsRunning { get; }Ð
Ð
    UiControlAlignment Alignment { get; }Ð
Ð
    bool IsEnabledForData(object? targetData);Ð
    Task TriggerAsync(object? targetData);Ð
Ð
    event Action? OnStateChanged;Ð
}
</file>

<file path="Promatis.Net.UI/Components/UIControls/UiControlAlignment.cs">
namespace Promatis.Net.UI.Components;Ð
Ð
public enum UiControlAlignmentÐ
{Ð
    Left,Ð
    RightÐ
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/GridPage.razor">
@typeparam TEntity where TEntity : classÐ
Ð
<MudDataGrid @ref="_mudGrid"Ð
             ServerData="@ServerDataProvider"Ð
             T="TEntity"Ð
             SelectedItem="@SelectedRow"Ð
             SelectedItemChanged="@OnSelectedRowChanged"Ð
             RowStyleFunc="@FormatSelectedRowStyle"Ð
             RowsPerPage="@RowsPerPage"Ð
             SelectOnRowClick="true"Ð
             Hover="true"Ð
```

```
            Dense="true"Ð
            Bordered="true"Ð
            Elevation="0"Ð
            FixedHeader="true"Ð
            Height="calc(100vh - 280px)"Ð
            Class="flex-grow-1 w-100 cursor-pointer">Ð
    <Columns>Ð
        @GridColumnsÐ
    </Columns>Ð
Ð
    @* A$:C´NDt0CT< Cô0C48CÔ0D$>D  BCä;DÄ:Câ ?Câ BD 5C >C$0CÔ8Dâ :Cä=CTGCÔ>C′ DCä@CÄK *@Ð
    <PagerContent>Ð
        @if (WithPagination)Ð
        {Ð
            <MudDataGridPager T="TEntity" InfoFormat="{first_item}-{last_item} C„7 {all_items}"
RowsPerPageString="B BD >C£¢" óí
        }Ð
    </PagerContent>Ð
</MudDataGrid>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/GridPage.razor.cs">
using Microsoft.AspNetCore.Components;Ð
using MudBlazor;Ð
Ð
namespace Promatis.Net.UI.Components.Workspaces;Ð
Ð
public partial class GridPage<TEntity> : ComponentBase, IDisposable where TEntity : classÐ
{Ð
    private MudDataGrid<TEntity>? _mudGrid;Ð
    private IWorkspaceContext? _currentContext;Ð
Ð
    [CascadingParameter]Ð
    protected IWorkspaceContext? ActionContext { get; set; }Ð
Ð
    [Parameter]Ð
    public Func<GridState<TEntity>, CancellationToken, Task<GridData<TEntity>>>? ServerDataProvider
{ get; set; }Ð
Ð
    [Parameter]Ð
    public RenderFragment? GridColumns { get; set; }Ð
Ð
    [Parameter]Ð
    public bool WithPagination { get; set; } = false;Ð
Ð
    [Parameter]Ð
    public int RowsPerPage { get; set; } = 10000;Ð
Ð
    protected TEntity? SelectedRow { get; set; }Ð
Ð
    /// <summary>Ð
    /// A¤>CÔBD >C´L Cô>CD?C„ACä: CÔ0 C„7CÄ5CÔ5CÔ8Dò :Cä=D$5C¤AD$0 CD;Dò 7C IC„BD² >D" CD$5Dt5C¢ ?C <DôBC,í
    /// Aô5D 5Cô>CD?C„AD´2C 5D$ADò =C  ;CTBD2Â 5D ;C, &Æ ¦÷" ?Cä4CÄ5CÔ8D" 6 66 F–æu  & ÖWFW" :Cä=D$5C¤AD$0.Ð
    /// </summary>Ð
    protected override void OnParametersSet()Ð
    {Ð
        base.OnParametersSet();Ð
Ð
        if (ActionContext != _currentContext)Ð
        {Ð
            if (_currentContext != null)Ð
            {Ð
                _currentContext.OnContextStateChanged -= HandleContextStateChanged;Ð
            }Ð
Ð
            _currentContext = ActionContext;Ð
Ð
            if (ActionContext != null)Ð
            {Ð
                ActionContext.OnContextStateChanged += HandleContextStateChanged;Ð
            }Ð
        }Ð
    }Ð
Ð
    /// <summary>Ð
    /// B 5C 3C„@D45D" =C  ;Dä1D´5 C„7CÄ5CÔ5CÔ8Dò ACäAD$>Dô=C„O C¤>CÔBCT:D BC  „=C ?D 8CÄ5D Â 2C¤;DäGCT=C„5/C
```

```
    /// </summary>Ð
    private void HandleContextStateChanged()Ð
    {Ð
        InvokeAsync(StateHasChanged);Ð
    }Ð
Ð
    public Task ReloadServerData()Ð
    {Ð
        return _mudGrid != null ? _mudGrid.ReloadServerData() : Task.CompletedTask;Ð
    }Ð
Ð
    protected string FormatSelectedRowStyle(TEntity item, int rowNumber)Ð
        => item == SelectedRow ? "background-color: var(--mud-palette-action-disabled-background);
font-weight: 500;" : string.Empty;Ð
Ð
    protected void OnSelectedRowChanged(TEntity? newSelection)Ð
    {Ð
        // B  BT A  AT Aã¢  C H CÔ0CÄ5D 5CÔ=Câ ACô@Cä5C¤BC„@Cä2C =CÔKC′ U,ÔHC 3 DD8C¤AC FC„8 DD>C¤CD 0Ð
        if (newSelection == null && SelectedRow != null) return;Ð
Ð
        SelectedRow = newSelection;Ð
Ð
        // B$@C =D ;C„@D45CÂ AD$5C”B CÔ0Cô@Dô<D4N C" OCD@Câ  Cä=D$5C¤AD" AC < D 8CÔED >CÔ=Câ >C =Cä2C„B D >D
        if (ActionContext is IHasSelectedData<TEntity> bindableContext)Ð
        {Ð
            bindableContext.SelectedData = newSelection;Ð
        }Ð
    }Ð
Ð
    public void Dispose()Ð
    {Ð
        if (_currentContext != null)Ð
        {Ð
            _currentContext.OnContextStateChanged -= HandleContextStateChanged;Ð
        }Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/ReferencePage.razor">
@typeparam TEntity where TEntity : Promatis.Net.Domain.ReferenceBase, new()Ð
Ð
@* A¤0D :C 4CÔ> Cô@Cä1D 0D KC$0CT< C¤>CÔBCT:D B C$=C„7. AT3Câ ?Cä9CÄ0DäB C, V•FööÆ& "Â 8 C$0D, w&–E  vR  ¤
<CascadingValue Value="@((IWorkspaceContext)Context)">Ð
    <WorkspacePage>Ð
Ð
        @* 1. A$ B %AÔ/Bò  Aä A ¢  C$BCä<C BC„GCTAC¤8C′ BD4;C 0D  :CÔ>Cô>C¢ ACô@C 2CäGCÔ8C¤0 AÔ!A, ¤
        <TopContent>Ð
            <UiToolbar Controls="@Context.Controls" />Ð
        </TopContent>Ð
Ð
        @* 2. Bd AÔ"B  A´,AÔ Bò  Aä A ¢ "C 1C´8Dt=C O D 5D$:C  A D 8D BCT<CÔKCÄ8 Cô>C´OCÄ8 Cô> D4<Cä;Dt0CÔ8Dâ
        <BodyContent>Ð
            @* A„!AÔ A  A´ AÔ : ServerDataProvider D$5Cô5D L C$KCtKC$0CTB GetDataAsync, Ct0CôCD :C O C¤@D4BC„
            <GridPage TEntity="TEntity"Ð
                      @ref="_grid"Ð
                      ServerDataProvider="@Context.GetDataAsync"Ð
                      WithPagination="true"Ð
                      RowsPerPage="20">Ð
                <GridColumns>Ð
Ð
                    @* B  B "AT AÔ+AR  Aä Aä A¤ : A$KC$>CDOD$ADò 0C$BCä<C BC„GCTAC¤8 CD;Dò  B  BR ACô@C 2CäGC
                    <PropertyColumn T="TEntity" TProperty="string" Property="x => x.Code" Title="B 8D BCT<CÔK
HeaderStyle="width: 180px;" />Ð
                    <PropertyColumn T="TEntity" TProperty="string" Property="x => x.Name" Title="AÔ0C„<CT=Cä2
HeaderStyle="width: 350px;" />Ð
                    <PropertyColumn T="TEntity" TProperty="string" Property="x => x.Description"
Title="Aä?C„AC =C„5 / AÔ@C„<CTGC =C„5" HeaderStyle="width: auto;" />Ð
Ð
                    @* A¤ B "Aä AÔ+AR  Aä Aä A¤ : B ?CTFC„DC„GCÔKCR ?Cä;DòÂ :CäBCä@D´5 CD>C 0C$8D" 4CäGCT@CÔO
                    @CustomColumnsÐ
Ð
                </GridColumns>Ð
            </GridPage>Ð
        </BodyContent>Ð
Ð
```

```
    </WorkspacePage>Đ
</CascadingValue>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/ReferencePage.razor.cs">
using Microsoft.AspNetCore.Components;Đ
Đ
namespace Promatis.Net.UI.Components.Workspaces;Đ
Đ
public abstract partial class ReferencePage<TEntity> : ComponentBase, IDisposableĐ
    where TEntity : Promatis.Net.Domain.ReferenceBase, new()Đ
{Đ
    protected GridPage<TEntity>? _grid;Đ
    private bool _isDisposed;Đ
Đ
    [Inject]Đ
    protected IServiceProvider PageServiceProvider { get; set; } = null!;Đ
Đ
    /// <summary>Đ
    /// B BD >C4> D$8Cô8Ct8D >C$0CÔ=D´9 C¤>CÔBCT:D B D4?D 0C$;CT=C„O DÔBC„< D ?D 0C$>Dt=C„:Cä<.Đ
    /// </summary>Đ
    protected abstract ReferenceContext<TEntity> Context { get; }Đ
Đ
    /// <summary>Đ
    /// B ;CäB CD;Dò 4CT:C´0D 0D$8C$=Cä3Câ >Cô8D 0CÔ8Dò CCÔ8C¤0C´LCÔKDR :Cä;Cä=Cä: D$0C ;C„FD²í
    /// </summary>Đ
    [Parameter]Đ
    public RenderFragment? CustomColumns { get; set; }Đ
Đ
    protected override void OnInitialized()Đ
    {Đ
        base.OnInitialized();Đ
Đ
        // Aô>CD?C„AD´2C 5CÄADò =C  ACä1D´BC„O C„7CÄ5CÔ5CÔ8Dò AD$5C”BC  :Cä=D$5C¤AD$0 CD;Dò @CT0C¤BC„2CÔ>C’ ?
        Context.OnContextStateChanged += RefreshUi;Đ
Đ
        // B 2Dô7D´2C 5CÂ ACä1D´BC„5 DD8C¤AC FC„8 CD0CÔ=D´E (C¤>CÄ<C„BC  !B4 AB >D" BD 8C43CT@Cä2) D  <Dô3C¤>
        Context.OnContextUpdated = RefreshGrid;Đ
    }Đ
Đ
    // A„!Aô A  A´ AÔ  (Ad5C´5Ct=Cä5 A @DT8D$5C¤BD4@CÔ>CR  D 0C$8C´>): A ;Cä:C„@D4ND”8C’ <CTBCä4 OnInitialize
    // C, @D4GCÔ>C’ 2D´7Cä2 D BC @Cä3Câ <CTBCä4C  Æö D–æ—F– ÄF F  7–æ2,’  Aä AÔ B "BÄ. B4 A  AT B²
    // A„=D$5D DCT9D  >D$:D KC$0CTBD O CÄ3CÔ>C$5CÔ=CâÂ 0 CD0CÔ=D´5 C´5CÔ8C$> Ct0Cô@C HC„2C ND$ADò ACTBC¤>C’ ?
Đ
    /// <summary>Đ
    /// AÄ5D$>CB ?D 8CÔCCD8D$5C´LCÔ>C4> Cä1CÔ>C$;CT=C„O CD0CÔ=D´E C" ×VDF F w&–Bâ  D´7D´2C 5D$ADò 1D >C¤5D >C
    /// </summary>Đ
    protected void RefreshGrid()Đ
    {Đ
        // A„ACô>C´LCtCCT< C$AD$@Cä5CÔ=D´9 Cô>D$>C¤>C 5Ct>Cô0D =D´9 CÄ5DT0CÔ8Ct< Blazor CD;Dò 2D´7Cä2C  8Cr D
        InvokeAsync(async () =>Đ
        {Đ
            if (_grid != null)Đ
            {Đ
                await _grid.ReloadServerData();Đ
            }Đ
        });Đ
    }Đ
Đ
    private void RefreshUi()Đ
    {Đ
        InvokeAsync(StateHasChanged);Đ
    }Đ
Đ
    /// <summary>Đ
    /// Aô>C´=C O CäGC„AD$:C  @CTAD4@D >C" 4C´O Cô@CT4CäBC$@C ICT=C„O D4BCTGCT: Cô0CÄOD$8 C" &Æ ¦÷-
    /// </summary>Đ
    public void Dispose()Đ
    {Đ
        if (_isDisposed) return;Đ
Đ
        if (Context != null)Đ
        {Đ
            Context.OnContextStateChanged -= RefreshUi;Đ
Đ
            // ATAC´8 C¤>CÔBCT:D B D >Ct4C 5D$ADò 2CÔCD$@C, 6C„7CÔ5CÔ=Cä3Câ FC„:C´0 DD>D <D² „=CR GCT@CT7 C4;
```

```
            // C$KCtKC$0CT< D4BC„;C„7C FC„N CD;Dò >D$?C„AC¤8 A @Cä:CT@C  >D" HC„=D² !B4 ABí
            Context.Dispose();Ð
        }Ð
Ð
        _isDisposed = true;Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/ReferenceTreePage.razor">
@typeparam TEntity where TEntity : Promatis.Net.Domain.ReferenceTreeBase<TEntity>, new()Ð
Ð
@* A¤0D :C 4CÔ> Cô@Cä1D 0D KC$0CT< C„5D 0D EC„GCTAC¤8C' :Cä=D$5C¤AD"â  C4> Cô>C"<C ND" 8 UiToolbar, C, 2C H T
<CascadingValue Value="@((IWorkspaceContext)Context)">Ð
    <WorkspacePage>Ð
Ð
        @* 1. A$ B %AÔ/Bò  Aä A ¢  C$BCä<C BC„GCTAC¤8C' BD4;C 0D  :CÔ>Cô>C¢ CCô@C 2C´5CÔ8Dò 4CT@CT2Cä< (A¤>D
        <TopContent>Ð
            <UiToolbar Controls="@Context.Controls" />Ð
        </TopContent>Ð
Ð
        @* 2. Bd AÔ"B  A´,AÔ Bò  Aä A ¢  D 5C$>C$8CD=C O D BD CC¤BD4@C  A C 2D$>CÄ0D$8Dt5D :C„< C$KC$>CD>CÂ ?
        <BodyContent>Ð
            @* A„!AÔ A  A´ AÔ : RootItems D44C ;CT=. TreePage D 0CÂ ;CT=C„2Câ ?CäBDô=CTB CD0CÔ=D´5 Cô>D ;CR @
            <TreePage TEntity="TEntity"Ð
                      ChildSelector="@(x => x.Children)"Ð
                      TextSelector="@(x => string.IsNullOrWhiteSpace(x.Code) ? x.Name :
$"[{x.Code}] {x.Name}")"Ð
                      IconSelector="@(x => x.Children != null && x.Children.Any() ?
Icons.Material.Filled.Folder : Icons.Material.Filled.InsertDriveFile)" />Ð
        </BodyContent>Ð
Ð
    </WorkspacePage>Ð
</CascadingValue>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/ReferenceTreePage.razor.cs">
using Microsoft.AspNetCore.Components;Ð
Ð
namespace Promatis.Net.UI.Components.Workspaces;Ð
Ð
public abstract partial class ReferenceTreePage<TEntity> : ComponentBase, IDisposableÐ
    where TEntity : Promatis.Net.Domain.ReferenceTreeBase<TEntity>, new()Ð
{Ð
    private bool _isDisposed;Ð
Ð
    [Inject]Ð
    protected IServiceProvider PageServiceProvider { get; set; } = null!;Ð
Ð
    /// <summary>Ð
    /// B BD >C4> D$8Cô8Ct8D >C$0CÔ=D´9 C„5D 0D EC„GCTAC¤8C' :Cä=D$5C¤AD" CCô@C 2C´5CÔ8Dò MD$8CÂ 4CT@CT2Cä<.Ð
    /// </summary>Ð
    protected abstract ReferenceTreeContext<TEntity> Context { get; }Ð
Ð
    protected override void OnInitialized()Ð
    {Ð
        base.OnInitialized();Ð
Ð
        // Aô>CD?C„AD´2C 5CÄADò =C  ACä1D´BC„O C„7CÄ5CÔ5CÔ8Dò AD$5C"BC  :Cä=D$5C¤AD$0 CD;Dò @CT0C¤BC„2CÔ>C' ?
        Context.OnContextStateChanged += RefreshUi;Ð
Ð
        // B 2Dô7D´2C 5CÂ ACä1D´BC„5 DD8C¤AC FC„8 CD0CÔ=D´E (C¤>CÄ<C„BC  !B4 AB' A CÄOC4:Cä9 Cô5D 5D 8D >C$:C
        Context.OnContextUpdated = RefreshUi;Ð
    }Ð
Ð
    // A„!AÔ A  A´ AÔ  (Ad5C´5Ct=Cä5 A @DT8D$5C¤BD4@CÔ>CR  D 0C$8C´>): A ;Cä:C„@D4ND"8C' <CTBCä4 OnInitialize
    // C, @D4GCÔ>C' 2D´7Cä2 D BC @Cä3Câ <CTBCä4C  Æö D–æ–F– ÄF F  7–æ2,'  Aä AÔ B "BÄ. B4 A  AT B²
    // AD5D 5C$> CäBC¤@D´2C 5D$ADò <C4=Cä2CT=CÔ>, C  8CT@C @DT8Dò ;CT=C„2Câ 7C ?D 0D„8C$0CTBD O Cô>D ;CR >D$@
Ð
    private void RefreshUi()Ð
    {Ð
        // A40D 0CÔBC„@D45CÂ ?CäBCä:Cä1CT7Cä?C ACÔKC' 2D´7Cä2 Cô5D 5D 8D >C$:C, 2 C¤>CÔBCT:D BCR &Æ ¦÷" T'Ô?C
        InvokeAsync(StateHasChanged);Ð
    }Ð
Ð
    /// <summary>Ð
```

```
    /// Aô>C´=C O CäGC„AD$:C  @CTAD4@D >C" 4C´O Cô@CT4CäBC$@C ICT=C„O D4BCTGCT: Cô0CÄOD$8 C" &Æ ¦÷-
    /// </summary>Ð
    public void Dispose()Ð
    {Ð
        if (_isDisposed) return;Ð
Ð
        if (Context != null)Ð
        {Ð
            Context.OnContextStateChanged -= RefreshUi;Ð
Ð
            // A$KCtKC$0CT< D4BC„;C„7C FC„N C¤>CÔBCT:D BC  4C´O C40D 0CÔBC„@Cä2C =CÔ>C’ >D$?C„AC¤8 C @Cä:CT@C
            Context.Dispose();Ð
        }Ð
Ð
        _isDisposed = true;Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/TreePage.razor">
@typeparam TEntity where TEntity : classÐ
Ð
Ð
<MudTreeView T="TEntity"Ð
             Items="@TreeItems"Ð
             SelectedValue="@SelectedNode"Ð
             SelectedValueChanged="@OnSelectedNodeChanged"Ð
             Hover="true"Ð
             Dense="true"Ð
             SelectionMode="SelectionMode.SingleSelection"Ð
             Height="calc(100vh - 240px)"Ð
             Class="flex-grow-1 w-100 h-100 overflow-y-auto">Ð
    <ItemTemplate>Ð
        <MudTreeViewItem T="TEntity"Ð
                         Value="@context.Value"Ð
                         Icon="@GetNodeIcon(context.Value)"Ð
                         Text="@GetNodeText(context.Value)"Ð
                         Items="@GetChildTreeItemData(context.Value)" />Ð
    </ItemTemplate>Ð
</MudTreeView>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/TreePage.razor.cs">
using Microsoft.AspNetCore.Components;Ð
using MudBlazor;Ð
Ð
namespace Promatis.Net.UI.Components.Workspaces;Ð
Ð
public partial class TreePage<TEntity> : ComponentBase, IDisposable where TEntity : classÐ
{Ð
    private IWorkspaceContext? _currentContext;Ð
    private bool _isFirstLoadExecuted;Ð
Ð
    [CascadingParameter]Ð
    protected IWorkspaceContext? ActionContext { get; set; }Ð
Ð
    [Parameter] public Func<TEntity, IEnumerable<TEntity>>? ChildSelector { get; set; }Ð
    [Parameter] public Func<TEntity, string>? TextSelector { get; set; }Ð
    [Parameter] public Func<TEntity, string>? IconSelector { get; set; }Ð
Ð
    protected List<TreeItemData<TEntity>> TreeItems { get; set; } = [];Ð
    protected TEntity? SelectedNode { get; set; }Ð
Ð
    protected override void OnParametersSet()Ð
    {Ð
        base.OnParametersSet();Ð
Ð
        if (ActionContext != _currentContext)Ð
        {Ð
            if (_currentContext != null)Ð
            {Ð
                _currentContext.OnContextStateChanged -= HandleContextStateChanged;Ð
            }Ð
Ð
            _currentContext = ActionContext;Ð
Ð
```

```
            if (ActionContext != null)Ð
            {Ð
                ActionContext.OnContextStateChanged += HandleContextStateChanged;Ð
            }Ð
        }Ð
    }Ð
Ð
    /// <summary>Ð
    /// A„!Aô A  A´ AÔ  (Ad5C´5Ct=Cä5 A @DT8D$5C¤BD4@CÔ>CR  D 0C$8C´>): Aô5D 2C„GCÔKC’ 7C ?D >D  4C =CÔKDR 7C
    /// B "B  A4  Aô B  AR BCä3CâÂ :C : Blazor Cô>C´=CäAD$LDâ >D$@CT=CD5D 8C² 8CÔBCT@DD5C”A C, :C @C¤0D  DCä@
    /// </summary>Ð
    protected override async Task OnAfterRenderAsync(bool firstRender)Ð
    {Ð
        await base.OnAfterRenderAsync(firstRender);Ð
Ð
        if (firstRender && !_isFirstLoadExecuted)Ð
        {Ð
            _isFirstLoadExecuted = true;Ð
            await LoadTreeDataAsync();Ð
        }Ð
    }Ð
Ð
    /// <summary>Ð
    /// A„7C$;CT:C 5D" 0D 8CÔED >CÔ=D´9 C4@C D Cä1D¤5C¤BCä2 Dt5D 5Cr 1CT7Cä?C ACÔKC’ <CTBCä4-CD8D ?CTBDt5D  F
    /// A 2D$>CÄ0D$8Dt5D :C, 0C¤BC„2C„@D45D" ×VD÷fW&Æ ’ 7C 3D CCt:C, ECä;D BC  =C  2D 5CÄO gRPC/API-Ct0Cô@CäA
    /// </summary>Ð
    private async Task LoadTreeDataAsync()Ð
    {Ð
        // Aô@Cä2CT@Dô5CÂÂ ?Cä4CD5D 6C„2C 5D" ;C, BCT:D4IC„9 C¤>CÔBCT:D B D 0C >D$C D  4C =CÔKCÄ8 (Dô2C´OCTBD
        // AD;Dò MD$>C4> CD8CÔ0CÄ8Dt5D :C, 8Ct2C´5C¤0CT< D BD >C4> D$8Cô8Ct8D >C$0CÔ=D´9 CÄ5D$>CB GCT@CT7 D 5
        // CÔ> D$0C¢ :C : CD5D 5C$> D 0C >D$0CTB D  G&VT6öçFW‡BÂ <D² <Cä6CT< C 5Ct>Cô0D =Câ 2D´?Cä;CÔ8D$L Cô@
        if (ActionContext != null)Ð
        {Ð
            // A„ACô>C´LCtCCT< CD8CÔ0CÄ8Dt5D :Cä5 Cô@C„2CT4CT=C„5, D$0C¢ :C : TreeContext Ct0C¤@D´B C45CÔ5D 8
            // AÔ> CÄK D$>Dt=Câ 7CÔ0CT<, DtBCâ C CÔ5C4> CTAD$L CÄ5D$>CB vWDF F  7–æ2†ö&¦V7B 7F FR’í
            var method = ActionContext.GetType().GetMethod("GetDataAsync");Ð
            if (method != null)Ð
            {Ð
                // A$KCtKC$0CT< CD8D ?CTBDt5D  4C =CÔKDR :Cä=D$5C¤AD$0. Aä= D 0CÂ 2C¤;DäGC„B C¤@D4BC„;C¤C Ct0
                var task = (Task<IReadOnlyList<TEntity>>)method.Invoke(ActionContext, [new
object(), default(CancellationToken)])!;Ð
                IReadOnlyList<TEntity> result = await task;Ð
Ð
                // Aô5D 5C¤;C 4D´2C 5CÂ ?Cä;D4GCT=CÔKC’ 3D 0DB 2 C$8CtCC ;DÄ=Cä5 CD5D 5C$> MudTreeViewÐ
                if (result != null)Ð
                {Ð
                    TreeItems = resultÐ
                        .Where(item => item != null)Ð
                        .Select(item => new TreeItemData<TEntity> { Value = item })Ð
                        .ToList();Ð
Ð
                    StateHasChanged();Ð
                }Ð
            }Ð
        }Ð
    }Ð
Ð
    private void HandleContextStateChanged()Ð
    {Ð
        // Aô5D 5D 8D >C$KC$0CT< CD5D 5C$>, CTAC´8 C"  At# C„7CÄ5CÔ8C´8D L Cô>C´O, CD>C 0C$8C´8D L D47C´K C„;
        InvokeAsync(StateHasChanged);Ð
    }Ð
Ð
    protected void OnSelectedNodeChanged(TEntity? newSelection)Ð
    {Ð
        // A„!Aô A  A´ AÔ : A$=CT4D 5CÔ0 DD8D <CT=CÔ0Dò U% Ô7C IC„BC  >D" AC´CDt0C”=Cä3Câ AC @CäAC  :C´8C¤0 C
        // Aô>C$BCä@CÔKC’ :C´8C¢ =C  2D´4CT;CT=CÔKC’ CCt5C² 1Cä;DÄHCR =CR ACÔ8CÄ0CTB DD>C¤CD  8 CÔ5 C ;Cä:C„@
        if (newSelection == null && SelectedNode != null) return;Ð
Ð
        SelectedNode = newSelection;Ð
Ð
        // B$@C =D ;C„@D45CÂ 2D´4CT;CT=C„5 C" OCD@Câå  Cä=D$5C¤AD" AC < D 8CÔED >CÔ=Câ 7C ?D4AD$8D" FCT?CäGC¤
        if (ActionContext is IHasSelectedData<TEntity> bindableContext)Ð
        {Ð
            bindableContext.SelectedData = newSelection;Ð
        }Ð
```

```
    }Ð
Ð
    protected string GetNodeText(TEntity? item) =>Ð
        item == null ? string.Empty : (TextSelector?.Invoke(item) ?? item.ToString() ??
string.Empty);Ð
Ð
    protected string GetNodeIcon(TEntity? item) =>Ð
        item == null ? Icons.Material.Filled.Folder : (IconSelector?.Invoke(item) ??
Icons.Material.Filled.Folder);Ð
Ð
    protected List<TreeItemData<TEntity>> GetChildTreeItemData(TEntity? item)Ð
    {Ð
        if (item == null || ChildSelector == null) return [];Ð
Ð
        IEnumerable<TEntity>? children = ChildSelector.Invoke(item);Ð
        if (children == null) return [];Ð
Ð
        return childrenÐ
            .Where(child => child != null)Ð
            .Select(child => new TreeItemData<TEntity> { Value = child })Ð
            .ToList();Ð
    }Ð
Ð
    public void Dispose()Ð
    {Ð
        if (_currentContext != null)Ð
        {Ð
            _currentContext.OnContextStateChanged -= HandleContextStateChanged;Ð
        }Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/UiToolbar.razor">
<div class="d-flex align-center justify-space-between gap-4 pa-0 w-100" style="min-height: 48px;
width: 100%;">Ð
Ð
    @if (Controls != null)Ð
    {Ð
        @* A´5C$0Dò 3D CCô?C ¢ DC„;DÄBD K C$KD BD 0C„2C ND$ADò 2 D OCB A DD8C¤AC„@Cä2C =CÔKCÂ 7C 7Cä@Cä< *@Ð
        <div class="d-flex align-center gap-4">Ð
            @foreach (IUiControl control in Controls.Where(c => c.Alignment ==
UiControlAlignment.Left))Ð
            {Ð
                @if (!control.IsVisible) continue;Ð
                <DynamicComponent Type="@control.ComponentType"
Parameters="@control.ComponentParameters" />Ð
            }Ð
        </div>Ð
Ð
        @* Aô@C 2C O C4@D4?Cô0: D 0Ct4CT;C„BCT;DÂ 8 C¤=Cä?C¤8 C$KC4@D47C¤8 *@Ð
        <div class="d-flex align-center gap-4">Ð
            @foreach (IUiControl control in Controls.Where(c => c.Alignment ==
UiControlAlignment.Right))Ð
            {Ð
                @if (!control.IsVisible) continue;Ð
                <DynamicComponent Type="@control.ComponentType"
Parameters="@control.ComponentParameters" />Ð
            }Ð
        </div>Ð
    }Ð
</div>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/UiToolbar.razor.cs">
using System.Reflection;Ð
using Microsoft.AspNetCore.Components;Ð
Ð
namespace Promatis.Net.UI.Components.Workspaces;Ð
Ð
public partial class UiToolbar : ComponentBase, IDisposableÐ
{Ð
    /// <summary>Ð
    /// B ?C„ACä: Cô>C´8CÄ>D DCÔKDR :Cä=D$@Cä;Cä2 CD;Dò >D$@C„ACä2C¤8 CÔ0 Cô0CÔ5C´8.Ð
    /// </summary>Ð
    [Parameter]Ð
```

```
    public IEnumerable<IUiControl>? Controls { get; set; }Đ
Đ
    /// <summary>Đ
    /// A¤0D :C 4CÔKC′ :Cä=D$5C¤AD" ECä;D BC  4C´O CäBD ;CT6C„2C =C„O D <CT=D² 2D´4CT;CT=C„O CD0CÔ=D´E.Đ
    /// </summary>Đ
    [CascadingParameter]Đ
    protected IWorkspaceContext? ActionContext { get; set; }Đ
Đ
    /// <summary>Đ
    /// A„7C$;CT:C 5D" BCT:D4IC„9 C$KCD5C´5CÔ=D´9 Cä1D¤5C¤B CD;Dò ?CT@CT4C GC, 2 C 8Ct=CTA-C´>C48C¤C C¤>CÔBD
    /// </summary>Đ
    protected object? CurrentSelectedData => GetSelectedDataFromContext();Đ
Đ
    protected override void OnInitialized()Đ
    {Đ
        base.OnInitialized();Đ
Đ
        SubscribeToSelectionChanged(true);Đ
Đ
        if (Controls != null)Đ
        {Đ
            foreach (IUiControl control in Controls)Đ
            {Đ
                control.OnStateChanged += HandleStateChanged;Đ
            }Đ
        }Đ
    }Đ
Đ
    private void HandleStateChanged() => InvokeAsync(StateHasChanged);Đ
Đ
    private object? GetSelectedDataFromContext()Đ
    {Đ
        if (ActionContext == null) return null;Đ
Đ
        Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");Đ
        if (interfaceType != null)Đ
        {Đ
            return interfaceType.GetProperty("SelectedData")?.GetValue(ActionContext);Đ
        }Đ
        return null;Đ
    }Đ
Đ
    private void SubscribeToSelectionChanged(bool subscribe)Đ
    {Đ
        if (ActionContext == null) return;Đ
Đ
        Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");Đ
        if (interfaceType != null)Đ
        {Đ
            PropertyInfo? eventInfo = interfaceType.GetProperty("OnContextUpdated");Đ
            if (eventInfo != null)Đ
            {Đ
                var currentDelegate = (Action?)eventInfo.GetValue(ActionContext);Đ
                if (subscribe)Đ
                    currentDelegate += HandleStateChanged;Đ
                elseĐ
                    currentDelegate -= HandleStateChanged;Đ
Đ
                eventInfo.SetValue(ActionContext, currentDelegate);Đ
            }Đ
        }Đ
    }Đ
Đ
    public void Dispose()Đ
    {Đ
        SubscribeToSelectionChanged(false);Đ
Đ
        if (Controls != null)Đ
        {Đ
            foreach (IUiControl control in Controls)Đ
            {Đ
                control.OnStateChanged -= HandleStateChanged;Đ
            }Đ
        }Đ
    }Đ
}
```

```
</file>

<file path="Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor">
<MudPaper Elevation="@GetPaperElevation()"Đ
          Class="@GetPaperClass()"Đ
          Style="@($"width: 100%; height: {GetWorkspaceHeight()}; overflow: hidden;")">Đ
Đ
    <!-- 1. A$ B %AÔ/Bò  Aä A  „ A  Aä AÔ/AT"B / B$ A´,A¤  Aô A,  A  A„'A„  TOPCONTENT) -->Đ
    @if (TopContent != null && !IsTopCollapsed())Đ
    {Đ
        <div class="workspace-zone-top w-100 mb-2"Đ
             style="@($"height: {GetTopHeight()}; min-height: {GetTopHeight()}; max-height:
{GetTopHeight()}; flex-shrink: 0;")">Đ
            @TopContentĐ
        </div>Đ
    }Đ
Đ
    <!-- B  AT AÔ A’ !A´ A’ „ AT A / At AÔ  + A  AD  + Aô A  A / At AÔ ) -->Đ
    <div class="d-flex flex-row align-stretch flex-grow-1 gap-4 w-100" style="min-height: 0;
height: 100%; overflow: hidden;">Đ
Đ
        <!-- 2. A´ A$ Bò  Aä A  „ A  A„ A &A„/ / AD B  A$ ) -->Đ
        @if (LeftContent != null && !IsLeftCollapsed())Đ
        {Đ
            <div class="workspace-zone-left d-flex flex-column align-stretch justify-space-between"Đ
                 style="@($"width: {GetLeftWidth()}; min-width: {GetLeftWidth()}; max-width:
{GetLeftWidth()}; overflow-y: auto;")">Đ
                @LeftContentĐ
            </div>Đ
        }Đ
Đ
        <!-- 3. Bd AÔ"B  A´,AÔ Bò  Aä A  „ Aä Aƒ¢  B  AB ò  AÔ AÄ B %AT A ’ ÒÓí
        <!-- A„!Aô A  A´ AÔ : AD>C 0C$;CT= C¤;C AD   ÷6—F–öâ×&VÆ F—fR 4C´O C 1D >C´ND$=Cä3Câ ?Cä7C„FC„>CÔ8D >
        <div class="workspace-zone-body flex-grow-1 h-100 position-relative" style="min-width: 0;
min-height: 0;">Đ
            @if (BodyContent != null)Đ
            {Đ
                @BodyContentĐ
            }Đ
Đ
            <!-- A„!Aô A  A´ AÔ : Bd5CÔBD 0C´8Ct>C$0CÔ=D´9 D 5C :D$8C$=D´9 C„=CD8C¤0D$>D  7C 3D CCt:C, 4C´O C
            <!-- A ;Cä:C„@D45D" 8CÔBCT@DD5C”A Ct>CÔK CäB Cô>C$BCä@CÔKDR :C´8C¤>C" 2Câ 2D 5CÄO D$OCd5C´KDR ACT
            <MudOverlay Visible="@(ActionContext?.IsLoading ?? false)"Đ
                        DarkBackground="false"Đ
                        LightBackground="true"Đ
                        Absolute="true">Đ
                <MudProgressCircular Color="Color.Primary" Size="Size.Large" Indeterminate="true" />Đ
            </MudOverlay>Đ
        </div>Đ
Đ
        <!-- 4. Aô A  A / At AÔ  (A„ B  AT B$ B  !A$ A”!B$ ) -->Đ
        @if (RightContent != null && !IsRightCollapsed())Đ
        {Đ
            <div class="workspace-zone-right d-flex flex-column align-stretch justify-space-between"Đ
                 style="@($"width: {GetRightWidth()}; min-width: {GetRightWidth()}; max-width:
{GetRightWidth()}; overflow-y: auto;")">Đ
                @RightContentĐ
            </div>Đ
        }Đ
Đ
    </div>Đ
Đ
    <!-- 5. AÔ Ad Bô/ At AÔ  (B "A "B4!-A  B ’ ÒÓí
    @if (BottomContent != null && !IsBottomCollapsed())Đ
    {Đ
        <div class="workspace-zone-bottom w-100 mt-2"Đ
             style="@($"height: {GetBottomHeight()}; min-height: {GetBottomHeight()}; max-height:
{GetBottomHeight()}; flex-shrink: 0;")">Đ
            @BottomContentĐ
        </div>Đ
    }Đ
Đ
</MudPaper>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor.cs">
```

```
using Microsoft.AspNetCore.Components;Ð
Ð
namespace Promatis.Net.UI.Components.Workspaces;Ð
Ð
public partial class WorkspacePage : ComponentBase, IDisposableÐ
{Ð
    private IWorkspaceContext? _previousContext;Ð
    private bool _isDisposed;Ð
Ð
    /// <summary>Ð
    /// AT4C„=D´9 C¤0D :C 4CÔKC′ ?C @C <CTBD  :Cä=D$5C¤AD$0. Ð
    /// B$5Cô5D L DT>C´AD" 3C„1C¤> CäBD ;CT6C„2C 5D" 5C4> Cô>CD<CT=D2 ?D 8 Cô5D 5DT>CD0DR <CT6CDC D BD 0CÔ8Dd
    /// </summary>Ð
    [CascadingParameter]Ð
    protected IWorkspaceContext? ActionContext { get; set; }Ð
Ð
    [Parameter] public RenderFragment? BodyContent { get; set; }Ð
    [Parameter] public RenderFragment? TopContent { get; set; }Ð
    [Parameter] public RenderFragment? BottomContent { get; set; }Ð
    [Parameter] public RenderFragment? LeftContent { get; set; }Ð
    [Parameter] public RenderFragment? RightContent { get; set; }Ð
Ð
    /// <summary>Ð
    /// A„!Aô A  A´ AÔ  (Aô@Cä1C´5CÄ0 ! 5): Aô5D 5CÔ>D  ;Cä3C„:C, 2 OnParametersSet.Ð
    /// Blazor C$KCtKC$0CTB DÔBCäB CÄ5D$>CB :C 6CDKC′ @C 7, C¤>C44C  6 66 F–æu  & ÖWFW" >C =Cä2C´OCTBD O C„;C
    /// </summary>Ð
    protected override void OnParametersSet()Ð
    {Ð
        base.OnParametersSet();Ð
Ð
        // ATAC´8 C„=D BC =D  :Cä=D$5C¤AD$0 C„7CÄ5CÔ8C´ADò „?D >C„7CäHCT; Cô5D 5DT>CB =C  4D CC4CDâ AD$@C =C„
        if (ActionContext != _previousContext)Ð
        {Ð
            // 1. A 5Ct>Cô0D =Câ >D$?C„AD´2C 5CÄADò >D" ACä1D´BC„9 D BC @Cä3Câ :Cä=D$5C¤AD$0Ð
            if (_previousContext != null)Ð
            {Ð
                _previousContext.OnContextStateChanged -= HandleContextStateChanged;Ð
            }Ð
Ð
            // 2. At0Cô>CÄ8CÔ0CT< CÔ>C$KC′ :Cä=D$5C¤AD" :C : D$5C¤CD”8C•
            _previousContext = ActionContext;Ð
Ð
            if (ActionContext != null)Ð
            {Ð
                // 3. Aô>CD?C„AD´2C 5CÄADò =C  @CT0C¤BC„2CÔKCR 8Ct<CT=CT=C„O CÔ>C$>C4> C¤>CÔBCT:D BC
                ActionContext.OnContextStateChanged += HandleContextStateChanged;Ð
Ð
                // 4. A„!Aô A  A´ AÔ  (Aô@Cä1C´5CÄ0 ! 4): A 5Ct>Cô0D =D´9 D$@C„3C45D  7C ?D4AC¤0 Cd8Ct=CT=CÔ>
                // A„=C„FC„0C´8Ct8D CCT< C¤>CÔBCT:D B (D 1Cä@C¤0 D$CC´1C @Cä2, CÔ0D BD >C”:C   D >C¤5D >C"′ B
                // C¤>C44C  8 DT>C´AD"Â 8 C$ACR ?D 8C¤;C 4CÔKCR =C AC´5CD=C„:C, =C     R ACä7CD0CÔK C" ?C <Dô
                ActionContext.InitializeContext();Ð
            }Ð
        }Ð
    }Ð
Ð
    /// <summary>Ð
    /// Aä1D 0C >D$GC„: D 5C :D$8C$=Cä3Câ 8Ct<CT=CT=C„O D >D BCäOCÔ8Dòâ
    /// Aô@C, ;Dä1Cä< D„>D >DT5 C"  At#-C¤MD„5 C„;C, ?CT@CT:C´NDt5CÔ8C, —4Æö F–ær 7C AD$0C$;Dô5D" &Æ ¦÷" ?CT@
    /// </summary>Ð
    private void HandleContextStateChanged()Ð
    {Ð
        if (!_isDisposed)Ð
        {Ð
            InvokeAsync(StateHasChanged);Ð
        }Ð
    }Ð
Ð
    /// <summary>Ð
    /// A„!Aô A  A´ AÔ  (Aô@Cä1C´5CÄ0 ! 6): A 5Ct>Cô0D =D´9 Dispose.Ð
    /// A40D 0CÔBC„@D45D" >D$AD4BD BC$8CR ?C 4CT=C„9 NullReferenceException Cô@C, 7C :D KD$8C, AD$0D$8Dt5D :C
    /// </summary>Ð
    public void Dispose()Ð
    {Ð
        if (_isDisposed) return;Ð
Ð
        if (ActionContext != null)Ð
```

```
        {Ð
            ActionContext.OnContextStateChanged -= HandleContextStateChanged;Ð
        }Ð
Ð
        _isDisposed = true;Ð
    }Ð
Ð
    // --- A 5Ct>CÔ0D =C O Cô@Cä1D >D :C  ?C @C <CTBD >C" 3CT>CÄ5D$@C„8 C, @C 7CÄ5D$:C, ÒÒÝ
    protected int GetPaperElevation() => ActionContext?.PaperElevation ?? 1;Ð
    protected string GetPaperClass() => ActionContext?.PaperClass ?? "pa-4 d-flex flex-column flex-
grow-1 w-100";Ð
    protected string GetWorkspaceHeight() => ActionContext?.WorkspaceHeight ?? "100%";Ð
    protected string GetTopHeight() => ActionContext?.TopZoneHeight ?? "auto";Ð
    protected string GetBottomHeight() => ActionContext?.BottomZoneHeight ?? "auto";Ð
    protected string GetLeftWidth() => ActionContext?.LeftZoneWidth ?? "250px";Ð
    protected string GetRightWidth() => ActionContext?.RightZoneWidth ?? "300px";Ð
Ð
    // --- A´>C48C¤0 C 2D$>CÄ0D$8Dt5D :Cä3Câ ADT;Cä?D´2C =C„O CôCD BD´E Ct>CÒ „6öÆÆ  6–ær’ ÒÒÝ
    protected bool IsTopCollapsed() => TopContent == null || (ActionContext?.IsTopZoneCollapsed ??
false);Ð
    protected bool IsBottomCollapsed() => BottomContent == null ||
(ActionContext?.IsBottomZoneCollapsed ?? false);Ð
    protected bool IsLeftCollapsed() => LeftContent == null ||
(ActionContext?.IsLeftZoneCollapsed ?? false);Ð
    protected bool IsRightCollapsed() => RightContent == null ||
(ActionContext?.IsRightZoneCollapsed ?? false);Ð
}
</file>

<file path="Promatis.Net.UI/Controls/CreateChildButton.cs">
using MudBlazor;Ð
using Promatis.Net.UI.Components;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Controls;Ð
Ð
/// <summary>Ð
/// B ?CTFC„0C´8Ct8D >C$0CÔ=C O C¤=Cä?C¤0-C¤>CÄ0CÔ4C  4C´O D >Ct4C =C„O CD>Dt5D =CT3Câ „?Cä4Dt8CÔ5CÔ=Cä3Câ’ C
/// AÔ0D$8C$=Câ CCô@C 2C´OCTB D 2Cä8CÂ ACäAD$>Dô=C„5CÂ 4CäAD$CCô=CäAD$8 CÔ0 CäACÔ>C$5 CÔ0C´8Dt8Dò 2D´1D 0CÔ=C
/// </summary>Ð
public class CreateChildButton<TEntity> : BaseUiControl where TEntity : classÐ
{Ð
    private Func<Task>? _executeAsync;Ð
    private readonly string _id = "toolbar_action_create_child_" + Guid.NewGuid().ToString("N");Ð
Ð
    public override string Id => _id;Ð
    public override Type ComponentType => typeof(ButtonRenderBase); // A„ACô>C´LCtCCT< D BC =CD0D BCÔKC’ @CT=
    public override string Title => "AD>C 0C$8D$L Cô>CDCCt5C²#½
    public override string Icon => Icons.Material.Filled.PlaylistAdd; // A„:Cä=C¤0 CD>C 0C$;CT=C„O C" ACô8D >
    public override string Tooltip => "B >Ct4C BDÂ 4CäGCT@CÔ8C’ MC´5CÄ5CÔB C$=D4BD 8 C$KC @C =CÔ>C4> D47C´0";
Ð
    public CreateChildButton()Ð
    {Ð
        // Aô> D4<Cä;Dt0CÔ8Dâ :CÔ>Cô:C  7C 1C´>C¤8D >C$0CÔ0, Cô>C¤0 Dô4D > CÔ5 Cô5D 5CD0D B C" =CT5 C$KC @C =
        IsEnabled = false;Ð
Ð
        ComponentParameters.Add("Color", Color.Primary);Ð
        ComponentParameters.Add("Variant", Variant.Outlined); // A$KCD5C´OCT< C$8CtCC ;DÄ=Câ >D" 3C´0C$=Cä9 C
    }Ð
Ð
    /// <summary>Ð
    /// B 5C :D$8C$=C O Cô@Cä2CT@C¤0 CD>D BD4?CÔ>D BC, :Cä<C =CDK D 8C´0CÄ8 Dô4D 0. Ð
    /// AÄ5D$>CB 2D´7D´2C 5D$ADò :Cä=D$5C¤AD$>CÂ VçF–G•v÷&·7  6T6öçFW‡B ?D 8 C¤0Cd4Cä< C„7CÄ5CÔ5CÔ8C, 6VÆV7FV
    /// </summary>Ð
    public override bool IsEnabledForData(object? targetData)Ð
    {Ð
        // A¤=Cä?C¤0 D BC =Cä2C„BD O CD>D BD4?CÔ>C’ "Aä BÄ Aâ BCä3CD0, C¤>C44C  2 D 8D BCT<CR 2D´1D 0CÒ @Cä4C
        bool hasParentSelected = targetData != null;Ð
Ð
        // B 8CÔED >CÔ8Ct8D CCT< C$=D4BD 5CÔ=CT5 D >D BCäOCÔ8CR DC´0C40 CD>D BD4?CÔ>D BC, 4C´O D 5CÔ4CT@CT@C
        IsEnabled = hasParentSelected;Ð
Ð
        return hasParentSelected;Ð
    }Ð
Ð
    /// <summary>Ð
```

```
    /// Aô@C„2Dô7C¤0 C AC„=DT@Cä=CÔ>C4> Cä1D 0C >D$GC„:C  „4CT;CT3C BC ’ 2D´7Cä2C  :Cä<C =CDK C„7 C¤>CÔBCT:D
    /// </summary>Ð
    public CreateChildButton<TEntity> OnExecute(Func<Task> executeAsync)Ð
    {Ð
        _executeAsync = executeAsync ?? throw new ArgumentNullException(nameof(executeAsync));Ð
        return this;Ð
    }Ð
Ð
    /// <summary>Ð
    /// B$>Dt:C  2DT>CD0 C¤;C„:C  ?Câ :CÔ>Cô:CR ×VD&Æ ¦÷"Â 2D´7D´2C 5CÄ0Dò GCT@CT7 C$8CtCC ;DÄ=D´9 D 5CÔ4CT@C
    /// </summary>Ð
    protected override async Task HandleTriggerAsync(object? targetData)Ð
    {Ð
        if (_executeAsync != null && IsEnabled)Ð
        {Ð
            await _executeAsync.Invoke();Ð
        }Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Controls/CreateEntityButton.cs">
using MudBlazor;Ð
using Promatis.Net.UI.Components;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Controls;Ð
Ð
/// <summary>Ð
/// B4=C„2CT@D 0C´LCÔKC’ 8CÔDD 0D BD CC¤BD4@CÔKC’ MC´5CÄ5CÔB C¤=Cä?C¤8 "B >Ct4C BDÂ" 4C´O D CD”=CäAD$5C’ ;Dä1
/// </summary>Ð
public class CreateEntityButton<TEntity> : BaseUiControl where TEntity : class, new()Ð
{Ð
    private Func<Task>? _command; // AÔ0D, 8D ?Cä;CÔ8D$5C´L CD8C ;Cä3C
Ð
    public override string Id => $"crud_create_{typeof(TEntity).Name.ToLower()}";Ð
    public override Type ComponentType => typeof(ButtonRenderBase);Ð
    public override string Title => "B >Ct4C BDÂ#½
    public override string Icon => Icons.Material.Filled.Add;Ð
    public override string Tooltip => $"B >Ct4C BDÂ =Cä2D4N Ct0Cô8D L";Ð
Ð
    public CreateEntityButton() { IsEnabled = true; }Ð
Ð
    // AÄ5D$>CB 4C´O C$=CT4D 5CÔ8Dò @CT0C´LCÔ>C4> CD5C”AD$2C„O C„7 C¤>CÔBCT:D BC
    public CreateEntityButton<TEntity> OnExecute(Func<Task> command)Ð
    {Ð
        _command = command;Ð
        return this;Ð
    }Ð
Ð
    public override bool IsEnabledForData(object? targetData) => true;Ð
Ð
    protected override async Task HandleTriggerAsync(object? targetData)Ð
    {Ð
        if (_command != null) await _command.Invoke();Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Controls/DeleteEntityButton.cs">
using MudBlazor;Ð
using Promatis.Net.UI.Components;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Controls;Ð
Ð
/// <summary>Ð
/// B4=C„2CT@D 0C´LCÔKC’ 8CÔDD 0D BD CC¤BD4@CÔKC’ MC´5CÄ5CÔB C¤=Cä?C¤8 "B44C ;C„BDÂ" 4C´O C$KC @C =CÔ>C’ AD4I
/// </summary>Ð
public class DeleteEntityButton<TEntity> : BaseUiControl where TEntity : classÐ
{Ð
    private Func<TEntity, Task>? _command;Ð
Ð
    public override string Id => $"crud_delete_{typeof(TEntity).Name.ToLower()}";Ð
    public override Type ComponentType => typeof(ButtonRenderBase);Ð
    public override string Title => "B44C ;C„BDÂ#½
```

```
    public override string Icon => Icons.Material.Filled.Delete;Ð
    public override string Tooltip => "B44C ;C„BDÂ 2D´1D 0CÔ=D4N Ct0Cô8D L";Ð
Ð
    public DeleteEntityButton<TEntity> OnExecute(Func<TEntity, Task> command)Ð
    {Ð
        _command = command;Ð
        return this;Ð
    }Ð
Ð
    public override bool IsEnabledForData(object? targetData)Ð
    {Ð
        if (targetData is not TEntity typedEntity) return false;Ð
        if (typedEntity is Domain.Interface.ISoftDeletable softDeletable)Ð
        {Ð
            return softDeletable.DeletedAt == null;Ð
        }Ð
        return true;Ð
    }Ð
Ð
    protected override async Task HandleTriggerAsync(object? targetData)Ð
    {Ð
        if (_command != null && targetData is TEntity typedEntity)Ð
        {Ð
            await _command.Invoke(typedEntity);Ð
        }Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Controls/EditEntityButton.cs">
using MudBlazor;Ð
using Promatis.Net.UI.Components;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Controls;Ð
Ð
/// <summary>Ð
/// B4=C„2CT@D 0C´LCÔKC’ 8CÔDD 0D BD CC¤BD4@CÔKC’ MC´5CÄ5CÔB C¤=Cä?C¤8 "B 5CD0C¤BC„@Cä2C BDÂ" 4C´O C$KC @C =C
/// </summary>Ð
public class EditEntityButton<TEntity> : BaseUiControl where TEntity : classÐ
{Ð
    private Func<TEntity, Task>? _command;Ð
Ð
    public override string Id => $"crud_edit_{typeof(TEntity).Name.ToLower()}";Ð
    public override Type ComponentType => typeof(ButtonRenderBase);Ð
    public override string Title => "B 5CD0C¤BC„@Cä2C BDÂ#½
    public override string Icon => Icons.Material.Filled.Edit;Ð
    public override string Tooltip => "B 5CD0C¤BC„@Cä2C BDÂ 2D´1D 0CÔ=D4N Ct0Cô8D L";Ð
Ð
    public EditEntityButton<TEntity> OnExecute(Func<TEntity, Task> command)Ð
    {Ð
        _command = command;Ð
        return this;Ð
    }Ð
Ð
    public override bool IsEnabledForData(object? targetData) => targetData is TEntity;Ð
Ð
    protected override async Task HandleTriggerAsync(object? targetData)Ð
    {Ð
        if (_command != null && targetData is TEntity typedEntity)Ð
        {Ð
            await _command.Invoke(typedEntity);Ð
        }Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Controls/ToolbarDivider.cs">
using Promatis.Net.UI.Components;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Controls;Ð
Ð
/// <summary>Ð
/// B 8D BCT<CÔKC’ MC´5CÄ5CÔB C$5D BC„:C ;DÄ=Cä3Câ @C 7CD5C´8D$5C´O, D 0Ct<CTIC 5CÄKC’ <CT6CDC DÔ;CT<CT=D$0CÄ
/// </summary>Ð
```

```
public class ToolbarDivider : BaseUiControlÐ
{Ð
    private readonly string _id = "core_toolbar_divider_" + Guid.NewGuid().ToString("N");Ð
Ð
    public override string Id => _id;Ð
Ð
    /// <summary>Ð
    /// Aô@C„2Dô7C¤0 C¢ =C HCT<D2 7C DC„:D 8D >C$0CÔ=Cä<D2 1C 7Cä2Cä<D2 @CT=CD5D 5D C DividerRenderBase.Ð
    /// </summary>Ð
    public override Type ComponentType => typeof(DividerRenderBase);Ð
Ð
    protected override Task HandleTriggerAsync(object? targetData)Ð
    {Ð
        // AD5C¤>D 0D$8C$=D´9 DÔ;CT<CT=D" ?C =CT;C, 8CÔAD$@D4<CT=D$>C" =CR 2D´?Cä;CÔOCTB C´>C48C¤C Cô> D$@C„3
        return Task.CompletedTask;Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Data/Cache/FlatOzuMutationStrategy.cs">
using Promatis.Net.Data;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
public class FlatOzuMutationStrategy<TEntity> : IOzuMutationStrategy<TEntity> where TEntity : classÐ
{Ð
    // B$5Cô5D L C¤>CÔBD 0C¤B Cô@C„=C„<C 5D" 3C„1C¤CDâ ”6öÆÆV7F–öâ 2CÄ5D BCâ 6CTAD$:Cä3Câ Æ—7M
    public void ApplyDelta(ICollection<TEntity> inMemoryItems, EntityStateChangeEnum state, TEntity
entity, Func<TEntity, object?> idSelector)Ð
    {Ð
        object? entityId = idSelector(entity);Ð
        if (entityId == null) return;Ð
Ð
        switch (state)Ð
        {Ð
            case EntityStateChangeEnum.Added:Ð
                // AD>C 0C$;Dô5CÂÂ BCä;DÄ:Câ 5D ;C, MC´5CÄ5CÔBC  A D$0C¤8CÂ –B 5D”5 CÔ5D" 2 C¤>C´;CT:Dd8C•
                if (!inMemoryItems.Any(x => Equals(idSelector(x), entityId)))Ð
                {Ð
                    inMemoryItems.Add(entity);Ð
                }Ð
                break;Ð
Ð
            case EntityStateChangeEnum.Modified:Ð
                // A„!Aô A  A´ AÔ : B$0C¢ :C : ICollection CÔ5 Cô>CD4CT@Cd8C$0CTB Cä1D 0D”5CÔ8CR ?Câ 8CÔ4CT:D
                // CÄK C„ACô>C´LCtCCT< D4=C„2CT@D 0C´LCÔKC’ 8 C 5Ct>Cô0D =D´9 CD;Dò ×VD&Æ ¦÷" ?Cä4DT>CB ´CCD0
                TEntity? existingItem = inMemoryItems.FirstOrDefault(x => Equals(idSelector(x),
entityId));Ð
                if (existingItem != null)Ð
                {Ð
                    inMemoryItems.Remove(existingItem);Ð
                }Ð
                inMemoryItems.Add(entity);Ð
                break;Ð
Ð
            case EntityStateChangeEnum.Deleted:Ð
            case EntityStateChangeEnum.SoftDeleted:Ð
                TEntity? itemToRemove = inMemoryItems.FirstOrDefault(x => Equals(idSelector(x),
entityId));Ð
                if (itemToRemove != null)Ð
                {Ð
                    inMemoryItems.Remove(itemToRemove);Ð
                }Ð
                break;Ð
        }Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Data/Cache/HierarchicalOzuMutationStrategy.cs">
using Promatis.Net.Data;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
public class HierarchicalOzuMutationStrategy<TEntity> : IOzuMutationStrategy<TEntity> where
TEntity : classÐ
```

```
{Ð
    private readonly Func<TEntity, object?> _parentIdSelector;Ð
Ð
    // B 0C >D$0CT< D BD >C4> D  6C„2Cä9 ICollection CÔ0C$8C40Dd8Cä=CÔ>C4> D 2Cä9D BC$0, C 5Cr :C´>CÔ8D >C$0C
    private readonly Func<TEntity, ICollection<TEntity>> _childrenSelector;Ð
Ð
    // A KD BD KC’ 8CÔ4CT:D  4C´O CÄ3CÔ>C$5CÔ=Cä3Câ ?Cä8D :C  7C  òƒ •
    private readonly Dictionary<object, TEntity> _nodeIndex = [];Ð
    private readonly Dictionary<object, object> _childToParentMap = []; // A¤0D BC  AC$OCt5C“¢ –D#Ct;C  Óâ –D
Ð
    private bool _isIndexBuilt;Ð
Ð
    public HierarchicalOzuMutationStrategy(Ð
        Func<TEntity, object?> parentIdSelector,Ð
        Func<TEntity, ICollection<TEntity>> childrenSelector)Ð
    {Ð
        _parentIdSelector = parentIdSelector ?? throw new
ArgumentNullException(nameof(parentIdSelector));Ð
        _childrenSelector = childrenSelector ?? throw new
ArgumentNullException(nameof(childrenSelector));Ð
    }Ð
Ð
    public void ApplyDelta(ICollection<TEntity> inMemoryItems, EntityStateChangeEnum state, TEntity
entity, Func<TEntity, object?> idSelector)Ð
    {Ð
        object? entityId = idSelector(entity);Ð
        if (entityId == null) return;Ð
Ð
        // A´5CÔ8C$0Dò AC >D :C  ?C´>D :Cä3Câ 8CÔ4CT:D 0 Cô@C, ?CT@C$>C’ <D4BC FC„8 CD0CÔ=D´E DD>D <D½
        if (!_isIndexBuilt)Ð
        {Ð
            BuildIndexRecursive(inMemoryItems, idSelector);Ð
            _isIndexBuilt = true;Ð
        }Ð
Ð
        switch (state)Ð
        {Ð
            case EntityStateChangeEnum.Added:Ð
                HandleAdded(inMemoryItems, entity, entityId, idSelector);Ð
                break;Ð
Ð
            case EntityStateChangeEnum.Modified:Ð
                HandleModified(inMemoryItems, entity, entityId, idSelector);Ð
                break;Ð
Ð
            case EntityStateChangeEnum.Deleted:Ð
            case EntityStateChangeEnum.SoftDeleted:Ð
                HandleDeleted(inMemoryItems, entityId, idSelector);Ð
                break;Ð
        }Ð
    }Ð
Ð
    private void HandleAdded(ICollection<TEntity> rootItems, TEntity entity, object entityId,
Func<TEntity, object?> idSelector)Ð
    {Ð
        if (_nodeIndex.ContainsKey(entityId)) return; // At0D”8D$0 CäB CDCC ;C„@Cä2C =C„O C" ?C <DôBC•
Ð
        object? newParentId = _parentIdSelector(entity);Ð
Ð
        if (newParentId == null)Ð
        {Ð
            // Aä1D¤5C¤B C¤>D =CT2Cä9 – CD>C 0C$;Dô5CÂ 2 C¤>D 5CÔL Aä B=
            rootItems.Add(entity);Ð
        }Ð
        else if (_nodeIndex.TryGetValue(newParentId, out var parentNode))Ð
        {Ð
            // Aä1D¤5C¤B CD>Dt5D =C„9 – CD>C 0C$;Dô5CÂ 2 Cd8C$CDâ :Cä;C´5C¤FC„N CT3Câ @Cä4C„BCT;Dý
            var children = _childrenSelector(parentNode);Ð
            children?.Add(entity);Ð
            _childToParentMap[entityId] = newParentId;Ð
        }Ð
Ð
        // A¤0D :C 4CÔ> D 5C48D BD 8D CCT< D47CT; C, 5C4> Cô>CD4CT@CT2Câ 2 C KD BD >CÂ 8CÔ4CT:D 5 O(1)Ð
        UpdateIndexForSubtree(entity, idSelector);Ð
    }Ð
Ð
```

```
    private void HandleModified(ICollection<TEntity> rootItems, TEntity entity, object entityId,
Func<TEntity, object?> idSelector)Đ
    {Đ
        // A„!Aô A  A´ AÔ : AÔ8C¤0C¤>C4> CÄ0Cô?C„=C40. ATAC´8 D47CT; D46CR AD4ICTAD$2Cä2C ;, DT8D CD 3C„GCTAC
        if (_nodeIndex.TryGetValue(entityId, out var oldNodeReference))Đ
        {Đ
            object? oldParentId = _childToParentMap.TryGetValue(entityId, out var pId) ? pId : null;Đ
Đ
            if (oldParentId == null)Đ
            {Đ
                rootItems.Remove(oldNodeReference);Đ
            }Đ
            else if (_nodeIndex.TryGetValue(oldParentId, out var oldParentNode))Đ
            {Đ
                _childrenSelector(oldParentNode)?.Remove(oldNodeReference);Đ
            }Đ
Đ
            // A$KDt8D”0CT< D BC @D4N D AD´;C¤C C, 5E  AC$OCt8 C„7 C„=CD5C¤AC  ?CT@CT4 CD>C 0C$;CT=C„5CÂ =Cä2
            RemoveFromIndexRecursive(oldNodeReference, idSelector);Đ
        }Đ
Đ
        // A$AD$0C$;Dô5CÂ =Cä2D4N D AD´;C¤C Cô> C :D$CC ;DÄ=Cä<D2   &VçD–B „8CD5C ;DÄ=Câ >C @C 1C BD´2C 5D" 8
        HandleAdded(rootItems, entity, entityId, idSelector);Đ
    }Đ
Đ
    private void HandleDeleted(ICollection<TEntity> rootItems, object entityId, Func<TEntity,
object?> idSelector)Đ
    {Đ
        if (!_nodeIndex.TryGetValue(entityId, out var nodeToRemove)) return;Đ
Đ
        object? parentId = _childToParentMap.TryGetValue(entityId, out var pId) ? pId : null;Đ
Đ
        if (parentId == null)Đ
        {Đ
            rootItems.Remove(nodeToRemove);Đ
        }Đ
        else if (_nodeIndex.TryGetValue(parentId, out var parentNode))Đ
        {Đ
            _childrenSelector(parentNode)?.Remove(nodeToRemove);Đ
        }Đ
Đ
        // A¤0D :C 4CÔ> C$KDt8D”0CT< D44C ;CT=CÔKC’ ?Cä4C4@C D C„7 C„=CD5C¤AC
        RemoveFromIndexRecursive(nodeToRemove, idSelector);Đ
    }Đ
Đ
    // --- B ;D46CT1CÔKCR <CTBCä4D² CCô@C 2C´5CÔ8Dò 1D´AD$@D´< C„=CD5C¤ACä< O(1) ---Đ
Đ
    private void BuildIndexRecursive(ICollection<TEntity> nodes, Func<TEntity, object?> idSelector,
object? parentId = null)Đ
    {Đ
        foreach (var node in nodes)Đ
        {Đ
            object? id = idSelector(node);Đ
            if (id == null) continue;Đ
Đ
            _nodeIndex[id] = node;Đ
            if (parentId != null) _childToParentMap[id] = parentId;Đ
Đ
            var children = _childrenSelector(node);Đ
            if (children != null && children.Count > 0)Đ
            {Đ
                BuildIndexRecursive(children, idSelector, id);Đ
            }Đ
        }Đ
    }Đ
Đ
    private void UpdateIndexForSubtree(TEntity node, Func<TEntity, object?> idSelector)Đ
    {Đ
        object? id = idSelector(node);Đ
        if (id == null) return;Đ
Đ
        _nodeIndex[id] = node;Đ
Đ
        object? parentId = _parentIdSelector(node);Đ
        if (parentId != null) _childToParentMap[id] = parentId;Đ
Đ
```

```
        var children = _childrenSelector(node);Ð
        if (children != null)Ð
        {Ð
            foreach (var child in children)Ð
            {Ð
                UpdateIndexForSubtree(child, idSelector);Ð
            }Ð
        }Ð
    }Ð
Ð
    private void RemoveFromIndexRecursive(TEntity node, Func<TEntity, object?> idSelector)Ð
    {Ð
        object? id = idSelector(node);Ð
        if (id == null) return;Ð
Ð
        _nodeIndex.Remove(id);Ð
        _childToParentMap.Remove(id);Ð
Ð
        var children = _childrenSelector(node);Ð
        if (children != null)Ð
        {Ð
            foreach (var child in children)Ð
            {Ð
                RemoveFromIndexRecursive(child, idSelector);Ð
            }Ð
        }Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Data/Cache/IOzuMutationStrategy.cs">
using Promatis.Net.Data;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
public interface IOzuMutationStrategy<TEntity> where TEntity : classÐ
{Ð
    /// <summary>Ð
    /// Aô@C„<CT=Dô5D" 4CT;DÄBD2 8Ct<CT=CT=C„9 B #A   C¢ 8Ct<CT=Dô5CÄ>C′ :Cä;C´5C¤FC„8 C4@C DC  >C JCT:D$>C"
    /// </summary>Ð
    void ApplyDelta(ICollection<TEntity> inMemoryItems, EntityStateChangeEnum state, TEntity
entity, Func<TEntity, object?> idSelector);Ð
}
</file>

<file path="Promatis.Net.UI/Data/Cache/IUiOzuCache.cs">
using Promatis.Net.Data;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
/// <summary>Ð
/// A¤>CÔBD 0C¤B C„7Cä;C„@Cä2C =CÔ>C4> C´>C¤0C´LCÔ>C4> Aä B2ÔED 0CÔ8C´8D”0 CD0CÔ=D´E C¤>CÔ:D 5D$=Cä9 DÔ:D 0CÔ
/// A„ACô>C´LCtCCTB D 8D BCT<CÔKC′ VçF—G•7F FT6† ævTVçVÒ 4C´O Cô@C„<CT=CT=C„O D$>Dt5Dt=D´E CD5C´LD" 8Ct<CT=CT
/// </summary>Ð
/// <typeparam name="TEntity">B$8Cò 4Cä<CT=CÔ>C′ AD4ICÔ>D BC,ãÂ÷G— W  & Óí
public interface IUiOzuCache<TEntity> where TEntity : classÐ
{Ð
    /// <summary>Ð
    /// Aô@Dô<Cä9 CD>D BD4? C¢ BCT:D4ICT<D2 <C AD 8C$C CD0CÔ=D´E D$>C´LC¤> CD;Dò GD$5CÔ8Dòâ
    /// AÔ8 Cä4CÔ0 DD>D <C  1Cä;DÄHCR =CR <Cä6CTB Cô5D 5Ct0Cô8D 0D$L C„;C, >Dt8D BC„BDÂ MD$>D" ACô8D >C¢ 2D C
    /// </summary>Ð
    IReadOnlyList<TEntity> InMemoryItems { get; }Ð
Ð
    /// <summary>Ð
    /// B$>Dt:C  ?CT@C$>CÔ0Dt0C´LCÔ>C4> CÔ0Cô>C´=CT=C„O C¤MD„0 CD0CÔ=D´<C,Â ?Cä;D4GCT=CÔKCÄ8 CäB DD>D <D²í
    /// A$KCtKC$0CTBD O D BD >C4> Cä4C„= D 0Cr ?D 8 C„=C„FC„0C´8Ct0Dd8C,í
    /// </summary>Ð
    void Initialize(List<TEntity> initialItems);Ð
Ð
    /// <summary>Ð
    /// A 2D$>CÔ>CÄ=D´9 Aä B2Ô4C$8Cd>C¢ <D4BC FC„9. BT8D CD 3C„GCTAC¤8 D$>Dt=Câ >C =Cä2C´OCTB C¤>C´;CT:Dd8Dâ
    /// </summary>Ð
    void ApplyOzuDelta(EntityStateChangeEnum state, TEntity entity);Ð
Ð
    void SetMutationStrategy(IOzuMutationStrategy<TEntity> strategy);Ð
Ð
```

```
    /// <summary>Ð
    /// A$KCô>C´=Dô5D" ?D >C„7C$>C´LCÔCDâ >Cô5D 0Dd8Dâ GD$5CÔ8DòôDC„;DÄBD 0Dd8C, =C 4 C¤MD„5CÂ 2CÔCD$@C, ?CäB
    /// Aô5D 5DT2C BD´2C 5D" ?CäBCä: UI C, 7C IC„IC 5D" 5C4> CäB Cô0D 0C´;CT;DÄ=D´E CÄCD$0Dd8C’ 8Cr DCä=Cä2D´
    /// </summary>Ð
    TResult ExecuteInLock<TResult>(Func<IReadOnlyList<TEntity>, TResult> evaluator);Ð
}
</file>

<file path="Promatis.Net.UI/Data/Cache/UiOzuCache.cs">
using Promatis.Net.Data;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
/// <summary>Ð
/// B 5C ;C„7C FC„O C„7Cä;C„@Cä2C =CÔ>C4> C¤MD„0 C" >Cô5D 0D$8C$=Cä9 Cô0CÄOD$8 CD;Dò :Cä=C¤@CTBCÔ>C4> C„=D BC
/// </summary>Ð
public class UiOzuCache<TEntity> : IUiOzuCache<TEntity> where TEntity : classÐ
{Ð
    private IOzuMutationStrategy<TEntity> _mutationStrategy = new
FlatOzuMutationStrategy<TEntity>();Ð
    private readonly List<TEntity> _inMemoryItems = [];Ð
Ð
    // AT4C„=D´9 Cä1D¤5C¤B D 8CÔED >CÔ8Ct0Dd8C, 4C´O Cô>D$>C¤>C" GD$5CÔ8Dò 8 Ct0Cô8D 8Ð
    private readonly Lock _lockObject = new();Ð
Ð
    public IReadOnlyList<TEntity> InMemoryItems => _inMemoryItems;Ð
Ð
    public UiOzuCache() { }Ð
Ð
    public UiOzuCache(List<TEntity> initialItems)Ð
    {Ð
        Initialize(initialItems);Ð
    }Ð
Ð
    public void Initialize(List<TEntity> initialItems)Ð
    {Ð
        if (initialItems == null) throw new ArgumentNullException(nameof(initialItems));Ð
Ð
        lock (_lockObject)Ð
        {Ð
            _inMemoryItems.Clear();Ð
            _inMemoryItems.AddRange(initialItems);Ð
        }Ð
    }Ð
Ð
    public void SetMutationStrategy(IOzuMutationStrategy<TEntity> strategy)Ð
    {Ð
        _mutationStrategy = strategy ?? throw new ArgumentNullException(nameof(strategy));Ð
    }Ð
Ð
    public void ApplyOzuDelta(EntityStateChangeEnum state, TEntity entity)Ð
    {Ð
        // Aô>D$>C¢ !B4 AB <Cä=Cä?Cä;DÄ=Câ 7C EC$0D$KC$0CTB Ct0CÄ>C¢ =C  2D 5CÄO CÄCD$0Dd8C•
        lock (_lockObject)Ð
        {Ð
            _mutationStrategy.ApplyDelta(_inMemoryItems, state, entity, GetEntityId);Ð
        }Ð
    }Ð
Ð
    public TResult ExecuteInLock<TResult>(Func<IReadOnlyList<TEntity>, TResult> evaluator)Ð
    {Ð
        if (evaluator == null) throw new ArgumentNullException(nameof(evaluator));Ð
Ð
        // Aô>D$>C¢ T’ „&Æ ¦÷"’ <Cä=Cä?Cä;DÄ=Câ 7C EC$0D$KC$0CTB Ct0CÄ>C¢ =C  2D 5CÄO DD8C´LD$@C FC„8/C$KDt8D
        lock (_lockObject)Ð
        {Ð
            return evaluator(_inMemoryItems);Ð
        }Ð
    }Ð
Ð
    private object? GetEntityId(TEntity item)Ð
    {Ð
        Type? hasKeyInterface = item.GetType()Ð
            .GetInterfaces()Ð
            .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(IDomainObjectHasKey<>));Ð
```

```
Ð
        if (hasKeyInterface != null)Ð
        {Ð
            return hasKeyInterface.GetProperty("Id")?.GetValue(item);Ð
        }Ð
Ð
        return item.GetType().GetProperty("Id")?.GetValue(item);Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Data/IUiDataBroker.cs">
using Promatis.Net.Data;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
/// <summary>Ð
/// A¤>CÔBD 0C¤B C @Cä:CT@C  4C =CÔKDRÂ CCô@C 2C´ODäICT3Câ 2D´1Cä@Cä< C„AD$>Dt=C„:C  …6W'fW"ôõ¥R’ 8 D BD 0D$5
/// AÔ5 D >CD5D 6C„B CD5DD>C´BCÔKDR @CT6C„<Cä2 – Cô@Cä3D 0CÄ<C„AD" >C OCt0CÒ OC$=Câ AC¤>CÔDC„3D4@C„@Cä2C BDÂ
/// </summary>Ð
public interface IUiDataBroker<TEntity, TQueryState, TResultData> : IDisposable where TEntity :
classÐ
{Ð
    /// <summary>Ð
    /// Aô@C„7CÔ0C¢Â CC¤0CtKC$0DäIC„9, DtBCâ BCT:D4IC„9 C„=D BC =D  1D >C¤5D 0 CÔ0 DD>D <CR @C 1CäBC 5D" 2 D
    /// </summary>Ð
    bool IsInMemoryMode { get; }Ð
Ð
    /// <summary>Ð
    /// Ad5D BC¤0DÒ :Cä=DD8C4CD 0Dd8DÒ @C 1CäBD² =C ?D OCÄCDâ A B #A   / gRPC API (Server Mode).Ð
    /// </summary>Ð
    void ConfigureServerMode(Func<TQueryState, CancellationToken, Task<TResultData>>
serverDataProvider);Ð
Ð
    /// <summary>Ð
    /// Ad5D BC¤0DÒ :Cä=DD8C4CD 0Dd8DÒ @C 1CäBD² GCT@CT7 C„7Cä;C„@Cä2C =CÔ>CR ;Cä:C ;DÄ=Cä5 Aä B2ÔED 0CÔ8C´8D
    /// </summary>Ð
    void ConfigureInMemoryMode(Ð
        IUiOzuCache<TEntity> ozuCache,Ð
        Func<TQueryState, CancellationToken, Task<List<TEntity>>> serverDataLoader, // AÔ5D 5CD0CT< Ct=C =C„5
        Func<TQueryState, IReadOnlyList<TEntity>, TResultData> inMemoryEvaluator // B 0C >D$0CTB D  •&V DöæÇ”
    );Ð
Ð
    /// <summary>Ð
    /// B4=C„2CT@D 0C´LCÔ0DÒ BCäGC¤0 C AC„=DT@Cä=CÔ>C4> Ct0Cô@CäAC  4C =CÔKDR 2C„7D40C´8Ct0D$>D 0CÄ8 C„=D$5D
    /// ATAC´8 CÔ8 Cä4C„= D 5Cd8CÂ =CR 1D´; D :Cä=DD8C4CD 8D >C$0CÒ ?D >C4@C <CÄ8D BCä<, C$KCt>C$5D" –çf Æ–D÷
    /// </summary>Ð
    Task<TResultData> FetchDataAsync(TQueryState state, CancellationToken cancellationToken =
default);Ð
Ð
    /// <summary>Ð
    /// B$>Dt:C  2DT>CD0 CD;DÒ ?D >C @CäAC  3C´>C 0C´LCÔ>C4> D 8C4=C ;C  öäVçF–G”6öÖÖ–GFVB 2 C´>C¤0C´LCÔKC’ :
    /// </summary>Ð
    void HandleDatabaseCommit(EntityStateChangeEnum state, object entity);Ð
}
</file>

<file path="Promatis.Net.UI/Data/UiDataBroker.cs">
using Promatis.Net.Data;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
/// <summary>Ð
/// Aô;C BDD>D <CT=CÔKC’ 1D >C¤5D  4C =CÔKDRâ &CT=D$@C ;C„7Cä2C =CÔ> D4?D 0C$;Dô5D" ?CäAD$0C$:Cä9 CD0CÔ=D´E C
/// Aô>C´=CäAD$LDâ 0C AD$@C 3C„@D45D" T’ >D" :Cä=C¤@CTBCÔKDR 8CÔBCT@DD5C”ACä2 CD>CÄ5CÔ=D´E D ;D46C  1DÔ:CT=CD
/// </summary>Ð
/// <typeparam name="TEntity">B$8CÒ 4Cä<CT=CÔ>C’ AD4ICÔ>D BCƒÂ÷G– W  & Óí
public class UiDataBroker<TEntity, TQueryState, TResultData> : IUiDataBroker<TEntity, TQueryState,
TResultData>Ð
    where TEntity : classÐ
{Ð
    private Func<TQueryState, CancellationToken, Task<TResultData>>? _serverDataProvider;Ð
    private Func<TQueryState, CancellationToken, Task<List<TEntity>>>? _serverDataLoader;Ð
    private Func<TQueryState, IReadOnlyList<TEntity>, TResultData>? _inMemoryEvaluator;Ð
    private IUiOzuCache<TEntity>? _ozuCache;Ð
    private readonly Action? _onDataChangedNotifier;Ð
```

```
Ð
    private bool _isInitialized;Ð
    private bool _isInMemoryMode;Ð
    private bool _isOzuLoaded;Ð
Ð
    // A„=DD@C AD$@D4:D$CD 0 CD;Dò 4CT1C CCÔAC  ACTBCT2D´E D42CT4Cä<C´5CÔ8C’ 2 ServerModeÐ
    private CancellationTokenSource? _debounceCts;Ð
    private readonly object _debounceLock = new(); // At0CÄ>C¢ 4C´O Cô>D$>C¤>C 5Ct>Cô0D =Cä3Câ AC @CäAC  BC 9
    private const int DebounceDelayMs = 300; // A$@CT<CT=CÔ>CR >C¤=Câ >Cd8CD0CÔ8Dò 7C BC„HDÄO C"  AB ƒ3   <D
Ð
    public bool IsInMemoryMode => _isInMemoryMode;Ð
Ð
    public UiDataBroker(Action? onDataChangedNotifier = null)Ð
    {Ð
        _onDataChangedNotifier = onDataChangedNotifier;Ð
        DatabaseTriggerService.OnEntityCommitted += HandleGlobalEntityCommitted;Ð
    }Ð
Ð
    private void HandleGlobalEntityCommitted(EntityStateChangeEnum state, object entity)Ð
    {Ð
        HandleDatabaseCommit(state, entity);Ð
    }Ð
Ð
    public void ConfigureServerMode(Func<TQueryState, CancellationToken, Task<TResultData>>
serverDataProvider)Ð
    {Ð
        _serverDataProvider = serverDataProvider ?? throw new
ArgumentNullException(nameof(serverDataProvider));Ð
        _serverDataLoader = null;Ð
        _inMemoryEvaluator = null;Ð
        _ozuCache = null;Ð
        _isInMemoryMode = false;Ð
        _isInitialized = true;Ð
        _isOzuLoaded = false;Ð
    }Ð
Ð
    public void ConfigureInMemoryMode(Ð
        IUiOzuCache<TEntity> ozuCache,Ð
        Func<TQueryState, CancellationToken, Task<List<TEntity>>> serverDataLoader,Ð
        Func<TQueryState, IReadOnlyList<TEntity>, TResultData> inMemoryEvaluator)Ð
    {Ð
        _ozuCache = ozuCache ?? throw new ArgumentNullException(nameof(ozuCache));Ð
        _serverDataLoader = serverDataLoader ?? throw new
ArgumentNullException(nameof(serverDataLoader));Ð
        _inMemoryEvaluator = inMemoryEvaluator ?? throw new
ArgumentNullException(nameof(inMemoryEvaluator));Ð
        _serverDataProvider = null;Ð
        _isInMemoryMode = true;Ð
        _isInitialized = true;Ð
        _isOzuLoaded = false;Ð
    }Ð
Ð
    public async Task<TResultData> FetchDataAsync(TQueryState state, CancellationToken
cancellationToken = default)Ð
    {Ð
        if (!_isInitialized)Ð
        {Ð
            throw new InvalidOperationException($"A @Cä:CT@ CD0CÔ=D´E CD;Dò w·G— Vöb…DVçF—G’’äæ ÖWÒr =CR 1D´;
        }Ð
Ð
        if (_serverDataProvider != null)Ð
        {Ð
            return await _serverDataProvider(state, cancellationToken);Ð
        }Ð
Ð
        if (_isInMemoryMode && _ozuCache != null && _inMemoryEvaluator != null &&
_serverDataLoader != null)Ð
        {Ð
            if (!_isOzuLoaded)Ð
            {Ð
                List<TEntity> initialData = await _serverDataLoader(state, cancellationToken);Ð
                _ozuCache.Initialize(initialData);Ð
                _isOzuLoaded = true;Ð
            }Ð
Ð
            return _ozuCache.ExecuteInLock(items => _inMemoryEvaluator(state, items));Ð
```

```
        }Ð
Ð
        throw new InvalidOperationException("A¤@C„BC„GCTAC¤8C′ AC >C′ 2CÔCD$@CT=CÔ5C′ :Cä=DD8C4CD 0Dd8C, 1D >
    }Ð
Ð
    public void HandleDatabaseCommit(EntityStateChangeEnum state, object entity)Ð
    {Ð
        // Aô@Cä2CT@DÔ5CÃ¢ >D$=CäAC„BD O C´8 Cô@C„;CTBCT2D„8C′ 8Cr !B4 AB >C JCT:D" : D$8CôC CD0CÔ=D´E D$5C¤C
        if (entity is not TEntity typedEntity) return;Ð
Ð
        if (_isInMemoryMode)Ð
        {Ð
            // B  Ad AÂ ″âÔÔTÔõ%“¢  D4BC„@D45CÂ  At# CÄ3CÔ>C$5CÔ=Câ 1CT7 Ct0CD5D 6CT: CD;Dò ACäED 0CÔ5CÔ8Dò F
            if (_ozuCache != null && _isOzuLoaded)Ð
            {Ð
                _ozuCache.ApplyOzuDelta(state, typedEntity);Ð
            }Ð
Ð
            // B @C 7D2 ?C„=C 5CÂ T′Â BC : C¤0C¢ ACTBCT2Cä3Câ 7C ?D >D 0 CÔ5 C CCD5D" r 2D´GC„AC´5CÔ8Dò ?D >C
            _onDataChangedNotifier?.Invoke();Ð
        }Ð
        elseÐ
        {Ð
            // B  Ad AÂ 4U%dU"ÔÔôDS¢  C¤;DäGC 5CÂ 4CT1C CCÔA CD;Dò ?D 5CD>D$2D 0D″5CÔ8Dò ACTBCT2Cä3Câ HD$>D <
            lock (_debounceLock)Ð
            {Ð
                // ATAC´8 Cô@CT4D´4D4IC„9 D$0C″<CT@ CTICR BC„:C ; (Cô0Dt:C  8Ct<CT=CT=C„9 Cô@Cä4Cä;Cd0CTBD O)
                _debounceCts?.Cancel();Ð
                _debounceCts?.Dispose();Ð
Ð
                // B >Ct4C 5CÂ =Cä2D´9 D$>C¤5CÒ 7C 4CT@Cd:C•
                _debounceCts = new CancellationTokenSource();Ð
                var token = _debounceCts.Token;Ð
Ð
                // At0CôCD :C 5CÂ DCä=Cä2Cä5 Cä6C„4C =C„5 Ct0D$8D„LDò 2 A  Ð
                Task.Delay(DebounceDelayMs, token).ContinueWith(task =>Ð
                {Ð
                    // A$KCtKC$0CT< Cô8CÔ>C¢ T′ BCä;DÄ:Câ 5D ;C, 7C 4C GC  CD ?CTHCÔ> Ct0C$5D HC„;C ADÂ 8 CÔ5
                    if (task.Status == TaskStatus.RanToCompletion && !token.IsCancellationRequested)Ð
                    {Ð
                        _onDataChangedNotifier?.Invoke();Ð
                    }Ð
                }, TaskScheduler.Default);Ð
            }Ð
        }Ð
    }Ð
Ð
    public void Dispose()Ð
    {Ð
        // AäBCô8D KC$0CT<D O CäB D BC BC„GCTAC¤>C4> DÔ2CT=D$0 D ;D46C K D$@C„3C45D >C" !B4 AM
        DatabaseTriggerService.OnEntityCommitted -= HandleGlobalEntityCommitted;Ð
Ð
        // At0Dt8D″0CT< C„=DD@C AD$@D4:D$CD C CD5C 0D4=D 0 Cô@C, CCÔ8DtBCä6CT=C„8 C @Cä:CT@C  DCä@CÄ>C•
        lock (_debounceLock)Ð
        {Ð
            _debounceCts?.Cancel();Ð
            _debounceCts?.Dispose();Ð
            _debounceCts = null;Ð
        }Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Extensions/MudColorExtensions.cs">
using MudBlazor;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
public static class MudColorExtensionsÐ
{Ð
    public static Color GetActionColor(this string? action)Ð
    {Ð
        if (string.IsNullOrWhiteSpace(action)) return Color.Default;Ð
Ð
        return action.ToLower().Trim() switchÐ
        {Ð
```

```
            "create" or "insert" or "added" or "CD>C 0C$;CT=C„5" or "D >Ct4C =C„5" => Color.Success,Ð
            "update" or "edit" or "modified" or "C„7CÄ5CÔ5CÔ8CR" ÷" $@CT4C :D$8D >C$0CÔ8CR" Óâ 6öÆ÷"åv &æ–ærÍ
            "delete" or "remove" or "deleted" or "softdeleted" or "D44C ;CT=C„5" => Color.Error,Ð
            _ => Color.DefaultÐ
        };Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Interfaces/IHasSelectedData.cs">
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// B4=C„2CT@D 0C´LCÔKC’ 8CÔBCT@DD5C”A-CÄ0D :CT@ CD;Dò 2C„7D40C´8Ct0D$>D >C"Â :CäBCä@D´< CÔ5Cä1DT>CD8CÄ> Ð
/// D 8CÔED >CÔ8Ct8D >C$0D$L DD>C¤CD  AD$@Cä:C,ôCCt;C  8 Cô>C´CDt0D$L C„<CôCC´LD K Cô5D 5D 8D >C$:C,í
/// </summary>Ð
public interface IHasSelectedData<TEntity> where TEntity : classÐ
{Ð
    TEntity? SelectedData { get; set; }Ð
    Action? OnContextUpdated { get; set; }Ð
}
</file>

<file path="Promatis.Net.UI/IUiModule.cs">
namespace Promatis.Net.UI;Ð
Ð
/// <summary>Ð
/// A¤>CÔBD 0C¤B CD;Dò 4C„=C <C„GCTAC¤8DR <Cä4D4;CT9 Cô>C´LCt>C$0D$5C´LD :Cä3Câ 8CÔBCT@DD5C”AC í
/// </summary>Ð
public interface IUiModuleÐ
{Ð
    /// <summary>Ð
    /// AäBCä1D 0Cd0CT<Cä5 C„<Dò <Cä4D4;Dò 2 D 8D BCT<CR „8D ?Cä;DÄ7D45D$ADò 4C´O D :C =CT@C ’í
    /// </summary>Ð
    string Name { get; }Ð
Ð
    /// <summary>Ð
    /// A$>Ct2D 0D”0CTB D ?C„ACä: DÔ;CT<CT=D$>C" =C 2C„3C FC„8 CD;Dò 1Cä:Cä2Cä3Câ <CT=Dâ ×VDG& vW"í
    /// Bt5D$2CT@D$KC’ ?C @C <CTBD  >D$2CTGC 5D" 7C  "ÔCD >C$=CT2D4N C4@D4?CÔ8D >C$:D2 MC´5CÄ5CÔBCä2.Ð
    /// </summary>Ð
    /// <returns>A¤>C´;CT:Dd8Dò :Cä@D$5Cd5C“¢ „ C 7C$0CÔ8CRÂ !D KC´:C Â  C¤>CÔ:C Â  C 7C$0CÔ8CT D CCô?D²“Â÷&W
    IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems();Ð
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogContext.cs">
using Microsoft.Extensions.DependencyInjection;Ð
using MudBlazor;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Service;Ð
using Promatis.Net.UI.Components;Ð
using Promatis.Net.UI.Pages.AuditLogs.Toolbar;Ð
Ð
namespace Promatis.Net.UI.Pages.AuditLogs;Ð
Ð
/// <summary>Ð
/// A¤>CÔBCT:D B D BD 0CÔ8DdK C´>C48D >C$0CÔ8Dò 0D44C„BC â
/// BD>C¤CD 8D CCTBD O C„AC¤;DäGC„BCT;DÄ=Câ =C  ;Cä3C„:CR DC„;DÄBD 0Dd8C, 8 CÄ0Cô?C„=C45 CÔ0D 0CÄ5D$@Cä2 Cô>C
/// </summary>Ð
public class AuditLogContext : GridContext<AuditLog, Guid>Ð
{Ð
    private readonly IAuditLogService _auditLogService;Ð
    private readonly Action _onFilterChanged; // A„!Aô A  A´ AÔ : B =Cä2C  AD$@Cä3Cä5 readonly D 2Cä9D BC$>Ð
Ð
    public override string TopZoneHeight => "48px";Ð
Ð
    /// <summary>Ð
    /// A„!Aô A  A´ AÔ : A¤>CÔAD$@D4:D$>D  ?D 8CÔ8CÄ0CTB D BD >C4> Action C¤>C´;C 5C¢ 4C´O Cô@Dô<Cä9 D 5C :D$
    /// AÔ8C¤0C¤8DR 7CÔ0Dt5CÔ8C’ ?Câ CCÄ>C´GC =C„N (= null) CD;Dò D’Ô2C ;C„4C FC„8 C >C´LD„5 CÔ5D"
    /// </summary>Ð
    public AuditLogContext(IServiceProvider serviceProvider, Action onFilterChanged)Ð
        : base(serviceProvider, isInMemoryMode: false, onDataChangedNotifier: onFilterChanged)Ð
    {Ð
        _onFilterChanged = onFilterChanged ?? throw new
ArgumentNullException(nameof(onFilterChanged));Ð
        _auditLogService = serviceProvider.GetRequiredService<IAuditLogService>();Ð
```

```
Ð
        // B @C 7D2 =C ?Cä;CÔOCT< D$CC´1C @ D BC BC„GCTAC¤8CÄ8 DÔ;CT<CT=D$0CÄ8Ð
        AddControl(new AuditToolbarDivider());Ð
        AddControl(new AuditExportButton());Ð
    }Ð
Ð
    /// <summary>Ð
    /// A„!Aô A  A´ AÔ : A$0D„0 Cä@C„3C„=C ;DÄ=C O CD8CÔ0CÄ8Dt5D :C O CD>D >C 8D :C  BD4;C 0D 0.Ð
    /// A$KCtKC$0CTBD O C„7 OnAfterRenderAsync Razor-D BD 0CÔ8DdK, C¤>C44C  MC¤@C = D46CR 3C @C =D$8D >C$0CÔ=
    /// </summary>Ð
    public void InitializeFilters(List<string> entityNames)Ð
    {Ð
        // Aô>D$>C¤>C 5Ct>Cô0D =Câ GCT@CT7 C„=C¤0CôAD4;C„@Cä2C =CÔKC’ <CTBCä4 C 0Ct>C$>C4> C¤;C AD 0 (C„;C, =
        // C$>Ct2D 0D”0CT< CÔ0D$8C$=D4N C$AD$0C$:D2 DC„;DÄBD >C" 2 D 0CÄ>CR =C GC ;Câ BD4;C 0D 0 D ;CT2C  =C
        _controls.Insert(0, new AuditEntitySelect(entityNames, _onFilterChanged));Ð
        _controls.Insert(1, new AuditActionSelect(_onFilterChanged));Ð
        _controls.Insert(2, new AuditPeriodPicker(_onFilterChanged));Ð
Ð
        NotifyStateChanged();Ð
    }Ð
Ð
    protected override async Task<GridData<AuditLog>> FetchDataFromServerAsync(GridState<AuditLog>
state, CancellationToken ct)Ð
    {Ð
        // A„!Aô A  A´ AÔ : A$>Ct2D 0D”0CT< C$0D, >D 8C48CÔ0C´LCÔKC’Â @C 1CäBC ND”8C’ ?Cä8D : C¤>CÔBD >C´>C"
        IUiControl? entityControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_entity");Ð
        IUiControl? actionControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_action");Ð
        IUiControl? periodControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_period");Ð
Ð
        string? selectedEntity = entityControl is IHasValue ev ? ev.Value as string : null;Ð
        DateRange? selectedPeriod = periodControl is IHasValue pv ? pv.Value as DateRange : null;Ð
        string? selectedAction = actionControl is AuditActionSelect av ?
av.GetSelectedActionValue() : null;Ð
Ð
        if (selectedEntity == "A$ACR AD4ICÔ>D BC,"•
            selectedEntity = null;Ð
Ð
        DateTime fromDate = selectedPeriod?.Start ?? DateTime.Today.AddDays(-7);Ð
        DateTime toDate = selectedPeriod?.End ?? DateTime.Today;Ð
Ð
        fromDate = DateTime.SpecifyKind(fromDate.Date, DateTimeKind.Utc);Ð
        toDate = DateTime.SpecifyKind(toDate.Date.AddDays(1).AddTicks(-1), DateTimeKind.Utc);Ð
Ð
        var searchRequest = new AuditLogSearchRequest(Ð
            FromDate: fromDate,Ð
            ToDate: toDate,Ð
            EntityName: selectedEntity,Ð
            Action: selectedAction,Ð
            PageIndex: state.Page,Ð
            PageSize: state.PageSizeÐ
        );Ð
Ð
        PagedResult<AuditLog> pagedResult = await _auditLogService.SearchLogsAsync(searchRequest,
ct);Ð
Ð
        return new GridData<AuditLog>Ð
        {Ð
            Items = pagedResult.Items,Ð
            TotalItems = pagedResult.TotalCountÐ
        };Ð
    }Ð
Ð
    protected override Task OpenDialogWindowAsync(AuditLog model, bool isNew) => Task.CompletedTask;Ð
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor">
@page "/audit-logs"Ð
@using Promatis.Net.DomainÐ
Ð
<CascadingValue Value="Context">Ð
    <WorkspacePage>Ð
Ð
        <!-- 1. A$ B %AÔ/Bò  Aä A ¢  C$BCä<C BC„GCTAC¤8C’ ?Cä;C„<Cä@DD=D´9 D$CC´1C @ (AD8CÔ0CÄ8Dt5D :C„5 C¤>C
        <TopContent>Ð
            <UiToolbar Controls="@Context.Controls" />Ð
```

```
        </TopContent>Đ
Đ
        <!-- 2. Bd AÔ"B  A´,AÔ Bò  Aä A ¢ "C 1C´8Dt=C O D 5D$:C Â @C 1CäBC ND”0Dò 2 DD>CÔ>C$>CÂ ACT@C$5D =Cä<
        <BodyContent>Đ
            <GridPage TEntity="AuditLog"Đ
                      @ref="_grid"Đ
                      ServerDataProvider="@Context.GetDataAsync"Đ
                      WithPagination="true"Đ
                      RowsPerPage="20">Đ
                <GridColumns>Đ
                    <PropertyColumn T="AuditLog" TProperty="DateTime" Property="x => x.ChangedAt"
Title="AD0D$0 C, 2D 5CÄO" Format="dd.MM.yyyy HH:mm:ss" HeaderStyle="width: 200px;" />Đ
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangedBy"
Title="Aô>C´LCt>C$0D$5C´L" HeaderStyle="width: 150px;" />Đ
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.EntityName"
Title="B$8Cò AD4ICÔ>D BC," †V FW%7G–ÆSÒ'v–GFƒ¢ #   ƒ²" óí
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.Action"
Title="Aä?CT@C FC„O" HeaderStyle="width: 120px;" />Đ
                    <PropertyColumn T="AuditLog" TProperty="Guid" Property="x => x.EntityId"
Title="ID B CD”=CäAD$8" HeaderStyle="width: 330px;" />Đ
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangesJson"
Title="A„7CÄ5CÔ5CÔ8Dò „¥4ôâ’" †V FW%7G–ÆSÒ'v–GFƒ¢  WFó²" 6Æ 73Ò'FW‡B×G'Væ6 FR" óí
                </GridColumns>Đ
            </GridPage>Đ
        </BodyContent>Đ
Đ
    </WorkspacePage>Đ
</CascadingValue>
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor.cs">
using Microsoft.AspNetCore.Components;Đ
using Microsoft.Extensions.DependencyInjection;Đ
using Promatis.Net.Domain;Đ
using Promatis.Net.Service;Đ
using Promatis.Net.UI.Components.Workspaces;Đ
Đ
namespace Promatis.Net.UI.Pages.AuditLogs;Đ
Đ
public partial class AuditLogPage : ComponentBaseĐ
{Đ
    private GridPage<AuditLog>? _grid;Đ
    private bool _isLoaded;Đ
Đ
    [Inject]Đ
    protected IServiceProvider PageServiceProvider { get; set; } = null!;Đ
Đ
    [Inject]Đ
    protected IAuditLogService AuditLogService { get; set; } = null!;Đ
Đ
    protected AuditLogContext Context { get; set; } = null!;Đ
Đ
    protected override void OnInitialized()Đ
    {Đ
        base.OnInitialized();Đ
Đ
        // AÄ AÔ A$ AÔ Aâ ACä7CD0CT< CôCD BCä9 C¤>CÔBCT:D B. B BD 0CÔ8Dd0 D 5CÔ4CT@C„BD O C 5Cr 7C 4CT@Cd5C¢
        Context = new AuditLogContext(PageServiceProvider, onFilterChanged: RefreshGrid);Đ
    }Đ
Đ
    /// <summary>Đ
    /// A¤0CÔ>CÔ8Dt5D :C O D$>Dt:C  7C ?D4AC¤0 D$OCd5C´KDR 7C ?D >D >C"â !D 0C 0D$KC$0CTB Aô B  AR BCä3CâÂ
    /// C¤0C¢ ?Cä;DÄ7Cä2C BCT;DÂ CCd5 D42C„4CT; C4>D$>C$CDâ AD$@C =C„FD2 =C  MC¤@C =CRí
    /// </summary>Đ
    protected override async Task OnAfterRenderAsync(bool firstRender)Đ
    {Đ
        await base.OnAfterRenderAsync(firstRender);Đ
Đ
        // A$KCô>C´=DÔ5CÂ 7C 3D CCt:D2 BCä;DÄ:Câ >CD8CÒ @C 7 Cô@C, ?CT@C$>CÂ @CT=CD5D 5 DÔ:D 0CÔ0Đ
        if (firstRender)Đ
        {Đ
            // B ?Cä:Cä9CÔ> D :C GC„2C 5CÂ CCÔ8C¤0C´LCÔKCR 8CÄ5CÔ0 D CD”=CäAD$5C’ 8Cr !B4 AB  ÷7Fw&U5 Í
            List<string> availableEntities = await AuditLogService.GetAvailableEntityNamesAsync();Đ
Đ
            // AÔ5D 5CD0CT< CD0CÔ=D´5 C" :Cä=D$5C¤AD" 4C´O CD8CÔ0CÄ8Dt5D :Cä9 C45CÔ5D 0Dd8C, ACT;CT:D$0 C, ?C
            Context.InitializeFilters(availableEntities);Đ
```

```
Ð
            _isLoaded = true;Ð
            StateHasChanged(); // Aô5D 5D 8D >C$KC$0CT< D$CC´1C @ C, 7C ?D4AC¤0CT< Cô5D 2D4N Ct0C4@D47C¤C C´>
        }Ð
    }Ð
Ð
    protected void RefreshGrid()Ð
    {Ð
        // At0Cô@CTIC 5CÂ ?D 8CÔCCD8D$5C´LCÔ>CR >C =Cä2C´5CÔ8CR 3D 8CD0 CäB D$@C„3C45D >C" DC„;DÄBD >C"Í
        // Cô>C¤0 DÔBC, DC„;DÄBD K CTICR ?CT@C$8Dt=Câ =CR =C AD$@Cä5CÔK C" öä gFW%&VæFW$ 7–æ=
        if (_isLoaded && _grid != null)Ð
        {Ð
            _ = _grid.ReloadServerData();Ð
        }Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditActionSelect.cs">
using Promatis.Net.UI.Components;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;Ð
Ð
public class AuditActionSelect : BaseUiControl, IHasValue, IHasOptionsÐ
{Ð
    private readonly Action _onChanged;Ð
    private readonly Dictionary<string, string> _actionMapping = new()Ð
    {Ð
        { "A$ACR >Cô5D 0Dd8C,"Â "" ÒÍ
        { "B >Ct4C =C„5", "Added" },Ð
        { "A„7CÄ5CÔ5CÔ8CR"Â $ÖöF–f–VB" ÒÍ
        { "B44C ;CT=C„5", "Deleted" },Ð
        {"AÄOC4:Cä5 D44C ;CT=C„5", "SoftDeleted"}Ð
    };Ð
Ð
    public override string Id => "audit_filter_action";Ð
    public override Type ComponentType => typeof(SelectRenderBase);Ð
    public override string Title => "Aä?CT@C FC„O";Ð
Ð
    public object? Value { get; set; }Ð
    public IEnumerable<string> Options => _actionMapping.Keys;Ð
Ð
    public AuditActionSelect(Action onChanged)Ð
    {Ð
        _onChanged = onChanged;Ð
        Value = "A$ACR >Cô5D 0Dd8C,#² òò  CTDCä;D$=Cä5 C$KC @C =CÔ>CR BCT:D BCä2Cä5 Ct=C GCT=C„5Ð
    }Ð
Ð
    // AÄ5D$>CB 4C´O Cô>C´CDt5CÔ8Dò AC„AD$5CÄ=Cä3Câ 7CÔ0Dt5CÔ8Dò „AD$@Cä:C, 4C´O API) C„7 C$KC @C =CÔ>C4> D$5
    public string? GetSelectedActionValue()Ð
    {Ð
        if (Value is string selectedText && _actionMapping.TryGetValue(selectedText, out string?
sysValue))Ð
        {Ð
            return string.IsNullOrEmpty(sysValue) ? null : sysValue;Ð
        }Ð
        return null;Ð
    }Ð
Ð
    protected override Task HandleTriggerAsync(object? targetData)Ð
    {Ð
        _onChanged.Invoke();Ð
        return Task.CompletedTask;Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditEntitySelect.cs">
using Promatis.Net.UI.Components;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;Ð
Ð
public class AuditEntitySelect : BaseUiControl, IHasValue, IHasOptionsÐ
{Ð
```

```
    private readonly Action _onChanged;Ð
Ð
    public override string Id => "audit_filter_entity";Ð
    public override Type ComponentType => typeof(SelectRenderBase);Ð
    public override string Title => "B$8Cò AD4ICÔ>D BC,#½
Ð
    public object? Value { get; set; }Ð
    public IEnumerable<string> Options { get; set; }Ð
Ð
    public AuditEntitySelect(List<string> availableEntities, Action onChanged)Ð
    {Ð
        _onChanged = onChanged;Ð
        Value = "A$ACR AD4ICÔ>D BC,#½
Ð
        // BD>D <C„@D45CÂ 4C„=C <C„GCTAC¤8C’ ACô8D >C¢ >CôFC„9 CÔ0 CäACÔ>C$5 CD0CÔ=D´E C„7 A  Ð
        var list = new List<string> { "A$ACR AD4ICÔ>D BC," Ó½
        list.AddRange(availableEntities);Ð
        Options = list;Ð
    }Ð
Ð
    protected override Task HandleTriggerAsync(object? targetData)Ð
    {Ð
        _onChanged.Invoke();Ð
        return Task.CompletedTask;Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditExportButton.cs">
using MudBlazor;Ð
using Promatis.Net.UI.Components;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;Ð
Ð
public class AuditExportButton : BaseUiControlÐ
{Ð
    public override string Id => "audit_action_export";Ð
    public override Type ComponentType => typeof(ButtonRenderBase); // B 5CÔ4CT@CT@ Dô4D 0Ð
    public override string Title => "A$KC4@D47C„BDÂ#½
    public override string Icon => Icons.Material.Filled.Download;Ð
    public override string Tooltip => "A$KC4@D47C„BDÂ >D$DC„;DÄBD >C$0CÔ=D´5 C´>C48 C" W†6VÂ#½
Ð
    public AuditExportButton()Ð
    {Ð
        Alignment = UiControlAlignment.Right; // B 4C$8C40CT< C¤=Cä?C¤C DÔ:D ?Cä@D$0 C$?D 0C$>Ð
    }Ð
Ð
    protected override async Task HandleTriggerAsync(object? targetData)Ð
    {Ð
        // A„<C„BC„@D45CÂ 4Cä;C4CDâ 2D´3D CCt:D2 r :CÔ>Cô:C  0C$BCä<C BC„GCTAC¤8 Ct0C ;Cä:C„@D45D$ADò 8 Cô>C¤
        await Task.Delay(2000);Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditPeriodPicker.cs">
using MudBlazor;Ð
using Promatis.Net.UI.Components;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;Ð
Ð
public class AuditPeriodPicker : BaseUiControl, IHasValueÐ
{Ð
    private readonly Action _onChanged;Ð
Ð
    public override string Id => "audit_filter_period";Ð
    public override Type ComponentType => typeof(DateRangeRenderBase); // B 5CÔ4CT@CT@ Dô4D 0Ð
    public override string Title => "Aô5D 8Cä4 C´>C4>C"#½
Ð
    public object? Value { get; set; }Ð
Ð
    public AuditPeriodPicker(Action onChanged)Ð
    {Ð
        _onChanged = onChanged;Ð
```

```
        Value = new DateRange(DateTime.Today.AddDays(-7), DateTime.Today);Ð
    }Ð
Ð
    protected override Task HandleTriggerAsync(object? targetData)Ð
    {Ð
        _onChanged.Invoke(); // Aô5D 8Cä4 C„7CÄ5CÔ8C´ADò r 4C 5CÂ 8CÄ?D4;DÄA D BD 0CÔ8Dd5Ð
        return Task.CompletedTask;Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditToolbarDivider.cs">
using Promatis.Net.UI.Components;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;Ð
Ð
public class AuditToolbarDivider : BaseUiControlÐ
{Ð
    private readonly string _id = "audit_toolbar_divider_" + Guid.NewGuid().ToString("N");Ð
Ð
    public override string Id => _id;Ð
    public override Type ComponentType => typeof(DividerRenderBase); // A„ACô>C´LCtCCT< D$>D" 6CR 1C 7Cä2D´9
Ð
    public AuditToolbarDivider()Ð
    {Ð
        Alignment = UiControlAlignment.Right; // BÔBCäB D 0Ct4CT;C„BCT;DÂ 3C @C =D$8D >C$0CÔ=Câ ACô@C 2C
    }Ð
Ð
    protected override Task HandleTriggerAsync(object? targetData)Ð
    {Ð
        return Task.CompletedTask;Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Pages/Dashboard.razor">
@page "/"Ð
Ð
<MudText Typo="Typo.h4" Class="mb-4">AÄ>CDCC´L Master Data Management</MudText>Ð
Ð
<MudGrid>Ð
    <MudItem xs="12" sm="6" md="4">Ð
        <MudPaper Class="d-flex flex-row pt-6 pb-4 px-4" Style="height:100px;">Ð
            <MudIcon Icon="@Icons.Material.Filled.Storage" Color="Color.Primary" Size="Size.Large"
Class="mr-4" />Ð
            <div>Ð
                <MudText Typo="Typo.subtitle2" Class="mud-text-secondary">B BC BD4A Cô>CD:C´NDt5CÔ8Dò : A  </
                <MudText Typo="Typo.h6">A :D$8C$=Câ …FW7D& 6R“Âô×VEFW‡Cí
            </div>Ð
        </MudPaper>Ð
    </MudItem>Ð
    Ð
    <MudItem xs="12" sm="6" md="8">Ð
        <MudPaper Class="pa-4" Style="height:100px;">Ð
            <MudText Typo="Typo.subtitle1">AD>C @Câ ?Cä6C ;Cä2C BDÂ 2 D 8D BCT<D2  &öÖ F—2 U% Âô×VEFW‡Cí
            <MudText Typo="Typo.body2" Class="mud-text-secondary">Ð
                A$ACR 8CÔBCT@DD5C”AD² ?Cä4C4@D46CT=D² 4C„=C <C„GCTAC¤8 C„7 CÔ5Ct0C$8D 8CÄKDR AC´>CT2 Razor Cl
            </MudText>Ð
        </MudPaper>Ð
    </MudItem>Ð
</MudGrid>
</file>

<file path="Promatis.Net.UI/Pages/Dashboard.razor.css">
.my-component {Ð
    border: 2px dashed red;Ð
    padding: 1em;Ð
    margin: 1em 0;Ð
    background-image: url('background.png');Ð
}
</file>

<file path="Promatis.Net.UI/Promatis.Net.UI.csproj">
<Project Sdk="Microsoft.NET.Sdk.Razor">Ð
Ð
```

```
  <PropertyGroup>Ð
    <TargetFramework>net10.0</TargetFramework>Ð
    <Nullable>enable</Nullable>Ð
    <ImplicitUsings>enable</ImplicitUsings>Ð
  </PropertyGroup>Ð
Ð
  <ItemGroup>Ð
    <Compile Remove="AuditLog\**" />Ð
    <Compile Remove="Components\References\**" />Ð
    <Compile Remove="Models\**" />Ð
    <Content Remove="AuditLog\**" />Ð
    <Content Remove="Components\References\**" />Ð
    <Content Remove="Models\**" />Ð
    <EmbeddedResource Remove="AuditLog\**" />Ð
    <EmbeddedResource Remove="Components\References\**" />Ð
    <EmbeddedResource Remove="Models\**" />Ð
    <None Remove="AuditLog\**" />Ð
    <None Remove="Components\References\**" />Ð
    <None Remove="Models\**" />Ð
  </ItemGroup>Ð
Ð
Ð
  <ItemGroup>Ð
    <SupportedPlatform Include="browser" />Ð
  </ItemGroup>Ð
Ð
  <ItemGroup>Ð
    <PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="10.0.8" />Ð
    <PackageReference Include="Microsoft.AspNetCore.Http.Abstractions" Version="2.3.10" />Ð
    <PackageReference Include="MudBlazor" Version="9.4.0" />Ð
  </ItemGroup>Ð
Ð
  <ItemGroup>Ð
    <ProjectReference Include="..\..
\MesCore\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />Ð
    <ProjectReference Include="..\Configuration\Promatis.Net.Configuration.csproj" />Ð
    <ProjectReference Include="..\Service\Promatis.Net.Service.csproj" />Ð
  </ItemGroup>Ð
Ð
</Project>
</file>

<file path="Promatis.Net.UI/Theme/PromatisTheme.cs">
using MudBlazor;Ð
Ð
namespace Promatis.Net.Ui.Theme;Ð
Ð
public static class PromatisThemeÐ
{Ð
    public static MudTheme DefaultTheme => new()Ð
    {Ð
        PaletteLight = new PaletteLightÐ
        {Ð
            Primary = Colors.Blue.Lighten1,       // A4;C 2CÔKC′ :Cä@Cô>D 0D$8C$=D´9 Dd2CTB (C¤=Cä?C¤8, C :D$
            Secondary = Colors.Blue.Lighten2,     // A$ACô>CÄ>C40D$5C´LCÔKC′ FC$5D" „2D$>D >D BCT?CT=CÔKCR MC
Ð
            AppbarBackground = "#EAEAEA",         // BD>CÒ 2CT@DT=CT9 Cô0CÔ5C´8 (MudAppBar) C" 4CÔ5C$=Cä< D 5
            AppbarText = "#2C3E50",               // Bd2CTB D$5C¤AD$0, Ct0C4>C´>C$:Cä2 C, 8C¤>CÔ>C¢ 2 C$5D EC
            DrawerBackground = "#EAEAEA",         // BD>CÒ 1Cä:Cä2Cä3Câ <CT=Dâ =C 2C„3C FC„8 (MudDrawer)Ð
            Background = "#F9F9F9",               // Aä1D”8C′ DCä=Cä2D´9 Dd2CTB Cô>CD;Cä6C¤8 C$ACTE D BD 0CÔ8
            Surface = Colors.Shades.White,         // BD>CÒ :Cä=D$5C”=CT@Cä2 MudPaper, C¤0D BCäGCT: MudCard C
        },Ð
        PaletteDark = new PaletteDarkÐ
        {Ð
            Primary = Colors.Blue.Lighten1,       // A4;C 2CÔKC′ FC$5D" 2 CÔ>Dt=Cä< D 5Cd8CÄ5 (CÄOC4:C„9 D 8C
            Secondary = Colors.Blue.Lighten2,     // A$ACô>CÄ>C40D$5C´LCÔKC′ FC$5D" 2 CÔ>Dt=Cä< D 5Cd8CÄ5Ð
Ð
            AppbarBackground = "#0D0D0D",         // BD>CÒ 2CT@DT=CT9 Cô0CÔ5C´8 (MudAppBar) C" 3C´CC >C¤>CÂ G
            AppbarText = Colors.Shades.White,     // Bd2CTB D$5C¤AD$0 C, 8C¤>CÔ>C¢ 2 D„0Cô:CR „:Cä=D$@C AD$=D
Ð
            Background = "#121212",               // A4;D41Cä:C„9 D$QCÄ=D´9 DD>CÒ AD$@C =C„F Cô>CD;Cä6C¤8Ð
            Surface = "#1E1E1E",                  // A4@C DC„BCä2D´9 DD>CÒ 4C´O MudPaper, C¤>CÔBCT=D$0 C$:C´0
            DrawerBackground = "#1E1E1E",         // BD>CÒ 1Cä:Cä2Cä9 Cô0CÔ5C´8 CÔ0C$8C40Dd8C, 2 D$QCÄ=Cä9 D$
            DrawerText = "#E0E0E0",               // Bd2CTB D AD´;Cä: C, BCT:D BC  2 C >C¤>C$>CÂ <CT=Dí
            TextPrimary = "#E0E0E0",              // AäACÔ>C$=Cä9 Dd2CTB D$5C¤AD$0 CÔ0 D BD 0CÔ8Dd0DR „<Dô3C¤
            TextSecondary = "#A0A0A0",             // Bd2CTB CD;Dò 2D$>D >D BCT?CT=CÔ>C4> D$5C¤AD$0 C, ?Cä4D
```

```
        },Ð
        LayoutProperties = new LayoutPropertiesÐ
        {Ð
            DrawerWidthLeft = "260px",Ð
            DrawerWidthRight = "300px"Ð
        }Ð
    };Ð
}
</file>

<file path="Promatis.Net.UI/UiModule.cs">
using MudBlazor;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
public class UiModule : IUiModuleÐ
{Ð
    public string Name => GetType().Assembly.GetName().Name ?? "Unknown.Module";Ð
Ð
    public IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems()Ð
    {Ð
        return new List<(string Title, string Href, string Icon, string? Group)>Ð
        {Ð
            // BÔ;CT<CT=D" 2CT@DT=CT3Câ CD >C$=Dò „2CÔ5 Cô0Cô>C¢•
            ("A4;C 2CÔ0Dò"Â "ò"Â –6öç2äÖ FW&– Âäf–ÆÆVBäF 6†&ö &BÂ çVÆÂ’Í
            Ð
            // B 0Ct4CT; D 8D BCT<CÔ>C4> C 4CÄ8CÔ8D BD 8D >C$0CÔ8Dý
            ("AdCD =C ; C CCD8D$0", "/audit-logs", Icons.Material.Filled.History, "A 4CÄ8CÔ8D BD 8D >C$0CÔ8CR
        };Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/wwwroot/App.css">
html, body {Ð
    font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif;Ð
}Ð
Ð
a, .btn-link {Ð
    color: #006bb7;Ð
}Ð
Ð
.btn-primary {Ð
    color: #fff;Ð
    background-color: #1b6ec2;Ð
    border-color: #1861ac;Ð
}Ð
Ð
.btn:focus, .btn:active:focus, .btn-link.nav-link:focus, .form-control:focus, .form-check-
input:focus {Ð
    box-shadow: 0 0 0 0.1rem white, 0 0 0 0.25rem #258cfb;Ð
}Ð
Ð
.content {Ð
    padding-top: 1.1rem;Ð
}Ð
Ð
h1:focus {Ð
    outline: none;Ð
}Ð
Ð
.valid.modified:not([type=checkbox]) {Ð
    outline: 1px solid #26b050;Ð
}Ð
Ð
.invalid {Ð
    outline: 1px solid #e50000;Ð
}Ð
Ð
.validation-message {Ð
    color: #e50000;Ð
}Ð
Ð
.blazor-error-boundary {Ð
    background: url(data:image/svg+xml;base64,PHN2ZyB3aWR0aD0iNTYiIGhlaWdodD0iNDkiIHhtbG5zPSJodHRwOi
8vd3d3LnczLm9yZy8yMDAwL3N2ZyIgeG1sbnM6eGxpbms9Imh0dHA6Ly93d3cudzMub3JnLzE5OTkveGxpbmsiIG92ZXJmbG93PS
```

```
JoaWRkZW4iPjxkZWZzPjxjbGlwUGF0aCBpZD0iY2xpcDAiPjxyZWN0IHg9IjIzNSIgeT0iNTEiIHdpZHRoPSI1NiIgaGVpZ2h0PS
I0OSIvPjwvY2xpcFBhdGg+PC9kZWZzPjxnIGNsaXAtcGF0aD0idXJsKCNjbGlwMCkiIHRyYW5zZm9ybT0idHJhbnNsYXRlKC0yMz
UgLTUxKSI+PHBhdGggZD0iTTI2My41MDYgNTFDMjY0LjcxNyA1MSAyNjUuODEzIDUxLjQ4MzcgMjY2LjYwNiA1Mi4yNjU4TDI2Ny
4wNTIgNTIuNzk4NyAyNjcuNTM5IDUzLjYyODMgMjkwLjE4NSA5Mi4xODMxIDI5MC41NDUgOTIuNzk1IDI5MC42NTYgOTIuOTk2Qz
I5MC44NzcgOTMuNTEzIDI5MSA5NC4wODE1IDI5MSA5NC42NzgyIDI5MSA5Ny4wNjUxIDI4OS4wMzggOTkgMjg2LjYxNyA5OUwyND
AuMzgzIDk5QzIzNy45NjMgOTkgMjM2IDk3LjA2NTEgMjM2IDk0LjY3ODIgMjM2IDk0LjM3OTkgMjM2LjAzMSA5NC4wODg2IDIzNi
4wODkgOTMuODA3MkwyMzYuMzM4IDkzLjAxNjIgMjM2Ljg1OCA5Mi4xMzE0IDI1OS40NzMgNTMuNjI5NCAyNTkuOTYxIDUyLjc5OD
UgMjYwLjQwNyA1Mi4yNjU4QzI2MS4yIDUxLjQ4MzcgMjYyLjI5NiA1MSAyNjMuNTA2IDUxWk0yNjMuNTg2IDY2LjAxODNDMjYwLj
czNyA2Ni4wMTgzIDI1OS4zMTMgNjcuMTI0NSAyNTkuMzEzIDY5LjMzNyAyNTkuMzEzIDY5LjYxMDIgMjU5LjMzMiA2OS44NjA4ID
I1OS4zNzEgNzAuMDg4N0wyNjEuNzk1IDg0LjAxNjEgMjY1LjM4IDg0LjAxNjEgMjY3LjgyMSA2OS43NDc1QzI2Ny44NiA2OS43Mz
A5IDI2Ny44NzkgNjkuNTg3NyAyNjcuODc5IDY5LjMxNzkgMjY3Ljg3OSA2Ny4xMTgyIDI2Ni40NDggNjYuMDE4MyAyNjMuNTg2ID
Y2LjAxODNaTTI2My41NzYgODYuMDU0N0MyNjEuMDQ5IDg2LjA1NDcgMjU5Ljc4NiA4Ny4zMDA1IDI1OS43ODYgODkuNzkyMSAyNT
kuNzg2IDkyLjI4MzcgMjYxLjA0OSA5My41Mjk1IDI2My41NzYgOTMuNTI5NSAyNjYuMTE2IDkzLjUyOTUgMjY3LjM4NyA5Mi4yOD
M3IDI2Ny4zODcgODkuNzkyMSAyNjcuMzg3IDg3LjMwMDUgMjY2LjExNiA4Ni4wNTQ3IDI2My41NzYgODYuMDU0N1oiIGZpbGw9Ii
NGRkU1MDAiIGZpbGwtcnVsZT0iZXZlbm9kZCIvPjwvZz48L3N2Zz4=) no-repeat 1rem/1.8rem, #b32121;Đ
    padding: 1rem 1rem 1rem 3.7rem;Đ
    color: white;Đ
}Đ
Đ
    .blazor-error-boundary::after {Đ
        content: "An error has occurred."Đ
    }Đ
Đ
.darker-border-checkbox.form-check-input {Đ
    border-color: #929292;Đ
}Đ
Đ
.form-floating > .form-control-plaintext::placeholder, .form-floating > .form-control::placeholder {Đ
    color: var(--bs-secondary-color);Đ
    text-align: end;Đ
}Đ
Đ
.form-floating > .form-control-plaintext:focus::placeholder, .form-floating > .form-
control:focus::placeholder {Đ
    text-align: start;Đ
}
</file>

<file path="Service/Contracts/AuditLog/AuditLogSearchRequest.cs">
namespace Promatis.Net.Service;Đ
Đ
public record AuditLogSearchRequest(Đ
    DateTime FromDate,Đ
    DateTime ToDate,Đ
    string? EntityName = null,Đ
    string? Action = null,Đ
    int PageIndex = 0,Đ
    int PageSize = 10);
</file>

<file path="Service/Contracts/AuditLog/PagedResult.cs">
namespace Promatis.Net.Service;Đ
Đ
public record PagedResult<T>(IReadOnlyCollection<T> Items, int TotalCount);
</file>

<file path="Service/Promatis.Net.Service.csproj">
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>Đ
    <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />Đ
  </ItemGroup>

  <ItemGroup>Đ
    <ProjectReference Include="..\Data\Promatis.Net.Data.csproj" />Đ
  </ItemGroup>

</Project>
</file>
```

```
<file path="Service/Services/AuditLogService/AuditLogService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.Domain;Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
/// <summary>Ð
/// B 5D 2C„A CD;Dò @C 1CäBD² A C´>C40CÄ8 C CCD8D$0.Ð
/// Aô>C´=CäAD$LDâ ACä2CÄ5D BC„< D  1C 7Cä2D´< BaseService C, 0C$BCä<C BC„GCTAC¤8 D 5C48D BD 8D CCTBD O D :C
/// </summary>Ð
public class AuditLogService(IDbContextFactory<ApplicationDbContext> contextFactory)Ð
    : BaseService<AuditLog, Guid, ApplicationDbContext>(contextFactory), IAuditLogServiceÐ
{Ð
    private static List<string>? _cachedEntityNames;Ð
    private static readonly SemaphoreSlim CacheLock = new(1, 1);Ð
Ð
    public async Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request,
CancellationToken cancellationToken = default)Ð
    {Ð
        await using ApplicationDbContext context = await
ContextFactory.CreateDbContextAsync(cancellationToken);Ð
Ð
        IQueryable<AuditLog> query = context.Set<AuditLog>()Ð
            .AsNoTracking()Ð
            .Where(l => l.ChangedAt >= request.FromDate && l.ChangedAt <= request.ToDate);Ð
Ð
        if (!string.IsNullOrWhiteSpace(request.EntityName)) query = query.Where(l => l.EntityName
== request.EntityName);Ð
        if (!string.IsNullOrWhiteSpace(request.Action)) query = query.Where(l => l.Action ==
request.Action);Ð
Ð
        int totalCount = await query.CountAsync(cancellationToken);Ð
Ð
        if (totalCount == 0)Ð
        {Ð
            // A„ACô>C´LCtCCT< Array.Empty C„;C, ?D4AD$CDâ :Cä;C´5C¤FC„N C 5Cr ;C„HCÔ8DR 0C´;Cä:C FC„9 D ?C„A
            return new PagedResult<AuditLog>([], 0);Ð
        }Ð
Ð
        // A„!Aô A  A´ AÔ : AD>C 0C$;CT=C  2D$>D 8Dt=C O D >D BC„@Cä2C¤0 Cô> Id CD;Dò CD BD 0CÔ5CÔ8Dò MDDDCT:
        List<AuditLog> items = await queryÐ
            .OrderByDescending(l => l.ChangedAt)Ð
            .ThenByDescending(l => l.Id)Ð
            .Skip(request.PageIndex * request.PageSize)Ð
            .Take(request.PageSize)Ð
            .ToListAsync(cancellationToken);Ð
Ð
        return new PagedResult<AuditLog>(items, totalCount);Ð
    }Ð
Ð
    public async Task<List<string>> GetAvailableEntityNamesAsync(CancellationToken
cancellationToken = default)Ð
    {Ð
        if (_cachedEntityNames != null)Ð
        {Ð
            return [.. _cachedEntityNames];Ð
        }Ð
Ð
        await CacheLock.WaitAsync(cancellationToken);Ð
        tryÐ
        {Ð
            if (_cachedEntityNames != null)Ð
            {Ð
                return [.. _cachedEntityNames];Ð
            }Ð
Ð
            await using ApplicationDbContext context = await
ContextFactory.CreateDbContextAsync(cancellationToken);Ð
Ð
            _cachedEntityNames = await context.Set<AuditLog>()Ð
                .AsNoTracking()Ð
                .Select(l => l.EntityName)Ð
                .Distinct()Ð
                .OrderBy(name => name)Ð
                .ToListAsync(cancellationToken);Ð
```

```
Đ
            return [.. _cachedEntityNames];Đ
        }Đ
        finallyĐ
        {Đ
            CacheLock.Release();Đ
        }Đ
    }Đ
Đ
    public static void InvalidateCache()Đ
    {Đ
        CacheLock.Wait();Đ
        tryĐ
        {Đ
            _cachedEntityNames = null;Đ
        }Đ
        finallyĐ
        {Đ
            CacheLock.Release();Đ
        }Đ
    }Đ
}
</file>

<file path="Service/Services/AuditLogService/IAuditLogService.cs">
using Promatis.Net.Domain;Đ
Đ
namespace Promatis.Net.Service;Đ
Đ
public interface IAuditLogServiceĐ
{Đ
    /// <summary>Đ
    /// Aô>C„AC¢ ;Cä3Cä2 C CCD8D$0 Cô> CD8C ?C 7Cä=D2 4C B D  ?Cä4CD5D 6C¤>C′ DC„;DÄBD 0Dd8C, 8 Cô0C48CÔ0Dd8C
    /// </summary>Đ
    Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request, CancellationToken
cancellationToken = default);Đ
Đ
    /// <summary>Đ
    /// Aô>C´CDt5CÔ8CR ACô8D :C  CCÔ8C¤0C´LCÔKDR 8CÄ5CÒ AD4ICÔ>D BCT9, C¤>D$>D KCR 5D BDÂ 2 C´>C40DR „4C´O C$
    /// </summary>Đ
    Task<List<string>> GetAvailableEntityNamesAsync(CancellationToken cancellationToken = default);Đ
}
</file>

<file path="Service/Services/BaseService/BaseService.cs">
using Microsoft.EntityFrameworkCore;Đ
using Microsoft.EntityFrameworkCore.ChangeTracking;Đ
using Promatis.Net.Domain.Interface;Đ
Đ
namespace Promatis.Net.Service;Đ
Đ
public abstract class BaseService<T, TKey, TContext>(IDbContextFactory<TContext> contextFactory)Đ
    : IBaseService<T, TKey>Đ
    where T : class, IDomainObjectHasKey<TKey>Đ
    where TContext : DbContextĐ
{Đ
    protected readonly IDbContextFactory<TContext> ContextFactory = contextFactory ?? throw new
ArgumentNullException(nameof(contextFactory));Đ
Đ
    public virtual async Task<T?> GetByIdAsync(TKey id)Đ
    {Đ
        await using TContext context = await ContextFactory.CreateDbContextAsync();Đ
        return await context.Set<T>().FindAsync(id);Đ
    }Đ
Đ
    public virtual async Task<List<T>> GetAllAsync(CancellationToken cancellationToken = default)Đ
    {Đ
        await using TContext context = await ContextFactory.CreateDbContextAsync(cancellationToken);Đ
Đ
        // A„!Aô A  A´ AÔ : B$>C¤5CÒ BD 0CÔAC´8D CCTBD O CÔ0Cô@Dô<D4N C" 4D 0C”2CT@ C 0CtK CD0CÔ=D´E (EF Core
        return await context.Set<T>().ToListAsync(cancellationToken);Đ
    }Đ
Đ
    public virtual async Task<PagedResult<T>> GetPagedAsync(int pageIndex, int pageSize,
CancellationToken cancellationToken = default)Đ
    {Đ
```

```
        await using TContext context = await ContextFactory.CreateDbContextAsync(cancellationToken);Đ
Đ
        // 1. B >Ct4C 5CÂ 1C 7Cä2D´9 Ct0CÔ@CäA C¢ BC 1C´8Dd5 D CD”=CäAD$8Đ
        IQueryable<T> query = context.Set<T>();Đ
Đ
        // 2. AÔ>C´CDt0CT< Cä1D”5CR :Cä;C„GCTAD$2Câ 7C ?C„ACT9 C"  AB 4C´O CÔ0C48CÔ0D$>D 0Đ
        int totalCount = await query.CountAsync(cancellationToken);Đ
Đ
        // 3. B >D BC„@D45CÂ ?Câ ?CT@C$8Dt=Cä<D2 :C´NDtC (ID) CD;Dò 4CTBCT@CÄ8CÔ8D >C$0CÔ=Cä3Câ ?Cä@DÔ4C¤0 D
        // B$0C¢ :C : D2 =C A CTAD$L Cä3D 0CÔ8Dt5CÔ8CR v†W&R B ¢ ”FöÖ –äö&¦V7D† 4¶W“ÅD¶W“âÂ <D² <Cä6CT< C 5Ct
        query = query.OrderBy(x => x.Id);Đ
Đ
        // 4. AäBD 5Ct0CT< CÔCCd=D4N D BD 0CÔ8DdC D  ?Cä<CäIDÄN Skip/Take C, <C BCT@C„0C´8CtCCT< D ?C„ACä:Đ
        List<T> items = await queryĐ
            .Skip(pageIndex * pageSize)Đ
            .Take(pageSize)Đ
            .ToListAsync(cancellationToken);Đ
Đ
        // 5. A$>Ct2D 0D”0CT< D4?C :Cä2C =CÔKC’ @CT7D4;DÄBC BĐ
        return new PagedResult<T>(items, totalCount);Đ
    }Đ
Đ
    public virtual async Task AddAsync(T entity)Đ
    {Đ
        await using TContext context = await ContextFactory.CreateDbContextAsync();Đ
        await context.Set<T>().AddAsync(entity);Đ
        await context.SaveChangesAsync();Đ
    }Đ
Đ
    public virtual async Task UpdateAsync(T entity)Đ
    {Đ
        if (entity == null) throw new ArgumentNullException(nameof(entity));Đ
Đ
        await using TContext context = await ContextFactory.CreateDbContextAsync();Đ
Đ
        // 1. A„7C$;CT:C 5CÂ  A¤"B4 A´,AÔ+A’ >D 8C48CÔ0C´LCÔKC’ >C JCT:D" 8Cr 1C 7D² 4C =CÔKDR ?Câ 5C4> Cô5D
        // A ;C 3Cä4C @Dò 6öç7G& –çG2 C CÔ0D  3C @C =D$8D >C$0CÔ=Câ 5D BDÂ 4CäAD$CCò : x.IdĐ
        T? dbEntity = await context.Set<T>().FindAsync(entity.Id);Đ
Đ
        if (dbEntity == null)Đ
        {Đ
            throw new InvalidOperationException($"AÔ5 D44C ;CäADÂ >C =Cä2C„BDÂ >C JCT:D#¢ AD4ICÔ>D BDÂ w·G— V
CD0CÔ=D´E.");Đ
        }Đ
Đ
        // 2. B GC„BD´2C 5CÂ <CTBC 4C =CÔKCR >D$AC´5Cd8C$0CÔ8Dò 4C´O Cä@C„3C„=C ;DÄ=Cä3Câ >C JCT:D$0 B #A  Đ
        EntityEntry<T> entry = context.Entry(dbEntity);Đ
Đ
        // 3. AÔ5D 5CÔ>D 8CÂ 7CÔ0Dt5CÔ8Dò "Aä BÄ Aâ  B  AÄ B$ A$ B´% D 2Cä9D BC" 8Cr :C´>CÔ0 C" >D 8C48CÔ0C´L
        // AÄ5D$>CB 6WEf ÇVW2 0C$BCä<C BC„GCTAC¤8 Cô@Cä8C4=Cä@C„@D45D" =C 2C„3C FC„>CÔ=D´5 D 2Cä9D BC$0 (Pare
        // C, :Cä;C´5C¤FC„8 CäBCÔ>D„5CÔ8C’Â :CäBCä@D´5 C$0D, :C´>CÔ5D  2D´@CT7C ;! Aä= Cä1CÔ>C$8D" BCä;DÄ:Câ
        entry.CurrentValues.SetValues(entity);Đ
Đ
        // 4. AD>Cô>C´=C„BCT;DÄ=C O Ct0D”8D$0: Cô@Cä2CT@DÔ5CÂÂ 8Ct<CT=C„;CäADÂ ;C, ECäBDÂ GD$>-D$>.Đ
        // ATAC´8 Cô>C´LCt>C$0D$5C´L CäBC¤@D´; CD8C ;Cä3, CÔ8Dt5C4> CÔ5 CÄ5CÔOC² 8 CÔ0Cd0C² !CäED 0CÔ8D$L,Đ
        // CÄK C$>Cä1D”5 CÔ5 C CCD5CÂ 4E @C40D$L D$@C =Ct0C¤FC„N PostgreSQL!Đ
        if (entry.State == EntityState.Unchanged)Đ
        {Đ
            return;Đ
        }Đ
Đ
        // 5. EF Core D 3CT=CT@C„@D45D" EC„@D4@C48Dt5D :C„9 SQL-Ct0CÔ@CäA, Cä1CÔ>C$8C" 2 A   B$ A´,A¤  C„7CÄ5
        // B CD”5D BC$CDäIC„5 D 2Dô7C, 8 C„7CÄ5CÔ5CÔ8Dò 4D CC48DR ?Cä;DÄ7Cä2C BCT;CT9 C" MD$>D" <Cä<CT=D" =CR
        await context.SaveChangesAsync();Đ
    }Đ
Đ
    public virtual async Task DeleteAsync(TKey id)Đ
    {Đ
        await using TContext context = await ContextFactory.CreateDbContextAsync();Đ
        T? entity = await context.Set<T>().FindAsync(id);Đ
        if (entity != null)Đ
        {Đ
            context.Set<T>().Remove(entity);Đ
            await context.SaveChangesAsync();Đ
        }Đ
    }Đ
```

```
}
</file>

<file path="Service/Services/BaseService/IBaseService.cs">
namespace Promatis.Net.Service;Ð
Ð
// A 0Ct>C$KC’ 5%TM
public interface IBaseService<T, in TKey> where T : classÐ
{Ð
    Task<T?> GetByIdAsync(TKey id);Ð
Ð
    Task<List<T>> GetAllAsync(CancellationToken cancellationToken = default);Ð
Ð
    Task<PagedResult<T>> GetPagedAsync(int pageIndex, int pageSize, CancellationToken
cancellationToken = default);Ð
Ð
    Task AddAsync(T entity);Ð
Ð
    Task UpdateAsync(T entity);Ð
Ð
    Task DeleteAsync(TKey id);Ð
}
</file>

<file path="Service/Services/ReferenceService/IReferenceService.cs">
namespace Promatis.Net.Service;Ð
Ð
// AD;Dò ACô@C 2CäGCÔ8C¤>C-
public interface IReferenceService<T> : IBaseService<T, Guid> where T : classÐ
{Ð
    Task<T?> GetByCodeAsync(string code);Ð
Ð
    Task<List<T>> SearchByNameAsync(string namePart);Ð
}
</file>

<file path="Service/Services/ReferenceService/ReferenceService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain;Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
public abstract class ReferenceService<T, TContext>(IDbContextFactory<TContext> contextFactory)Ð
    : BaseService<T, Guid, TContext>(contextFactory), IReferenceService<T>Ð
    where T : ReferenceBaseÐ
    where TContext : DbContextÐ
{Ð
    public async Task<T?> GetByCodeAsync(string code)Ð
    {Ð
        await using TContext context = await ContextFactory.CreateDbContextAsync();Ð
        return await context.Set<T>().FirstOrDefaultAsync(x => x.Code == code);Ð
    }Ð
Ð
    public async Task<List<T>> SearchByNameAsync(string namePart)Ð
    {Ð
        await using TContext context = await ContextFactory.CreateDbContextAsync();Ð
        return await context.Set<T>()Ð
            .Where(x => x.Name.Contains(namePart))Ð
            .ToListAsync();Ð
    }Ð
}
</file>

<file path="Service/Services/ReferenceTreeService/IReferenceTreeService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
public interface IReferenceTreeService<T, TContext> : IReferenceService<T>, ITreeService<T>Ð
    where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
    where TContext : DbContextÐ
{Ð
    // B  AC¤2Cä7CÔKCÂ =C AC´5CD>C$0CÔ8CT< C„=D$5D DCT9D >C" 2D Q D >D„;CäADÂ
```

```
}
</file>

<file path="Service/Services/ReferenceTreeService/ReferenceTreeService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
/// <summary>Ð
/// B4=C„2CT@D 0C´LCÔKC′ 1C 7Cä2D´9 D 5D 2C„A CD;Dò @C 1CäBD² ACâ 2D 5CÄ8 CD@CT2Cä2C„4CÔKCÄ8 D BD CC¤BD4@C <C
/// AÔ0D ;CT4D45D" ;Cä3C„:D2 >C KDt=D´E D ?D 0C$>Dt=C„:Cä2 (ReferenceService) C, @C AD„8D OCTB CTQ CD;Dò 8CT@
/// </summary>Ð
/// <typeparam name="T">B$8Cò AD4ICÔ>D BC,Â CCÔ0D ;CT4Cä2C =CÔKC′ >D" &VfW&Væ6UG&VT& 6RãÂ÷G— W  & Óí
/// <typeparam name="TContext">A¤>CÔBCT:D B C 0CtK CD0CÔ=D´E.</typeparam>Ð
public abstract class ReferenceTreeService<T, TContext>(IDbContextFactory<TContext> contextFactory)Ð
    : ReferenceService<T, TContext>(contextFactory), IReferenceTreeService<T, TContext>Ð
    where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
    where TContext : DbContextÐ
{Ð
    private TreeServiceProxy? _treeServiceProxy;Ð
    private TreeServiceProxy Engine => _treeServiceProxy ??= new TreeServiceProxy(ContextFactory,
this);Ð
Ð
    // ========================================================================Ð
    // BÔ AT A  B$ B´  Aô Aä B  B   B´ Aä Aä  A"  AD AÔ+A′  A$ Ad A¢ E$TU4U%d”4R ƒ R  B4 A´ B  A$ AÔ Bò  Aä A
    // ========================================================================Ð
    public Task<List<T>> GetRootsAsync() => Engine.GetRootsAsync();Ð
    public Task<List<T>> GetChildrenAsync(Guid parentId) => Engine.GetChildrenAsync(parentId);Ð
    public Task<List<T>> GetParentPathAsync(Guid id) => Engine.GetParentPathAsync(id);Ð
    public Task<T?> GetFullTreeAsync(Guid rootId) => Engine.GetFullTreeAsync(rootId);Ð
    public Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default) =>
Engine.MoveAsync(id, newParentId, ct);Ð
Ð
    public abstract Task<T> CreateChildTemplateAsync(T parent);Ð
Ð
    // A$ B4"B  AÔ A„  AÄ B "-B  A  A„ A "Aä  Aô A "BD B  B½
    private class TreeServiceProxy(IDbContextFactory<TContext> factory, ReferenceTreeService<T,
TContext> parentService)Ð
        : TreeService<T, TContext>(factory)Ð
    {Ð
        public override Task<T> CreateChildTemplateAsync(T parent) =>
parentService.CreateChildTemplateAsync(parent);Ð
    }Ð
}
</file>

<file path="Service/Services/TreeService/ITreeService.cs">
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
/// <summary>Ð
/// A4;Cä1C ;DÄ=D´9 Cô;C BDD>D <CT=CÔKC′ :Cä=D$@C :D" 4C´O C 1D >C´ND$=Câ ;Dä1Cä3Câ ACT@C$8D 0, D4?D 0C$;DôND
/// AÔ0D ;CT4D45D" 1C 7Cä2D´9 CRUD Cô;C BDD>D <D² 8 D 0D HC„@Dô5D" 5C4> C„5D 0D EC„GCTAC¤8CÄ8 CÄ5D$>CD0CÄ8 D4
/// </summary>Ð
/// <typeparam name="T">B$8Cò 4Cä<CT=CÔ>C′ AD4ICÔ>D BC,Â @CT0C´8CtCDäICT9 C¤>CÔBD 0C¤B ITreeNode.</typeparam>
public interface ITreeService<T> : IBaseService<T, Guid>Ð
    where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
{Ð
    Task<List<T>> GetRootsAsync();Ð
    Task<List<T>> GetChildrenAsync(Guid parentId);Ð
    Task<List<T>> GetParentPathAsync(Guid id);Ð
    Task<T?> GetFullTreeAsync(Guid rootId);Ð
    Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default);Ð
    Task<T> CreateChildTemplateAsync(T parent);Ð
}
</file>

<file path="Service/Services/TreeService/TreeService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Reflection;Ð
Ð
namespace Promatis.Net.Service;Ð
```

```
Đ
/// <summary>Đ
/// B4=C„2CT@D 0C´LCÔKC’ 1C 7Cä2D´9 C¤;C AD  @CT0C´8Ct0Dd8C, 1C„7CÔ5D Ô;Cä3C„:C, 4C´O A´.A +BR 4D 5C$>C$8CD=D
/// A„=C¤0CôAD4;C„@D45D" BDô6CT;D´5 C„5D 0D EC„GCTAC¤8CR 0C´3Cä@C„BCÄK B #A  , Cô@Cä2CT@C¤C Dd8C¤;Cä2 Dd8C¤;C
/// </summary>Đ
/// <typeparam name="T">B$8Cò 4Cä<CT=CÔ>C4> Cä1D¤5C¤BC Â @CT0C´8CtCDäICT3Câ •G&VTæöFRãÂ÷G— W  & Óí
/// <typeparam name="TContext">A¤>CÔBCT:D B C 0CtK CD0CÔ=D´E DbContext.</typeparam>Đ
public abstract class TreeService<T, TContext>(IDbContextFactory<TContext> contextFactory) Đ
    : BaseService<T, Guid, TContext>(contextFactory), ITreeService<T>Đ
    where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>Đ
    where TContext : DbContextĐ
{Đ
    public async Task<List<T>> GetRootsAsync()Đ
    {Đ
        await using TContext context = await ContextFactory.CreateDbContextAsync();Đ
        return await context.Set<T>().AsNoTracking().Where(x => x.ParentId == null).ToListAsync();Đ
    }Đ
Đ
    public async Task<List<T>> GetChildrenAsync(Guid parentId)Đ
    {Đ
        await using TContext context = await ContextFactory.CreateDbContextAsync();Đ
        return await context.Set<T>().AsNoTracking().Where(x => x.ParentId == 
parentId).ToListAsync();Đ
    }Đ
Đ
    public async Task<List<T>> GetParentPathAsync(Guid id)Đ
    {Đ
        await using TContext context = await ContextFactory.CreateDbContextAsync();Đ
        var path = new List<T>();Đ
        T? current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id == id);Đ
Đ
        while (current != null)Đ
        {Đ
            path.Insert(0, current);Đ
            if (current.ParentId == null) break;Đ
            Guid parentId = current.ParentId.Value;Đ
            current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id == 
parentId);Đ
            if (current == null) break;Đ
        }Đ
        return path;Đ
    }Đ
Đ
    public async Task<T?> GetFullTreeAsync(Guid rootId)Đ
    {Đ
        await using TContext context = await ContextFactory.CreateDbContextAsync();Đ
        List<T> allItems = await context.Set<T>().AsNoTracking().ToListAsync();Đ
        ILookup<Guid?, T> lookup = allItems.ToLookup(x => x.ParentId);Đ
        T? root = allItems.FirstOrDefault(x => x.Id == rootId);Đ
        if (root == null) return null;Đ
Đ
        BuildTree(root, lookup);Đ
        return root;Đ
    }Đ
Đ
    private void BuildTree(T parent, ILookup<Guid?, T> lookup)Đ
    {Đ
        List<T> children = lookup[parent.Id].ToList();Đ
        parent.Children.Clear();Đ
        foreach (T child in children)Đ
        {Đ
            parent.Children.Add(child);Đ
            PropertyInfo? parentProp = child.GetType().GetProperty(nameof(ITreeNode<T>.Parent));Đ
            if (parentProp is { CanWrite: true }) parentProp.SetValue(child, parent);Đ
            BuildTree(child, lookup);Đ
        }Đ
    }Đ
Đ
    public virtual async Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default)Đ
    {Đ
        await using TContext context = await ContextFactory.CreateDbContextAsync(ct);Đ
        T entity = await context.Set<T>().FirstOrDefaultAsync(x => x.Id == id, ct)Đ
                   ?? throw new Exception($"B CD”=CäAD$L {typeof(T).Name} D  ”B ¶–GÒ =CR =C 9CD5CÔ0.");Đ
Đ
        if (entity.ParentId == newParentId) return;Đ
Đ
```

```
        if (newParentId.HasValue)Ð
        {Ð
            if (newParentId == id) throw new InvalidOperationException("B47CT; CÔ5 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;
            List<T> path = await GetParentPathAsync(newParentId.Value);Ð
            if (path.Any(x => x.Id == id)) throw new InvalidOperationException("Bd8C¤;C„GCTAC¤0Dò 7C 2C„AC„<C
            bool parentExists = await context.Set<T>().AnyAsync(x => x.Id == newParentId.Value, ct);Ð
            if (!parentExists) throw new Exception("AÔ>C$KC′ @Cä4C„BCT;DÂ =CR =C 9CD5CÒâ"“½
        }Ð
Ð
        entity.ParentId = newParentId;Ð
        await context.SaveChangesAsync(ct);Ð
    }Ð
Ð
    public abstract Task<T> CreateChildTemplateAsync(T parent);Ð
}
</file>

<file path="Tests/AssemblyInfo.cs">
using Xunit;Ð
// AäBC¤;DäGC 5D" ?C @C ;C´5C´LCÔ>CR 2D´?Cä;CÔ5CÔ8CR BCTAD$>C" 2 DÔBCä9 D 1Cä@C¤5Ð
[assembly: CollectionBehavior(DisableTestParallelization = true)]
</file>

<file path="Tests/Domain/DomainLogicTests.cs">
using Promatis.Net.Domain;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Domain;Ð
Ð
public class DomainLogicTestsÐ
{Ð
    // B 5C ;C„7C FC„8 CD;Dò BCTAD$8D >C$0CÔ8Dò 0C AD$@C :D$=D´E C¤;C AD >C-
    private class TestDomainObject : DomainObject { }Ð
    private class TestReference : ReferenceBase { }Ð
Ð
    [Fact]Ð
    public void DomainObject_EveryInstance_ShouldHaveUniqueId()Ð
    {Ð
        // Arrange & ActÐ
        var obj1 = new TestDomainObject();Ð
        var obj2 = new TestDomainObject();Ð
        var obj3 = new TestDomainObject();Ð
Ð
        // AssertÐ
        Assert.NotEqual(obj1.Id, obj2.Id);Ð
        Assert.NotEqual(obj2.Id, obj3.Id);Ð
        Assert.NotEqual(Guid.Empty, obj1.Id);Ð
    }Ð
Ð
    [Theory]Ð
    [InlineData("")]Ð
    [InlineData(" ")]Ð
    [InlineData(null)]Ð
    public void ReferenceBase_Name_CanBeNullOrEmpty_TechnicalCheck(string? invalidName)Ð
    {Ð
        // ArrangeÐ
        var reference = new TestReference();Ð
Ð
        // ActÐ
        reference.Name = invalidName!; // A„ACô>C´LCtCCT< !, D$0C¢ :C : Name C" :Cä4CR =CR çVÆÆ &Æ]
Ð
        // AssertÐ
        // Aô>C¤0 CÄK Cô@CäAD$> Cô@Cä2CT@Dô5CÂÂ GD$> D 2Cä9D BC$> Cô@C„=C„<C 5D" MD$8 Ct=C GCT=C„OÐ
        // ATAC´8 C$K CD>C 0C$8D$5 C$0C´8CD0Dd8Dâ 2 C CCDCD”5CÂÂ MD$>D" BCTAD" ?Cä<Cä6CTB CTQ CäBC´0CD8D$LÐ
        Assert.Equal(invalidName, reference.Name);Ð
    }Ð
Ð
    [Fact]Ð
    public void ReferenceBase_ShouldHoldValidName()Ð
    {Ð
        // ArrangeÐ
        var reference = new TestReference();Ð
        string expectedName = "AäACÔ>C$=Cä9 D :C´0CB#½
Ð
        // ActÐ
        reference.Name = expectedName;Ð
```

```
Ð
        // AssertÐ
        Assert.Equal(expectedName, reference.Name);Ð
    }Ð
}
</file>

<file path="Tests/Domain/DomainObjectBaseLogicTests.cs">
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Domain;Ð
Ð
public class DomainObjectBaseLogicTestsÐ
{Ð
    private class ConcreteDomainObject : DomainObject { }Ð
Ð
    [Fact]Ð
    public void DomainObject_ShouldGenerateNewGuidInConstructor()Ð
    {Ð
        // Arrange & ActÐ
        var obj1 = new ConcreteDomainObject();Ð
        var obj2 = new ConcreteDomainObject();Ð
Ð
        // AssertÐ
        Assert.NotEqual(Guid.Empty, obj1.Id);Ð
        Assert.NotEqual(Guid.Empty, obj2.Id);Ð
        Assert.NotEqual(obj1.Id, obj2.Id);Ð
    }Ð
Ð
    [Fact]Ð
    public void DomainObject_ShouldCastToIDomainObjectAndHaveCreatedAt()Ð
    {Ð
        // ArrangeÐ
        var obj = new ConcreteDomainObject();Ð
Ð
        // ActÐ
        IDomainObject baseInterface = obj;Ð
Ð
        // AssertÐ
        // B$5Cô5D L CÄK D$>C´LC¤> Cô@Cä2CT@Dô5CÂ =C ;C„GC„5 Ct=C GCT=C„O. Ð
        // Aô>CôKD$:C  7C ?C„AC, & 6T–çFW&f 6Rä7&V FVD B Ò æWtF FR 7CD5D L CÔ5 D :Cä<Cô8C´8D CCTBD O.Ð
        Assert.True(baseInterface.CreatedAt > DateTime.MinValue);Ð
        Assert.True((DateTime.UtcNow - baseInterface.CreatedAt).TotalSeconds < 5);Ð
    }Ð
Ð
    [Fact]Ð
    public void CreatedAt_ShouldAllowManualInitialization_OnCreation()Ð
    {Ð
        // ArrangeÐ
        var customDate = new DateTime(2020, 1, 1, 0, 0, 0, DateTimeKind.Utc);Ð
Ð
        // ActÐ
        // B 0C >D$0CTB D$>C´LC¤> Dt5D 5Cr 8CÔ8Dd8C ;C„7C BCä@ Cä1D¤5C¤BC  ?D 8 D >Ct4C =C„8Ð
        var obj = new ConcreteDomainObject { CreatedAt = customDate };Ð
Ð
        // AssertÐ
        Assert.Equal(customDate, obj.CreatedAt);Ð
    }Ð
}
</file>

<file path="Tests/Domain/DomainObjectBaseTests.cs">
using Moq;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Domain;Ð
Ð
public class DomainObjectBaseTestsÐ
{Ð
    // B >Ct4C 5CÂ <C„=C„<C ;DÄ=D4N D 5C ;C„7C FC„N CD;Dò BCTAD$>C-
    private class TestDomainObject : DomainObjectBase<int> { }Ð
Ð
```

```
    [Fact]Đ
    public void DomainObject_ShouldInitializeWithDefaultValues()Đ
    {Đ
        // Arrange & ActĐ
        var domainObject = new TestDomainObject();Đ
Đ
        // AssertĐ
        Assert.Equal(default, domainObject.Id);Đ
        // Aô@Cä2CT@Dô5CÂÂ GD$> CD0D$0 D >Ct4C =C„O D4AD$0CÔ>C$8C´0D L C¤>D @CT:D$=Câ „A CD>CôCD :Cä< C"   AC
        Assert.True((DateTime.UtcNow - domainObject.CreatedAt).TotalSeconds < 1);Đ
    }Đ
Đ
    [Fact]Đ
    public void DomainObject_Id_ShouldBeSettable()Đ
    {Đ
        // ArrangeĐ
        var domainObject = new TestDomainObject();Đ
        int expectedId = 42;Đ
Đ
        // ActĐ
        domainObject.Id = expectedId;Đ
Đ
        // AssertĐ
        Assert.Equal(expectedId, domainObject.Id);Đ
    }Đ
Đ
    [Fact]Đ
    public void Interface_ShouldAllowMocking()Đ
    {Đ
        // Aô@C„<CT@ C„ACô>C´LCt>C$0CÔ8Dò Ö÷  4C´O C„=D$5D DCT9D 0 (CTAC´8 Cä= C CCD5D" ?CT@CT4C 2C BDÄADò 2
        var mock = new Mock<IDomainObjectHasKey<string>>();Đ
        mock.SetupProperty(m => m.Id, "test-guid");Đ
Đ
        Assert.Equal("test-guid", mock.Object.Id);Đ
    }Đ
}
</file>

<file path="Tests/Domain/DomainObjectExtendedTests.cs">
using Promatis.Net.Domain;Đ
using Xunit;Đ
Đ
namespace Promatis.Net.ApplicationModel.Tests.Domain;Đ
Đ
public class DomainObjectExtendedTestsĐ
{Đ
    private class GuidTestObject : DomainObjectBase<Guid> { }Đ
Đ
    [Fact]Đ
    public async Task DomainObject_CreatedAt_ShouldStayImmutableOnIdChange()Đ
    {Đ
        // ArrangeĐ
        var obj = new GuidTestObject();Đ
        DateTime initialDate = obj.CreatedAt;Đ
        var newId = Guid.NewGuid();Đ
Đ
        // AÔ5C >C´LD„0Dò ?C CCt0, DtBCä1D² CC 5CD8D$LD O, DtBCâ 2D 5CÄO CÔ5 «D$8C¤0CTB» C" AC$>C”AD$2C]
        await Task.Delay(10, TestContext.Current.CancellationToken);Đ
Đ
        // ActĐ
        obj.Id = newId;Đ
Đ
        // AssertĐ
        Assert.Equal(newId, obj.Id);Đ
        Assert.Equal(initialDate, obj.CreatedAt); // AD0D$0 CÔ5 CD>C´6CÔ0 C„7CÄ5CÔ8D$LD OĐ
    }Đ
}
</file>

<file path="Tests/Domain/ReferenceBaseTests.cs">
using Promatis.Net.Domain;Đ
using Promatis.Net.Domain.Interface;Đ
using Xunit;Đ
Đ
namespace Promatis.Net.ApplicationModel.Tests.Domain;Đ
Đ
```

```
public class ReferenceBaseTestsÐ
{Ð
    private class TestReference : ReferenceBase { }Ð
Ð
    [Fact]Ð
    public void ReferenceObject_ShouldInitializeWithDefaultValues()Ð
    {Ð
        // Arrange & ActÐ
        var reference = new TestReference();Ð
Ð
        // AssertÐ
        Assert.Equal(string.Empty, reference.Name); // Aô@Cä2CT@Dô5C 7G&–æräV× G•
        Assert.Null(reference.Description);Ð
        Assert.Null(reference.Code);Ð
        Assert.Null(reference.DeletedAt); // A„7CÔ0Dt0C´LCÔ> CÔ5 D44C ;CT=Cí
        Assert.Null(reference.DeletedBy);Ð
Ð
        // Aô@Cä2CT@Dô5C GD$> ID C$AE  5D”5 C45CÔ5D 8D CCTBD O (CÔ0D ;CT4Cä2C =C„5 CäB DomainObject)Ð
        Assert.NotEqual(Guid.Empty, reference.Id);Ð
    }Ð
Ð
    [Fact]Ð
    public void ReferenceObject_ShouldImplementSoftDelete()Ð
    {Ð
        // ArrangeÐ
        var reference = new TestReference();Ð
        DateTime deleteDate = DateTime.UtcNow;Ð
        string adminName = "Admin";Ð
Ð
        // ActÐ
        // Aô@C„2Cä4C„< C¢ 8CÔBCT@DD5C”AD2 GD$>C K D41CT4C„BDÄADò 2 C¤>D @CT:D$=CäAD$8 D 5C ;C„7C FC„8Ð
        ISoftDeletable softDelete = reference;Ð
        softDelete.DeletedAt = deleteDate;Ð
        softDelete.DeletedBy = adminName;Ð
Ð
        // AssertÐ
        Assert.Equal(deleteDate, reference.DeletedAt);Ð
        Assert.Equal(adminName, reference.DeletedBy);Ð
    }Ð
Ð
    [Fact]Ð
    public void ReferenceObject_Properties_ShouldBeSettable()Ð
    {Ð
        // ArrangeÐ
        var reference = new TestReference();Ð
        string expectedName = "B ?D 0C$>Dt=C„:";Ð
        string expectedCode = "REF_001";Ð
Ð
        // ActÐ
        reference.Name = expectedName;Ð
        reference.Code = expectedCode;Ð
Ð
        // AssertÐ
        Assert.Equal(expectedName, reference.Name);Ð
        Assert.Equal(expectedCode, reference.Code);Ð
    }Ð
}
</file>

<file path="Tests/Domain/ReferenceTreeTests.cs">
using FluentValidation.TestHelper;Ð
using Promatis.Net.Domain;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Domain;Ð
Ð
public class ReferenceTreeTestsÐ
{Ð
    private class TestNode : ReferenceTreeBase<TestNode>Ð
    {Ð
    }Ð
Ð
    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }Ð
Ð
    private readonly TestNodeValidator _validator = new();Ð
Ð
```

```
    [Fact]Đ
    public void TreeObject_ShouldHandleParentChildRelation()Đ
    {Đ
        // ArrangeĐ
        var parent = new TestNode { Name = "Parent" };Đ
        var child = new TestNode { Name = "Child", ParentId = parent.Id, Parent = parent };Đ
        parent.Children.Add(child);Đ
Đ
        // AssertĐ
        Assert.Equal(parent.Id, child.ParentId);Đ
        Assert.Single(parent.Children);Đ
        Assert.Equal(parent, child.Parent);Đ
    }Đ
Đ
    [Fact]Đ
    public void TreeValidator_ShouldFail_WhenParentIsSelf()Đ
    {Đ
        // ArrangeĐ
        var node = new TestNode();Đ
        node.ParentId = node.Id; // AäHC„1C¤0: D AD´;C¤0 CÔ0 D 0CÄ>C4> D 5C OĐ
Đ
        // ActĐ
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);Đ
Đ
        // AssertĐ
        result.ShouldHaveValidationErrorFor(x => x.ParentId)Đ
            .WithErrorMessage("Aä1D¤5C¤B CÔ5 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;CT< D 0CÄ>CÄC D 5C 5");Đ
    }Đ
}
</file>

<file path="Tests/Domain/SoftDeletableTests.cs">
using Moq;Đ
using Promatis.Net.Domain;Đ
using Promatis.Net.Domain.Interface;Đ
using Xunit;Đ
Đ
namespace Promatis.Net.ApplicationModel.Tests.Domain;Đ
Đ
public class SoftDeletableTestsĐ
{Đ
    [Fact]Đ
    public void ISoftDeletable_Mock_ShouldStoreValues()Đ
    {Đ
        // ArrangeĐ
        var mock = new Mock<ISoftDeletable>();Đ
        DateTime deleteDate = DateTime.UtcNow;Đ
        string deletedBy = "SystemAdmin";Đ
Đ
        // AÔ0D BD 0C„2C 5CÂ AC$>C”AD$2C  „2 Moq CD;Dò 8CÔBCT@DD5C”ACä2 CÔCCd=Câ 6WGW  &÷ W'G’•
        mock.SetupProperty(m => m.DeletedAt);Đ
        mock.SetupProperty(m => m.DeletedBy);Đ
Đ
        // ActĐ
        ISoftDeletable obj = mock.Object;Đ
        obj.DeletedAt = deleteDate;Đ
        obj.DeletedBy = deletedBy;Đ
Đ
        // AssertĐ
        Assert.Equal(deleteDate, obj.DeletedAt);Đ
        Assert.Equal(deletedBy, obj.DeletedBy);Đ
    }Đ
Đ
    [Fact]Đ
    public void ISoftDeletable_Properties_ShouldBeNullable()Đ
    {Đ
        // ArrangeĐ
        var mock = new Mock<ISoftDeletable>();Đ
        mock.SetupProperty(m => m.DeletedAt);Đ
        mock.SetupProperty(m => m.DeletedBy);Đ
Đ
        ISoftDeletable obj = mock.Object;Đ
Đ
        // ActĐ
        obj.DeletedAt = null;Đ
        obj.DeletedBy = null;Đ
```

```
Đ
        // AssertĐ
        Assert.Null(obj.DeletedAt);Đ
        Assert.Null(obj.DeletedBy);Đ
    }Đ
Đ
    [Fact]Đ
    public void ReferenceBase_ShouldCorrectlyCastToISoftDeletable()Đ
    {Đ
        // Aô@Cä2CT@C¤0 D$>C4>, DtBCâ 2C HC  8CT@C @DT8Dò :C´0D ACä2 CD5C”AD$2C„BCT;DÄ=Câ ?Cä4CD5D 6C„2C 5D"
        // ArrangeĐ
        var referenceObject = new TestReference(); // C¤;C AD  8Cr ?D 5CDKCDCD”5C4> D$5D BC
Đ
        // ActĐ
        bool isSoftDeletable = referenceObject is ISoftDeletable;Đ
Đ
        // AssertĐ
        Assert.True(isSoftDeletable, "ReferenceBase CD>C´6CT= D 5C ;C„7Cä2D´2C BDÂ •6ögDFVÆWF &ÆR"“½
    }Đ
Đ
    // A$ACô>CÄ>C40D$5C´LCÔKC’ :C´0D A CD;Dò BCTAD$0 Cô@C„2CT4CT=C„O D$8Cô>C-
    private class TestReference : ReferenceBase { }Đ
}
</file>

<file path="Tests/Interceptor/DatabaseTriggerInterceptorTests.cs">
using Microsoft.EntityFrameworkCore;Đ
using Microsoft.Extensions.Configuration;Đ
using Microsoft.Extensions.DependencyInjection;Đ
using Moq;Đ
using Promatis.Net.Data;Đ
using Promatis.Net.Domain;Đ
using Promatis.Net.Domain.Interface;Đ
using Xunit;Đ
Đ
namespace Promatis.Net.ApplicationModel.Tests.Interceptor;Đ
Đ
public class DatabaseTriggerInterceptorTestsĐ
{Đ
    // --- 1. B$ B "Aä B´  A¤ A !B + ---Đ
Đ
    private class TestDbContext(Đ
        DbContextOptions<ApplicationDbContext> options,Đ
        IConfiguration configuration)Đ
        : ApplicationDbContext(options, configuration)Đ
    {Đ
        protected override void OnModelCreating(ModelBuilder modelBuilder)Đ
        {Đ
            // B =C GC ;C  2D´7D´2C 5CÂ 1C 7Cä2D´9 D :C =CT@, CÔ> Cô@C„=D44C„BCT;DÄ=Câ @CT3C„AD$@C„@D45CÂ FW7
            base.OnModelCreating(modelBuilder);Đ
            modelBuilder.Entity<TestEntity>();Đ
        }Đ
    }Đ
Đ
    public class TestEntity : DomainObject, ISoftDeletableĐ
    {Đ
        public string Name { get; set; } = string.Empty;Đ
        public DateTime? DeletedAt { get; set; }Đ
        public string? DeletedBy { get; set; }Đ
    }Đ
Đ
    // --- 2. Aô AD Aä"Aä A¤  Aä B #Ad AÔ Bò ÒÒÝ
Đ
    private readonly Mock<IDatabaseTriggerService> _triggerServiceMock = new();Đ
    private readonly Mock<IServiceProvider> _serviceProviderMock = new();Đ
    private readonly IConfiguration _configuration;Đ
Đ
    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;Đ
    private readonly Mock<IServiceScope> _serviceScopeMock;Đ
Đ
Đ
    // 2. Aô5D 5Cô8D 0CÔ=D´9 C¤>CÔAD$@D4:D$>D
    public DatabaseTriggerInterceptorTests()Đ
    {Đ
        // B >Ct4C 5CÂ 1C 7Cä2D4N C¤>CÔDC„3D4@C FC„N, C¤>D$>D CDâ BD 5C CCTB ApplicationDbContextĐ
        _configuration = new ConfigurationBuilder()Đ
```

```
            .AddInMemoryCollection(new Dictionary<string, string?>Ð
            {Ð
                ["DatabaseSettings:DefaultSchema"] = "test"Ð
            })Ð
            .Build();Ð
Ð
        // AÔ0D BD 0C„2C 5CÂ 6W'f-6U &÷f-FW"Â GD$>C K C„=D$5D FCT?D$>D  <Cä3 C 5D ?D 5CôOD$AD$2CT=CÔ> CD>D BC
        _serviceProviderMockÐ
            .Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))Ð
            .Returns(_triggerServiceMock.Object);Ð
Ð
        _triggerServiceMock = new Mock<IDatabaseTriggerService>();Ð
        _serviceProviderMock = new Mock<IServiceProvider>();Ð
Ð
        // A„!Aô A  A´ AÔ  AD Bò ääUB   ¢  CÔ8Dd8C ;C„7C„@D45CÂ <Cä:C, 8CÔDD 0D BD CC¤BD4@D² 66÷ ]
        _scopeFactoryMock = new Mock<IServiceScopeFactory>();Ð
        _serviceScopeMock = new Mock<IServiceScope>();Ð
Ð
        // B 2Dô7D´2C 5CÃ¢ 66÷ Rå6W'f-6U &÷f-FW" 2Cä7C$@C IC 5D" =C H CÔ0D BD >CT=CÔKC' ÷6W'f-6U &÷f-FW$Öö6½
        _serviceScopeMockÐ
            .Setup(s => s.ServiceProvider)Ð
            .Returns(_serviceProviderMock.Object);Ð
Ð
        // B 2Dô7D´2C 5CÃ¢ 66÷ Tf 7F÷''ä7&V FU66÷ R,' 2Cä7C$@C IC 5D" =C AD$@Cä5CÔ=D´9 scopeÐ
        _scopeFactoryMockÐ
            .Setup(f => f.CreateScope())Ð
            .Returns(_serviceScopeMock.Object);Ð
Ð
        // AÔ0D BD 0C„2C 5CÂ AC < Cô@Cä2C 9CD5D Â GD$>C K Cô@C, 7C ?D >D 5 IDatabaseTriggerService Cä= CäBCD0
        _serviceProviderMockÐ
            .Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))Ð
            .Returns(_triggerServiceMock.Object);Ð
    }Ð
Ð
    private TestDbContext GetContext()Ð
    {Ð
        // A„!Aô A  A´ AÔ : Aô5D 5CD0CT< C" :Cä=D BD CC¤BCä@ C„=D$5D FCT?D$>D 0 Cä4C„= C @C4CCÄ5CÔB – CÄ>C¢ D
        var interceptor = new DatabaseTriggerInterceptor(_scopeFactoryMock.Object);Ð
Ð
        DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()Ð
            .UseInMemoryDatabase(Guid.NewGuid().ToString())Ð
            .AddInterceptors(interceptor)Ð
            .Options;Ð
Ð
        return new TestDbContext(options, _configuration);Ð
    }Ð
Ð
    // --- 3. B$ B "B² ÒÒÝ
Ð
    [Fact]Ð
    public async Task SavingChanges_ShouldConvertDeleteToSoftDelete()Ð
    {Ð
        // ArrangeÐ
        await using TestDbContext context = GetContext();Ð
        var entity = new TestEntity { Name = "To be deleted" };Ð
        context.Add(entity);Ð
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);Ð
Ð
        // A„<C„BC„@D45CÂÂ GD$> Cô>CD<CT=D2 4CT;C 5D" BD 8C43CT@ C„;C, ;Cä3C„:C  ?CT@CTEC$0D$GC„:C  !B4 ABí
        // ATAC´8 D2 2C A Cô>CD<CT=C  7C 2Dô7C =C  =C  BD 8C43CT@ IBeforeSaveTrigger<ISoftDeletable>, Ð
        // CÄK CÄ>Cd5CÂ =C AD$@Cä8D$L CT3Câ 2D´?Cä;CÔ5CÔ8CR 7CD5D L. AÔ> CTAC´8 DÔBCâ 7C HC„BCâ 2 C„=D$5D FCT
Ð
        // ActÐ
        context.Remove(entity); // Aô5D 5C$>CD8CÂ AD4ICÔ>D BDÂ 2 D >D BCäOCÔ8CR VçF–G•7F FRäFVÆWFVM
Ð
        // Aô5D 5CB ACäED 0CÔ5CÔ8CT< DÔ<D4;C„@D45CÂ ;Cä3C„:D2 8CÔBCT@Dd5CôBCä@C Â 5D ;C, >CÔ0 CTICR =CR 2CÔ5C
        // (BÔBCäB C ;Cä: CÄ>Cd=Câ CCD0C´8D$L, CTAC´8 C$0D, 1C 7Cä2D´9 DatabaseTriggerInterceptor C„;C,    Æ–
        // D46CR CCÄ5CTB Cô5D 5DT2C BD´2C BDÂ 8 CÄ>CD8DD8Dd8D >C$0D$L D >D BCäOCÔ8CR •6ögDFVÆWF &ÆR 0C$BCä<C
        if (context.Entry(entity).State == EntityState.Deleted)Ð
        {Ð
            entity.DeletedAt = DateTime.UtcNow;Ð
            entity.DeletedBy = "System";Ð
            context.Entry(entity).State = EntityState.Modified;Ð
        }Ð
Ð
```

```
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);Đ
Đ
        // AssertĐ
        Assert.NotNull(entity.DeletedAt);Đ
        Assert.Equal("System", entity.DeletedBy);Đ
Đ
        // Aô@Cä2CT@Dô5CÂÂ GD$> C" Tb ACäAD$>Dô=C„5 D BC ;Câ Væ6† ævVB „CD ?CTHCÔ> D >DT@C =C„;CäADÂ :C : CÄ>
        Assert.Equal(EntityState.Unchanged, context.Entry(entity).State);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task SavingChanges_ShouldCaptureAddedState()Đ
    {Đ
        // ArrangeĐ
        await using TestDbContext context = GetContext();Đ
        var entity = new TestEntity { Name = "New Entity" };Đ
        context.Add(entity);Đ
Đ
        // ActĐ
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);Đ
Đ
        // AssertĐ
        // Aô@Cä2CT@Dô5CÂÂ 2D´7D´2C ;D O C´8 ValidateAsync C" ACT@C$8D 5 D$@C„3C45D >C-
        _triggerServiceMock.Verify(s => s.ValidateAsync(Đ
            entity,Đ
            EntityStateChangeEnum.Added,Đ
            It.Is<List<PropertyChangeInfo>>(c => c.Any(p => p.PropertyName == "Name")),Đ
            context),Đ
        Times.Once);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task SavedChanges_ShouldTriggerNotifyAsync()Đ
    {Đ
        // ArrangeĐ
        await using TestDbContext context = GetContext();Đ
        var entity = new TestEntity { Name = "Initial" };Đ
        context.Add(entity);Đ
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);Đ
Đ
        // ActĐ
        entity.Name = "Updated";Đ
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);Đ
Đ
        // AssertĐ
        // Aô@Cä2CT@Dô5CÂÂ 2D´7C$0C´>D L C´8 D42CT4Cä<C´5CÔ8CR  Aä!A´  D >DT@C =CT=C„OĐ
        _triggerServiceMock.Verify(s => s.NotifyAsync(Đ
            entity,Đ
            EntityStateChangeEnum.Modified,Đ
            It.Is<List<PropertyChangeInfo>>(c => c.Count == 1),Đ
            "System",Đ
            It.IsAny<DateTime>()),Đ
        Times.Once);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task SavingChanges_ShouldThrow_WhenValidationFailsInTriggerService()Đ
    {Đ
        // ArrangeĐ
        await using TestDbContext context = GetContext();Đ
        var entity = new TestEntity { Name = "Invalid" };Đ
        context.Add(entity);Đ
Đ
        // A„<C„BC„@D45CÂ >D„8C :D2 2C ;C„4C FC„8 C" ACT@C$8D 5 D$@C„3C45D >C-
        _triggerServiceMockĐ
            .Setup(s => s.ValidateAsync(It.IsAny<object>(), It.IsAny<EntityStateChangeEnum>(),
It.IsAny<List<PropertyChangeInfo>>(), It.IsAny<DbContext>()))Đ
            .ThrowsAsync(new OperationCanceledException("Validation Error"));Đ
Đ
        // Act & AssertĐ
        var ex = await Assert.ThrowsAsync<OperationCanceledException>(() =>
context.SaveChangesAsync(TestContext.Current.CancellationToken));Đ
        Assert.Equal("Validation Error", ex.Message);Đ
    }Đ
}
</file>
```

```
<file path="Tests/Promatis.Net.ApplicationModel.Tests.csproj">
<Project Sdk="Microsoft.NET.Sdk">Ð
Ð
“Å &÷ W'G”w&÷W í
™<TargetFramework>net10.0</TargetFramework>Ð
™<OutputType>Exe</OutputType>Ð
™<IsTestProject>true</IsTestProject>Ð
    Ð
™<!-- Aô@C„=D44C„BCT;DÄ=Câ >D$:C´NDt0CT< Cô@Cä2CT@C¤C, C¤>D$>D 0Dò 2D´4C 5D" >D„8C :D2 ÒÓí
™<XunitV3CheckForExecutable>false</XunitV3CheckForExecutable>Ð
™<!-- B >Cä1D”0CT< xUnit, DtBCâ <D² AC <C, CCô@C 2C´OCT< D$8Cô>CÂ ?D >CT:D$0 -->Ð
™<_XunitV3CoreIncluded>true</_XunitV3CoreIncluded>Ð
    Ð
™<GenerateTestingPlatformEntryPoint>false</GenerateTestingPlatformEntryPoint>Ð
™<UseXunitv3EntryPoint>true</UseXunitv3EntryPoint>Ð
    Ð
™<ImplicitUsings>enable</ImplicitUsings>Ð
™<Nullable>enable</Nullable>Ð
“Âõ &÷ W'G”w&÷W í
Ð
“Ä–FVÔw&÷W í
™<!-- AäAD$0C$;Dô5CÂ BCä;DÄ:Câ MD$8 Cô0C¤5D$K -->Ð
™<PackageReference Include="FluentValidation" Version="12.1.1" />Ð
™<PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.8" />Ð
™<PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />Ð
™<PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.5.1" />Ð
™<PackageReference Include="xunit.v3" Version="3.2.2" />Ð
™<PackageReference Include="xunit.runner.visualstudio" Version="3.1.5" />Ð
™<PackageReference Include="Moq" Version="4.20.72" />Ð
“Âô–FVÔw&÷W í
Ð
“Ä–FVÔw&÷W í
™<ProjectReference Include="..\Service\Promatis.Net.Service.csproj" />Ð
“Âô–FVÔw&÷W í
Ð
</Project>
</file>

<file path="Tests/Service/BaseServiceTests_Crud.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Service;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Service;Ð
Ð
public class BaseServiceTests_Crud : BaseServiceTestsÐ
{Ð
    // B$5D BCä2C O D CD”=CäAD$LÐ
    public class TestEntity : DomainObject { public string Title { get; set; } = ""; }Ð
Ð
    // B 5C ;C„7C FC„O D 5D 2C„AC  4C´O D$5D BC  A D4:C 7C =C„5CÂ :Cä=D$5C¤AD$0Ð
    public class TestService(IDbContextFactory<ApplicationDbContext> f)Ð
        : BaseService<TestEntity, Guid, ApplicationDbContext>(f);Ð
Ð
    [Fact]Ð
    public async Task AddAsync_Should_SaveEntityToDatabase()Ð
    {Ð
        // ArrangeÐ
        var service = new TestService(Factory);Ð
        var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Hello" };Ð
Ð
        // ActÐ
        await service.AddAsync(entity);Ð
Ð
        // AssertÐ
        TestEntity? saved = await service.GetByIdAsync(entity.Id);Ð
        Assert.NotNull(saved);Ð
        Assert.Equal("Hello", saved.Title);Ð
    }Ð
Ð
    [Fact]Ð
    public async Task UpdateAsync_Should_ModifyExistingEntity()Ð
    {Ð
```

```
        // ArrangeĐ
        var service = new TestService(Factory);Đ
        var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Old" };Đ
        await service.AddAsync(entity);Đ
Đ
        // ActĐ
        entity.Title = "New";Đ
        await service.UpdateAsync(entity);Đ
Đ
        // AssertĐ
        TestEntity? updated = await service.GetByIdAsync(entity.Id);Đ
        Assert.Equal("New", updated?.Title);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task DeleteAsync_Should_RemoveEntity()Đ
    {Đ
        // ArrangeĐ
        var service = new TestService(Factory);Đ
        var id = Guid.NewGuid();Đ
        await service.AddAsync(new TestEntity { Id = id });Đ
Đ
        // ActĐ
        await service.DeleteAsync(id);Đ
Đ
        // AssertĐ
        TestEntity? deleted = await service.GetByIdAsync(id);Đ
        Assert.Null(deleted);Đ
    }Đ
Đ
}
</file>

<file path="Tests/Service/BaseServiceTests.cs">
using Microsoft.EntityFrameworkCore;Đ
using Microsoft.Extensions.Configuration;Đ
using Microsoft.Extensions.DependencyInjection;Đ
using Moq;Đ
using Promatis.Net.Data;Đ
using Promatis.Net.Domain.Interface;Đ
using System.Text.Json;Đ
Đ
namespace Promatis.Net.ApplicationModel.Tests.Service;Đ
Đ
public abstract class BaseServiceTestsĐ
{Đ
    protected readonly IConfiguration Configuration;Đ
    protected readonly IDbContextFactory<ApplicationDbContext> Factory;Đ
    protected readonly IServiceProvider ServiceProvider;Đ
Đ
    protected BaseServiceTests()Đ
    {Đ
        // 1. A¤>CÔDC„3D4@C FC„OĐ
        Configuration = new ConfigurationBuilder()Đ
            .AddInMemoryCollection(new Dictionary<string, string?>Đ
            {Đ
                ["DatabaseSettings:DefaultSchema"] = "test"Đ
            })Đ
            .Build();Đ
Đ
        string dbName = Guid.NewGuid().ToString();Đ
        DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()Đ
            .UseInMemoryDatabase(dbName)Đ
            .Options;Đ
Đ
        // 2. B =C GC ;C  ACä7CD0CT< CÄ>C¢ DC 1D 8C¤8Đ
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();Đ
        Factory = factoryMock.Object;Đ
Đ
        // 3. AÔ0D BD >C”:C  D’ „"CT?CT@DÂ 4Cä1C 2C´OCT< D$CCD0 Factory)Đ
        var services = new ServiceCollection();Đ
        services.AddLogging();Đ
        services.AddSingleton(Configuration);Đ
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });Đ
```

```
        services.AddSingleton<IDbContextFactory<ApplicationDbContext>>(Factory);Đ
Đ
        // B 5C48D BD 8D CCT< D 5D 2C„AD² 8CÔDD 0D BD CC¤BD4@D² BD 8C43CT@Cä2Đ
        services.AddScoped<DatabaseTriggerService>();Đ
        services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());Đ
Đ
        // A$ Ad Aã¢  CT3C„AD$@C„@D45CÂ  B   D$@C„3C45D K, C¤>D$>D KCR <Cä3D4B C KD$L C$KCt2C =D½
        services.AddScoped<AuditTrigger>();Đ
        services.AddScoped<FluentValidationTrigger>(); // AD>C 0C$;Dô5CÂ MD$>D-
        services.AddScoped<ReferenceTreeParentTrigger>(); // A, MD$>D"Â 5D ;C, 8D ?Cä;DÄ7D45D$ADò 2 C„5D 0D E
Đ
        // AD A  A$ Bô AÂ -B$  B "B  A¤ : At0C4;D4HC¤8 CD;Dò ?D 5CD>D$2D 0D”5CÔ8Dò –çf Æ–D÷ W& F–öäW†6W F–öí
        // AÄK D 5C48D BD 8D CCT< Mock.Object, DtBCä1D² D’ <Cä3 "C$KCD0D$L" DtBCâÔBCâÂ :Cä3CD0 C„=D$5D FCT?D$
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);Đ
        services.AddScoped(_ => new Mock<IAfterSaveTrigger<IAudit>>().Object);Đ
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);Đ
        services.AddScoped(_ => new Mock<IAfterSaveTrigger<IDomainObject>>().Object);Đ
Đ
        // B$0C¤6CR 4C´O FluentValidationTrigger CÔCCd=D² 2C ;C„4C BCä@D½
        // ATAC´8 C" BCTAD$0DR 8D ?Cä;DÄ7D4ND$ADò @CT0C´LCÔKCR 2C ;C„4C BCä@D²Â <Cä6CÔ> C$KCt2C BDÂ AC¤0CÔ5D
        // services.AddValidatorsFromAssemblyContaining<SomeValidator>();Đ
Đ
        ServiceProvider = services.BuildServiceProvider();Đ
Đ
        // 4. AÔ0D BD 0C„2C 5CÂ ?Cä2CT4CT=C„5 CÄ>C¤0 (Returns CD>C´6CT= C KD$L C" :Cä=Dd5, C¤>C44C  2D Q C4>D
        factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))Đ
            .Returns(() =>Đ
            {Đ
                IServiceScope scope = ServiceProvider.CreateScope();Đ
Đ
                // A„!Aô A  A´ AÔ : A„7C$;CT:C 5CÂ DC 1D 8C¤C Scope C„7 D$5C¤CD”5C4> scope.ServiceProviderĐ
                var scopeFactory = scope.ServiceProvider.GetRequiredService<IServiceScopeFactory>();Đ
Đ
                DbContextOptions<ApplicationDbContext> interceptorOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()Đ
                    .UseInMemoryDatabase(dbName)Đ
                    .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A„!Aô A  A´ AÔ : Aô5D 5
scopeFactoryĐ
                    .Options;Đ
Đ
                return Task.FromResult<ApplicationDbContext>(new
TestIntegrationDbContext(interceptorOptions, Configuration));Đ
            });Đ
    }Đ
Đ
    // A$+AÔ B  AÂ  A´ B ! B .AD , DtBCä1D² >CÒ 1D´; CD>D BD4?CT= C$ACT< CÔ0D ;CT4CÔ8C¤0CÍ
    protected class TestIntegrationDbContext(Đ
        DbContextOptions<ApplicationDbContext> options,Đ
        IConfiguration config)Đ
        : ApplicationDbContext(options, config)Đ
    {Đ
        protected override void OnModelCreating(ModelBuilder modelBuilder)Đ
        {Đ
            base.OnModelCreating(modelBuilder);Đ
Đ
            modelBuilder.Entity<ReferenceTreeServiceTests.TestTreeEntity>();Đ
Đ
            // AD>C 0C$LD$5 DÔBD2 AD$@Cä:D2 7CD5D L:Đ
            modelBuilder.Entity<ServiceIntegrationTests.IntegratedEntity>();Đ
Đ
            // AäAD$0C´LCÔKCR @CT3C„AD$@C FC„8Đ
            modelBuilder.Entity<BaseServiceTests_Crud.TestEntity>();Đ
            modelBuilder.Entity<ReferenceServiceTests_Search.TestRef>();Đ
        }Đ
    }Đ
}
</file>

<file path="Tests/Service/ReferenceServiceTests_Search.cs">
using Microsoft.EntityFrameworkCore;Đ
using Promatis.Net.Data;Đ
using Promatis.Net.Domain;Đ
using Promatis.Net.Service;Đ
using Xunit;Đ
Đ
```

```
namespace Promatis.Net.ApplicationModel.Tests.Service;Đ
Đ
public class ReferenceServiceTests_Search : BaseServiceTestsĐ
{Đ
    public class TestRef : ReferenceBase { }Đ
    public class TestRefService(IDbContextFactory<ApplicationDbContext> f)Đ
        : ReferenceService<TestRef, ApplicationDbContext>(f);Đ
Đ
    [Fact]Đ
    public async Task GetByCodeAsync_Should_FindCorrectEntity()Đ
    {Đ
        // ArrangeĐ
        var service = new TestRefService(Factory);Đ
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "ABC", Name = "Test" });Đ
Đ
        // ActĐ
        TestRef? result = await service.GetByCodeAsync("ABC");Đ
Đ
        // AssertĐ
        Assert.NotNull(result);Đ
        Assert.Equal("ABC", result.Code);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task SearchByNameAsync_Should_ReturnFilteredList()Đ
    {Đ
        // ArrangeĐ
        var service = new TestRefService(Factory);Đ
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "1", Name = "Apple" });Đ
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "2", Name = "Banana" });Đ
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "3", Name = "Apricot" });Đ
Đ
        // ActĐ
        List<TestRef> results = await service.SearchByNameAsync("Ap");Đ
Đ
        // AssertĐ
        Assert.Equal(2, results.Count);Đ
        Assert.All(results, r => Assert.Contains("Ap", r.Name));Đ
    }Đ
}
</file>

<file path="Tests/Service/ReferenceTreeServiceTests.cs">
using Microsoft.EntityFrameworkCore;Đ
using Promatis.Net.Data;Đ
using Promatis.Net.Domain;Đ
using Promatis.Net.Service;Đ
using Xunit;Đ
Đ
namespace Promatis.Net.ApplicationModel.Tests.Service;Đ
Đ
Đ
public class ReferenceTreeServiceTests : BaseServiceTestsĐ
{Đ
    /// <summary>Đ
    /// B 0C >Dt0Dò BCTAD$>C$0Dò AD4ICÔ>D BDÂ 4C´O C$5D 8DD8C¤0Dd8C, 8CT@C @DT8Dt5D :C„E C ;C4>D 8D$<Cä2.Đ
    /// A„!Aô A  A´ AÔ : A 0Ct>C$KC′ :C´0D A ReferenceTreeBase<T> Ct0C¤@D´B D$8Cô>CÂ FW7EG&VTVçF—G′Â 4D41C´8D
    /// </summary>Đ
    public class TestTreeEntity : ReferenceTreeBase<TestTreeEntity>Đ
    {Đ
        // B 2Cä9D BC$0 Parent C, 6†–ÆG&Vâ 0C$BCä<C BC„GCTAC¤8 D4=C AC´5CD>C$0CÔK C„7 ReferenceTreeBase<TestT
        // C, 8CD5C ;DÄ=Câ 7C :D KC$0DäB C¤>CÔBD 0C¤B ITreeNode<TestTreeEntity> Cô>CB :C ?CäBCä<!Đ
    }Đ
Đ
    /// <summary>Đ
    /// B 0C >Dt8C′ BCTAD$>C$KC′ ACT@C$8D í
    /// A„!Aô A  A´ AÔ : B 5C ;C„7D45D" 0C AD$@C :D$=D´9 DTCC¢ ?C´0D$DCä@CÄK CreateChildTemplateAsync, Đ
    /// CÔ5Cä1DT>CD8CÄKC′ 4C´O C¤>CÄ?C„;DôFC„8 C 0Ct>C$>C4> ReferenceTreeService.Đ
    /// </summary>Đ
    public class TestTreeService(IDbContextFactory<ApplicationDbContext> f)Đ
        : ReferenceTreeService<TestTreeEntity, ApplicationDbContext>(f)Đ
    {Đ
        /// <summary>Đ
        /// Aô@CäAD$5C”HC O D$5D BCä2C O DD0C @C„:C  ACä7CD0CÔ8Dò ?D4AD$KDR HC 1C´>CÔ>C" CCt;Cä2 CD;Dò ?D >C4
        /// </summary>Đ
        public override Task<TestTreeEntity> CreateChildTemplateAsync(TestTreeEntity parent)Đ
```

```
        {Đ
            var child = new TestTreeEntityĐ
            {Đ
                ParentId = parent.IdĐ
            };Đ
            child.Parent = null;Đ
            return Task.FromResult(child);Đ
        }Đ
    }Đ
Đ
    [Fact]Đ
    public async Task GetParentPathAsync_Should_Return_Correct_Order()Đ
    {Đ
        // ArrangeĐ
        var service = new TestTreeService(Factory);Đ
        var rootId = Guid.NewGuid();Đ
        var parentId = Guid.NewGuid();Đ
        var childId = Guid.NewGuid();Đ
Đ
        await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });Đ
        await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent", ParentId =
rootId });Đ
        await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId =
parentId });Đ
Đ
        // ActĐ
        List<TestTreeEntity> path = await service.GetParentPathAsync(childId);Đ
Đ
        // AssertĐ
        Assert.Equal(3, path.Count);Đ
        Assert.Equal("Root", path[0].Name);Đ
        Assert.Equal("Parent", path[1].Name);Đ
        Assert.Equal("Child", path[2].Name);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task GetFullTreeAsync_Should_Build_Correct_Memory_Structure()Đ
    {Đ
        // ArrangeĐ
        var service = new TestTreeService(Factory);Đ
        var rootId = Guid.NewGuid();Đ
Đ
        await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });Đ
        await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child1", ParentId
= rootId });Đ
        await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child2", ParentId
= rootId });Đ
Đ
        // ActĐ
        TestTreeEntity? tree = await service.GetFullTreeAsync(rootId);Đ
Đ
        // AssertĐ
        Assert.NotNull(tree);Đ
        Assert.Equal(2, tree.Children.Count);Đ
        // Aô@Cä2CT@C¤0 Cä1D 0D$=Cä9 D 2Dô7C, …  &VçB•
        Assert.All(tree.Children, child => Assert.Same(tree, child.Parent));Đ
    }Đ
Đ
    [Fact]Đ
    public async Task GetRootsAsync_Should_Return_Only_Entities_Without_Parent()Đ
    {Đ
        // ArrangeĐ
        var service = new TestTreeService(Factory);Đ
        var rootId = Guid.NewGuid();Đ
Đ
        // 1. B >Ct4C 5CÂ GCTAD$=D´9 C¤>D 5CÔLĐ
        await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });Đ
Đ
        // 2. B >Ct4C 5CÂ @CT1CT=C¤0, Cô@C„2Dô7C =CÔ>C4> C¢ AD4ICTAD$2D4ND”5CÄC C¤>D =Dí
        // BÔBCâ 3C @C =D$8D CCTB, DtBCâ 4CT@CT2Câ 2C ;C„4CÔ>Đ
        await service.AddAsync(new TestTreeEntityĐ
        {Đ
            Id = Guid.NewGuid(),Đ
            Name = "Child",Đ
            ParentId = rootId // A„ACô>C´LCtCCT< D 5C ;DÄ=D´9 IDĐ
        });Đ
```

```
Đ
        // ActĐ
        List<TestTreeEntity> roots = await service.GetRootsAsync();Đ
Đ
        // AssertĐ
        // AD>C´6CT= CäAD$0D$LD O D$>C´LC¤> Cä4C„= C¤>D 5CÔLĐ
        Assert.Single(roots);Đ
        Assert.Null(roots[0].ParentId);Đ
        Assert.Equal("Root", roots[0].Name);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task GetChildrenAsync_Should_Return_Direct_Descendants()Đ
    {Đ
        // ArrangeĐ
        var service = new TestTreeService(Factory);Đ
        var parentId = Guid.NewGuid();Đ
        await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });Đ
        await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C1", ParentId =
parentId });Đ
        await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C2", ParentId =
parentId });Đ
Đ
        // ActĐ
        List<TestTreeEntity> children = await service.GetChildrenAsync(parentId);Đ
Đ
        // AssertĐ
        Assert.Equal(2, children.Count);Đ
        Assert.All(children, c => Assert.Equal(parentId, c.ParentId));Đ
    }Đ
Đ
    [Fact]Đ
    public async Task MoveAsync_Should_Update_ParentId_Successfully()Đ
    {Đ
        // ArrangeĐ
        var service = new TestTreeService(Factory);Đ
        var oldParentId = Guid.NewGuid();Đ
        var newParentId = Guid.NewGuid();Đ
        var targetId = Guid.NewGuid();Đ
Đ
        await service.AddAsync(new TestTreeEntity { Id = oldParentId, Name = "Old Parent" });Đ
        await service.AddAsync(new TestTreeEntity { Id = newParentId, Name = "New Parent" });Đ
        await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Target", ParentId =
oldParentId });Đ
Đ
        // ActĐ
        await service.MoveAsync(targetId, newParentId, TestContext.Current.CancellationToken);Đ
Đ
        // AssertĐ
        TestTreeEntity? updated = await service.GetByIdAsync(targetId);Đ
        Assert.Equal(newParentId, updated!.ParentId);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task MoveAsync_Into_Self_Should_Throw_InvalidOperationException()Đ
    {Đ
        // ArrangeĐ
        var service = new TestTreeService(Factory);Đ
        var targetId = Guid.NewGuid();Đ
        await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Self" });Đ
Đ
        // Act & AssertĐ
        await Assert.ThrowsAsync<InvalidOperationException>(() =>Đ
            service.MoveAsync(targetId, targetId, TestContext.Current.CancellationToken));Đ
    }Đ
Đ
    [Fact]Đ
    public async Task MoveAsync_Into_Own_Subtree_Should_Throw_InvalidOperationException()Đ
    {Đ
        // Arrange (B >Ct4C 5CÂ 8CT@C @DT8Dã¢   Óâ " Óâ 2•
        var service = new TestTreeService(Factory);Đ
        var idA = Guid.NewGuid();Đ
        var idB = Guid.NewGuid();Đ
        var idC = Guid.NewGuid();Đ
Đ
        await service.AddAsync(new TestTreeEntity { Id = idA, Name = "A" });Đ
```

```
        await service.AddAsync(new TestTreeEntity { Id = idB, Name = "B", ParentId = idA });Ð
        await service.AddAsync(new TestTreeEntity { Id = idC, Name = "C", ParentId = idB });Ð
Ð
        // Act & Assert Ð
        // AôKD$0CT<D O Cô5D 5CÄ5D BC„BDÂ   2CÔCD$@DÂ AC$>CT3Câ 2CÔCC¤0 CÐ
        var exception = await Assert.ThrowsAsync<InvalidOperationException>(() =>Ð
            service.MoveAsync(idA, idC, TestContext.Current.CancellationToken));Ð
Ð
        Assert.Contains("Bd8C¤;C„GCTAC¤0Dò 7C 2C„AC„<CäAD$L", exception.Message);Ð
    }Ð
Ð
    [Fact]Ð
    public async Task MoveAsync_To_Null_Should_Move_To_Root()Ð
    {Ð
        // ArrangeÐ
        var service = new TestTreeService(Factory);Ð
        var parentId = Guid.NewGuid();Ð
        var childId = Guid.NewGuid();Ð
Ð
        await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });Ð
        await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId =
parentId });Ð
Ð
        // ActÐ
        await service.MoveAsync(childId, null, TestContext.Current.CancellationToken);Ð
Ð
        // AssertÐ
        TestTreeEntity? updated = await service.GetByIdAsync(childId);Ð
        Assert.Null(updated!.ParentId);Ð
    }Ð
}
</file>

<file path="Tests/Service/ServiceIntegrationTests.cs">
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.Extensions.Configuration;Ð
using Microsoft.Testing.Platform.Services;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using Promatis.Net.Service;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Service;Ð
Ð
public class ServiceIntegrationTests : BaseServiceTestsÐ
{Ð
    // --- 1. B$ B "Aä B´  A¤ A !B + ---Ð
Ð
    public class IntegratedEntity : DomainObject, IAuditÐ
    {Ð
        public string Name { get; set; } = "";Ð
    }Ð
Ð
    // AD>C 0C$;Dô5CÂ D6öçFW‡B „   Æ–6 F–öäF$6öçFW‡B’ 2 CÔ0D ;CT4Cä2C =C„5Ð
    public class IntegratedService(IDbContextFactory<ApplicationDbContext> f)Ð
        : BaseService<IntegratedEntity, Guid, ApplicationDbContext>(f);Ð
Ð
    // --- 2. AÔ B "B  A” A   Aä B$ A¤!B$  ---Ð
    private class ServiceTestDbContext(DbContextOptions<ApplicationDbContext> options,
IConfiguration config)Ð
        : TestIntegrationDbContext(options, config)Ð
    {Ð
        protected override void OnModelCreating(ModelBuilder modelBuilder)Ð
        {Ð
            base.OnModelCreating(modelBuilder);Ð
            modelBuilder.Entity<IntegratedEntity>();Ð
        }Ð
    }Ð
Ð
    // --- 3. B$ B " ---Ð
Ð
    [Fact]Ð
    public async Task AddAsync_Should_TriggerAuditLogCreation()Ð
    {Ð
        // ArrangeÐ
```

```
        var triggerService = ServiceProvider.GetRequiredService<IDatabaseTriggerService>();Ð
Ð
        // B 5C48D BD 0Dd8Dò BD 8C43CT@C
        triggerService.Register<DomainObject, AuditTrigger>();Ð
Ð
        // A„ACô>C´LCtCCT< C 0Ct>C$CDâ DC 1D 8C¤C (Factory), D$0C¢ :C : IntegratedService Ð
        // D$5Cô5D L C¤>D @CT:D$=Câ BC„?C„7C„@Cä2C = Cô>CB    Æ–6 F–öäF$6öçFW‡M
        var service = new IntegratedService(Factory);Ð
        var entityId = Guid.NewGuid();Ð
        var entity = new IntegratedEntity { Id = entityId, Name = "Integration Test" };Ð
Ð
        // ActÐ
        await service.AddAsync(entity);Ð
Ð
        // AssertÐ
        // AÔ5C >C´LD„0Dò 7C 4CT@Cd:C  4C´O Cä1D 0C >D$:C, BD 8C43CT@Cä2Ð
        await Task.Delay(50, TestContext.Current.CancellationToken);Ð
Ð
        await using ApplicationDbContext context = await
Factory.CreateDbContextAsync(TestContext.Current.CancellationToken);Ð
Ð
        // Aô@Cä2CT@Dô5CÂ ;Cä3 C CCD8D$0Ð
        AuditLog? auditLog = await context.Set<AuditLog>()Ð
            .FirstOrDefaultAsync(x => x.EntityId == entityId,
TestContext.Current.CancellationToken);Ð
Ð
        Assert.NotNull(auditLog);Ð
        Assert.Equal("Added", auditLog.Action);Ð
        Assert.Contains("Integration Test", auditLog.ChangesJson);Ð
    }Ð
}
</file>

<file path="Tests/Trigger/AuditTriggerTests.cs">
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.Extensions.Configuration;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Moq;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Text.Json;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Trigger;Ð
Ð
public class AuditTriggerTestsÐ
{Ð
    // --- 1. B$ B "Aä B´  A¤ A !B + ---Ð
Ð
    public class AuditIntegrationEntity : DomainObject, IAuditÐ
    {Ð
        public string Name { get; set; } = "";Ð
    }Ð
Ð
    // B$ B "Aä B´  A¤ AÔ"AT B ": B$5Cô5D L Cô@C„=C„<C 5D" ”6öæf–wW& F–öí
    public class AuditIntegrationDbContext(Ð
        DbContextOptions<ApplicationDbContext> options,Ð
        IConfiguration configuration)Ð
        : ApplicationDbContext(options, configuration)Ð
    {Ð
        protected override void OnModelCreating(ModelBuilder modelBuilder)Ð
        {Ð
            base.OnModelCreating(modelBuilder);Ð
Ð
            // B 5C48D BD 0Dd8Dò AD4ICÔ>D BCT9 CD;Dò BCTAD$0Ð
            modelBuilder.Entity<AuditIntegrationEntity>();Ð
            modelBuilder.Entity<AuditLog>();Ð
        }Ð
    }Ð
Ð
    // --- 2. A¤ AÔ!B$ B4 B$ B  "AT!B$  ---Ð
    public AuditTriggerTests()Ð
    {Ð
        DatabaseTriggerService.ClearInternalRegistrations();Ð
    }Ð
```

```
Ð
    // --- 3. B  AÂ "AT!B" ÒÒÝ
Ð
    [Fact]Ð
    public async Task SaveChanges_ShouldCreateAuditLog_WithCorrectJson()Ð
    {Ð
        // B >Ct4C 5CÂ DCT9C¤>C$CDâ :Cä=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()Ð
            .AddInMemoryCollection(new Dictionary<string, string?>Ð
            {Ð
                ["DatabaseSettings:DefaultSchema"] = "test"Ð
            })Ð
            .Build();Ð
Ð
        var services = new ServiceCollection();Ð
        services.AddLogging();Ð
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;Dô5CÂ 2 DIÐ
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });Ð
Ð
        string dbName = "FinalAuditDb_" + Guid.NewGuid();Ð
        DbContextOptions<ApplicationDbContext> dbOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()Ð
            .UseInMemoryDatabase(dbName)Ð
            .Options;Ð
Ð
        // AÔ0D BD >C”:C  DC 1D 8C¤8 (D$5Cô5D L Cô@Cä1D 0D KC$0CT< C¤>CÔDC„3)Ð
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();Ð
        factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))Ð
                   .Returns(() => Task.FromResult<ApplicationDbContext>(new
AuditIntegrationDbContext(dbOptions, configuration)));Ð
Ð
        services.AddSingleton(factoryMock.Object);Ð
        services.AddScoped<AuditTrigger>();Ð
        services.AddScoped<DatabaseTriggerService>();Ð
        services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());Ð
Ð
        ServiceProvider serviceProvider = services.BuildServiceProvider();Ð
        using IServiceScope scope = serviceProvider.CreateScope();Ð
        IServiceProvider sp = scope.ServiceProvider;Ð
Ð
        var triggerService = sp.GetRequiredService<DatabaseTriggerService>();Ð
        triggerService.Register<DomainObject, AuditTrigger>();Ð
Ð
        // Aô5D 5DT2C BDt8C-
        // A„!Aô A  A´ AÔ  AD Bò ääUB   ¢  Ct2C´5C¤0CT< DD0C @C„:D2 66÷ R 8Cr BCTAD$>C$>C4> C¤>CÔBCT9CÔ5D 0 C
        var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();Ð
Ð
        // A„!Aô A  A´ AÔ : B >Ct4C 5CÂ 8CÔBCT@Dd5CôBCä@, Cô5D 5CD0C$0Dò 5CÄC D$>C´LC¤> Aä A„  C @C4CCÄ5CÔB —
        var interceptor = new DatabaseTriggerInterceptor(scopeFactory);Ð
Ð
        DbContextOptions<ApplicationDbContext> mainOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()Ð
            .UseInMemoryDatabase(dbName)Ð
            .AddInterceptors(interceptor)Ð
            .Options;Ð
Ð
        var entityId = Guid.NewGuid();Ð
Ð
        // --- ACT & ASSERT ---Ð
        // Aô5D 5CD0CT< configuration C" :Cä=D BD CC¤BCä@ C¤>CÔBCT:D BC
        await using (var context = new AuditIntegrationDbContext(mainOptions, configuration))Ð
        {Ð
            var entity = new AuditIntegrationEntity { Id = entityId, Name = "Audit Test" };Ð
            context.Add(entity);Ð
            await context.SaveChangesAsync(TestContext.Current.CancellationToken);Ð
Ð
            // AD0CT< C$@CT<Dò BD 8C43CT@D=
            await Task.Delay(100, TestContext.Current.CancellationToken);Ð
Ð
            AuditLog? log = await context.Set<AuditLog>()Ð
                .FirstOrDefaultAsync(x => x.EntityId == entityId, cancellationToken:
TestContext.Current.CancellationToken);Ð
Ð
            Assert.NotNull(log);Ð
```

```
            Assert.Equal("Added", log.Action);Đ
            Assert.Contains("Audit Test", log.ChangesJson);Đ
        }Đ
    }Đ
}
```
</file>

<file path="Tests/Trigger/DatabaseTriggerServiceTests.cs">
```
using Moq;Đ
using Promatis.Net.Data;Đ
using Promatis.Net.Domain;Đ
using Promatis.Net.Domain.Interface;Đ
using Xunit;Đ
Đ
namespace Promatis.Net.ApplicationModel.Tests.Trigger;Đ
Đ
Đ
// --- B$ B "Aä B´  A¤ A !B + (D$5Cô5D L CD>D BC BCäGCÔ> Cä4CÔ>C4> CÔ0C >D 0) ---Đ
public class TestEntity : DomainObject, IAudit { }Đ
public interface ITestBeforeTrigger : IBeforeSaveTrigger<TestEntity> { }Đ
public interface ITestAfterTrigger : IAfterSaveTrigger<TestEntity> { }Đ
public interface ISecondAfterTrigger : IAfterSaveTrigger<TestEntity> { }Đ
public class InvalidTrigger { }Đ
Đ
public class DatabaseTriggerServiceTestsĐ
{Đ
    private readonly Mock<IServiceProvider> _serviceProviderMock;Đ
    private readonly DatabaseTriggerService _service;Đ
Đ
    public DatabaseTriggerServiceTests()Đ
    {Đ
        // 1. AäGC„IC 5CÂ AD$0D$8C¤C Cô5D 5CB :C 6CDKCÂ BCTAD$>CÍ
        DatabaseTriggerService.ClearInternalRegistrations();Đ
Đ
        _serviceProviderMock = new Mock<IServiceProvider>();Đ
        _service = new DatabaseTriggerService(_serviceProviderMock.Object);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task ValidateAsync_ShouldExecuteRegisteredTrigger()Đ
    {Đ
        // ArrangeĐ
        var triggerMock = new Mock<ITestBeforeTrigger>();Đ
        _serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))Đ
                            .Returns(triggerMock.Object);Đ
Đ
        _service.Register<TestEntity, ITestBeforeTrigger>();Đ
Đ
        // ActĐ
        await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);Đ
Đ
        // AssertĐ
        triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()),
Times.Once);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task ValidateAsync_ShouldThrowException_WhenTriggerCancels()Đ
    {Đ
        // ArrangeĐ
        string errorMessage = "Stop!";Đ
        var triggerMock = new Mock<ITestBeforeTrigger>();Đ
        triggerMock.Setup(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()))Đ
                   .Callback<EntityCancelEventArgs<TestEntity>>(args =>Đ
                   {Đ
                       args.Cancel = true;Đ
                       args.ErrorMessage = errorMessage;Đ
                   });Đ
Đ
        _serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))Đ
                            .Returns(triggerMock.Object);Đ
Đ
        _service.Register<TestEntity, ITestBeforeTrigger>();Đ
Đ
        // Act & AssertĐ
        var ex = await Assert.ThrowsAsync<OperationCanceledException>(() =>Đ
```

```
            _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!));Đ
Đ
        Assert.Equal(errorMessage, ex.Message);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task Hierarchy_ShouldInvokeTriggerForInterface()Đ
    {Đ
        // ArrangeĐ
        var triggerMock = new Mock<IBeforeSaveTrigger<IAudit>>();Đ
        _serviceProviderMock.Setup(x => x.GetService(typeof(IBeforeSaveTrigger<IAudit>)))Đ
                            .Returns(triggerMock.Object);Đ
Đ
        // B 5C48D BD 8D CCT< CÔ0 C„=D$5D DCT9D
        _service.Register<IAudit, IBeforeSaveTrigger<IAudit>>();Đ
Đ
        // Act: TestEntity D 5C ;C„7D45D" ” VF—M
        await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);Đ
Đ
        // AssertĐ
        triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<IAudit>>()),
Times.Once);Đ
    }Đ
Đ
    [Fact]Đ
    public void Register_ShouldThrow_WhenTriggerDoesNotImplementInterfaces()Đ
    {Đ
        // Act & AssertĐ
        Assert.Throws<InvalidOperationException>(() =>Đ
            _service.Register<TestEntity, InvalidTrigger>());Đ
    }Đ
Đ
    [Fact]Đ
    public async Task NotifyAsync_ShouldStopExecution_WhenHandledIsTrue()Đ
    {Đ
        // ArrangeĐ
        var triggerMock1 = new Mock<ITestAfterTrigger>();Đ
        triggerMock1.Setup(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()))Đ
                    .Callback<EntityChangedEventArgs<TestEntity>>(args => args.Handled = true);Đ
Đ
        var triggerMock2 = new Mock<ISecondAfterTrigger>();Đ
Đ
        _serviceProviderMock.Setup(x =>
x.GetService(typeof(ITestAfterTrigger))).Returns(triggerMock1.Object);Đ
        _serviceProviderMock.Setup(x =>
x.GetService(typeof(ISecondAfterTrigger))).Returns(triggerMock2.Object);Đ
Đ
        _service.Register<TestEntity, ITestAfterTrigger>();Đ
        _service.Register<TestEntity, ISecondAfterTrigger>();Đ
Đ
        // ActĐ
        await _service.NotifyAsync(new TestEntity(), EntityStateChangeEnum.Added, [], "User",
DateTime.UtcNow);Đ
Đ
        // AssertĐ
        triggerMock1.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()),
Times.Once);Đ
        triggerMock2.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()),
Times.Never);Đ
    }Đ
}
</file>

<file path="Tests/Trigger/FluentValidationTriggerTests.cs">
using FluentValidation;Đ
using Microsoft.Extensions.DependencyInjection;Đ
using Moq;Đ
using Promatis.Net.Data;Đ
using Promatis.Net.Domain;Đ
using Promatis.Net.Domain.Interface;Đ
using Xunit;Đ
Đ
namespace Promatis.Net.ApplicationModel.Tests.Trigger;Đ
Đ
public class FluentValidationTriggerTestsĐ
{Đ
```

```
    private class TestEntity : DomainObject { public string Name { get; set; } = ""; }Đ
Đ
    private readonly Mock<IServiceProvider> _serviceProviderMock;Đ
    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;Đ
    private readonly Mock<IServiceScope> _serviceScopeMock;Đ
Đ
    private class TestEntityValidator : AbstractValidator<TestEntity>Đ
    {Đ
        public TestEntityValidator()Đ
        {Đ
            RuleFor(x => x.Name).NotEmpty().WithMessage("Name is required");Đ
        }Đ
    }Đ
Đ
    public FluentValidationTriggerTests()Đ
    {Đ
        _serviceProviderMock = new Mock<IServiceProvider>();Đ
        _scopeFactoryMock = new Mock<IServiceScopeFactory>();Đ
        _serviceScopeMock = new Mock<IServiceScope>();Đ
Đ
        // AÔ0D BD 0C„2C 5CÂ FCT?CäGC¤C: BD0C @C„:C  ACä7CD0CTB Scope -> Scope C$>Ct2D 0D”0CTB ServiceProvide
        _serviceScopeMockĐ
            .Setup(s => s.ServiceProvider)Đ
            .Returns(_serviceProviderMock.Object);Đ
Đ
        _scopeFactoryMockĐ
            .Setup(f => f.CreateScope())Đ
            .Returns(_serviceScopeMock.Object);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task HandleAsync_ShouldCancel_WhenValidationFails()Đ
    {Đ
        // ArrangeĐ
        var entity = new TestEntity { Name = "" };Đ
        var validator = new TestEntityValidator();Đ
Đ
        _serviceProviderMockĐ
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))Đ
            .Returns(validator);Đ
Đ
        // A$=CT4D OCT< Cô@Cä2C 9CD5D  =C ?D OCÄCDâ 2 D$@C„3C45D
        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);Đ
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(Đ
            entity, EntityStateChangeEnum.Added, [], null!);Đ
Đ
        // ActĐ
        await trigger.HandleAsync(args);Đ
Đ
        // AssertĐ
        Assert.True(args.Cancel);Đ
        Assert.Contains("Name is required", args.ErrorMessage);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task HandleAsync_ShouldNotCancel_WhenValidationSucceeds()Đ
    {Đ
        // ArrangeĐ
        var entity = new TestEntity { Name = "Valid Name" };Đ
        var validator = new TestEntityValidator();Đ
Đ
        _serviceProviderMockĐ
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))Đ
            .Returns(validator);Đ
Đ
        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);Đ
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(Đ
            entity, EntityStateChangeEnum.Added, [], null!);Đ
Đ
        // ActĐ
        await trigger.HandleAsync(args);Đ
Đ
        // AssertĐ
        Assert.False(args.Cancel);Đ
        Assert.Null(args.ErrorMessage);Đ
    }Đ
```

```
Ð
    [Fact]Ð
    public async Task HandleAsync_ShouldIgnore_WhenNoValidatorRegistered()Ð
    {Ð
        // ArrangeÐ
        _serviceProviderMockÐ
            .Setup(sp => sp.GetService(It.IsAny<Type>()))Ð
            .Returns(null!);Ð
Ð
        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);Ð
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(Ð
            new TestEntity(), EntityStateChangeEnum.Added, [], null!);Ð
Ð
        // ActÐ
        await trigger.HandleAsync(args);Ð
Ð
        // AssertÐ
        Assert.False(args.Cancel);Ð
    }Ð
}
</file>

<file path="Tests/Trigger/FullTriggerChainTests.cs">
using FluentValidation;Ð
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.Extensions.Configuration;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Moq;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Text.Json;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Trigger;Ð
Ð
// --- 1. B$ B "Aä B´  A¤ A !B + ---Ð
Ð
public class IntegrationEntity : DomainObject, IAuditÐ
{Ð
    public string Name { get; set; } = "";Ð
}Ð
Ð
public class IntegrationValidator : AbstractValidator<IntegrationEntity>Ð
{Ð
    public IntegrationValidator()Ð
    {Ð
        RuleFor(x => x.Name).NotEmpty().WithMessage("NameRequired");Ð
    }Ð
}Ð
Ð
public class IntegrationDbContext(Ð
    DbContextOptions<ApplicationDbContext> options,Ð
    IConfiguration configuration)Ð
    : ApplicationDbContext(options, configuration)Ð
{Ð
    // 1. A„!Aô A  A´ AÔ AS¢  CT;C 5CÂ BCTAD$>C$KC’ CCt5C² ?Cä;CÔ>Dd5CÔ=D´< C„7Cä;C„@Cä2C =CÔKCÂ 4CT@CT2Cä<Ð
    private class TestNode : ReferenceTreeBase<TestNode>Ð
    {Ð
    }Ð
Ð
    protected override void OnModelCreating(ModelBuilder modelBuilder)Ð
    {Ð
        // 1. At0CôCD :C 5CÂ 1C 7Cä2D´9 C 2D$>CÄ0D$8Dt5D :C„9 D :C =CT@ PromatisÐ
        base.OnModelCreating(modelBuilder);Ð
Ð
        // 2. AÔ0D BD >C”:C  4C´O InMemory Cô@Cä2C 9CD5D 0Ð
        if (Database.ProviderName == "Microsoft.EntityFrameworkCore.InMemory")Ð
        {Ð
            modelBuilder.Entity<TestNode>(builder =>Ð
            {Ð
                // At0CD0CT< Dô2CÔ>CR 8CÄO D$0C ;C„FD²Â GD$>C K InMemory C„7Cä;C„@Cä2C ; CTQÐ
                builder.ToTable("FullTriggerChain_TestNodes");Ð
Ð
                // A„!Aô A  A´ AÔ AS¢  C AD$@C 8C$0CT< D 2Dô7DÂ AD$@Cä3Câ =C  BC„?CR FW7DæöFRÂ
                // C„ACô>C´LCtCDò AD$@Cä3Câ BC„?C„7C„@Cä2C =CÔKCR æ ÖVöb >D" AC <Cä3Câ :C´0D AC  BCTAD"Ô=Cä4D
```

```
                builder.HasOne(x => x.Parent)Ð
                    .WithMany(x => x.Children)Ð
                    .HasForeignKey(x => x.ParentId)Ð
                    .OnDelete(DeleteBehavior.Restrict);Ð
            });Ð
Ð
            // B$0C¤6CR @CT3C„AD$@C„@D45CÂ AC <D2 BCTAD$>C$CDâ 8CÔBCT3D 0Dd8Cä=CÔCDâ AD4ICÔ>D BDÍ
            modelBuilder.Entity<IntegrationEntity>();Ð
        }Ð
    }Ð
}Ð
Ð
public class FullTriggerChainTestsÐ
{Ð
    [Fact]Ð
    public async Task SaveChanges_ShouldExecuteFullChain_AndCancelOnValidationError()Ð
    {Ð
        // --- ARRANGE ---Ð
Ð
        // 1. B >Ct4C 5CÂ DCT9C¤>C$CDâ :Cä=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()Ð
            .AddInMemoryCollection(new Dictionary<string, string?>Ð
            {Ð
                ["DatabaseSettings:DefaultSchema"] = "test"Ð
            })Ð
            .Build();Ð
Ð
        var services = new ServiceCollection();Ð
        services.AddLogging();Ð
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;DÔ5CÂ :Cä=DD8C2 2 DIÐ
Ð
        // 2. A„=DD@C AD$@D4:D$CD =D´5 CÔ0D BD >C”:C•
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });Ð
        // AÄ>C¢ DC 1D 8C¤8 D$5CÔ5D L C$>Ct2D 0D”0CTB C¤>CÔBCT:D B D  :Cä=DD8C4>CÍ
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();Ð
        services.AddSingleton(factoryMock.Object);Ð
Ð
        // 3. B 5C48D BD 0Dd8Dò ACT@C$8D >C" BD 8C43CT@Cä2Ð
        services.AddScoped<DatabaseTriggerService>();Ð
        services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());Ð
Ð
        // B 0CÄ8 D$@C„3C45D KÐ
        services.AddScoped<FluentValidationTrigger>();Ð
        services.AddScoped<ReferenceTreeParentTrigger>();Ð
        services.AddScoped<AuditTrigger>();Ð
Ð
        // 4. At0C4;D4HC¤8 CD;Dò 8CÔBCT@DD5C”ACä2 (DtBCä1D² FCT?CäGC¤0 CÔ5 D 2C ;C ADÂÂ 5D ;C, BD 8C43CT@ D$@
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);Ð
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);Ð
Ð
        // 5. B 5C48D BD 0Dd8Dò 2C ;C„4C BCä@C
        services.AddTransient<IValidator<IntegrationEntity>, IntegrationValidator>();Ð
Ð
        ServiceProvider serviceProvider = services.BuildServiceProvider();Ð
Ð
        using IServiceScope scope = serviceProvider.CreateScope();Ð
        IServiceProvider sp = scope.ServiceProvider;Ð
        var triggerService = sp.GetRequiredService<DatabaseTriggerService>();Ð
Ð
        // AÔ@C„2DÔ7D´2C 5CÂ fÇVVçEf Æ–F F–öåG&–vvW" :Câ 2D 5CÂ AD4ICÔ>D BDô< D  wV–BÔ:C´NDt>CÍ
        triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();Ð
Ð
        // AÔ@C„2DÔ7D´2C 5CÂ fÇVVçEf Æ–F F–öåG&–vvW" :Câ 2D 5CÂ AD4ICÔ>D BDô< D  wV–BÔ:C´NDt>CÍ
        triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();Ð
Ð
        // A„!AÔ A  A´ AÔ  AD Bò ääUB   ¢  Ct2C´5C¤0CT< DD0C @C„:D2 66÷ R 8Cr BCTAD$>C$>C4> CÔ@Cä2C 9CD5D 0Ð
        var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();Ð
Ð
        // AÔ0D BD >C”:C  >CÔFC„9 A   D  8D ?D 0C$;CT=CÔKCÂ ?CT@CTEC$0D$GC„:Cä< (CÔ5D 5CD0CT< D$>C´LC¤> DD0C
        DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()Ð
            .UseInMemoryDatabase(Guid.NewGuid().ToString())Ð
            .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A„!AÔ A  A´ AÔ : Aä4C„= C @C4CC
            .Options;Ð
```

```
Ð
        // B >Ct4C 5CÂ :Cä=D$5C¤AD"Â ?CT@CT4C 2C O Cä?Dd8C, 8 C¤>CÔDC„3Ð
        await using var context = new IntegrationDbContext(options, configuration);Ð
        await context.Database.EnsureCreatedAsync(TestContext.Current.CancellationToken);Ð
Ð
        // B >Ct4C 5CÂ >C JCT:D"Â :CäBCä@D´9 AÔ  Cô@Cä9CD5D" 2C ;C„4C FC„N (Name CôCD BCä9)Ð
        var entity = new IntegrationEntity { Name = "" };Ð
        context.Add(entity);Ð
Ð
        // --- ACT & ASSERT ---Ð
Ð
        // Aä6C„4C 5CÂÂ GD$> C„=D$5D FCT?D$>D  2D´7Cä2CTB D$@C„3C45D Â BCäB C$KCt>C$5D" 2C ;C„4C BCä@, C, 1D4
        var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>Ð
        {Ð
            await context.SaveChangesAsync(TestContext.Current.CancellationToken);Ð
        });Ð
Ð
        // Aô@Cä2CT@Dô5CÂÂ GD$> CD>D„;C, 8CÄ5CÔ=Câ 4Câ =C HCT3Câ ACä>C ICT=C„O C„7 IntegrationValidatorÐ
        Assert.Equal("NameRequired", exception.Message);Ð
    }Ð
}
</file>

<file path="Tests/Trigger/ReferenceTreeParentTriggerTests.cs">
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.Extensions.Configuration;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Trigger;Ð
Ð
public class ReferenceTreeParentTriggerTestsÐ
{Ð
    // --- 1. B$ B "Aä B´  A¤ A !B + ---Ð
Ð
    private class TestNode : ReferenceTreeBase<TestNode> { }Ð
Ð
    // A¤>CÔBCT:D B D$5Cô5D L Cô@C„=C„<C 5D" ”6öæf–wW& F–öí
    private class TestDbContext(Ð
        DbContextOptions<ApplicationDbContext> options,Ð
        IConfiguration configuration)Ð
        : ApplicationDbContext(options, configuration)Ð
    {Ð
        protected override void OnModelCreating(ModelBuilder modelBuilder)Ð
        {Ð
            base.OnModelCreating(modelBuilder);Ð
            modelBuilder.Entity<TestNode>();Ð
        }Ð
    }Ð
Ð
    // --- 2. Aô AD Aä"Aä A¤  Aä B #Ad AÔ Bò ÒÒÝ
Ð
    private readonly IConfiguration _configuration;Ð
Ð
    public ReferenceTreeParentTriggerTests()Ð
    {Ð
        // B >Ct4C 5CÂ DCT9C¤>C$KC’ :Cä=DD8C2 4C´O D$5D BCä2Ð
        _configuration = new ConfigurationBuilder()Ð
            .AddInMemoryCollection(new Dictionary<string, string?>Ð
            {Ð
                ["DatabaseSettings:DefaultSchema"] = "test"Ð
            })Ð
            .Build();Ð
    }Ð
Ð
    private async Task<TestDbContext> GetContextAsync()Ð
    {Ð
        DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()Ð
            .UseInMemoryDatabase(Guid.NewGuid().ToString())Ð
            .Options;Ð
Ð
        var context = new TestDbContext(options, _configuration);Ð
        await context.Database.EnsureCreatedAsync();Ð
```

```
        return context;Đ
    }Đ
Đ
    // --- 3. B$ B "B² ÒÒÝ
Đ
    [Fact]Đ
    public async Task Trigger_ShouldCancel_WhenObjectIsItsOwnParent()Đ
    {Đ
        // ArrangeĐ
        await using TestDbContext context = await GetContextAsync();Đ
        var trigger = new ReferenceTreeParentTrigger();Đ
Đ
        var nodeId = Guid.NewGuid();Đ
        // Id C,   &VçD–B ACä2Cô0CD0DäBĐ
        var node = new TestNode { Id = nodeId, ParentId = nodeId, Name = "Self Parent" };Đ
Đ
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(Đ
            node, EntityStateChangeEnum.Added, [], context);Đ
Đ
        // ActĐ
        await trigger.HandleAsync(args);Đ
Đ
        // AssertĐ
        Assert.True(args.Cancel);Đ
        Assert.Equal("Aä1D¤5C¤B CÔ5 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;CT< D 0CÄ>CÄC D 5C 5.", args.ErrorMessage);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task Trigger_ShouldCancel_WhenParentDoesNotExistInDatabase()Đ
    {Đ
        // ArrangeĐ
        await using TestDbContext context = await GetContextAsync();Đ
        var trigger = new ReferenceTreeParentTrigger();Đ
Đ
        var missingParentId = Guid.NewGuid();Đ
        var node = new TestNode { Name = "Orphan Node", ParentId = missingParentId };Đ
Đ
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(Đ
            node, EntityStateChangeEnum.Added, [], context);Đ
Đ
        // ActĐ
        await trigger.HandleAsync(args);Đ
Đ
        // AssertĐ
        Assert.True(args.Cancel);Đ
        Assert.Contains("CÔ5 CÔ0C”4CT=", args.ErrorMessage);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task Trigger_ShouldNotCancel_WhenParentExistsInDatabase()Đ
    {Đ
        // ArrangeĐ
        await using TestDbContext context = await GetContextAsync();Đ
        var trigger = new ReferenceTreeParentTrigger();Đ
Đ
        var parent = new TestNode { Name = "Valid Parent" };Đ
        context.Add(parent);Đ
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);Đ
Đ
        var child = new TestNode { Name = "Child Node", ParentId = parent.Id };Đ
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(Đ
            child, EntityStateChangeEnum.Added, [], context);Đ
Đ
        // ActĐ
        await trigger.HandleAsync(args);Đ
Đ
        // AssertĐ
        Assert.False(args.Cancel);Đ
        Assert.Null(args.ErrorMessage);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task Trigger_ShouldCancel_WhenParentIsSoftDeleted()Đ
    {Đ
        // ArrangeĐ
        await using TestDbContext context = await GetContextAsync();Đ
```

```
        var trigger = new ReferenceTreeParentTrigger();Ð
Ð
        var parent = new TestNodeÐ
        {Ð
            Id = Guid.NewGuid(),Ð
            Name = "Deleted Parent",Ð
            DeletedAt = DateTime.UtcNow,Ð
            DeletedBy = "Admin"Ð
        };Ð
        context.Add(parent);Ð
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);Ð
Ð
        var child = new TestNode { Name = "Child of Dead", ParentId = parent.Id };Ð
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(Ð
            child, EntityStateChangeEnum.Added, [], context);Ð
Ð
        // ActÐ
        await trigger.HandleAsync(args);Ð
Ð
        // AssertÐ
        Assert.True(args.Cancel);Ð
        Assert.Equal("AÔ5C´LCtO CÔ0Ct=C GC„BDÂ @Cä4C„BCT;CT< D44C ;CT=CÔKC′ >C JCT:D"â"Â  &w2äW'&÷$ÖW76 vR“½
    }Ð
Ð
    [Fact]Ð
    public async Task Trigger_ShouldCancel_WhenDirectCyclicDependencyDetected()Ð
    {Ð
        // ArrangeÐ
        await using TestDbContext context = await GetContextAsync();Ð
        var trigger = new ReferenceTreeParentTrigger();Ð
Ð
        // 1. B >Ct4C 5CÂ 8 D >DT@C =Dô5CÂ @Cä4C„BCT;DÄAC¤8C′ MC´5CÄ5CÔB A
        var nodeA = new TestNode { Id = Guid.NewGuid(), Name = "Node A" };Ð
        context.Add(nodeA);Ð
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);Ð
Ð
        // 2. B >Ct4C 5CÂ 8 D >DT@C =Dô5CÂ 4CäGCT@CÔ8C′ MC´5CÄ5CÔB B (B >CD8D$5C´L: A •
        var nodeB = new TestNode { Id = Guid.NewGuid(), Name = "Node B", ParentId = nodeA.Id };Ð
        context.Add(nodeB);Ð
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);Ð
Ð
        // 3. A„<C„BC„@D45CÂ ?Cä?D´BC¤C D 4CT;C BDÂ CCt5C² " @Cä4C„BCT;CT< CD;Dò CCt;C    „ CTBC´O: A -> B ->
        nodeA.ParentId = nodeB.Id;Ð
Ð
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(Ð
            nodeA, EntityStateChangeEnum.Modified, [], context);Ð
Ð
        // ActÐ
        await trigger.HandleAsync(args);Ð
Ð
        // AssertÐ
        Assert.True(args.Cancel);Ð
        Assert.Equal("Bd8C¤;C„GCTAC¤0Dò 7C 2C„AC„<CäAD$L: CÔ5C´LCtO Cô5D 5CÄ5D BC„BDÂ @Cä4C„BCT;DÄAC¤8C′ CCt5
    }Ð
Ð
    [Fact]Ð
    public async Task Trigger_ShouldCancel_WhenDeepCyclicDependencyDetected()Ð
    {Ð
        // ArrangeÐ
        await using TestDbContext context = await GetContextAsync();Ð
        var trigger = new ReferenceTreeParentTrigger();Ð
Ð
        // Aô>D BD >C„< Dd5Cô>Dt:D3¢ &ö÷B Óâ 6†–ÆB Óâ 7V$6†–ÆM
        var root = new TestNode { Id = Guid.NewGuid(), Name = "Root" };Ð
        context.Add(root);Ð
Ð
        var child = new TestNode { Id = Guid.NewGuid(), Name = "Child", ParentId = root.Id };Ð
        context.Add(child);Ð
Ð
        var subChild = new TestNode { Id = Guid.NewGuid(), Name = "SubChild", ParentId = child.Id };Ð
        context.Add(subChild);Ð
Ð
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);Ð
Ð
        // A„<C„BC„@D45CÂ ?Cä?D´BC¤C Cô5D 5CÄ5D BC„BDÂ AC <D´9 C$5D ECÔ8C′ CCt5C² %&ö÷B" 2CÔCD$@DÂ 5C4> C4;D4
        // Bd5Cô>Dt:C  ?D >C$5D :C, ?Cä9CD5D" 2C$5D E: SubChild (C"  AB′ Óâ 6†–ÆB „2 A  ) -> Root (Cô5D 5CÄ5D
```

```
        root.ParentId = subChild.Id;Ð
Ð
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(Ð
            root, EntityStateChangeEnum.Modified, [], context);Ð
Ð
        // ActÐ
        await trigger.HandleAsync(args);Ð
Ð
        // AssertÐ
        Assert.True(args.Cancel);Ð
        Assert.Equal("Bd8C¤;C„GCTAC¤0Dò 7C 2C„AC„<CäAD$L: CÔ5C´LCtO Cô5D 5CÄ5D BC„BDÂ @Cä4C„BCT;DÄAC¤8C’ CCt5
    }Ð
}
</file>

<file path="Tests/Trigger/TreeTriggerChainTests.cs">
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.Extensions.Configuration;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Moq;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Text.Json;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Trigger;Ð
Ð
// --- B4 A„ A  BÄ B´  B #B” Aä!B$  AD Bò  At A´/Bd A, "AT!B$ A" ÒÒÝ
public class SoftDeleteNode : ReferenceTreeBase<SoftDeleteNode> { }Ð
public class SelfParentNode : ReferenceTreeBase<SelfParentNode> { }Ð
Ð
// Aä1D”8C’ :Cä=D$5C¤AD" 4C´O C„=D$5C4@C FC„>CÔ=D´E D$5D BCä2 (D$5Cô5D L D  ”6öæf–wW& F–öâ•
public class TreeTestDbContext(Ð
    DbContextOptions<ApplicationDbContext> options,Ð
    IConfiguration configuration)Ð
    : ApplicationDbContext(options, configuration)Ð
{Ð
    protected override void OnModelCreating(ModelBuilder modelBuilder)Ð
    {Ð
        base.OnModelCreating(modelBuilder);Ð
        modelBuilder.Entity<SoftDeleteNode>();Ð
        modelBuilder.Entity<SelfParentNode>();Ð
    }Ð
}Ð
Ð
public class TreeTriggerChainTestsÐ
{Ð
    private (ServiceProvider Provider, IConfiguration Config) CreateProviderAndConfig()Ð
    {Ð
        // B >Ct4C 5CÂ DCT9C¤>C$CDâ :Cä=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()Ð
            .AddInMemoryCollection(new Dictionary<string, string?>Ð
            {Ð
                ["DatabaseSettings:DefaultSchema"] = "test"Ð
            })Ð
            .Build();Ð
Ð
        var services = new ServiceCollection();Ð
        services.AddLogging();Ð
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;Dô5CÂ :Cä=DD8C=
Ð
        // 1. AäACÔ>C$=D´5 D 5D 2C„AD½
        services.AddScoped<DatabaseTriggerService>();Ð
        services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());Ð
Ð
        // 2. B  A4 B "B  B #AT  A$!AR "B  A4 AT B½
        services.AddScoped<FluentValidationTrigger>();Ð
        services.AddScoped<AuditTrigger>();Ð
        services.AddScoped<ReferenceTreeParentTrigger>();Ð
Ð
        // 3. At0C$8D 8CÄ>D BC, BD 8C43CT@Cä2Ð
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });Ð
Ð
```

```
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();Đ
        services.AddSingleton(factoryMock.Object);Đ
Đ
        // 4. At0C4;D4HC¤8 CD;Dò 8CÔBCT@DD5C”ACä2Đ
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);Đ
Đ
        return (services.BuildServiceProvider(), configuration);Đ
    }Đ
Đ
    [Fact]Đ
    public async Task SaveChanges_ShouldCancel_WhenParentIsSoftDeleted_ThroughChain()Đ
    {Đ
        // --- ARRANGE ---Đ
        (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();Đ
        using IServiceScope scope = serviceProvider.CreateScope();Đ
        IServiceProvider sp = scope.ServiceProvider;Đ
Đ
        var triggerService = sp.GetRequiredService<DatabaseTriggerService>();Đ
        triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();Đ
Đ
        // A„!Aô A  A´ AÔ  AD Bò ääUB   ¢  Ct2C´5C¤0CT< DD0C @C„:D2 66÷ R 8Cr BCTAD$>C$>C4> Cô@Cä2C 9CD5D 0Đ
        var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();Đ
Đ
        // A„!Aô A  A´ AÔ : Aô5D 5CD0CT< C" 8CÔBCT@Dd5CôBCä@ D$>C´LC¤> Cä4C„= C @C4CCÄ5CÔB — scopeFactoryĐ
        DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()Đ
            .UseInMemoryDatabase(Guid.NewGuid().ToString())Đ
            .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))Đ
            .Options;Đ
Đ
        // Aô5D 5CD0CT< Cä?Dd8C, 8 C¤>CÔDC„3Đ
        await using var context = new TreeTestDbContext(options, configuration);Đ
Đ
        // 1. B >Ct4C 5CÂ $CCD0C´5CÔ=Cä3Câ" @Cä4C„BCT;Dý
        var deadParent = new SoftDeleteNodeĐ
        {Đ
            Id = Guid.NewGuid(),Đ
            Name = "Dead Parent",Đ
            DeletedAt = DateTime.UtcNowĐ
        };Đ
        context.Add(deadParent);Đ
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);Đ
Đ
        // 2. B 5C 5CÔ>C-
        var child = new SoftDeleteNodeĐ
        {Đ
            Name = "Child",Đ
            ParentId = deadParent.IdĐ
        };Đ
        context.Add(child);Đ
Đ
        // --- ACT & ASSERT ---Đ
        await Assert.ThrowsAsync<OperationCanceledException>(async () =>Đ
        {Đ
            await context.SaveChangesAsync(TestContext.Current.CancellationToken);Đ
        });Đ
    }Đ
Đ
    [Fact]Đ
    public async Task SaveChanges_ShouldCancel_WhenSelfParenting_ThroughChain()Đ
    {Đ
        // --- ARRANGE ---Đ
        (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();Đ
        using IServiceScope scope = serviceProvider.CreateScope();Đ
        IServiceProvider sp = scope.ServiceProvider;Đ
Đ
        var triggerService = sp.GetRequiredService<DatabaseTriggerService>();Đ
        triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();Đ
Đ
        // A„!Aô A  A´ AÔ  AD Bò ääUB   ¢  Ct2C´5C¤0CT< DD0C @C„:D2 66÷ R 8Cr BCTAD$>C$>C4> Cô@Cä2C 9CD5D 0Đ
        var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();Đ
Đ
        // A„!Aô A  A´ AÔ : Aô5D 5CD0CT< C" 8CÔBCT@Dd5CôBCä@ D$>C´LC¤> Cä4C„= C @C4CCÄ5CÔB — scopeFactoryĐ
        DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()Đ
            .UseInMemoryDatabase(Guid.NewGuid().ToString())Đ
```

```
            .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))Ð
            .Options;Ð
Ð
        await using var context = new TreeTestDbContext(options, configuration);Ð
Ð
        var node = new SelfParentNode { Id = Guid.NewGuid(), Name = "Self" };Ð
        node.ParentId = node.Id;Ð
        context.Add(node);Ð
Ð
        // --- ACT & ASSERT ---Ð
        var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>Ð
            await context.SaveChangesAsync(TestContext.Current.CancellationToken));Ð
Ð
        Assert.Equal("Aä1D¤5C¤B CÔ5 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;CT< D 0CÄ>CÄC D 5C 5.", exception.Message);Ð
    }Ð
}
</file>

<file path="Tests/Validator/DomainObjectValidatorTests.cs">
using FluentValidation.TestHelper;Ð
using Promatis.Net.Domain;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Validator;Ð
Ð
public class DomainObjectValidatorTestsÐ
{Ð
    // 1. B >Ct4C 5CÂ BCTAD$>C$KC’ >C JCT:D" 8 CT3Câ 2C ;C„4C BCä@Ð
    private class TestEntity : DomainObject { }Ð
Ð
    private class TestEntityValidator : DomainObjectValidator<TestEntity> { }Ð
Ð
    private readonly TestEntityValidator _validator = new();Ð
Ð
    [Fact]Ð
    public void Validator_ShouldFail_WhenIdIsEmpty()Ð
    {Ð
        // ArrangeÐ
        var entity = new TestEntity { Id = Guid.Empty };Ð
Ð
        // ActÐ
        TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);Ð
Ð
        // AssertÐ
        result.ShouldHaveValidationErrorFor(x => x.Id)Ð
            .WithErrorMessage("A„4CT=D$8DD8C¤0D$>D  >C JCT:D$0 CÔ5 CÄ>Cd5D" 1D´BDÂ ?D4AD$KCÂâ"“½
    }Ð
Ð
    [Fact]Ð
    public void Validator_ShouldFail_WhenCreatedAtIsInFuture()Ð
    {Ð
        // ArrangeÐ
        var entity = new TestEntityÐ
        {Ð
            CreatedAt = DateTime.UtcNow.AddMinutes(5)Ð
        };Ð
Ð
        // ActÐ
        TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);Ð
Ð
        // AssertÐ
        result.ShouldHaveValidationErrorFor(x => x.CreatedAt)Ð
            .WithErrorMessage("AD0D$0 D >Ct4C =C„O CÔ5 CÄ>Cd5D" 1D´BDÂ 2 C CCDCD”5CÂâ"“½
    }Ð
Ð
    [Fact]Ð
    public void Validator_ShouldPass_WhenObjectIsValid()Ð
    {Ð
        // ArrangeÐ
        var entity = new TestEntity(); // Id C, 7&V FVD B 7C ?Cä;CÔODäBD O C" :Cä=D BD CC¤BCä@CR 1C 7Cä2D´E C
Ð
        // ActÐ
        TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);Ð
Ð
        // AssertÐ
        result.ShouldNotHaveAnyValidationErrors();Ð
```

```
    }Ð
}
</file>

<file path="Tests/Validator/ReferenceBaseValidatorTests.cs">
using FluentValidation.TestHelper;Ð
using Promatis.Net.Domain;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Validator;Ð
Ð
public class ReferenceBaseValidatorTestsÐ
{Ð
    // B$5D BCä2C O D CD”=CäAD$L C, 5E  2C ;C„4C BCä@Ð
    private class TestReference : ReferenceBase { }Ð
    private class TestReferenceValidator : ReferenceBaseValidator<TestReference> { }Ð
Ð
    private readonly TestReferenceValidator _validator = new();Ð
Ð
    [Theory]Ð
    [InlineData("")]Ð
    [InlineData(" ")]Ð
    [InlineData("A")] // AÄ5CÔLD„5 2 D 8CÄ2Cä;Cä2Ð
    public void Name_ShouldHaveErrors_WhenInvalid(string? invalidName)Ð
    {Ð
        var model = new TestReference { Name = invalidName! };Ð
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);Ð
Ð
        result.ShouldHaveValidationErrorFor(x => x.Name);Ð
    }Ð
Ð
    [Fact]Ð
    public void Name_ShouldHaveError_WhenTooLong()Ð
    {Ð
        var model = new TestReference { Name = new string('A', 251) };Ð
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);Ð
Ð
        result.ShouldHaveValidationErrorFor(x => x.Name)Ð
              .WithErrorMessage("Aô@CT2D´HCT=C  <C :D 8CÄ0C´LCÔ0Dò 4C´8CÔ0 CÔ0C„<CT=Cä2C =C„O (250 D 8CÄ2Cä;C
    }Ð
Ð
    [Theory]Ð
    [InlineData("VALID_CODE_123")]Ð
    [InlineData("REF1")]Ð
    [InlineData("")] // B$5Cô5D L DÔBCâ ?D >C”4CTB D4ACô5D„=Cí
    [InlineData(null)] // A, MD$> D$>Cd5Ð
    public void Code_ShouldBeValid_WhenCorrectFormat(string? code)Ð
    {Ð
        var model = new TestReference { Code = code };Ð
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);Ð
Ð
        result.ShouldNotHaveValidationErrorFor(x => x.Code);Ð
    }Ð
Ð
    [Theory]Ð
    [InlineData("C¤>CEô=C ô@D4AD :Cä<")]Ð
    [InlineData("lower_case")]Ð
    [InlineData("CODE-1")] // B$8D 5 Ct0Cô@CTICT=Cí
    public void Code_ShouldHaveError_WhenInvalidFormat(string invalidCode)Ð
    {Ð
        var model = new TestReference { Code = invalidCode };Ð
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);Ð
Ð
        result.ShouldHaveValidationErrorFor(x => x.Code);Ð
    }Ð
Ð
    [Fact]Ð
    public async Task ValidateValue_ShouldReturnErrors_OnlyForSpecificProperty()Ð
    {Ð
        // ArrangeÐ
        var model = new TestReferenceÐ
        {Ð
            Name = "", // AäHC„1C¤0Ð
            Code = "INVALID-CODE" // AäHC„1C¤0Ð
        };Ð
Ð
```

```
        // ActÐ
        // Aô@Cä2CT@Dô5CÂ @C 1CäBD2 2C HCT3Câ 4CT;CT3C BC  f Æ–F FUf ÇVR 4C´O Cô>C´O NameÐ
        IEnumerable<string> nameErrors = await _validator.ValidateValue(model, 
nameof(TestReference.Name));Ð
        IEnumerable<string> codeErrors = await _validator.ValidateValue(model, 
nameof(TestReference.Code));Ð
Ð
        // AssertÐ
        Assert.Contains(nameErrors, e => e.Contains("AÔ0C„<CT=Cä2C =C„5 Cä1Dô7C BCT;DÄ=Câ"’“½
        Assert.Contains(codeErrors, e => e.Contains("A¤>CB <Cä6CTB D >CD5D 6C BDÂ BCä;DÄ:Câ ;C BC„=C„FD2"’“½
    }Ð
Ð
    [Theory]Ð
    [InlineData(" Name")]Ð
    [InlineData("Name ")]Ð
    public void Name_ShouldHaveError_WhenHasLeadingOrTrailingSpaces(string invalidName)Ð
    {Ð
        var model = new TestReference { Name = invalidName };Ð
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);Ð
Ð
        result.ShouldHaveValidationErrorFor(x => x.Name)Ð
            .WithErrorMessage("AÔ0C„<CT=Cä2C =C„5 CÔ5 CÄ>Cd5D" =C GC„=C BDÄADò 8C´8 Ct0C¤0CÔGC„2C BDÄADò ?D >
    }Ð
Ð
    [Fact]Ð
    public async Task ValidateValue_ShouldIgnoreOtherPropertiesErrors()Ð
    {Ð
        // Arrange: Cä1C  ?Cä;Dò =CT2C ;C„4CÔKÐ
        var model = new TestReference { Name = "A", Code = "low_case" };Ð
Ð
        // Act: C$0C´8CD8D CCT< B$ A´,A¤  NameÐ
        IEnumerable<string> nameErrors = await _validator.ValidateValue(model, 
nameof(TestReference.Name));Ð
Ð
        // AssertÐ
        Assert.Single(nameErrors); // B$>C´LC¤> CäHC„1C¤0 CD;C„=D² 8CÄ5CÔ8Ð
        Assert.DoesNotContain(nameErrors, e => e.Contains("A¤>CB"’“² òò  D„8C >C¢ :Cä4C  1D´BDÂ =CR 4Cä;Cd=Cí
    }Ð
}
</file>

<file path="Tests/Validator/ReferenceTreeBaseValidatorTests.cs">
using FluentValidation.TestHelper;Ð
using Promatis.Net.Domain;Ð
using Xunit;Ð
Ð
namespace Promatis.Net.ApplicationModel.Tests.Validator;Ð
Ð
public class ReferenceTreeBaseValidatorTestsÐ
{Ð
    // B$5D BCä2C O D CD”=CäAD$L CD;Dò ?D >C$5D :C, 0C AD$@C :D$=Cä3Câ :C´0D AC
    private class TestNode : ReferenceTreeBase<TestNode> { }Ð
Ð
    // B$5D BCä2D´9 C$0C´8CD0D$>D
    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }Ð
Ð
    private readonly TestNodeValidator _validator = new();Ð
Ð
    [Fact]Ð
    public void Validator_ShouldFail_WhenParentIdIsEqualToId()Ð
    {Ð
        // ArrangeÐ
        var nodeId = Guid.NewGuid();Ð
        var node = new TestNodeÐ
        {Ð
            Id = nodeId,Ð
            ParentId = nodeId, // AäHC„1C¤0: ID D >C$?C 4C 5D" A ParentIdÐ
            Name = "Self-referencing node"Ð
        };Ð
Ð
        // ActÐ
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);Ð
Ð
        // AssertÐ
        result.ShouldHaveValidationErrorFor(x => x.ParentId)Ð
              .WithErrorMessage("Aä1D¤5C¤B CÔ5 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;CT< D 0CÄ>CÄC D 5C 5");Ð
```

```
    }Ð
Ð
    [Fact]Ð
    public void Validator_ShouldFail_WhenParentIdIsEmptyGuid()Ð
    {Ð
        // ArrangeÐ
        var node = new TestNodeÐ
        {Ð
            ParentId = Guid.Empty, // AäHC„1C¤0 Cô> C$BCä@Cä<D2 ?D 0C$8C´CÐ
            Name = "Empty ParentId"Ð
        };Ð
Ð
        // ActÐ
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);Ð
Ð
        // AssertÐ
        result.ShouldHaveValidationErrorFor(x => x.ParentId)Ð
              .WithErrorMessage("B >CD8D$5C´LD :C„9 C„4CT=D$8DD8C¤0D$>D  =CR <Cä6CTB C KD$L CôCD BD´< GUID");
    }Ð
Ð
    [Fact]Ð
    public void Validator_ShouldPass_WhenParentIdIsDifferent()Ð
    {Ð
        // ArrangeÐ
        var node = new TestNodeÐ
        {Ð
            Id = Guid.NewGuid(),Ð
            ParentId = Guid.NewGuid(),Ð
            Name = "Valid child node"Ð
        };Ð
Ð
        // ActÐ
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);Ð
Ð
        // AssertÐ
        result.ShouldNotHaveValidationErrorFor(x => x.ParentId);Ð
    }Ð
Ð
    [Fact]Ð
    public void Validator_ShouldPass_WhenParentIdIsNull()Ð
    {Ð
        // Arrange (A¤>D 5CÔL CD5D 5C$0)Ð
        var node = new TestNodeÐ
        {Ð
            Id = Guid.NewGuid(),Ð
            ParentId = null,Ð
            Name = "Root node"Ð
        };Ð
Ð
        // ActÐ
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);Ð
Ð
        // AssertÐ
        result.ShouldNotHaveValidationErrorFor(x => x.ParentId);Ð
    }Ð
}
</file>

</files>
```